

**BỘ GIAO THÔNG VẬN TẢI
TRƯỜNG ĐẠI HỌC HÀNG HẢI
BỘ MÔN: KHOA HỌC MÁY TÍNH
KHOA: CÔNG NGHỆ THÔNG TIN**

Bài giảng XỬ LÝ ẢNH

TÊN HỌC PHẦN : Xử lý ảnh

MÃ HỌC PHẦN : 17221

TRÌNH ĐỘ ĐÀO TẠO : ĐẠI HỌC CHÍNH QUY

DÙNG CHO SV NGÀNH : CÔNG NGHỆ THÔNG TIN

8. Đề cương chi tiết các học phần:

8.1. Tên học phần: Xử lý ảnh

Mã HP: 17221

a. Số tín chỉ: 03 TC

BTL ☐

ĐAMH ☐

b. Đơn vị giảng dạy: Bộ môn Khoa học máy tính

c. Phân bổ thời gian:

- Tổng số (TS): 45 tiết.

- Lý thuyết (LT): 28

- Thực hành (TH): 30 tiết.

- Bài tập (BT): 0

- Hướng dẫn BTL/ĐAMH (HD): 0 tiết.

- Kiểm tra (KT): 2

d. Điều kiện đăng ký học phần:

Đã học xong các học phần: Đại số tuyến tính, Đồ họa máy tính, Kỹ thuật lập trình C

e. Mục đích, yêu cầu của học phần:

Cung cấp nền tảng lý thuyết và các phương pháp của xử lý ảnh số (digital image processing).

f. Mô tả nội dung học phần:

Các phương pháp thu thập, lưu trữ ảnh, các phép biến đổi tăng chất lượng ảnh, biến đổi Fourier hai chiều, khôi phục ảnh (Image Restoration), biến đổi hình thái (Morphological), các phép lọc ảnh tuyến tính và không gian, phép lọc phi tuyến, ảnh nhị phân, phát hiện biên, phân đoạn ảnh là các nội dung chính của môn học

g. Người biên soạn: Nguyễn Hữu Tuân – Khoa Công nghệ Thông tin

h. Nội dung chi tiết học phần:

TÊN CHƯƠNG MỤC	PHÂN PHỐI SỐ TIẾT					
	TS	LT	BT	TH	HD	KT
Chương I. Tổng quan về xử lý ảnh	5	4	0	2	0	0
1.1. Khái niệm ảnh số		0.5				
1.2. Một số ứng dụng của xử lý ảnh số		0.5				
1.3. Các bước cơ bản trong xử lý ảnh số		0.5				
1.4. Lấy mẫu và lượng tử hóa ảnh		0.5				
1.5. Mối quan hệ giữa các điểm ảnh		0.5		0.5		
1.6. Nội suy ảnh		0.5		0.5		
1.7. Matlab và OpenCV		1		1		
Chương II. Các thao tác xử lý dựa trên điểm ảnh	6	3	0	4	0	1
2.1. Khái niệm		0.5				
2.2. Các thao tác dựa trên điểm ảnh cơ bản		1		1		
2.3. Các thao tác dựa trên histogram của ảnh		1		2		
2.4. Biến đổi logic và đại số		0.5		1		
Chương III. Nâng cao chất lượng ảnh bằng các bộ lọc không gian	7	4	0	6	0	0
3.1. Nguyên tắc của phép lọc ảnh		0.5				
3.2. Các phép lọc ảnh làm mịn ảnh		1		2		

3.3. Các phép lọc làm sắc nét ảnh		1.5		3		
3.4. Edge detection		1		1		
Chương IV. Xử lý ảnh trong miền tần số	8	6	0	4	0	0
4.1. Miền tần số và biến đổi Fourier 2 chiều		1.5				
4.2. Lọc ảnh trong miền tần số		0.5				
4.3. Làm mờ ảnh bằng bộ lọc thông thấp		1.5		2		
4.4. Làm sắc nét ảnh bằng bộ lọc thông cao		1.5		2		
4.5. Các vấn đề liên quan khác		1				
Chương V. Khôi phục ảnh	7	5	0	4	0	0
5.1. Khái niệm cơ bản		0.5				
5.2. Khôi phục ảnh bằng loại bỏ nhiễu		2		2		
5.3. Khôi phục ảnh theo mô hình về độ mờ ảnh		1.5		2		
5.4. Một số kỹ thuật khác		1				
Chương VI. Các phép xử lý hình thái	7	4	0	4	0	1
6.1. Giới thiệu		0.5				
6.2. Các phép toán hình thái trên ảnh nhị phân		1		1		
6.3. Các phép toán hình thái trên ảnh xám		1		2		
6.4. Một số ứng dụng của phép toán hình thái		1.5		2		
Chương VII. Phân đoạn ảnh	5	3	0	4	0	0
7.1. Khái niệm		0.5				
7.2. Phân đoạn dựa vào tách ngưỡng		1		2		
7.3. Phân đoạn dựa trên các vùng ảnh con		1		1		
7.4. Phân đoạn dựa vào kỹ thuật phân cụm (clustering)		0.5		1		
Tổng:	45	28	0	30	0	2

i. Mô tả cách đánh giá học phần:

Sinh viên phải tham dự tối thiểu 75% số giờ lên lớp và phải đạt các điểm thành phần X_1 , X_2 từ 4,0 trở lên (X_1 là điểm chuyên cần, X_2 là điểm trung bình các bài kiểm tra).

Điểm học phần (Z) được tính theo công thức: $Z = 0.3X + 0.7Y$

Trong đó:

- X : điểm quá trình, bằng trung bình cộng của X_1 , X_2 .
- Y : điểm bài kiểm tra kết thúc học phần

Thang điểm đánh giá: A^+ , A , B^+ , B , C^+ , C , D và F

k. Giáo trình:

R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Addison-Wesley, Third edition, 2008.

l. Tài liệu tham khảo:

1. R.C. Gonzalez, R.E. Woods, and S. Eddins, *Digital Image Processing Using MATLAB*, 2nd edition, Pearson Prentice Hall, 2008

2. C. Solomon and T. Breckon, *Fundamentals of Digital Image processing: A Practical Approach with Examples in MATLAB*, John Wiley & Sons, 2011.

Trưởng Khoa**Trưởng Bộ môn****Người biên soạn*****TS Lê Quốc Định******TS. Nguyễn Duy Trường Giang******TS. Nguyễn Hữu Tuân****o. Tiến trình cập nhật Đề cương:*

Cập nhật lần 1: ngày 26/06/2014 Nội dung: Rà soát theo kế hoạch Nhà trường gồm: - Chính sửa, làm rõ các Mục c, e, i theo các mục tiêu đổi mới căn bản. - Mục h : bổ sung Nội dung tự học cuối mỗi chương mục, chuyển một số nội dung giảng dạy sang phần tự học. - Bổ sung các mục m, n, o	Người cập nhật <i>Nguyễn Hữu Tuân</i> P. Trưởng Bộ môn <i>Nguyễn Văn Thủy</i>
Cập nhật lần 2: ngày / / Nội dung	Người cập nhật Trưởng Bộ môn

MỤC LỤC

DANH MỤC HÌNH VẼ	vii
DANH MỤC BẢNG	xi
LỜI NÓI ĐẦU	1
CHƯƠNG I: TỔNG QUAN VỀ XỬ LÝ ẢNH	2
1. Khái niệm ảnh số	3
2. Một số ứng dụng của xử lý ảnh số	5
3. Các bước cơ bản trong xử lý ảnh số	7
4. Lấy mẫu và lượng tử hóa ảnh	9
5. Mối quan hệ giữa các điểm ảnh	10
5.1. Quan hệ láng giềng (Neighbour)	10
5.2. Độ tương phản của ảnh (image contrast)	11
5.3. Khoảng cách giữa các điểm ảnh	11
6. Nội suy ảnh	11
7. Matlab và OpenCV	12
7.1. Matlab	12
7.2. OpenCV	13
7.3. Lập trình OpenCV với Visual Studio 2010	16
Bài tập	17
CHƯƠNG II: CÁC THAO TÁC XỬ LÝ DỰA TRÊN ĐIỂM ẢNH	18
1. Khái niệm	19
2. Một số thao tác biến đổi dựa trên điểm ảnh cơ bản	19
2.1. Lấy âm bản (Negative image)	19
2.2. Tăng độ tương phản (contrast enhancement)	19
2.3. Tách ngưỡng (thresholding)	20
2.4. Biến đổi logarithmic	21
2.5. Biến đổi Power-Law	21
2.6. Gamma correction	23
3. Các thao tác dựa trên histogram của ảnh	24
3.1. Histogram của ảnh	24
3.2. Cân bằng histogram	26
3.3. Một số ứng dụng của histogram	27
4. Biến đổi logic và số học	28
4.1. Biến đổi dựa trên phép toán số học	28
4.2. Biến đổi dựa trên phép toán logic	30
Bài tập	30
CHƯƠNG III: NÂNG CAO CHẤT LƯỢNG ẢNH BẰNG CÁC BỘ LỌC KHÔNG GIAN	32
1. Nguyên tắc của phép lọc ảnh	32
2.1. Phép nhân chập (convolution)	32
2. Các phép lọc làm mịn ảnh	34
2.1. Lọc tuyến tính	34
2.2. Lọc phi tuyến	36
3. Các phép lọc làm sắc nét ảnh	41
3.1. Bộ lọc Linear sharpening	41

3.2. Bộ lọc High boost.....	41
3.3. Đạo hàm của ảnh (Image derivative)	42
3.4. Giới thiệu một số bộ lọc phổ biến.....	45
4. Phát hiện biên (edge detection).....	47
4.1. Giới thiệu.....	47
4.2. Phát hiện biên dựa vào gradient của ảnh.	48
Bài tập	50
CHƯƠNG IV: XỬ LÝ ẢNH TRONG MIỀN TẦN SỐ	52
1. Miền tần số và biến đổi Fourier hai chiều	52
1.1. Các khái niệm cơ sở.....	52
1.2. Lý thuyết Fourier	54
1.3. Biến đổi Fourier rời rạc (Discrete Fourier Transform-DFT)	56
2. Các phép lọc ảnh trong miền tần số.....	62
2.1. Lọc ảnh trong miền tần số	62
2.2. Làm mượt ảnh bằng phép lọc thông thấp (Lowpass filter)	65
2.3. Làm sắc nét ảnh bằng phép lọc thông cao (Highpass filter)	69
2.4. Lọc dải thông (Bandpass filter).....	72
2.5. Lọc chặn dải thông (Band stop filter).....	73
2.6. Toán tử Laplacian trong miền tần số.....	75
2.7. Lọc đồng hình (Homomorphic filter)	77
Bài tập	79
CHƯƠNG V: KHÔI PHỤC ẢNH.....	80
1. Khái niệm cơ bản.....	80
1.1. Phân biệt khôi phục ảnh và tăng cường chất lượng ảnh	80
1.2. Mô hình suy thoái chất lượng ảnh (degradation model)	80
1.3. Đánh giá hiệu năng.....	81
2. Khôi phục ảnh bằng loại bỏ nhiễu	81
2.1. Mô hình nhiễu (Noise model)	81
2.2. Bộ lọc trung bình cho lọc nhiễu	86
2.3. Loại bỏ nhiễu bằng bộ lọc Non-Local Means.....	86
2.4. Bộ lọc trung vị linh hoạt (Adaptive median filter) cho lọc nhiễu xung	89
2.5. Lọc ảnh trong miền tần số để khử nhiễu định kỳ.....	90
3. Khôi phục ảnh dựa vào mô hình độ mờ ảnh.....	96
3.1. Mô hình tuyến tính (Linear Blur Model).....	96
3.2. Bộ lọc ngược (Inverse filter)	96
3.3. Bộ lọc Wiener.....	96
4. Một số kỹ thuật khôi phục ảnh khác	98
4.1. Kỹ thuật Super resolution	98
4.2. Khôi phục ảnh bằng biểu diễn thưa (Sparse representation).....	98
Bài tập	98
CHƯƠNG VI: CÁC PHÉP XỬ LÝ HÌNH THÁI.....	99
1. Giới thiệu.....	99
1.1. Lý thuyết tập hợp	99
1.2. Phép toán logic trên ảnh nhị phân.....	101
1.3. Tính liên thông	101

2. Các phép toán hình thái trên ảnh nhị phân	103
2.1. Phép giãn ảnh (dilation).....	103
2.2. Phép co ảnh (erosion)	103
2.3. Phép đóng ảnh (closing) và phép mở ảnh (opening).....	104
2.4. Biến đổi hit-and-miss	105
2.5. Một số kiểu phần tử cấu trúc thông dụng.....	106
3. Các phép toán hình thái trên ảnh xám	107
4. Ứng dụng các phép toán hình thái trong xử lý ảnh	109
4.1. Trích chọn đường bao (contour) sử dụng phép toán hình thái	109
4.2. Làm mảnh (thinning) sử dụng phép toán hình thái	109
4.3. Trích chọn thành phần liên thông	110
4.5. Làm mượt ảnh dựa vào phép toán hình thái	111
4.4. Phát hiện biên (edge detection) dựa vào phép toán hình thái.....	112
4.5. Tìm xương (skeleton) của ảnh	113
Bài tập	114
CHƯƠNG VII: PHÂN ĐOẠN ẢNH	115
1. Khái niệm	115
2. Phân đoạn dựa trên tách ngưỡng	115
2.1. Kỹ thuật chọn ngưỡng toàn cục cơ bản.....	116
2.2. Chọn ngưỡng tối ưu theo phương pháp của Otsu	118
2.3. Phân ngưỡng biến thiên (Variable thresholding)	119
3. Phân đoạn dựa trên vùng con của ảnh	121
4. Phân đoạn ảnh dựa vào kỹ thuật phân cụm	123
4.1. Thuật toán K-mean	124
4.2. Thuật toán Mean-shift.....	125
Bài tập	127
TÀI LIỆU THAM KHẢO	128

DANH MỤC HÌNH VẼ

Hình 1.1: Sự khác nhau giữa tăng cường chất lượng và biểu diễn đặc điểm ảnh	2
Hình 1.2: Hệ màu RGB	3
Hình 1.3: Cấu trúc của một ảnh màu RGB.....	3
Hình 1.4: Các loại ảnh số	4
Hình 1.5: Ảnh hưởng của độ sâu màu tới chất lượng ảnh.....	5
Hình 1.6: Phân biệt Xử lý ảnh với Thị giác máy tính với Đồ họa máy tính	6
Hình 1.7: Các bước chính trong một hệ thống xử lý ảnh.....	7
Hình 1.8: Các thành phần của một hệ thống xử lý ảnh	8
Hình 1.9: Xử lý ảnh màu RGB.....	8
Hình 1.10: Thiết bị CCD.....	9
Hình 1.11: Lấy mẫu và lượng tử hóa	10
Hình 1.12: Quan hệ láng giềng của một điểm ảnh	10
Hình 1.13: Nội suy bilinear.....	12
Hình 1.14: Giao diện Matlab 2010b.....	12
Hình 1.15: Logo của OpenCV	13
Hình 1.16: Tổng quan về các hàm của OpenCV	14
Hình 1.17: Thiết lập sử dụng OpenCV với Visual Studio 2010	15
Hình 2.1: Các kỹ thuật nâng cao chất lượng ảnh	18
Hình 2.2: Tăng cường chất lượng ảnh dựa vào các phép biến đổi không gian	18
Hình 2.3: Hàm biến đổi dựa trên điểm ảnh	19
Hình 2.4: Tách ngưỡng nhị phân và tách ngưỡng linh hoạt.....	20
Hình 2.5: Các phép biến đổi dựa trên điểm ảnh	22
Hình 2.6: Gamma correction trên thiết bị hiển thị.....	23
Hình 2.7: Ví dụ về Gamma Correction.....	23
Hình 2.8: Histogram của ảnh	24
Hình 2.9: Ảnh và histogram của các channel khác nhau.....	24
Hình 2.10: Histogram của ảnh sáng và ảnh tối.....	25
Hình 2.11: Histogram và độ tương phản của ảnh.....	25
Hình 2.12: Mối quan hệ giữa độ tương phản và histogram	26
Hình 2.13: Nguyên tắc cân bằng histogram	26
Hình 2.14: Cân bằng histogram ảnh.....	27
Hình 2.15: Các phép biến đổi số học	29
Hình 2.16: Các phép biến đổi logic.....	30
Hình 3.1: Phép nhân chập	32
Hình 3.2: Minh họa phép nhân chập	33
Hình 3.3: Các kiểu padding của phép nhân chập	33
Hình 3.4: Biểu diễn trực quan của bộ lọc Gaussian	34
Hình 3.5: Một ví dụ về bộ lọc Gaussian	34
Hình 3.6: Lọc trung bình và lọc Gaussian với các kích thước kernel khác nhau	35
Hình 3.7: Phân tích một bộ lọc Gaussian 2D thành tích	36
Hình 3.8: Minh họa lọc trung vị	36
Hình 3.9: Tác dụng của lọc trung vị.....	37
Hình 3.10: Bộ lọc Alpha-Trimmed Mean	37
Hình 3.11: Lọc Gaussian, lọc trung vị và lọc ATM	38

Hình 3.12: Sự kết hợp của bộ lọc spatial kernel và range kernel.....	39
Hình 3.13: Lọc Bilateral so với lọc Gaussian và Median	40
Hình 3.14: Các bộ lọc phi tuyến Min, Max và Mid-point.....	40
Hình 3.15: Kỹ thuật linear sharpening	42
Hình 3.16: Kết quả lọc ảnh bằng toán tử Laplacian.....	45
Hình 3.17: Một số hiệu ứng với các bộ lọc phổ biến	47
Hình 3.18: Ảnh và các thành phần Gradient	49
Hình 3.19: Phát hiện biên dựa trên gradient của ảnh	50
Hình 4.1: Các bộ lọc ảnh trong miền tần số	52
Hình 4.2: Biểu diễn số phức	52
Hình 4.3: Biểu diễn của hàm sin và cosin	53
Hình 4.4: Sinusoid.....	53
Hình 4.5: Biểu diễn của hàm $y=3\sin(t)$	54
Hình 4.6: Biểu diễn của hàm $y=\cos(4t)$	54
Hình 4.7: Biểu diễn tần số của tín hiệu 1-D	54
Hình 4.8: Lý thuyết Fourier	56
Hình 4.9: Tính tuần hoàn của DFT	57
Hình 4.10: Biểu diễn DFT trong một chu kỳ	57
Hình 4.11: Biểu diễn của DFT sau phép dịch	58
Hình 4.12: Tính tuần hoàn của biến đổi DFT 2-D	59
Hình 4.13: Biểu diễn 2-D DFT dùng dịch vòng	59
Hình 4.14: Phép dịch vòng trong biểu diễn 2-D DFT	60
Hình 4.15: Phase và Magnitude của ảnh trong biến đổi 2-D DFT.....	60
Hình 4.16: Vai trò của Magnitude và Phase trong khôi phục ảnh	61
Hình 4.17: Các bước của phép lọc ảnh trong miền tần số.....	63
Hình 4.18: Các phép lọc trong miền tần số	63
Hình 4.19: Các bước của phép lọc ảnh trong miền tần số.....	65
Hình 4.20: Tần số bị loại bỏ trong lọc thông thấp ILPF.....	66
Hình 4.21: Bán kính năng lượng trong bộ lọc thông thấp ILPF	66
Hình 4.22: Hiện tượng Ringing effect.....	67
Hình 4.23: Lọc thông thấp Butterworth và Gaussian	67
Hình 4.24: Biểu diễn trực quan 2D và 3D của các bộ lọc thông thấp.....	68
Hình 4.25: Ứng dụng thực tế của lọc thông thấp	69
Hình 4.26: Biểu diễn trong miền không gian của các bộ lọc thông cao	70
Hình 4.27: Kết quả thực hiện các phép lọc thông cao	71
Hình 4.28: Một ứng dụng của lọc thông cao	71
Hình 4.29: Biểu diễn các bộ lọc dải thông.....	72
Hình 4.30: Kết quả thực hiện phép lọc dải thông.....	73
Hình 4.31: Biểu diễn các bộ lọc chặn dải thông	74
Hình 4.32: Phép lọc chặn dải thông	75
Hình 4.33: Toán tử Laplacian trong miền tần số.....	76
Hình 4.34: Các bước của phép lọc đồng hình	77
Hình 4.35: Kết quả phép lọc đồng hình Gaussian	78
Hình 4.36: Loại bỏ tần số trong lọc đồng hình	79
Hình 5.1: Mô hình suy thoái ảnh.....	80
Hình 5.2: Biểu diễn của hàm mật độ xác suất nhiều Gaussian.....	82

Hình 5.3: Biểu diễn hàm mật độ của mô hình nhiễu Rayleigh.....	82
Hình 5.4: Biểu diễn hàm mật độ của mô hình nhiễu Gamma.....	83
Hình 5.5: Mô hình nhiễu hàm mũ	83
Hình 5.6: Biểu diễn của hàm mật độ mô hình nhiễu Uniform	84
Hình 5.7: Biểu diễn hàm mật độ xác suất của nhiễu xung	84
Hình 5.8: Ảnh hưởng của các mô hình nhiễu lên ảnh	85
Hình 5.9: Ước lượng tham số nhiễu.	86
Hình 5.10: Tính chất self-similarity của ảnh	87
Hình 5.11: Minh họa cách làm việc của thuật toán NLM	87
Hình 5.12: So sánh bộ lọc NLM với bộ lọc Gaussian và Median	88
Hình 5.13: Bộ lọc trung vị linh hoạt (Amed) vs. Gaussian (G), Median(M) và ATM.....	89
Hình 5.14: Bộ lọc Bandstop cho nhiễu định kỳ	90
Hình 5.15: Nhiễu định kỳ của ảnh trên biểu diễn phổ DFT	91
Hình 5.16: Các bộ lọc Notch Reject	92
Hình 5.17: Lọc nhiễu bằng bộ lọc Notch Reject.....	94
Hình 5.18: Khôi phục ảnh bằng bộ lọc tối ưu Gaussian Notch.....	95
Hình 5.19: Kết quả thực hiện bộ lọc ngược và bộ lọc Wiener.....	97
Hình 6.1: Ví dụ về các thành phần của ảnh	99
Hình 6.2: Các phép toán trên tập hợp	100
Hình 6.3: Phép toán logic trên ảnh nhị phân	101
Hình 6.4: Các mẫu lân cận của một điểm ảnh.....	102
Hình 6.5: Ví dụ về các thành phần liên thông	102
Hình 6.6: Phép giãn ảnh và co ảnh	104
Hình 6.7: Ứng dụng phép co ảnh để loại bỏ các phần tử không phù hợp	104
Hình 6.8: Phép đóng ảnh	105
Hình 6.9: Các kiểu phần tử cấu trúc.....	106
Hình 6.10: Qui tắc áp dụng phép toán hình thái với ảnh nhị phân	107
Hình 6.11: Qui tắc áp dụng phép toán hình thái với ảnh xám	107
Hình 6.12: Các phép xử lý hình thái với ảnh xám.....	108
Hình 6.13: Trích chọn đường bao sử dụng phép toán hình thái	109
Hình 6.14: Ví dụ trích chọn đường bao thực tế	109
Hình 6.15: Các bước của phép làm mảnh	110
Hình 6.16: Làm mảnh ảnh bằng phép xử lý hình thái	111
Hình 6.17: Trích chọn thành phần liên thông	111
Hình 6.18: Làm mượt ảnh bằng phép xử lý hình thái	111
Hình 6.19: Phát hiện biên bằng phép xử lý hình thái (ngưỡng=20)	112
Hình 6.20: Tìm xương của ảnh bằng phép xử lý hình thái	114
Hình 7.1: Chọn ngưỡng trong phân đoạn ảnh dựa trên kỹ thuật tách ngưỡng	115
Hình 7.2: Ảnh hưởng của nhiễu đối với việc chọn ngưỡng.....	116
Hình 7.3: Ảnh hưởng của nhiễu và ánh sáng đối với việc chọn ngưỡng.....	116
Hình 7.4: Phân đoạn sử dụng chọn ngưỡng toàn cục.....	117
Hình 7.5: Phân ngưỡng bằng phương pháp Otsu.....	119
Hình 7.6: Biến dữ liệu ảnh thành dãy zigzag	120
Hình 7.7: Tính ngưỡng cho dãy zigzag.....	120
Hình 7.8: So sánh phân ngưỡng cục bộ và Otsu.....	121
Hình 7.9: Phân đoạn ảnh dựa trên vùng Region Growing.....	122

Hình 7.10: Cơ sở của phân đoạn ảnh dựa vào phân cụm	123
Hình 7.11: Phân đoạn ảnh bằng thuật toán K-mean	124
Hình 7.12: Phân đoạn ảnh K-mean theo các tiêu chí độ tương đồng khác nhau	125
Hình 7.13: Thuật toán phân cụm Mean-shift	126
Hình 7.14: Kết quả phân đoạn ảnh bằng thuật toán Mean-shift	127

DANH MỤC BẢNG

Bảng 3.1: Các bộ lọc dùng để tính đạo hàm	43
Bảng 3.2: Toán tử Laplacian	44
Bảng 3.3: Bộ lọc làm sắc nét ảnh bằng toán tử Laplacian	45
Bảng 3.4: Bộ lọc làm mờ ảnh	46
Bảng 3.5: Bộ lọc tạo hiệu ứng blur motion	46
Bảng 3.6: Bộ lọc làm sắc nét ảnh.....	46
Bảng 3.7: Bộ lọc tạo hiệu ứng Emboss.....	47

LỜI NÓI ĐẦU

Sự phát triển nhanh chóng của các thiết bị thu nhận ảnh cầm tay (camera, camcorder, mobile phone, tablette ..), cũng như năng lực xử lý đồ họa của các hệ thống máy tính hiện đại (sử dụng các hệ thống vi xử lý đa nhân, luồng và các chip đồ họa chuyên nghiệp GPU) và sự bùng nổ của các hệ thống mạng xã hội (Facebook, Youtube ...) đòi hỏi các hệ thống có khả năng xử lý ảnh một cách nhanh chóng và tiện lợi ngày càng trở nên cấp thiết. Xử lý ảnh (hay xử lý ảnh số - Digital Image Processing) là ngành khoa học nghiên cứu các thuật toán thu nhận, xử lý ảnh số trên các hệ thống máy tính (computer) với mục đích làm tăng cường chất lượng của hình ảnh thu nhận được, và đồng thời có thể làm nền tảng cho các ngành khoa học ứng dụng khác như Thị giác máy tính (Computer Vision), Robotics hoặc viễn thám (Remote sensing), xử lý ảnh y học (Medical Imaging). Bài giảng Xử lý ảnh này được chia thành 7 chương nhằm cung cấp kiến thức về các phương pháp thu thập, lưu trữ ảnh, các kỹ thuật tăng cường chất lượng ảnh trong miền không gian dựa trên điểm ảnh, các phép lọc không gian dựa trên lân cận, phát hiện biên, xử lý ảnh trong miền tần số với biến đổi Fourier rời rạc, các thao tác xử lý hình thái, khôi phục và phân đoạn ảnh cho các sinh viên ngành Công nghệ Thông tin ở bậc Đại học. Mặc dù đã dành nhiều tâm huyết và cố gắng tối đa trong quá trình biên soạn nhưng vẫn có thể có một số thiếu sót, hy vọng sẽ được các bạn bè đồng nghiệp, các em sinh viên, các bạn độc giả góp ý chân thành để tôi có thể hoàn thiện hơn nữa tài liệu này. Mọi góp ý xin gửi về địa chỉ email: tuannhtn@gmail.com.

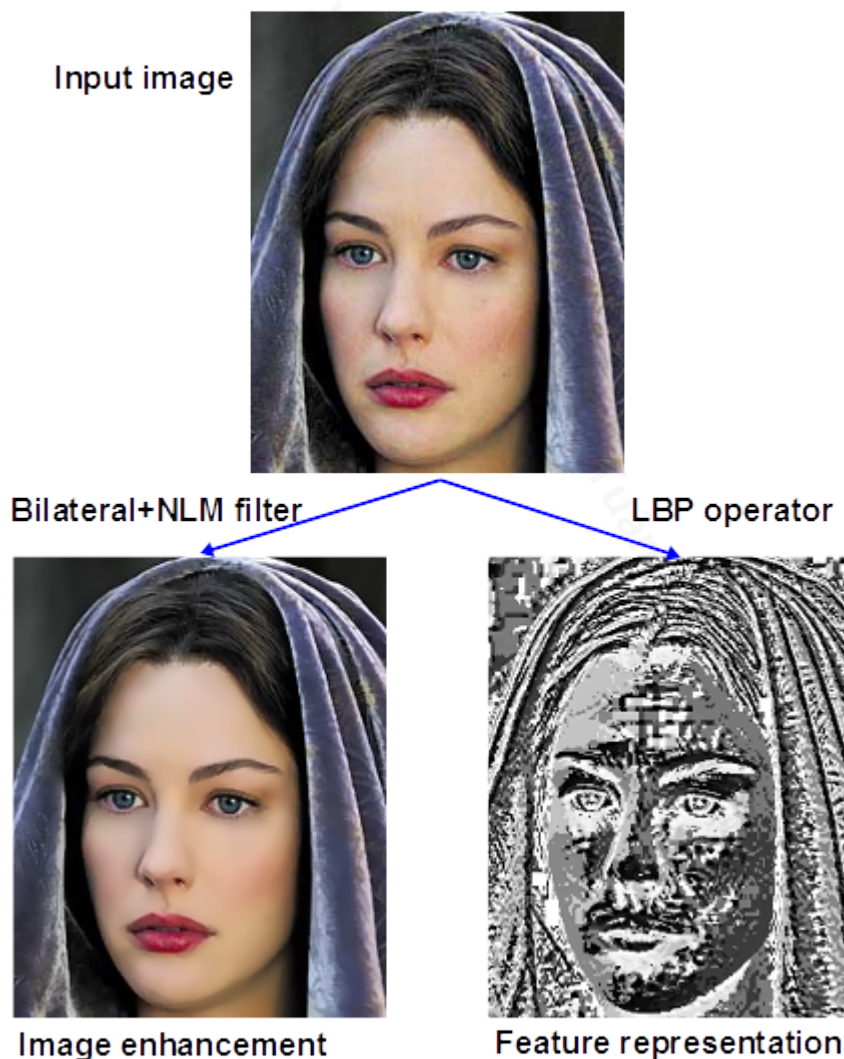
Tháng 12 năm 2013

Tác giả

Nguyễn Hữu Tuấn

CHƯƠNG I: TỔNG QUAN VỀ XỬ LÝ ẢNH

Xử lý ảnh là một lĩnh vực nghiên cứu quan trọng của công nghệ thông tin nói chung và xử lý tín hiệu số (Digital Signal Processing) nói riêng. Ngày nay, xử lý ảnh đã trở thành một công nghệ phát triển với các thuật toán được ứng dụng để xử lý các bức ảnh số lưu trên các hệ thống máy tính, mobile, tablette, camera nhằm tạo ra các bức ảnh có chất lượng tốt hơn (cho việc nhận thức về hình ảnh của con người) hoặc đưa ra các biểu diễn gồm các đặc trưng của ảnh làm cơ sở cho việc lưu trữ hoặc truyền ảnh và các ngành mang tính ứng dụng khác như thị giác máy tính, viễn thám, xử lý ảnh y học (medical imaging), robotics, đồ họa máy tính, hoặc thậm chí là các lĩnh vực pháp lý/hình sự. Tuy nhiên trên thực tế hai mục đích này có thể tạo ra các bức ảnh trái ngược nhau theo cảm nhận của con người: các bức ảnh được đánh giá là đẹp mắt đối với con người thường không giúp ích cho các thao tác nhận dạng của máy tính và ngược lại. Có thể thấy rõ điều này trong hình 1.1, một bức ảnh màu lọc bằng bộ lọc Bilateral và Non Local Means (NLM) cho kết quả đẹp mắt đối với hệ thống thị giác của con người nhưng giúp ích nhiều cho việc nhận dạng với máy tính, trong khi bức ảnh sinh ra từ việc áp dụng toán tử mẫu nhị phân cục bộ (Local Binary Patterns - LBP) có biểu diễn không đẹp mắt nhưng lại giúp ích cho việc nhận dạng mặt người của máy tính.

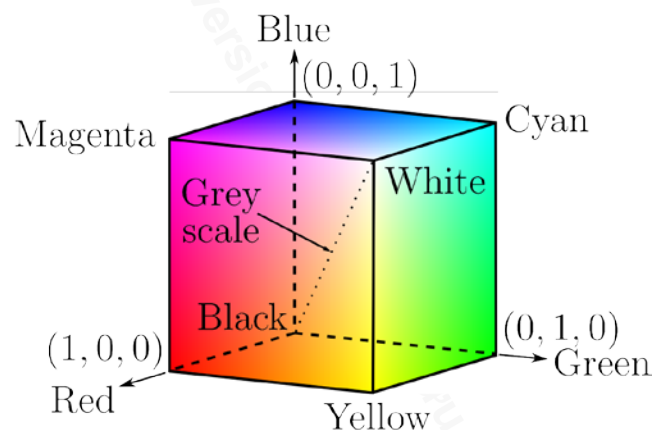


Hình 1.1: Sự khác nhau giữa tăng cường chất lượng và biểu diễn đặc điểm ảnh

1. Khái niệm ảnh số

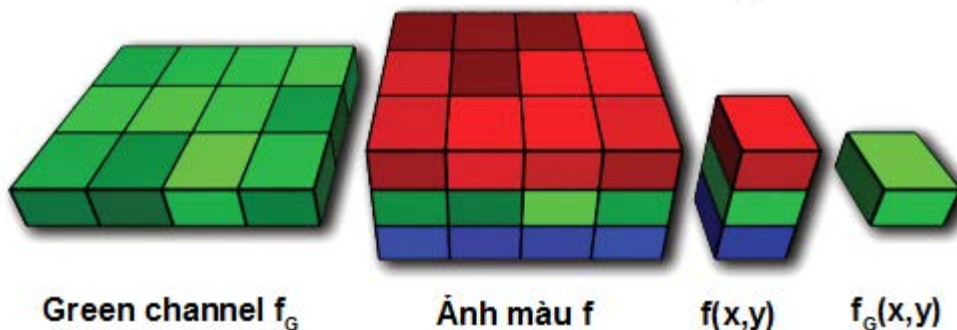
Đối tượng của xử lý ảnh là các bức ảnh số được biểu diễn trên máy tính bằng các điểm ảnh gọi là các pixels. Thông thường, cấu trúc ma trận với các phần tử là các điểm ảnh sẽ được dùng để biểu diễn ảnh trong bộ nhớ của máy tính. Khi đó mỗi điểm ảnh có một vị trí không gian (spatial location) trong bức ảnh gọi là tọa độ và được biểu diễn bằng chỉ số hàng và chỉ số cột của nó trong ma trận (x, y) . Giá trị của một điểm ảnh $f(x, y)$ được gọi là cường độ sáng (intensity) hoặc mức xám (gray level) thể hiện màu sắc của nó. Chỉ các số nguyên được sử dụng để biểu diễn giá trị của một điểm ảnh. Các điểm ảnh trong một bức ảnh màu sẽ được biểu diễn bằng các thành phần màu cơ sở (như đỏ-red, xanh lá cây-green và xanh nước biển-blue) và sử dụng một kỹ thuật tổng hợp để mô phỏng các màu sắc của thế giới tự nhiên. Sự khác biệt của một bức ảnh số (digital image) và một bức ảnh (photo) nằm tính liên tục (continuous) của bức ảnh (về không gian và màu sắc) so với tính không liên tục (discontinuous) của ảnh số vì các giá trị x , y và $f(x, y)$ hoàn toàn là rời rạc (discrete) và thực chất chỉ là một xấp xỉ của một hình ảnh thuộc thế giới thực.

Các màu sắc cơ sở dùng để biểu diễn một ảnh màu tạo thành một hệ màu (Color model) và được đề xuất dựa trên các nghiên cứu nhận thức thị giác của con người. Hệ màu RGB (red, green, blue) được đề xuất từ thế kỷ 19 là phổ biến nhất:



Hình 1.2: Hệ màu RGB

Trong hệ màu RGB mỗi điểm ảnh được biểu diễn bằng 3 giá trị số nguyên thuộc khoảng $[0..255]$ đại diện cho 3 màu sắc cơ bản, màu của điểm ảnh sẽ được tạo thành qua việc kết hợp ba giá trị màu này.



Hình 1.3: Cấu trúc của một ảnh màu RGB

Xét trên quan điểm màu sắc, tất cả các ảnh đều là ảnh màu (color image) vì màu đen và màu trắng đều là màu sắc. Tuy nhiên trong phạm vi học phần này, chúng ta phân biệt ba loại ảnh sau:

+ Ảnh màu (color image) là một ảnh có thể chứa bất cứ màu nào (xem hình minh họa 1.2 với ảnh RGB). Thường các ảnh này được tổ chức theo một hệ màu nào đó và có nhiều channels.

+ Ảnh xám (grayscale image) là ảnh mà giá trị màu của nó nằm trên đường chéo trắng-đen của không gian màu RGB. Cụ thể hơn trên đường chéo này (các điểm có màu xám), các giá trị màu thuộc 3 thành phần Red, Green, Blue đều bằng nhau nên chỉ cần một giá trị để biểu diễn. Thông thường mỗi giá trị cường độ ảnh (intensity) được biểu diễn bằng 8-bit sẽ cho một tập 256 giá trị mà xám từ 0 tới 255. Đây là loại ảnh số phổ biến nhất và trong tài liệu này, trừ khi có đề cập cụ thể tới ảnh màu hoặc ảnh nhị phân, các thao tác, thuật toán đều được ngầm định là thực hiện cho ảnh xám. Cũng có thể coi ảnh xám là ảnh có một channel trong hệ màu RGB.

+ Ảnh nhị phân (binary image) là ảnh mà mỗi giá trị điểm ảnh chỉ là trắng hoặc đen. Hình 1.4 cho ta sự phân biệt về ba loại ảnh số này.



Ảnh màu

Ảnh xám

Ảnh nhị phân

Hình 1.4: Các loại ảnh số

Độ sâu màu (Color depth) của ảnh: là đại lượng mô tả khả năng một ảnh số có khả năng biểu diễn các màu sắc khi biểu diễn (trên các thiết bị màn hình điện tử) hoặc in ấn. Đại lượng này được tính bằng số lượng các bit cần dùng để biểu diễn cho một điểm ảnh đơn bpp (bits per pixel). Có thể phân biệt ba loại ảnh màu, xám, và nhị phân qua khái niệm độ sâu màu với ảnh nhị phân là ảnh 2 bpp, ảnh xám là ảnh 8 bpp, ảnh màu là ảnh 24 bpp (xem hình 1.5). Trên thực tế hầu hết các ứng dụng không cần ảnh có độ sâu màu quá lớn (24 bpp là đủ) trừ các ứng dụng đặc biệt trong lĩnh vực y tế hoặc trong nghiên cứu khoa học.



Hình 1.5: Ảnh hưởng của độ sâu màu tới chất lượng ảnh

Độ phân giải (resolution) của ảnh: đại lượng đo mức độ chi tiết của ảnh và được tính bằng số cột nhân với số hàng của ảnh. Ảnh có độ phân giải càng cao thì khả năng biểu diễn các chi tiết càng lớn. Các độ phân giải thường được sử dụng là 640x480, 800x600, 1600x1200.

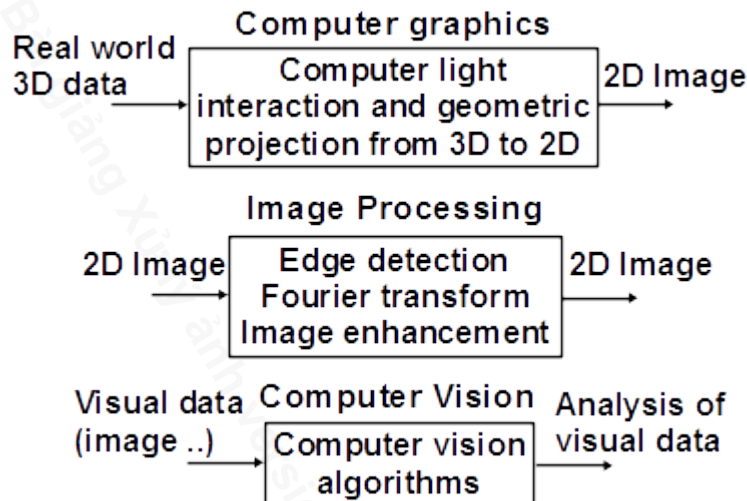
Tỉ lệ ảnh (aspect ratio): là giá trị số cột chia cho số hàng của ảnh. Ví dụ ảnh có độ phân giải 800x600 sẽ có tỉ lệ là 4:3 hay 1,33.

Tần số của ảnh (image frequency): Tần số của ảnh là một đại lượng đo lường thay đổi về màu sắc trên toàn bộ biểu diễn không gian của ảnh. Các thay đổi cường độ sáng trong một khoảng cách nhỏ là tần số cao. Các thay đổi cường độ sáng nhỏ trên một khoảng cách lớn là tần số thấp. Đại lượng tần số có liên quan chặt chẽ tới độ phân giải của ảnh. Ảnh có độ phân giải cao có thể biểu diễn các tần số cao tốt hơn các ảnh có độ phân giải thấp. Hầu hết các ảnh đều có các tần số cao và tần số thấp trong các vùng ảnh khác nhau.

2. Một số ứng dụng của xử lý ảnh số

Phân biệt xử lý ảnh với đồ họa máy tính và thị giác máy tính: Xử lý ảnh có liên quan rất mật thiết tới đồ họa máy tính (Computer Graphics) và thị giác máy tính (Computer Vision). Trong khi đồ họa máy tính nghiên cứu các phương pháp tổng hợp

thông tin từ thế giới thực và biểu diễn output dưới dạng ảnh thu được thì xử lý ảnh nghiên cứu các thuật toán biến đổi thông tin ảnh (điểm, đường, không gian, miền tần số ...) nhằm mục đích thu được ảnh có chất lượng tốt hơn cho một mục đích nào đó hoặc biểu diễn các đặc trưng của ảnh. Thị giác máy tính nghiên cứu các thuật toán giúp máy tính có thể nhìn thấy và hiểu được các hình ảnh từ thế giới thực như con người. Thị giác máy tính sử dụng các kiến thức từ đồ họa máy tính và xử lý ảnh làm nền tảng. Hình 1.6 cho ta một sự phân biệt về ba lĩnh vực.



Hình 1.6: Phân biệt Xử lý ảnh với Thị giác máy tính với Đồ họa máy tính

Sơ lược về lịch sử phát triển của xử lý ảnh: Sự phát triển của xử lý ảnh được đánh dấu bởi các sự kiện sau:

- + Đầu những năm 1920, ứng dụng đầu tiên của xử lý ảnh được triển khai trong ngành báo chí với dịch vụ truyền ảnh Bartlane dựa trên cáp giữa London và Newyork. Ảnh được mã hóa để truyền qua hệ thống cáp ngầm qua đại dương và được khôi phục tại đầu nhận rồi in ra giấy bằng một máy in điện báo.

- + Giữa đến cuối những năm 1920, hệ thống truyền ảnh được cải tiến để có thể truyền các bức ảnh có chất lượng cao hơn.

- + Những năm 1960: Sự phát triển của công nghệ máy tính và xu hướng nghiên cứu vũ trụ thúc đẩy mạnh ngành xử lý ảnh. Các thuật toán tăng cường chất lượng ảnh được sử dụng để cải thiện chất lượng ảnh chụp vệ tinh.

- + Những năm 1970: Các ứng dụng xử lý ảnh bắt đầu được sử dụng trong lĩnh vực y tế. Với việc phát minh ra máy chụp cắt lớp CAT (Computerised Axial Tomography), hai nhà khoa học Godfrey N. Hounsfield và Allan M. Cormack đã đạt giải Nobel y học.

- + Những năm 1980 – nay: Xử lý ảnh đã phát triển rất nhanh và trở thành được sử dụng trong rất nhiều lĩnh vực khác nhau từ khoa học tới thực tế của đời sống xã hội.

Ứng dụng của xử lý ảnh: Các ứng dụng thực tế của xử lý ảnh có thể thấy trong rất nhiều lĩnh vực khác nhau, việc đưa ra một thống kê đầy đủ vượt ra ngoài phạm vi của tài liệu này, vì thế ở đây chúng ta chỉ xem xét một số lĩnh vực chủ yếu.

- + Nhiếp ảnh (Photography): đây là lĩnh vực sử dụng các thiết bị điện tử để ghi lại các khung cảnh (scene) từ thế giới thực. Các thuật toán xử lý ảnh giúp cho việc thu được các hình ảnh một cách hiệu quả hơn.

+ Viễn thám (Remote sensing): việc nghiên cứu trái đất từ các bức ảnh chụp bởi các vệ tinh ngày càng phát triển, đặc biệt khi các ảnh hưởng của biến đổi khí hậu ngày càng ảnh hưởng nặng nề tới cuộc sống của con người. Các kỹ thuật của xử lý ảnh có thể giúp cho việc thu nhận, xử lý các bức ảnh vệ tinh (như xử lý nhiễu, khôi phục ảnh ...).

+ Xử lý ảnh y học (Medial Imaging): Xử lý ảnh cung cấp các thuật toán loại bỏ nhiễu, làm sắc nét cho các bức ảnh y tế (vốn thường được chụp bởi các thiết bị sử dụng sóng siêu âm, tia X hay cộng hưởng từ (Magnetic Resonance Image – MRI)) để các chuyên gia có thể đưa ra các kết luận chính xác về tình hình sức khỏe của con người.

+ Thị giác máy tính: Xử lý ảnh cung cấp các thuật toán nền tảng cho thị giác máy tính như tiền xử lý ảnh bằng các bộ lọc loại bỏ nhiễu, biểu diễn ảnh trong miền tần số, phát hiện biên và phân đoạn ảnh.

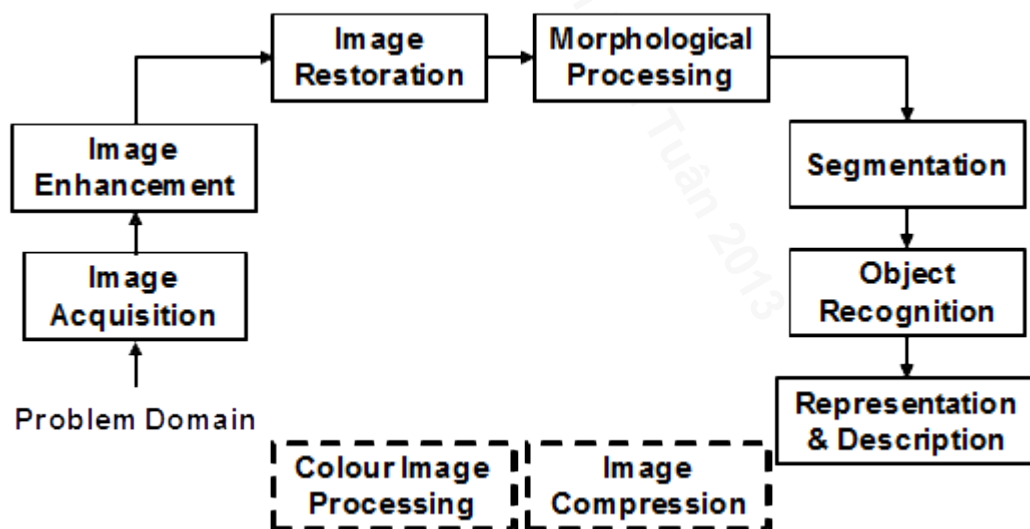
+ Robotics: Việc chế tạo các robot không thể thiếu được các thuật toán xử lý ảnh nhằm mục đích dò đường đi và nhận biết vật cản.

+ Giao thông vận tải: Các hệ thống trợ giúp quan sát giao thông và kiểm soát giao thông được xây dựng dựa nhiều vào các thuật toán xử lý ảnh.

+ Quân sự: Đây là lĩnh vực được phát triển rất mạnh, đặc biệt trong thời gian gần đây khi mà năng lực thu nhận của các thiết bị chụp và xử lý ảnh của các hệ thống máy tính đã tăng lên đáng kể. Các ứng dụng đã được triển khai trên thực tế có thể kể đến là các hệ thống phát hiện và theo dõi mục tiêu di động, hệ thống trợ giúp tác chiến trên chiến trường.

+ Điện ảnh: Việc xây dựng các bộ phim bom tấn với các kỹ xảo hoành tráng của Hollywood không thể thiếu được vai trò của các thuật toán xử lý ảnh.

3. Các bước cơ bản trong xử lý ảnh số



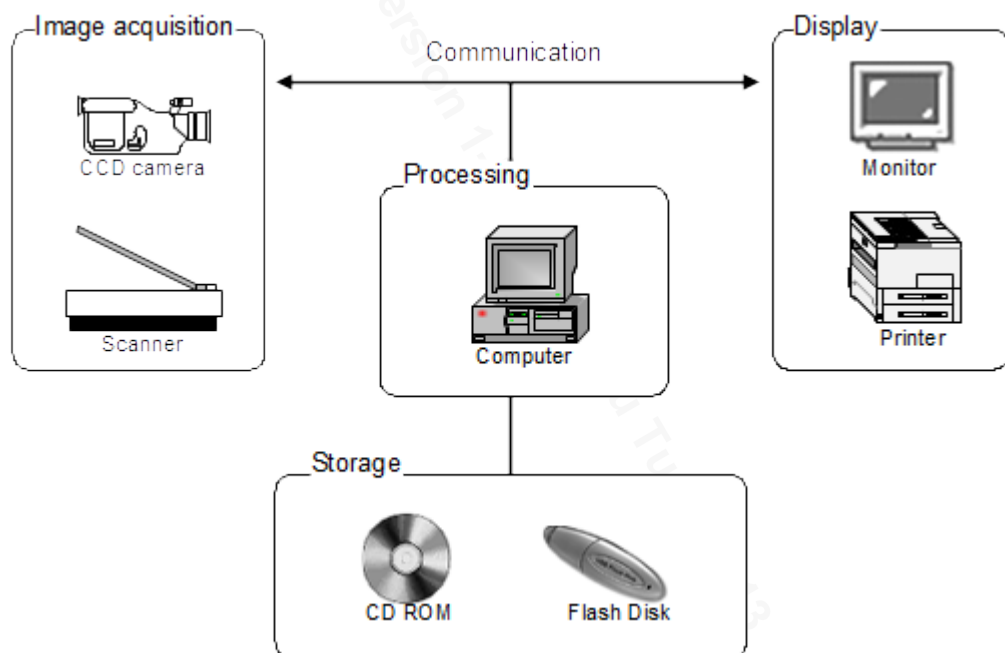
Hình 1.7: Các bước chính trong một hệ thống xử lý ảnh

Các bước chính trong xử lý ảnh gồm được minh họa trong hình 1.7, các bước này cũng là các lĩnh vực nhỏ của các kỹ thuật xử lý ảnh. Ban đầu ảnh được thu nhận từ thế giới thực, sau đó được tăng cường chất lượng nhờ các bộ lọc ảnh. Nếu có các nhiễu hoặc biến dạng trong ảnh, nó sẽ được xử lý bằng các kỹ thuật khôi phục nhằm loại bỏ các nhiễu và thu được ảnh gốc ban đầu. Tiếp đến ảnh được xử lý bằng các thuật toán

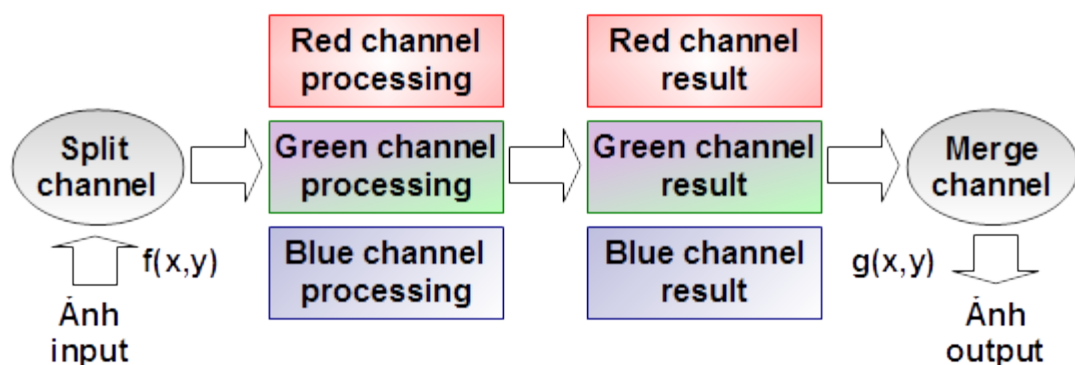
hình thái học nhằm chuẩn bị cho các bước xử lý ở mức cao hơn (các mức từ tăng cường chất lượng ảnh cho tới xử lý hình thái được gọi là các bước xử lý ở mức thấp). Ảnh được phân đoạn để phục vụ cho các bài toán nhận dạng, biểu diễn và mô tả ở mức cao hơn. Các kỹ thuật xử lý ảnh màu và nén ảnh cũng là các lĩnh vực nghiên cứu của xử lý ảnh. Trong phạm vi và giới hạn thời gian cho phép của học phần, tài liệu này chỉ đề cập tới các vấn đề: tăng cường chất lượng ảnh với các phép xử lý trong miền không gian (dựa trên các thao tác trực tiếp trên điểm ảnh và dựa vào lân cận) và trong miền tần số (với biến đổi 2D Fourier rời rạc làm nền tảng), các kỹ thuật khôi phục ảnh và xử lý nhiễu, các thuật toán xử lý hình thái cuối cùng là phân đoạn ảnh.

Các thành phần của hệ thống xử lý ảnh: Một hệ thống xử lý ảnh gồm các thành phần (xem hình minh họa 1.8): thiết bị thu nhận ảnh, hệ thống máy tính nơi diễn ra các thao tác xử lý, hệ thống lưu trữ ảnh và thiết bị xuất là màn hình điện tử hoặc máy in.

Xử lý ảnh màu: Đa số các thuật toán trình bày trong tài liệu này được thực hiện với các ảnh xám, việc áp dụng các kỹ thuật này với các ảnh màu (trong các trường hợp có thể) đơn giản là tách các channel của ảnh thành các ảnh xám rồi thực hiện cùng một thao tác với các ảnh đó (với ảnh màu là 3 channel cho các màu Red, Green, Blue) sau đó kết hợp lại để nhận được kết quả là một ảnh màu (xem minh họa ở hình 1.9).



Hình 1.8: Các thành phần của một hệ thống xử lý ảnh

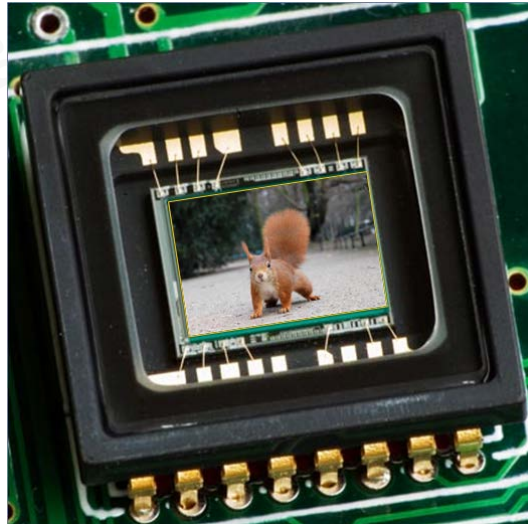


Hình 1.9: Xử lý ảnh màu RGB

4. Lấy mẫu và lượng tử hóa ảnh

Quá trình thu nhận ảnh (image acquisition) từ các khung cảnh (scene) của thế giới thực thành ảnh số được thực hiện qua các thiết bị phần cứng như máy ảnh số, máy copy, máy quét. Hầu hết các thiết bị này đều sử dụng một bộ cảm biến CCD (Charge Coupled Device) hoặc CMOS (Complementary Metal Oxide Semiconductor).

Một thiết bị CCD (hình 1.10) là một mạch điện tử với một lưới các photo-cells hình chữ nhật. Một ống kính quang học sẽ được sử dụng để chuyển hình ảnh từ thế giới thực vào các bộ cảm biến. Mỗi photo-cell sẽ đo lường ánh sáng mà nó nhận được và dữ liệu thu nhận được của các bộ cảm biến sẽ là biểu diễn cho một bức ảnh.



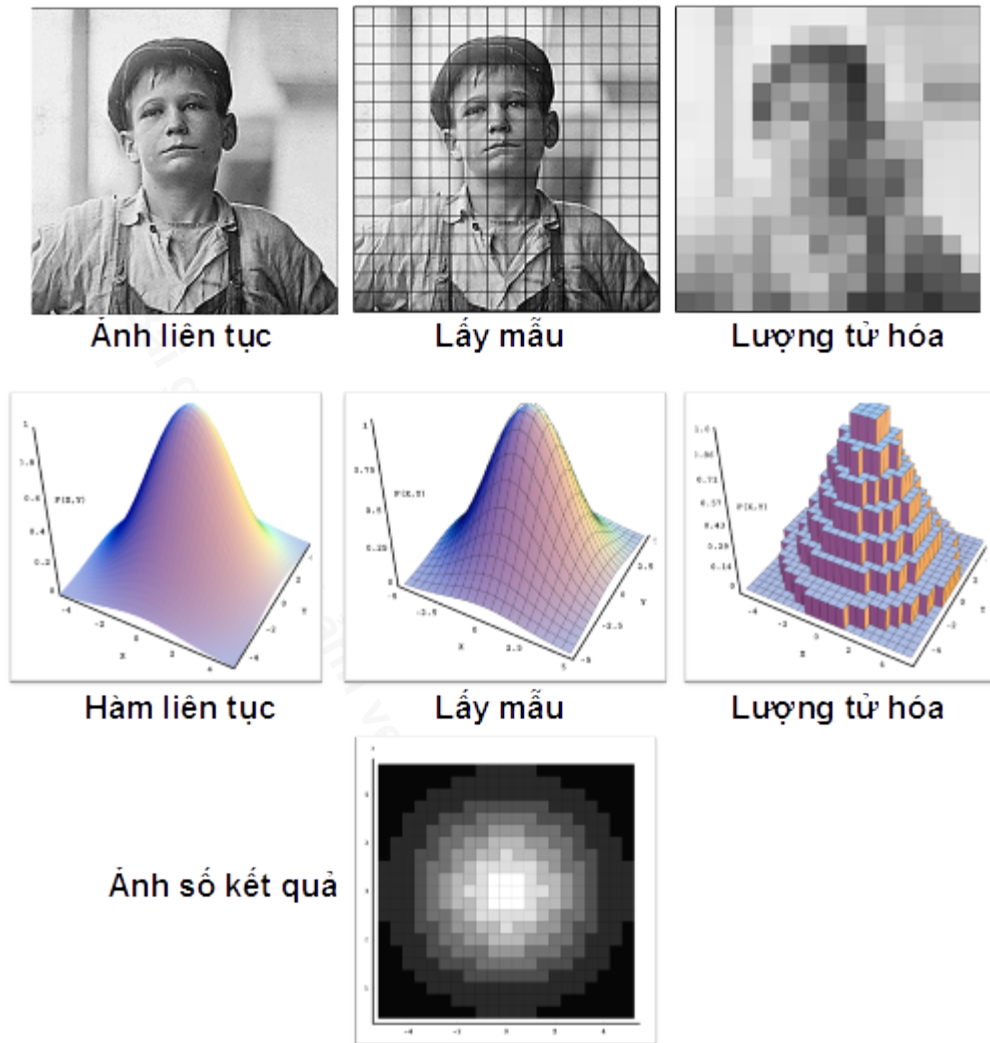
Hình 1.10: Thiết bị CCD

Ảnh số nhận được sẽ là một xấp xỉ của các tín hiệu liên tục của thế giới thực về cả ánh sáng và các đặc điểm không gian. Quá trình xấp xỉ này được thực hiện qua quá trình lấy mẫu (sampling) và lượng tử hóa (quantization) trong đó quá trình lấy mẫu là một phân hoạch không gian liên quan trực tiếp tới độ phân giải của ảnh còn quá trình lượng tử hóa là một phân hoạch các cấp độ ánh sáng và tương ứng với độ sâu màu của ảnh. Có thể hình dung hai quá trình này đối với ảnh thực và biểu diễn qua hàm trong hình 1.11.

Quá trình lấy mẫu được thực hiện qua kích thước vật lý và hình dáng (thường là hình chữ nhật) của các bộ cảm biến CCD/CMOS cụ thể. Quá trình lượng tử hóa được xác định qua mức độ nhạy cảm của các bộ cảm biến.

Như đã trình bày ở phần đầu chương, độ phân giải của ảnh quyết định mức độ chi tiết của ảnh. Tuy nhiên một câu hỏi đặt ra ở đây là cần có độ phân giải bằng bao nhiêu là đủ để thu được một bức ảnh cụ thể. Theo lý thuyết lấy mẫu về xử lý tín hiệu của Shannon và Nyquist thì nếu tần số cao nhất của tín hiệu liên tục là B thì độ phân giải cần thiết là $2B$.

Đối với ảnh màu, một máy ảnh sẽ phải thực hiện việc lấy mẫu 3 lần cho mỗi vị trí điểm ảnh để thu được 3 ảnh ở ba màu sắc khác nhau, sau đó các ảnh này sẽ được sử dụng để tạo ra ảnh kết quả.

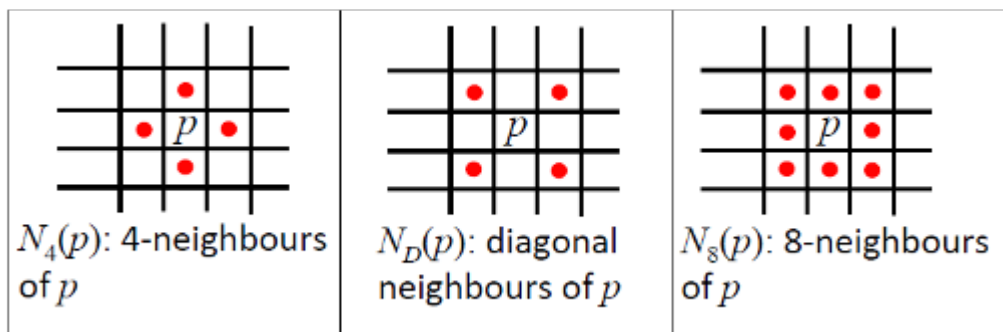


Hình 1.11: Lấy mẫu và lượng tử hóa

5. Mối quan hệ giữa các điểm ảnh

5.1. Quan hệ láng giềng (Neighbour)

Một điểm ảnh p ở vị trí (x, y) sẽ có các láng giềng, hàng xóm là các điểm nằm gần nó. Các điểm ở các tọa độ $(x-1, y)$, $(x+1, y)$, $(x, y-1)$, $(x, y+1)$ gọi là các điểm lân cận 4-neighbours của p . Các điểm ở các tọa độ $(x-1, y-1)$, $(x-1, y+1)$, $(x+1, y-1)$ và $(x+1, y+1)$ là các lân cận đường chéo của p . Tất cả các điểm lân cận 4-neighbor và đường chéo của p gọi là các điểm lân cận 8-neighbours của p .



Hình 1.12: Quan hệ láng giềng của một điểm ảnh

5.2. Độ tương phản của ảnh (image contrast)

Dựa trên độ chênh lệch giữa cường độ sáng (hoặc mức xám) của các điểm ảnh, khái niệm độ tương phản (contrast) của ảnh được định nghĩa.

Có 3 cách khác nhau để tính độ tương phản của ảnh:

- + Cách 1: $\max\{I(x,y)\} - \min\{I(x,y)\}$
- + Cách 2: $\text{var}\{I(x,y)\} = \text{mean}\{(I(x,y) - I_{av})^2\}$
- + Cách 3: $\text{std}\{I(x,y)\} = \sqrt{\text{var}\{I(x,y)\}}$

Độ tương phản cục bộ (local contrast) tại một điểm ảnh là khái niệm chỉ sự khác nhau về cường độ sáng (intensity) giữa điểm ảnh và lân cận của nó:

$$C = \left| \frac{I_p - I_n}{I_n} \right|$$

5.3. Khoảng cách giữa các điểm ảnh

Giả sử hai điểm ảnh có tọa độ là (x_1, y_1) và (x_2, y_2) . Một hàm khoảng cách là một đánh giá hình thức về mức độ gần gũi của hai điểm ảnh và mối quan hệ giữa chúng. Có nhiều hàm khoảng cách đã được đề xuất cho các mục đích khác nhau, ở đây chúng ta sẽ xem xét các hàm chung nhất.

Hàm khoảng cách Euclidean:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Hàm khoảng cách City-Block:

$$d = |x_1 - x_2| + |y_1 - y_2|$$

Hàm khoảng cách Chessboard:

$$d = \max(|x_1 - x_2|, |y_1 - y_2|)$$

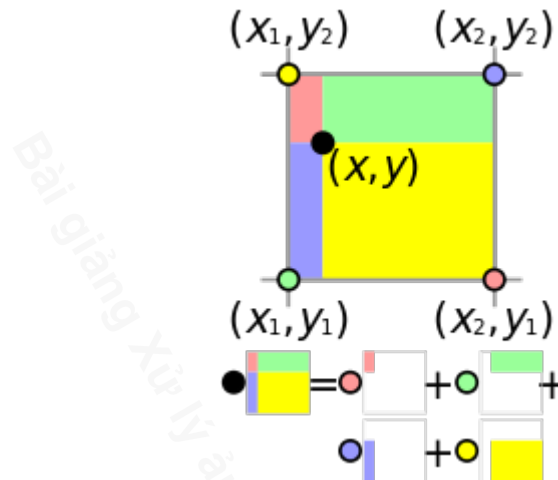
6. Nội suy ảnh

Nội suy (interpolation) là một kỹ thuật quan trọng để thực hiện các thao tác liên quan tới xử lý ảnh như thay đổi kích thước (resize), zoom, xoay (rotation), co ảnh (shrinking) hoặc thực hiện các phép khôi phục ảnh về mặt hình học. Các phép nội suy thường được thực hiện sau các phép biến đổi hình học và chúng ta chỉ nhận được một số điểm ảnh đã xác định được giá trị trong khi các điểm ảnh khác thì đã bị mất do đổi chỗ. Khi đó ta cần phải xác định các giá trị điểm ảnh nằm trên một lưới (grid) cho ảnh output và kỹ thuật này được gọi là nội suy. Ta sẽ xem xét hai phương pháp nội suy ảnh phổ biến là nearest neighbor và bilinear.

Nội suy nearest neighbor: Đây là phương pháp đơn giản nhất và cũng là tệ nhất. Mỗi điểm ảnh trên lưới sẽ được gán bằng với giá trị của điểm ảnh gần với nó nhất.

Nội suy bilinear: Giả sử ta cần tính giá trị chưa biết của một điểm ảnh tại vị trí (x, y) khi đã biết các giá trị tại các điểm $Q_{11}=(x_1, y_1)$, $Q_{12}=(x_1, y_2)$, $Q_{21}=(x_2, y_1)$, $Q_{22}=(x_2, y_2)$. Khi đó giá trị của $f(x,y)$ sẽ được tính theo công thức:

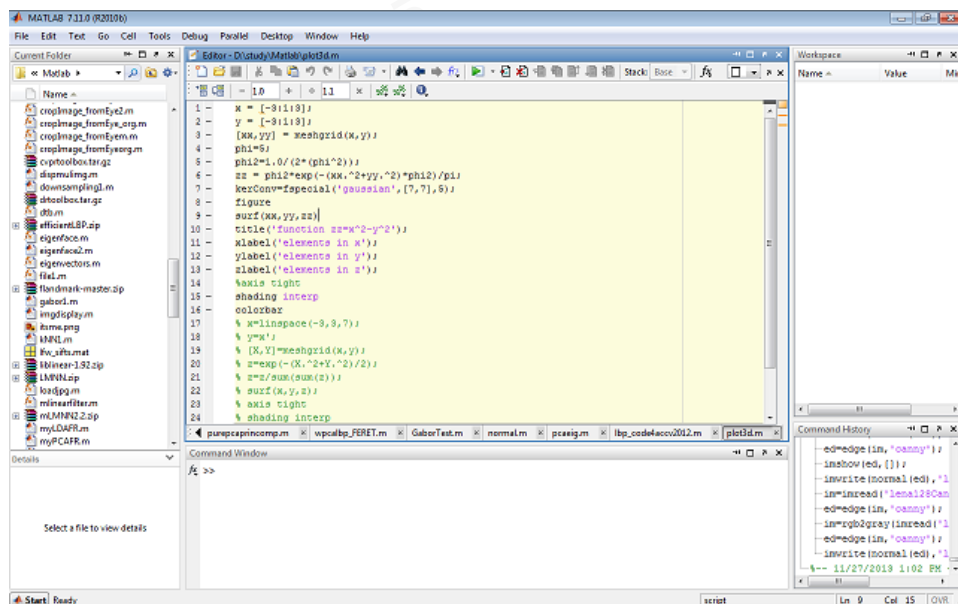
$$f(x,y) = \frac{1}{(x_2 - x_1)(y_2 - y_1)} (f(Q_{11})(x_2 - x)(y_2 - y) + f(Q_{21})(x - x_1)(y_2 - y) + f(Q_{12})(x_2 - x)(y - y_1) + f(Q_{22})(x - x_1)(y - y_1))$$



Hình 1.13: Nội suy bilinear

7. Matlab và OpenCV

7.1. Matlab



Hình 1.14: Giao diện Matlab 2010b

Matlab (MATrix LABoratory) là một công cụ tích hợp rất mạnh của công ty MathWorks. Matlab gồm nhiều thành phần khác nhau cho phép lập trình và xử lý với đơn vị cơ sở là các ma trận. Ngoài ra Matlab còn cho phép thực hiện các thao tác biểu diễn trực quan dữ liệu, đồ thị của các hàm số, tạo giao diện người dùng, liên kết với các module chương trình khác.

Ban đầu Matlab (cuối những năm 1970, Cleve Moler) là một chương trình được sử dụng chủ yếu trong nội bộ trường Stanford cho mục đích nghiên cứu, đến năm 1984, phiên bản thương mại Matlab 1.0 đầu tiên đã được đưa ra, hiện nay bản Matlab mới nhất là 2013b. Ngày nay Matlab đã tập hợp rất nhiều các thư viện, module khác nhau và tất cả

đều là những sản phẩm tốt nhất như FFTW, Intel MKL, Boost, Java ... và là một công cụ phổ biến phục vụ cho việc nghiên cứu ở các cơ sở nghiên cứu, giáo dục và thương mại trên thế giới. Đa số các nghiên cứu sinh, các nhà nghiên cứu thuộc lĩnh vực kỹ thuật đều sử dụng Matlab cho công việc của mình.

Ưu điểm nổi bật của Matlab là hệ thống các hàm thư viện phong phú, tin cậy, nhanh giúp cho việc triển khai thử nghiệm các ý tưởng trên máy tính được thực hiện một cách nhanh chóng với cấu trúc mã lệnh đơn giản, dễ hiểu. Để hình dung về điều này, có thể tưởng tượng một chương trình xử lý ảnh, nếu viết bằng C/C++ sẽ dài tới hàng trăm, thậm chí hàng nghìn dòng lệnh nhưng với Matlab, chỉ là một script cỡ vài chục dòng lệnh. Tất nhiên là các chương trình C/C++ sẽ nhanh hơn (nhưng không nhiều) khi chúng được tối ưu mã lệnh nhưng việc đó chỉ thực sự cần thiết với các thuật toán đã tường minh và hiệu quả, còn để thử nghiệm một ý tưởng mà phải viết hàng trăm, hàng ngàn dòng lệnh thì việc sử dụng Matlab là hợp lý hơn. Bên cạnh đó còn cần phải kể tới bộ soạn thảo chương trình tích hợp của Matlab cho phép viết code nhanh, kiểm soát và gỡ lỗi hiệu quả, cho phép theo dõi giá trị của các biến trong quá trình thực hiện chương trình để phát hiện lỗi. Ngoài ra còn phải kể tới công cụ vẽ hình trực quan và rất đẹp của Matlab.

Vì đơn vị xử lý của Matlab là ma trận, là dạng biểu diễn của ảnh 2 chiều nên việc dùng Matlab minh họa cho các thuật toán thuộc đồ họa máy tính, xử lý ảnh và thị giác máy tính là rất phù hợp. Các thuật toán được trình bày trong tài liệu này đều có thể minh họa qua Matlab, đa số các hình vẽ trong tài liệu này đều có thể tạo ra bằng cách sử dụng Matlab.

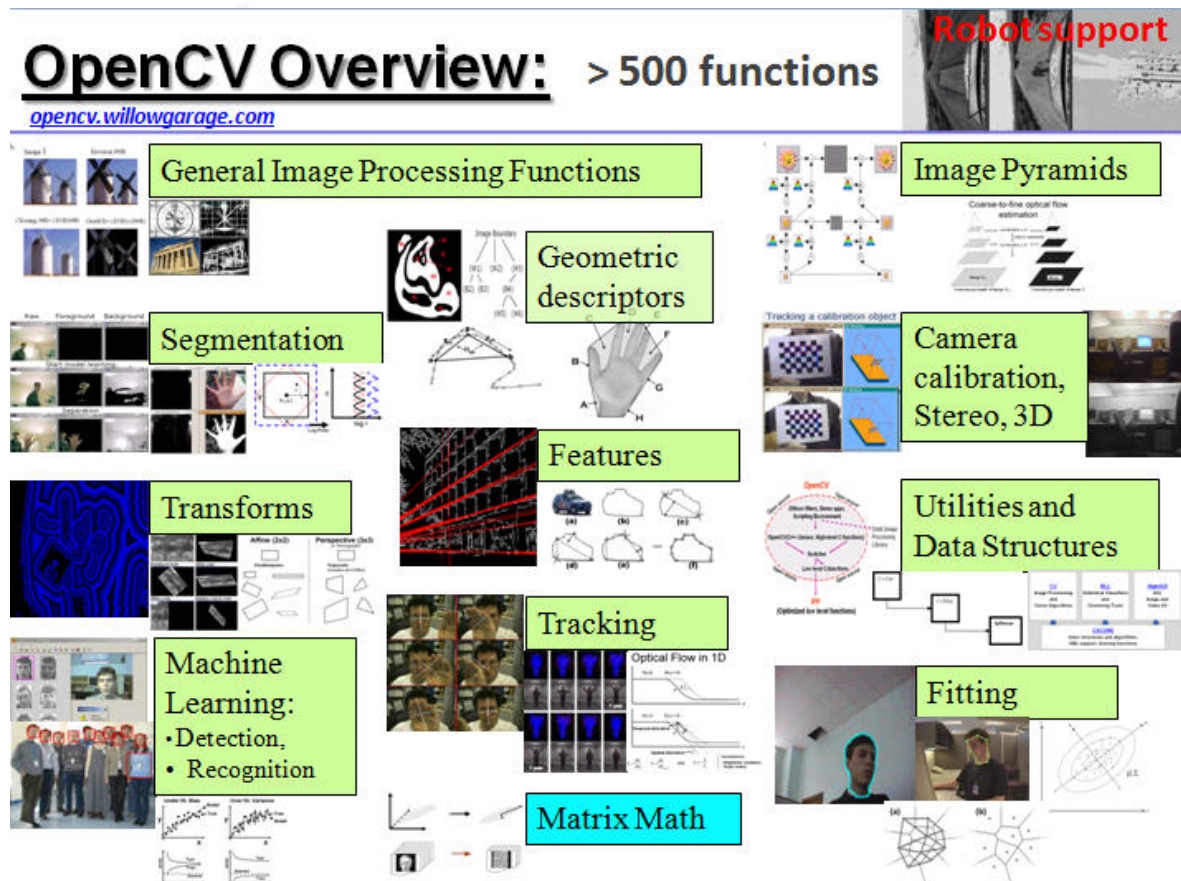
7.2. OpenCV



Hình 1.15: Logo của OpenCV

OpenCV (Open Computer Vision-opencv.org) là một thư viện cung cấp các hàm thư viện cho các bài toán xử lý ảnh và thị giác máy tính hoàn toàn miễn phí cho cả mục đích giáo dục và thương mại. So về qui mô, sự ổn định, đặc biệt là về gỡ lỗi thì OpenCV không thể bằng Matlab nhưng mặt khác OpenCV lại có các ưu điểm đáng kể. Đầu tiên, đây là một thư viện miễn phí, thứ hai là OpenCV được viết bằng C/C++ nên hiệu năng (không phải đối với tất cả các tác vụ) cao hơn Matlab, tiếp đến OpenCV là công cụ để triển khai thực tế các ứng dụng của thị giác máy tính, xử lý ảnh. OpenCV cung cấp giao diện để có thể lập trình với C/C++/C#, Java, Python, Matlab trên các nền tảng Windows, Linux, Mac OS, iOS và Adroid. Ngoài ra OpenCV còn hỗ trợ các giao diện lập trình song song như OpenCL và GPU. Các đặc tính này cho phép triển khai các ứng dụng viết bằng OpenCV một cách dễ dàng với các ngôn ngữ lập trình hàng đầu trên các nền tảng từ PC, tablette, mobile phone cho tới các hệ thống tích hợp như robot, các hệ thống tương tác

dựa trên thị giác máy tính. OpenCV hiện vẫn đang phát triển và được duy trì chính bởi công ty Itseez, một công ty của Nga (ban đầu OpenCV được phát triển bởi các nhà nghiên cứu của hãng Intel tại Nga) được xem là nhà sáng lập OpenCV chuyên về phát triển các ứng dụng thị giác máy tính, bên cạnh đó thư viện này cũng nhận được rất nhiều sự đóng góp từ cộng đồng mã nguồn mở, những người phát triển ứng dụng, nghiên cứu về xử lý ảnh và thị giác máy tính. Tất cả các thuật toán trình bày trong tài liệu này đều có thể cài đặt với ngôn ngữ C++ sử dụng thư viện OpenCV.



Hình 1.16: Tổng quan về các hàm của OpenCV

Tuy nhiên do vẫn còn đang trong giai đoạn phát triển và là miễn phí nên các tài liệu hỗ trợ của OpenCV vẫn chưa được nhiều lắm và đôi khi còn lỗi, không ổn định.

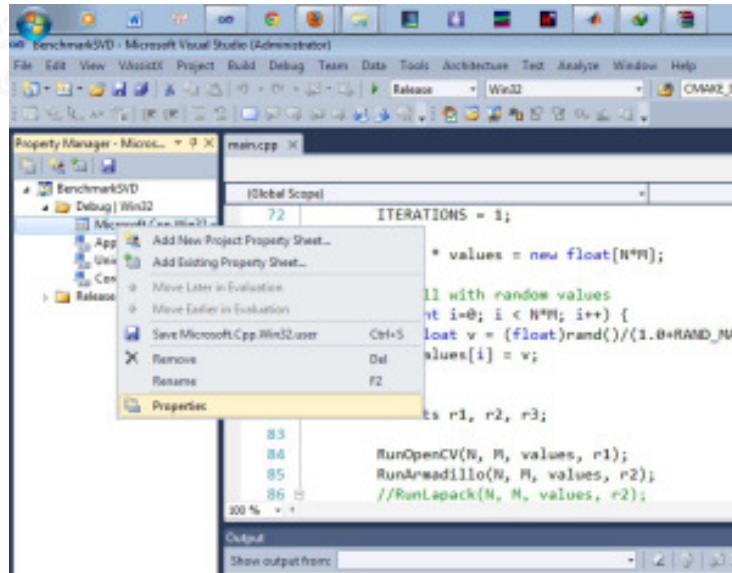
Phiên bản ổn định của OpenCV hiện nay là 2.4.7, bản phát triển là 3.0.0. Sau đây là một số hướng dẫn thiết lập môi trường cơ bản để có thể lập trình trên Visual Studio 2010 sử dụng OpenCV.

1) Download bản OpenCV về máy tính của mình, sau đó giải nén thành thư mục opencv (chẳng hạn trên ổ D:\). Ta có thư mục D:\opencv chứa các file cần thiết của thư viện.

2) Xóa bỏ một số phần không dùng đến trong thư mục build và build\common\tbb: chẳng hạn như phần Python, GPU (lớn nhất), x64, vc9 và staticlib. Chỉ cần giữ lại thư mục common, include, x86\vc10 (hai thư mục con là bin và lib) là đủ. Sau khi thực hiện thao tác này cả thư mục opencv của bạn chỉ còn khoảng hơn 300 Mb. Nhỏ hơn nhiều so với 2Gb ban đầu (lúc mới giải nén OpenCV).

3) Thiết lập đường dẫn môi trường của Windows: thêm hai đường dẫn D:\opencv\build\x86\vc10\bin (chứa file .dll của OpenCV) và D:\opencv\build\common\tbb\ia32\vc10 (chứa file .dll của TBB) vào biến môi trường system path của Windows.

4) Tạo hoặc mở một project bất kỳ với Visual Studio, vào mục Property Manager, kích đúp chuột lên mục Debug và nhấn chuột phải lên mục Microsoft Cpp Win32 như hình sau:



Hình 1.17: Thiết lập sử dụng OpenCV với Visual Studio 2010

Trong cửa sổ hiện ra, ở mục VC++ Directories, phần Include files, thêm đường dẫn D:\opencv\build\include vào cuối, trong mục Library files, thêm đường dẫn D:\opencv\build\x86\vc10\lib vào cuối. Trong mục Linker, phần Input, mục Additional Dependencies, thêm các file sau :

```
opencv_legacy247.lib;  
opencv_core247.lib;  
opencv_features2d247.lib;  
opencv_highgui247.  
lib;opencv_calib3d247.lib;  
opencv_imgproc247.lib;  
opencv_objdetect247.lib;  
opencv_video247.lib;  
opencv_haartraining_engine.lib
```

Đến đây là xong bước cấu hình, thoát khỏi Visual Studio, chương trình sẽ hỏi có muốn lưu lại cấu hình không, nhấn vào Yes và bắt đầu lập trình.

Lưu ý: Nên đặt thư mục chứa OpenCV là opencv vì sẽ ít phải sửa cấu hình khi thay đổi các phiên bản OpenCV khác nhau (chỉ cần thay tên các file .lib theo số phiên bản) và nên xóa các thứ không cần thiết để đỡ tốn bộ nhớ.

7.3. Lập trình OpenCV với Visual Studio 2010

Sau khi thiết lập các đường dẫn cần thiết, chúng ta có thể bắt đầu lập trình sử dụng OpenCV với Visual Studio theo các bước sau:

Bước 1: tạo 1 project Win32 Console Application bằng cách chạy Visual Studio, vào mục File chọn New->Project, trong cửa sổ hiện ra chọn mục Visual C++, ở mục Project Type chọn Win32 Console Application, gõ tên của Project vào ô Name, chẳng hạn như Vidu1. Chọn đường dẫn lưu Project và nhấn OK. Trong cửa sổ Win32 Application Wizard hiện ra sau đó, chọn mục Application Settings, đánh dấu chọn lên ô Empty project và nhấn Finish.

Bước 2: viết mã nguồn chương trình bằng cách nhấn chuột phải lên Project Name (vidu1) chọn mục Add->New Item, trong cửa sổ hiện ra chọn C++ File (.cpp) và gõ tên file là main.cpp và nhấn vào Add. Gõ mã lệnh cho file main.cpp như sau:

```
#include <windows.h>
#include <opencv2\imgproc\imgproc_c.h>
#include <opencv2\imgproc\imgproc.hpp>
#include <opencv2\highgui\highgui.hpp>
#include <opencv2\highgui\highgui_c.h>

using namespace cv;

int main( int argc, char** argv )
{
    char* imageName = argv[1];

    Mat image;
    // Read image file
    image = imread(imageName, 1);

    if( argc != 2 || !image.data )
    {
        printf( " No image data \n " );
        return -1;
    }

    Mat gray_image;
    // Convert image from BGR color to gray scale format
    cvtColor( image, gray_image, CV_BGR2GRAY );
    // write to image file
    imwrite( "Gray_Image.jpg", gray_image );

    namedWindow( imageName, CV_WINDOW_AUTOSIZE );
    namedWindow( "Gray image", CV_WINDOW_AUTOSIZE );
    // show image
```



```
imshow( imageName, image );  
imshow( "Gray image", gray_image );  
  
waitKey(0);  
  
return 0;  
}
```

Bước 3: dịch và chạy chương trình bằng các phím F7 (dịch) và F5.

Chương trình trên mặc dù đơn giản nhưng đã minh họa cho 4 thao tác cơ bản sau đây của một chương trình xử lý ảnh: đọc ảnh từ file, convert ảnh màu thành ảnh xám, ghi ảnh vào file kết quả, và hiển thị ảnh kết quả.

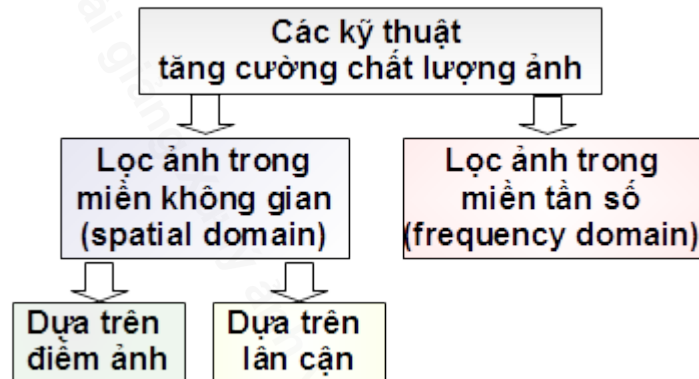
Bài tập

Bài tập 1: Cài đặt và cấu hình OpenCV trên máy tính, viết chương trình mở một file ảnh, chuyển đổi thành ảnh xám và ghi kết quả ra file.

CHƯƠNG II: CÁC THAO TÁC XỬ LÝ DỰA TRÊN ĐIỂM ẢNH

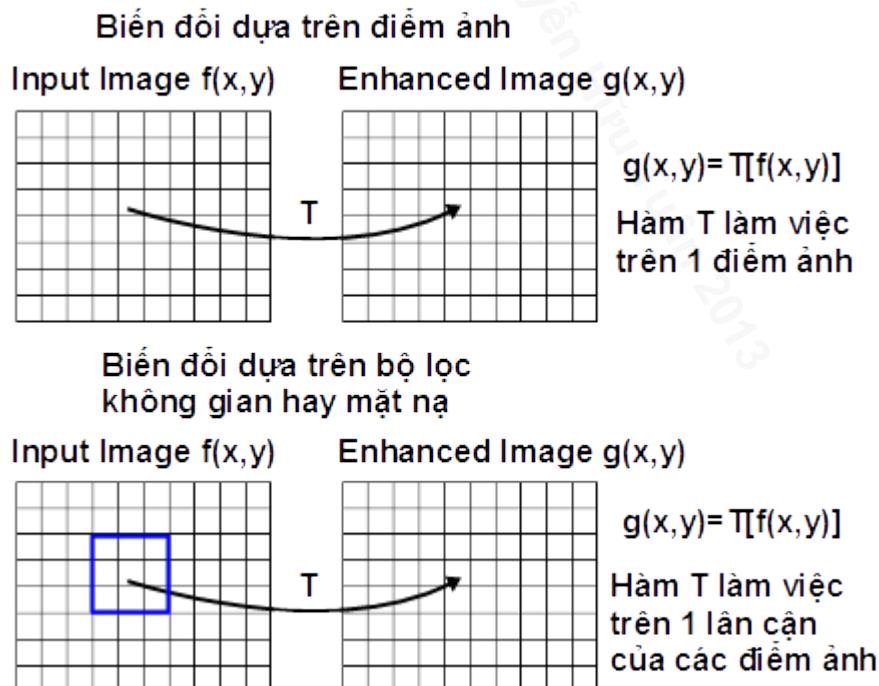
Như đã trình bày ở chương I, tăng cường chất lượng ảnh (*image enhancement*) là một trong các mục đích chính của xử lý ảnh và để đạt được điều này, có 2 kỹ thuật thường được sử dụng là:

- + Các phương pháp trong miền không/thời gian (*spatial/time domain methods*) sử dụng các đặc điểm không gian của ảnh như điểm ảnh, các lân cận của điểm ảnh.
- + Các phương pháp trong miền tần số (*frequency domain methods*) sử dụng biến đổi Fourier rời rạc làm căn bản.



Hình 2.1: Các kỹ thuật nâng cao chất lượng ảnh

Đối với các phương pháp trong miền không gian, có hai cách tiếp cận là cách tiếp cận dựa trên điểm ảnh (*point processing*, hay còn gọi là *intensity transformations*) và cách tiếp cận dựa trên mặt nạ (*mask*, hay còn gọi là các *kernel*, một tên khác là bộ lọc không gian *spatial filter*, bản chất là xét vùng lân cận của điểm ảnh). Trong chương này chúng ta sẽ xem xét các phương pháp dựa trên điểm ảnh.

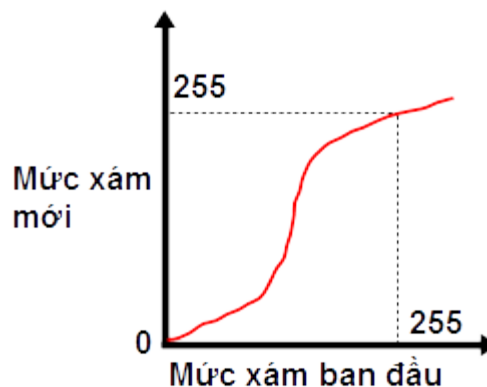


Hình 2.2: Tăng cường chất lượng ảnh dựa vào các phép biến đổi không gian

1. Khái niệm

Các phương pháp biến đổi không gian (spatial, từ này xuất phát từ thực tế là khi chúng ta xử lý trên các điểm ảnh theo các chiều của trục x và y, thực chất là chúng ta đã di chuyển qua vùng không gian của ảnh) có thể mô tả bởi hàm biến đổi: $g(x, y) = T[f(x, y)]$. Trong đó $f(x, y)$ là ảnh input, $g(x, y)$ là ảnh kết quả còn T là hàm biến đổi (xem hình 2.3). Đối với các thao tác dựa trên điểm ảnh, hàm T thực hiện với từng giá trị điểm ảnh (là tham số của hàm), trong khi với các thao tác biến đổi dựa trên bộ lọc không gian, hàm T làm việc trên lân cận của từng điểm ảnh. Cũng có thể coi các thao tác biến đổi dựa trên điểm ảnh là trường hợp đặc biệt của việc biến đổi dựa trên bộ lọc trong trường hợp đặc biệt khi kích thước của bộ lọc bằng 1×1 .

Về bản chất, hàm biến đổi T sẽ nhận tham số là mức xám của một điểm ảnh, sinh ra một giá trị mức xám mới và gán cho điểm ảnh đó (xem hình minh họa bên dưới).



Hình 2.3: Hàm biến đổi dựa trên điểm ảnh

2. Một số thao tác biến đổi dựa trên điểm ảnh cơ bản

2.1. Lấy âm bản (Negative image)

Thao tác lấy âm bản có tác dụng tăng cường các chi tiết màu trắng hoặc xám bị ẩn trong các vùng tối của ảnh. Cụ thể mức xám mới s sẽ được tính từ mức xám gốc r theo công thức:

$$s = 255 - r.$$

Đôi khi công thức này còn được thay thế bằng công thức sau:

$$s = \max_{\text{intensity}} - r.$$

trong đó $\max_{\text{intensity}}$ là mức xám lớn nhất của ảnh input.

Lấy âm bản của ảnh rất hay được dùng trong ảnh chụp y học nhằm làm rõ các chi tiết khó nhận biết bằng mắt thường với các ảnh tối màu (xem hình minh họa 2.4 bên dưới).

2.2. Tăng độ tương phản (contrast enhancement)

Thuật toán tăng độ tương phản được dùng đối với các ảnh mà các mức xám của nó được phân bố trong một miền cục bộ $[u1, u2]$ với $\min I = u1$ và $\max I = u2$ là giá trị mức xám nhỏ nhất và cao nhất của ảnh. Khi đó ta dùng công thức sau:

$$s = 255 * (r - \min I) / (\max I - \min I)$$

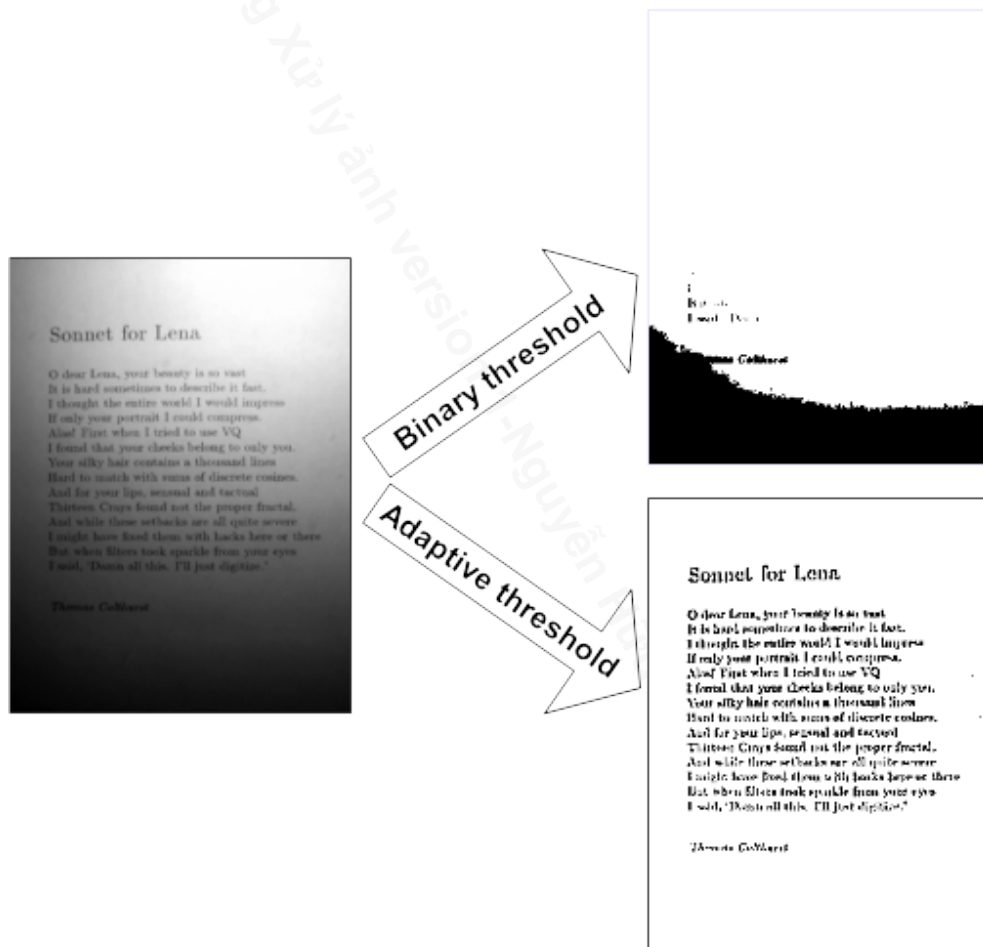
Đối với các ảnh có độ tương phản thấp (low contrast) các giá trị u_1 và u_2 khá gần nhau (ảnh chỉ có một số lượng ít các mức xám), xem hình minh họa 2.4.

2.3. Tách ngưỡng (thresholding)

Kỹ thuật tách ngưỡng thường được sử dụng cho bài toán phân đoạn ảnh khi ta muốn tách hoặc nhấn mạnh các đối tượng quan tâm khỏi nền của ảnh. Mức xám mới sẽ được tính theo công thức:

$$s = \begin{cases} 255 & \text{nếu } r > \text{threshold} \\ 0 & \text{nếu } r \leq \text{threshold} \end{cases}$$

Threshold là một hằng số gọi là ngưỡng còn ảnh nhận được gọi là ảnh nhị phân vì chỉ có 2 mức xám (xem hình minh họa 2.4). Phương pháp này còn được gọi là tách ngưỡng nhị phân.



Hình 2.4: Tách ngưỡng nhị phân và tách ngưỡng linh hoạt

Tách ngưỡng linh hoạt (adaptive thresholding): Có thể thấy rằng việc tách ngưỡng chỉ sử dụng một ngưỡng duy nhất cho cả một ảnh sẽ làm mất đi nhiều đặc điểm của ảnh (xem hình 2.4), vì vậy một thuật toán có tính chất linh hoạt hơn sẽ được dùng để tách ngưỡng cho ảnh. Trong kỹ thuật tách ngưỡng linh hoạt, thay vì dùng một ngưỡng duy nhất, giá trị ngưỡng dùng để tách ngưỡng sẽ thay đổi trên toàn bộ bức ảnh. Ảnh thu được vẫn là ảnh nhị phân nhưng sẽ giữ lại được nhiều đặc điểm hơn so với tách ngưỡng nhị phân. Đối với mỗi điểm ảnh, một giá trị ngưỡng sẽ được tính, nếu giá trị mức xám của điểm ảnh nhỏ hơn ngưỡng, nó sẽ thuộc về giá trị nền (background), ngược lại nó sẽ

được đưa vào vùng foreground (đối tượng cần quan tâm). Có hai thuật toán dùng để tìm ngưỡng là thuật toán “Chow and Kaneko” và thuật toán tìm ngưỡng cục bộ (local thresholding). Hai thuật toán này đều dựa trên một giả thuyết là các vùng nhỏ cục bộ của ảnh sẽ có xu hướng có giá trị sáng xấp xỉ bằng nhau và sẽ thích hợp để thực hiện tách ngưỡng. Thuật toán của Chow and Kaneko chia ảnh thành các vùng chồng nhau và thực hiện tìm ngưỡng tối ưu cho các vùng dựa trên histogram của từng vùng. Ngưỡng của mỗi điểm được tính bằng cách nội suy các kết quả tìm được (của vùng con tương ứng với điểm). Một nhược điểm lớn và dễ thấy là phương pháp này đòi hỏi khối lượng tính toán lớn, do đó hiển nhiên không phù hợp cho các bài toán đòi hỏi xử lý theo thời gian thực.

Thuật toán tìm ngưỡng cục bộ dựa vào các giá trị thống kê trên các mức xám của các điểm nằm trong một lân cận (neighborhood) của mỗi điểm ảnh. Các cách đơn giản và nhanh gồm có: tính giá trị trung bình (mean), tính giá trị median hoặc tính trung bình cộng của min và max, tính tổng có trọng số của các điểm lân cận với các trọng số được cho bởi một bộ lọc Gaussian. Việc chọn kích thước của vùng lân cận phải đủ lớn, nếu không ngưỡng chọn được sẽ không tốt. Ngược lại chọn ngưỡng quá lớn sẽ phá vỡ giả thiết về mức sáng xấp xỉ của các điểm trong một vùng cục bộ. Trên thực tế việc tách ngưỡng linh hoạt thường sử dụng các cửa sổ có kích thước là một số lẻ lớn hơn 3 (5, 7 ...) và một hằng số C thay cho giá trị trung bình (mean) gọi là mean_C vì giá trị trung bình của một vùng thường gần với giá trị của điểm trung tâm lân cận và kết quả thu được thường không tốt. Trong hình 2.3 là kết quả thực hiện với cửa sổ có kích thước 5x5 và mean_C = 5.

2.4. Biến đổi logarithmic

Biến đổi hàm logarithm ánh xạ (nén) một vùng hẹp của các mức xám thấp sang một vùng giá trị rộng hơn theo công thức:

$$s = c \cdot \log(1+r).$$

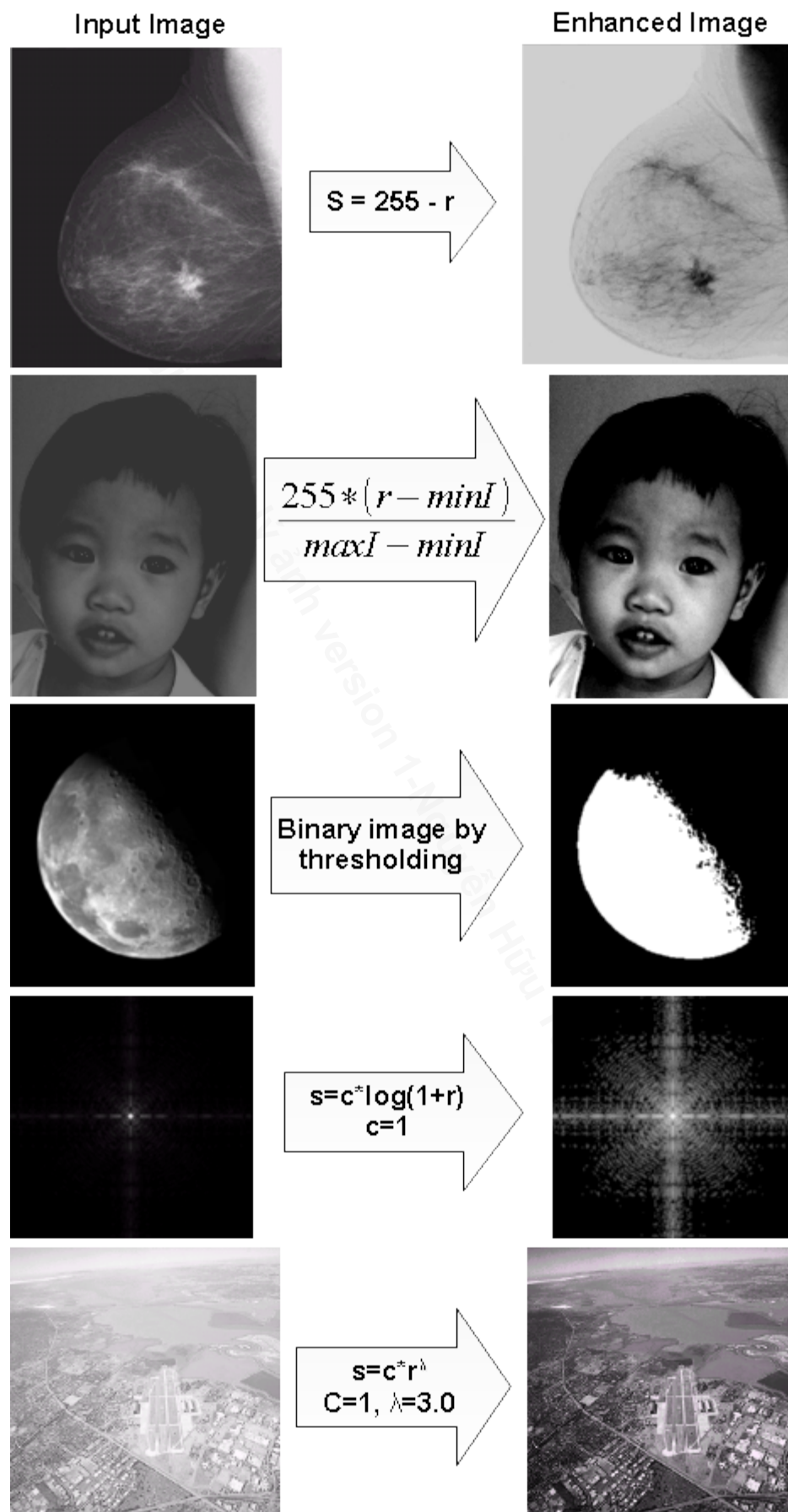
Trong đó c là một hằng số (thường được đặt bằng 1) và $r > 0$, r nằm trong khoảng [0.0, 1.0]. Biến đổi logarithm rất có ích đặc biệt trong các trường hợp mà các mức xám của ảnh có thể nằm trong một vùng giá trị cực lớn (vì thế nên phép biến đổi này còn được gọi là kỹ thuật nén vùng động – dynamic range compression - DRC). Ví dụ như kết quả tính magnitude của ảnh sau khi thực hiện biến đổi Fourier như hình 2.4.

2.5. Biến đổi Power-Law

Biến đổi Power-Law ánh xạ một vùng hẹp của các giá trị input tối vào một vùng giá trị rộng hơn và ngược lại theo công thức sau:

$$s = c \cdot r^\lambda$$

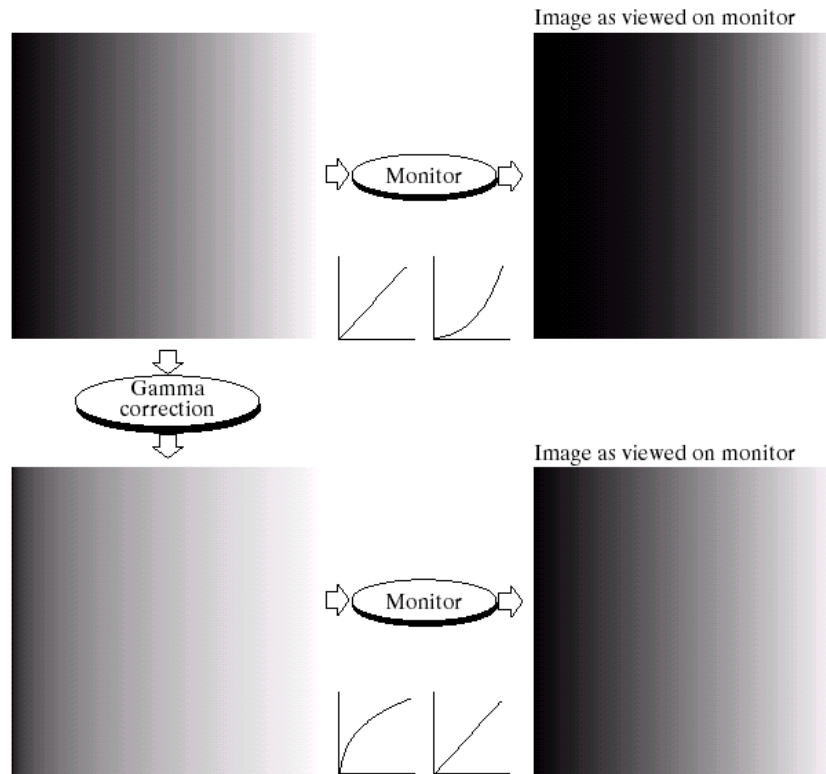
Hằng số λ tạo thành họ các đường cong khác nhau và sẽ làm nổi bật các đặc điểm khác nhau của ảnh gốc (xem hình minh họa 2.4). Cụ thể hơn với $\lambda > 1$ thì ảnh sẽ tối hơn, còn $\lambda < 1$ thì ảnh sẽ sáng hơn (có thể xem thêm trường hợp Gamma correction với giá trị λ chuẩn bằng 1/2.2 làm cho ảnh sáng hơn).



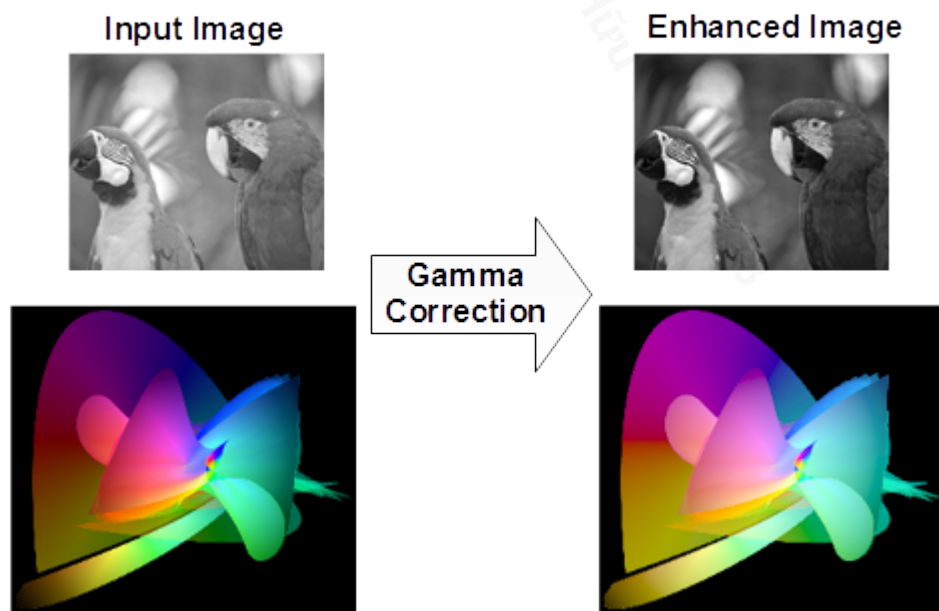
Hình 2.5: Các phép biến đổi dựa trên điểm ảnh

2.6. Gamma correction

Gamma correction là thao tác điều chỉnh lại mức xám của ảnh hiển thị trên các màn hình (máy tính, tivi) nhằm làm cho mắt người có thể nhận biết hình ảnh (ánh sáng và màu sắc) một cách tốt hơn bằng cách sử dụng biến đổi Power-Law. Vấn đề này xuất phát từ một thực tế là các thiết bị màn hình không hiển thị các cường độ điểm ảnh mà nó nhận được theo quan hệ tuyến tính (xem hình minh họa bên dưới):



Hình 2.6: Gamma correction trên thiết bị hiển thị



Hình 2.7: Ví dụ về Gamma Correction

Một mô hình xấp xỉ để tính ánh sáng hiển thị L_i được kích thích bởi súng bắn tia thứ i có mức điện áp V_i cho một màn hình được cho bởi công thức:

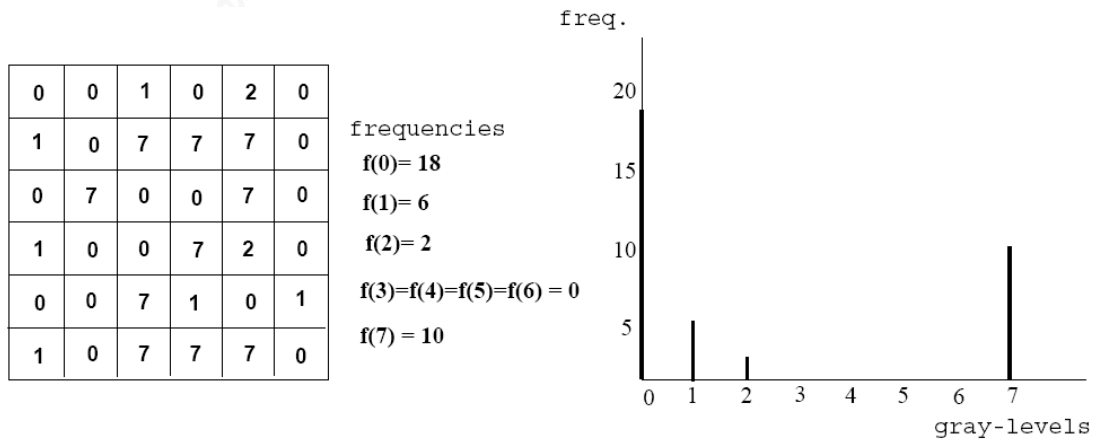
$$\begin{cases} L_i = k_i V_i^{\lambda_i} \\ \lambda_i = \text{giá trị gamma của s\acute{u}ng } i \\ k_i \text{ là hằng số} \end{cases}$$

Thường giá trị của λ nằm trong khoảng $(1, 2.5]$, giá trị Gamma chuẩn bằng $1/2.2$.

3. Các thao tác dựa trên histogram của ảnh

3.1. Histogram của ảnh

Histogram của một ảnh là biểu diễn trực quan về tần số xuất hiện của các mức xám trong ảnh. Một cách đơn giản nhất có thể hình dung như hình vẽ bên dưới:

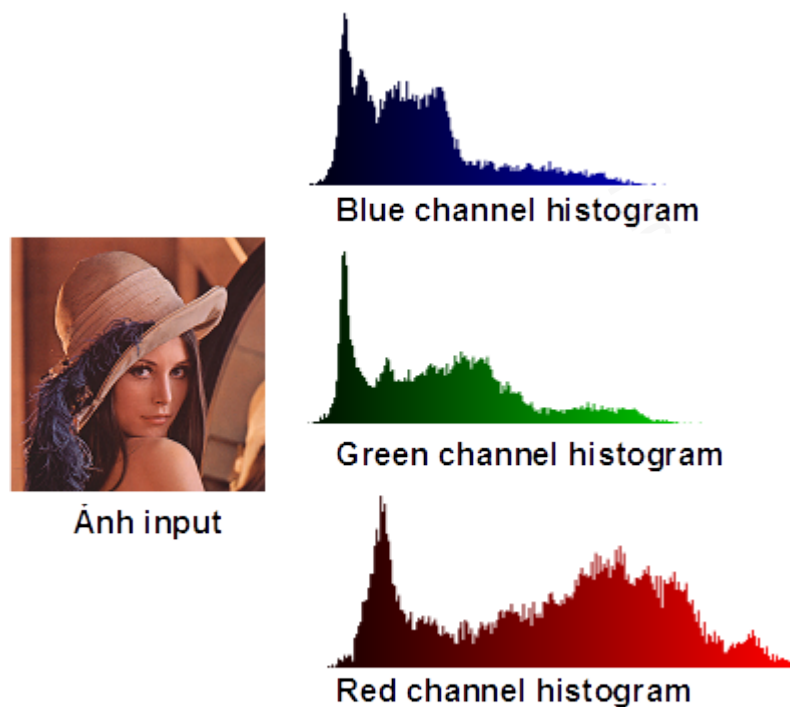


Hình 2.8: Histogram của ảnh

Như vậy để tính histogram của một ảnh, chúng ta đi đếm số lần xuất hiện của các mức xám trong ảnh và lưu vào một mảng một chiều có kích thước 256 phần tử:

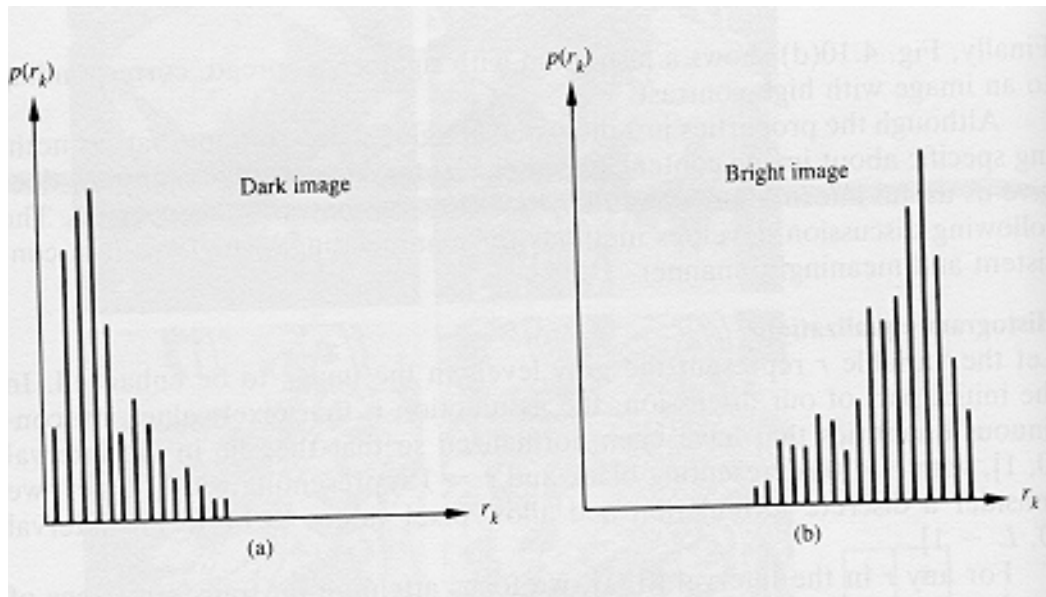
$$n_k = \text{hist}[k] = \sum_{f(x,y)=k} 1.$$

Mỗi ảnh màu có 3 channels nên có thể có 3 histogram khác nhau:



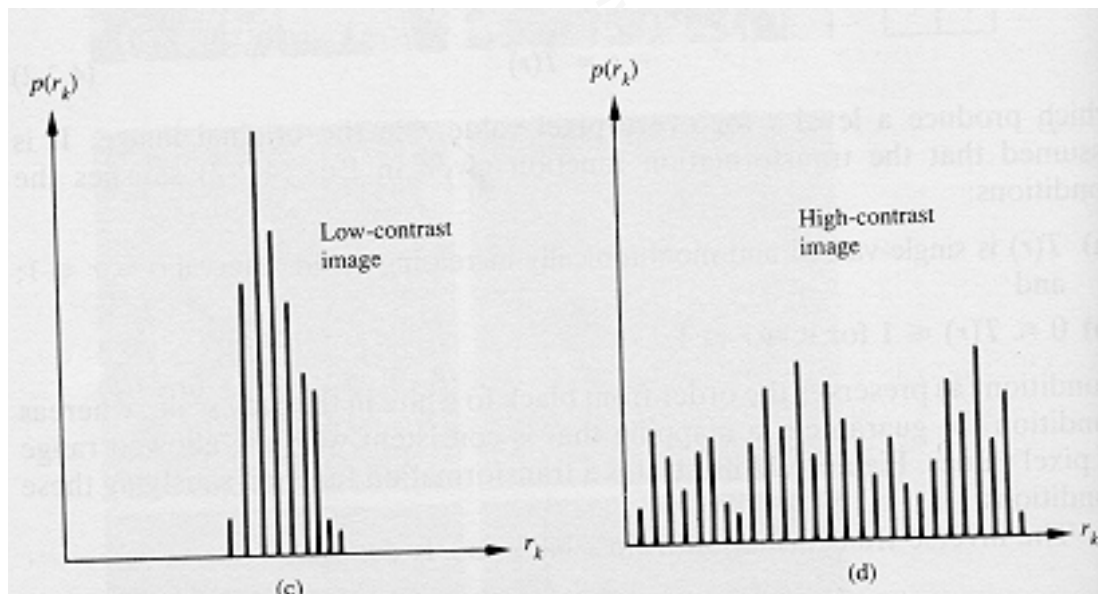
Hình 2.9: Ảnh và histogram của các channel khác nhau

Histogram của ảnh cho ta biết tần số xuất hiện của các mức xám trong ảnh. Dựa vào histogram ta có thể biết được đặc điểm của ảnh. Đối với các ảnh tối, các histogram sẽ tập trung ở các mức xám thấp (phía bên trái của histogram). Ngược lại đối với các ảnh sáng, histogram sẽ tập trung ở các mức xám cao (xem hình minh họa 2.10 bên dưới).



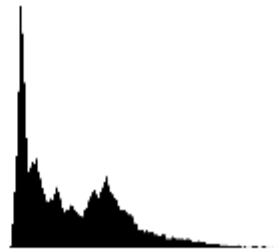
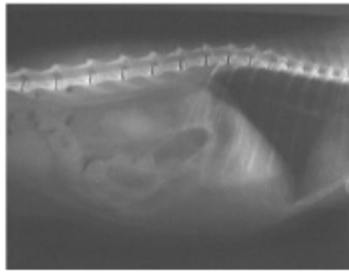
Hình 2.10: Histogram của ảnh sáng và ảnh tối

Histogram chỉ gồm các đường nằm trong một khoảng nhỏ sẽ tương ứng với các ảnh có độ tương phản thấp (hầu hết là tối, hoặc hầu hết là sáng, hoặc hầu hết là xám). Ngược lại các ảnh có histogram trải dài sẽ là các ảnh có độ tương phản cao (xem hình minh họa bên dưới).

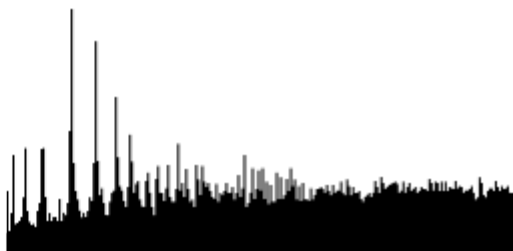
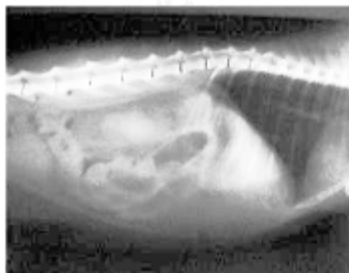


Hình 2.11: Histogram và độ tương phản của ảnh

Ảnh có độ tương phản thấp và Histogram



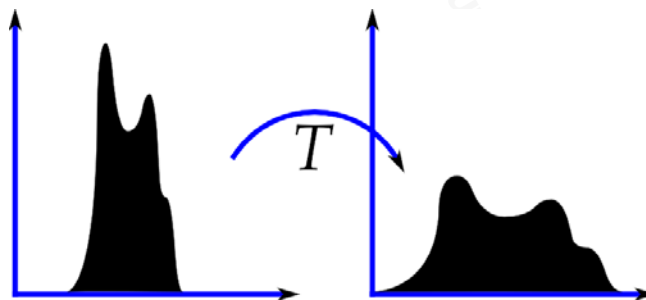
Ảnh có độ tương phản cao và Histogram



Hình 2.12: Mối quan hệ giữa độ tương phản và histogram

3.2. Cân bằng histogram

Dựa vào mối liên quan giữa các đặc điểm của ảnh và histogram, một kết luận có thể rút ra là có thể điều chỉnh histogram để nâng cao chất lượng ảnh (tăng cường độ tương phản và làm cho ảnh không bị tối hay sáng quá). Bên cạnh đó theo các nghiên cứu về thị giác con người thì hệ thống thị giác của chúng ta có thể nhìn được các chi tiết ảnh một cách rõ ràng hơn nếu histogram của ảnh được trải rộng và bằng phẳng hơn là nằm trong một dải hẹp và có các đỉnh. Để đạt được điều này, ý tưởng cơ bản là phân phối lại histogram sao cho tần số của các mức xám xuất hiện trong ảnh trở nên cân bằng hơn:



Hình 2.13: Nguyên tắc cân bằng histogram

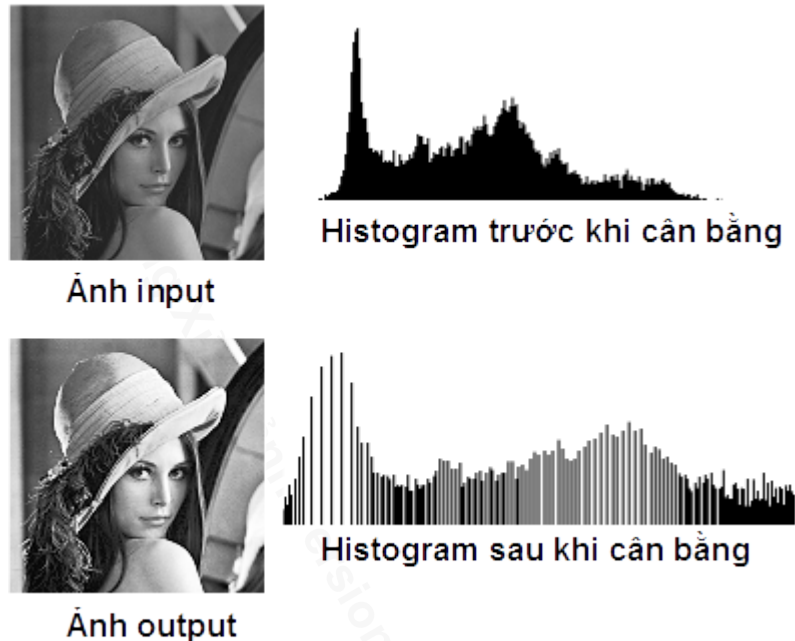
Thuật toán cân bằng histogram (histogram equalization) được thực hiện bằng cách chia tần số của mỗi điểm ảnh cho tổng số điểm ảnh để lấy xác suất của mức xám r_k tương ứng theo công thức sau (bước này gọi là chuẩn hóa histogram – histogram normalization):

$$p_k(r_k) = \frac{n_k}{N}, k = 0..L - 1$$

Trong đó $L-1$ là mức xám lớn nhất của ảnh, N là tổng số điểm ảnh, n_k là tổng số điểm ảnh có mức xám r_k . Hàm biến đổi để tính mức xám mới cho mỗi điểm ảnh là:

$$s_k = T(r_k) = (L - 1) \sum_{j \leq k} p_r(r_j) = (L - 1) \sum_{j \leq k} \left(\frac{n_j}{N} \right)$$

Ảnh output sẽ nhận được bằng cách ánh xạ mỗi giá mức xám r_k của ảnh input để nhận được giá trị mức xám mới s_k .



Hình 2.14: Cân bằng histogram ảnh

3.3. Một số ứng dụng của histogram

Được tính dựa trên giá trị các mức xám của ảnh và thể hiện các thông tin đặc trưng của ảnh (thông tin về không gian, về độ sáng, độ tương phản) nên histogram là một trong những công cụ *rất quan trọng* để biểu diễn ảnh trong các ứng dụng của thị giác máy tính, nhận dạng mẫu, phân tích hình ảnh. Các ứng dụng chính của histogram là:

- + Được sử dụng như một thuật toán tiền xử lý (preprocessing) trong nhiều hệ thống nhận dạng (mặt người, mắt người, vân tay ...) nhằm chuẩn hóa ánh sáng, đưa các ảnh về cùng một điều kiện sáng.
- + Ứng dụng trong phân đoạn ảnh (segmentation).
- + Được sử dụng để biểu diễn các đặc trưng cục bộ (micro patterns) của ảnh trong việc biểu diễn ảnh qua các descriptor, cách này dùng nhiều trong thị giác máy tính (ví dụ như các phương pháp dựa vào mẫu nhị phân cục bộ Local Binary Patterns - LBP).
- + Được sử dụng để dự đoán mô hình nhiễu của ảnh (xem thêm chương 5).
- + Biểu diễn ảnh trong các bài toán đối sánh mẫu (template matching), image retrieval, detection, tracking. Khi đó mỗi ảnh được biểu diễn dưới dạng một histogram và một hàm đo khoảng cách giữa hai histogram sẽ được dùng để đánh giá độ tương đồng (similarity) của hai ảnh. Các hàm sau có thể dùng để tính khoảng cách giữa hai histogram:

Hàm Minkowski:

$$D_P(A,B) = [\sum_k |P_A(k) - P_B(k)|^p]^{1/p}$$

Hàm Kullback-Leibler (KL):

$$D_{KL}(A,B) = - \sum_k P_A(k) \log \frac{P_A(k)}{P_B(k)}$$

Hàm Chi-Square:

$$\chi_w^2(S,M) = \sum_{i,j} w_j \frac{(S_{i,j} - M_{i,j})^2}{|S_{i,j} + M_{i,j}|}$$

Hàm Correlation:

$$D_{corr}(X,Y) = \frac{\sum_i (x_i - \text{mean}(X))(y_i - \text{mean}(Y))}{\sqrt{\sum_i (x_i - \text{mean}(X))^2 (y_i - \text{mean}(Y))^2}}$$

4. Biến đổi logic và số học

4.1. Biến đổi dựa trên phép toán số học

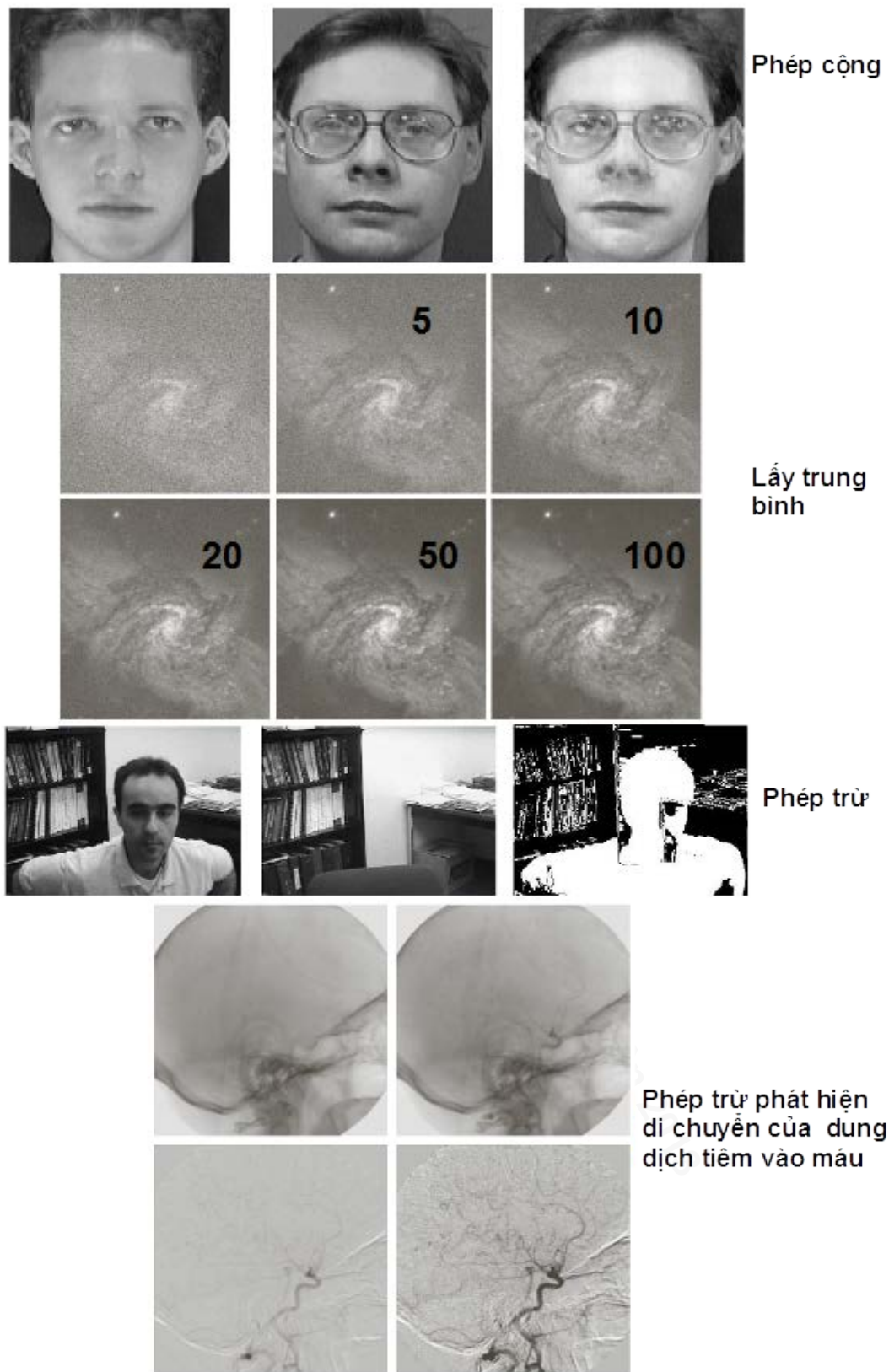
Phép cộng: được sử dụng để kết hợp các thông tin giữa hai ảnh

$$O(r, c) = \alpha I_1(r, c) + (1-\alpha) I_2(r, c), \quad (0 \leq \alpha \leq 1).$$

Phép lấy trung bình cộng: chất lượng của ảnh có thể được tăng cường bằng cách lấy trung bình cộng của một dãy các ảnh chụp cùng một cảnh (đặc biệt hữu ích trong các ứng dụng chụp ảnh thiên văn).

Phép trừ: trừ từng cặp các điểm ảnh của hai ảnh cùng kích thước, được dùng để phát hiện các thay đổi của ảnh trong các bài toán phát hiện chuyển động, phân tích hình ảnh y học

Phép nhân/chia: thực hiện nhân/chia mức xám của các điểm ảnh với một hằng số, được dùng để điều chỉnh độ sáng của ảnh.

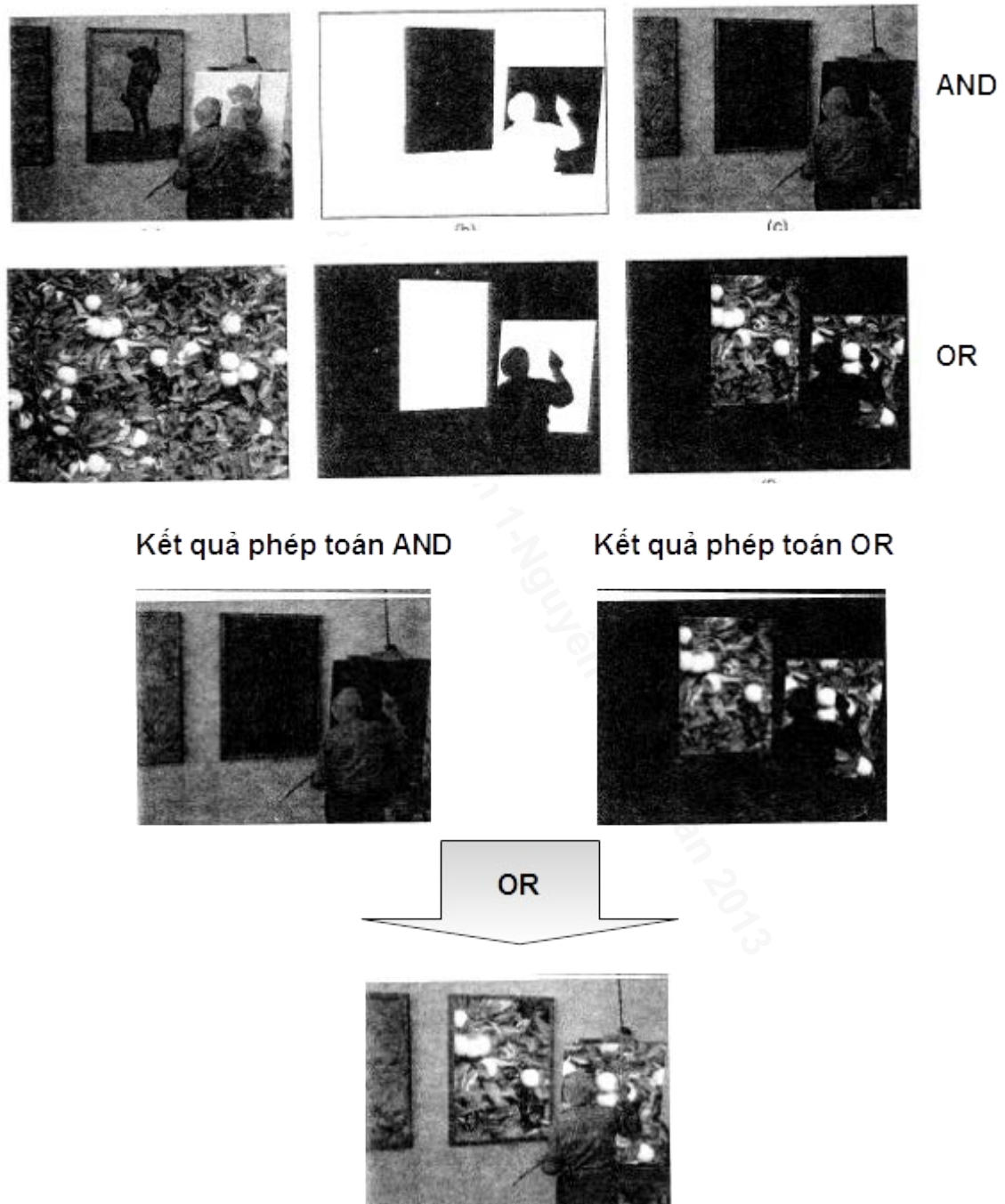


Hình 2.15: Các phép biến đổi số học

Lưu ý: Các phép toán số học có thể sinh các giá trị mức xám nằm ngoài khoảng $[0, 255]$, khi đó thuật toán cần sử dụng để đưa các giá trị mức xám về vùng $[0, 255]$.

4.2. Biến đổi dựa trên phép toán logic

Giữa hai ảnh cùng kích thước chúng ta có thể dùng các phép toán logic cho các cặp điểm ảnh ở cùng vị trí: AND, OR, NOT, AND-NOT, XOR. Hai phép toán logic hay dùng đối với các cặp điểm ảnh của hai ảnh là phép toán AND và phép toán OR trong đó phép toán AND thường được dùng để che phần mặt nạ trong ảnh gốc còn phép toán OR thực hiện thêm phần của ảnh khác vào trong một tấm ảnh, hai thao tác này cũng thường được sử dụng kết hợp với nhau (xem hình minh họa bên dưới):



Hình 2.16: Các phép biến đổi logic

Bài tập

Bài tập 1: Viết chương trình mở một file ảnh, thực hiện các thao tác sau đối với ảnh:

- + lấy ảnh âm bản
- + tăng độ tương phản
- + tách ngưỡng nhị phân
- + biến đổi logarithmic
- + biến đổi Power Law
- + Biến đổi Gamma

Ghi kết quả vào file có tên tương ứng với thao tác thực hiện.

Bài tập 2: Viết chương trình mở một file ảnh và thực hiện:

- + Hiển thị histogram của ảnh
- + Nếu là ảnh màu, hiển thị histogram của các channel khác nhau
- + Cân bằng histogram của ảnh xám

Cho phép ghi kết quả vào file

Bài tập 3: Viết chương trình mở hai file ảnh và thực hiện:

- + Tính khoảng cách giữa hai ảnh dựa trên histogram của ảnh theo các hàm đã học.

CHƯƠNG III: NÂNG CAO CHẤT LƯỢNG ẢNH BẰNG CÁC BỘ LỌC KHÔNG GIAN

Trong chương này chúng ta tiếp tục với các phương pháp nâng cao chất lượng ảnh trong miền không gian (*spatial domain*) bằng các phép lọc ảnh dựa trên các lân cận (*neighborhood*), hay chính xác là giá trị mức xám của các điểm lân cận, của mỗi điểm ảnh (xem lại hình 2.1 để phân biệt các phép lọc ảnh dựa trên mặt nạ với các phép biến đổi dựa trên điểm ảnh trong chương 2). Đồng thời ứng dụng phát hiện biên dựa trên các phép lọc cũng được trình bày.

1. Nguyên tắc của phép lọc ảnh

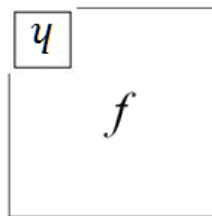
Lân cận của một điểm ảnh là tất cả các điểm ảnh nằm trong một khu vực con (sub-region) có hình chữ nhật với điểm ảnh đang xét là trung tâm. Một bộ lọc không gian (spatial filter, hay mặt nạ-mask, hay kernel filter) là một lân cận (về mặt hình ảnh) được biểu diễn dưới dạng ma trận hai chiều với mỗi phần tử của ma trận là một giá trị thực $w(i,j)$ gọi là trọng số. Các phép lọc này do đó còn được gọi là các kỹ thuật xử lý theo vùng (regional processing). Giá trị ảnh output $g(x,y)$ sẽ được tính từ ảnh input $f(x,y)$ dựa trên việc áp dụng bộ lọc h bằng cách di chuyển nó qua tất cả các điểm ảnh của $f(x,y)$ và tính giá trị tại các điểm tương ứng. Tùy thuộc vào cách thức giá trị output được tính tại mỗi điểm ảnh dựa trên lân cận của nó, người ta chia thành 2 kỹ thuật lọc tuyến tính (linear filtering) dựa vào phép nhân chập (convolution) và lọc phi tuyến (non-linear filtering). Các phép lọc này cũng có thể chia theo chức năng: một loại làm cho ảnh mượt (smooth) đồng thời loại bỏ các nhiễu với các bộ lọc mà tổng các trọng số bằng 1, một loại làm ảnh trở nên sắc nét (sharpen) với việc làm nổi bật các chi tiết của ảnh với các bộ lọc mà tổng các trọng số bằng 0.

2.1. Phép nhân chập (convolution)

Gọi f là ảnh và h là kernel filter, tích chập của f và h ký hiệu là $f * h$ hay $f \otimes h$ được cho bởi công thức:

$$(f * h)[m, n] = \sum_{k, l} f[m - k, n - l] h[k, l]$$

Với m, n là kích thước của kernel filter. Theo qui ước các filter kernel đều được lật ngược như hình vẽ sau:



Hình 3.1: Phép nhân chập

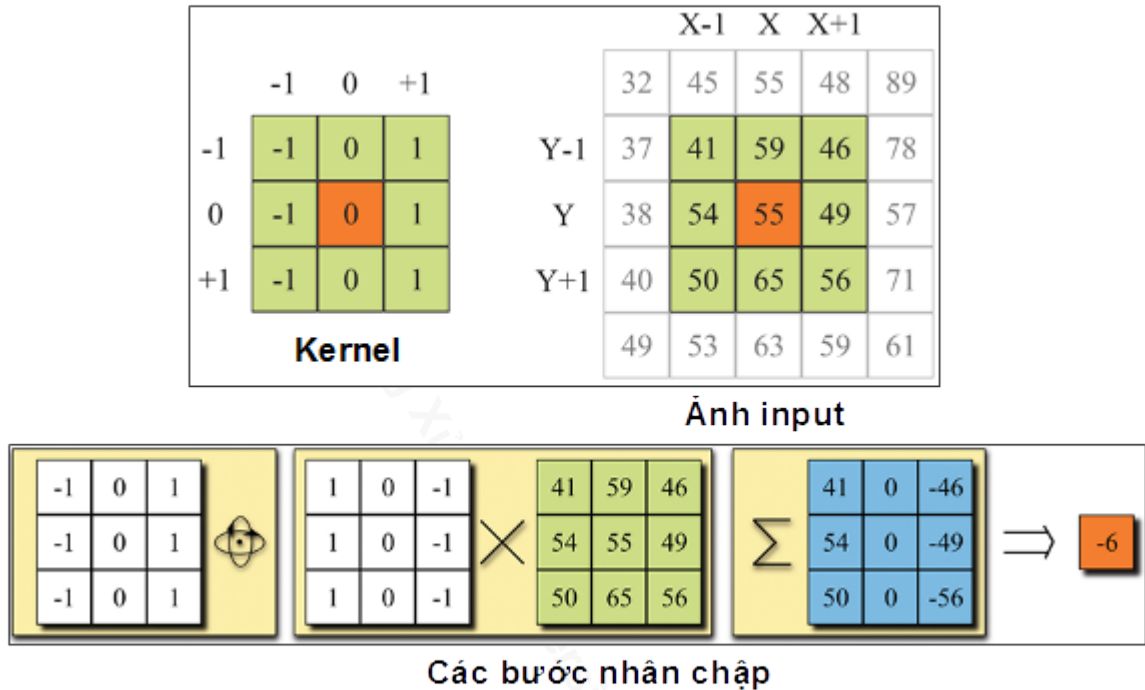
Các tính chất của phép nhân chập:

+ tuyến tính: $\text{filter}(f_1 + f_2) = \text{filter}(f_1) + \text{filter}(f_2)$

+ bất biến đối với phép dịch (shift invariance), không thay đổi dù vị trí các điểm ảnh có thể thay đổi sau phép dịch: $\text{filter}(\text{shift}(f)) = \text{shift}(\text{filter}(f))$.

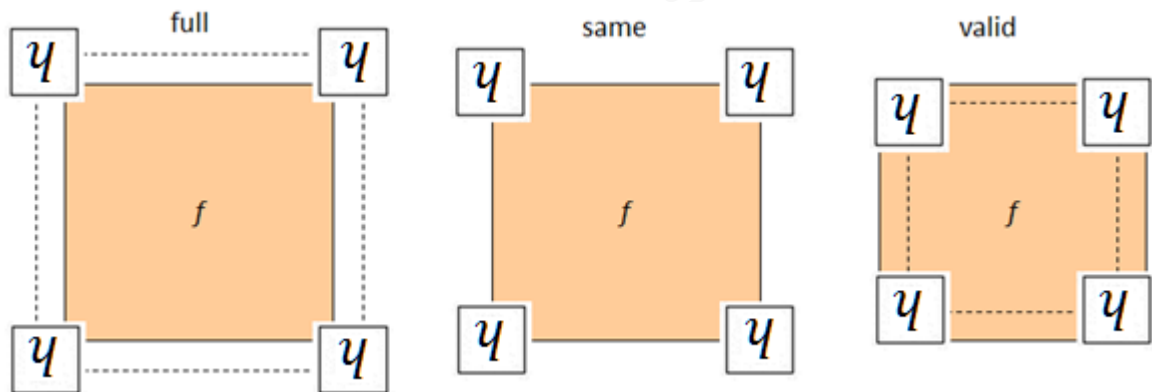
+ về lý thuyết tất cả các phép dịch bất biến tuyến tính đều có thể biểu diễn dưới dạng một phép nhân chập.

Các bước thực hiện tính giá trị output cho mỗi điểm ảnh được minh họa qua hình sau:



Hình 3.2: Minh họa phép nhân chập

Tùy theo kiểu padding cho các điểm không thuộc ảnh, chúng ta có 3 kiểu kết quả có thể nhận được từ phép nhân chập:



Hình 3.3: Các kiểu padding của phép nhân chập

Cài đặt của phép nhân chập: việc cài đặt phép nhân chập bằng ngôn ngữ lập trình C/C++/C# hay Matlab là khá đơn giản với 4 vòng lặp lồng nhau, thuật toán sẽ có độ phức tạp là $O(R \cdot C \cdot m \cdot n)$ trong đó $R \cdot C$ là kích thước của ảnh còn $m \cdot n$ là kích thước của kernel. Tuy nhiên với các ảnh lớn và số thao tác xử lý đòi hỏi sử dụng lặp lại nhiều lần thì cài đặt này là chưa đủ tốt, giải pháp ở đây là sử dụng giải thuật song song để tận dụng thực tế là các hệ thống ngày nay thường có nhiều CPU Core hoặc chip đồ họa riêng (GPU). Khi $m \cdot n > \log(R \cdot C)$ ta có thể cân nhắc việc dùng DFT để cài đặt cho thuật toán này vì sẽ nhanh hơn (với độ phức tạp là $O(R \cdot C \cdot \log(R \cdot C))$, xem chương IV để biết rõ hơn).

Đối với Matlab hàm nhân chập là `conv2()`, OpenCV là `filter2D()`, các hàm này đều được tối ưu cài đặt và có hiệu năng tốt.

2. Các phép lọc làm mịn ảnh

Các bộ lọc làm mịn ảnh (còn gọi là các bộ lọc thông thấp trong miền không gian, spatial lowpass filters) nhằm mục đích giảm nhiễu và loại bỏ các chi tiết nhỏ khỏi ảnh.

2.1. Lọc tuyến tính

Một bộ lọc tuyến tính là bộ lọc mà giá trị mỗi điểm ảnh output là một hàm phụ thuộc tuyến tính vào giá trị mức xám của các điểm lân cận của nó. Hàm phụ thuộc này được biểu diễn bằng một ma trận 2 chiều gọi là kernel filter với các phần tử là các trọng số tương ứng cho điểm ảnh ở vị trí đó. Các phần tử trong bộ lọc phải có giá trị dương và được chuẩn hóa để có tổng bằng 1.

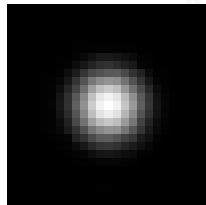
Trong phần này chúng ta sẽ xem xét hai kỹ thuật lọc dựa vào phép nhân chập một ảnh với bộ lọc là phép lọc trung bình và lọc Gaussian.

2.1.1. Lọc trung bình (mean filter)

Bộ lọc trung bình (còn gọi là normalized box filter) sử dụng các kernel mà tất cả các phần tử đều có giá trị bằng nhau. Nếu kích thước của bộ lọc là $M \times N$ thì một phần tử trong bộ lọc sẽ có giá trị (thực) là $1/(M \times N)$. Bộ lọc trung bình làm giảm nhiễu trong ảnh nhưng cũng đồng thời làm cho ảnh trở nên “*mềm mại*” (softened, hay mượt smooth) hơn. Điều này đồng thời cũng làm *mất đi các chi tiết sắc nét của ảnh*.

2.1.2. Lọc Gaussian

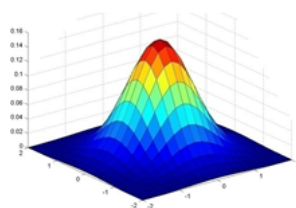
Phép lọc Gaussian dựa trên ý tưởng về một phép lọc làm mượt ảnh sử dụng một filter kernel với các trọng số phân bố của các điểm lân cận dựa trên khoảng cách của nó tới điểm trung tâm, về mặt hình ảnh, trông như sau:



Hình 3.4: Biểu diễn trực quan của bộ lọc Gaussian

Các trọng số của một Gaussian filter được tính theo công thức sau:

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

Hình 3.5: Một ví dụ về bộ lọc Gaussian

Hằng số σ được sử dụng để đảm bảo tổng các trọng số của kernel luôn bằng 1. Các bộ lọc Gaussian về mặt lý thuyết có thể có kích thước vô cùng nhưng trên thực tế các thường là các kernel có kích thước cố định.

Qui tắc xác định kích thước của bộ lọc Gaussian: một nửa độ rộng của kernel nên xấp xỉ giá trị 3σ .



Hình 3.6: Lọc trung bình và lọc Gaussian với các kích thước kernel khác nhau

Một số tính chất của bộ lọc Gaussian:

- + Loại bỏ các thành phần có tần số cao (high frequency) khỏi ảnh, có nghĩa là việc áp dụng bộ lọc Gaussian giống như một bộ lọc thông thấp (lowpass filter).
- + Việc áp dụng liên tiếp hai lần cùng một bộ lọc Gaussian sẽ tương đương với việc áp dụng bộ lọc Gaussian cùng kích thước nhưng giá trị $\sigma_{\text{new}} = \sigma\sqrt{2}$.
- + Một bộ lọc 2D có thể phân tích thành tích của hai bộ lọc 1D.

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}} = \left(\frac{1}{2\pi\sigma^2} e^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{2\pi\sigma^2} e^{-\frac{y^2}{2\sigma^2}} \right)$$

Ví dụ:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Hình 3.7: Phân tích một bộ lọc Gaussian 2D thành tích

Điều này rất có ích trên thực tế vì khi đó phép lọc sẽ được thực hiện bằng hai bộ lọc 1 chiều với số thao tác cần tính toán ít hơn đáng kể.

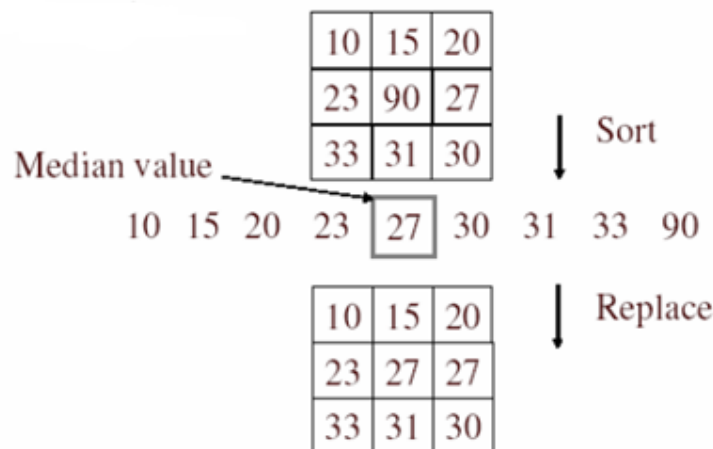
+ So với một bộ lọc trung bình có cùng kích thước, bộ lọc Gaussian sẽ giữ lại được nhiều chi tiết sắc của ảnh hơn. Khi giá trị σ rất lớn so với kích thước của nó thì bộ lọc Gaussian có thể xem như bộ lọc trung bình.

2.2. Lọc phi tuyến

Các bộ lọc phi tuyến là các bộ lọc mà giá trị output của mỗi điểm ảnh không phụ thuộc tuyến tính vào các điểm lân cận của nó. So với các phép lọc tuyến tính, các kỹ thuật lọc phi tuyến có hai ưu điểm đáng kể: thứ nhất là chúng loại bỏ các nhiễu (đặc biệt là nhiễu dạng muối tiêu - salt and pepper) tốt hơn, thứ hai là ảnh *vẫn giữ được các chi tiết sắc nét sau khi lọc* (edge preserving).

2.2.1. Lọc trung vị (median filter)

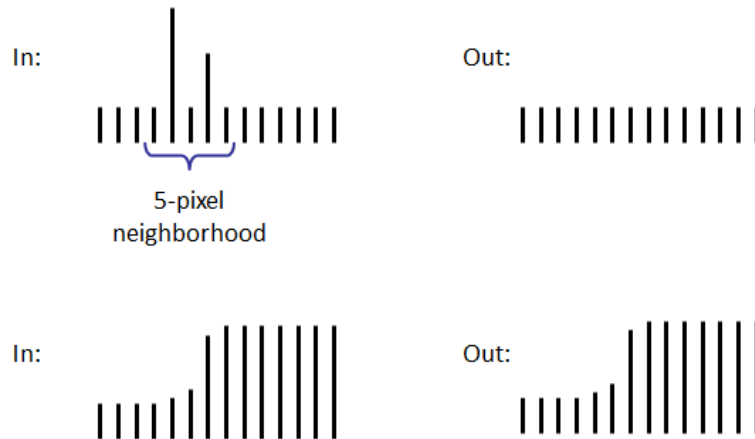
Phép lọc trung vị thực hiện thay thế mỗi điểm ảnh bằng một giá trị median của các điểm trong một lân cận (cửa sổ) có vị trí trung tâm là điểm đang xét, cụ thể như hình sau :



Hình 3.8: Minh họa lọc trung vị

Về mặt tổng quan, một phép lọc trung vị giống như một bộ lọc dựa trên thứ tự sắp xếp của các mức xám, điều này hoàn toàn khác với trường hợp bộ lọc trung bình, ví dụ $\text{median}([1 \ 6 \ 1 \ 3 \ 1])=1$ nhưng $\text{mean}([1 \ 6 \ 1 \ 3 \ 1])=2.4$;

Vậy tác dụng của lọc trung vị là gì ? Chúng ta hãy xem hình minh họa sau :



Hình 3.9: Tác dụng của lọc trung vị

Các nhiễu biến đổi đột ngột (spikde noise) sẽ bị loại bỏ trong khi các đường biến đổi đơn điệu (mononic edges) vẫn được giữ nguyên. Có thể nhận thấy rõ sự khác nhau giữa lọc Gaussian và lọc trung vị qua hình minh họa 3.11.

So với lọc trung bình, lọc trung vị mạnh hơn vì các điểm ảnh không có tính chất đại diện (xa so với giá trị trung bình) sẽ không ảnh hưởng nhiều tới giá trị trung vị. Hơn nữa do giá trị điểm ảnh output là một giá trị thực sự (của một điểm ảnh) chứ không phải là một giá trị không thực sinh ra từ các điểm trong kernel filter nên bộ lọc trung vị vẫn giữ lại được các chi tiết sắc nét của ảnh.

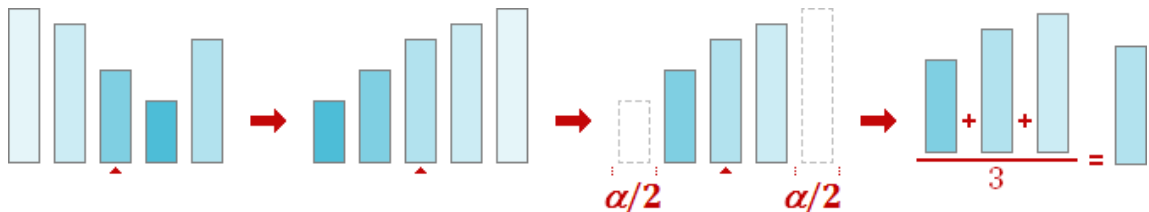
Mặc dù vậy bộ lọc trung vị cũng có những nhược điểm. Nhược điểm lớn nhất là độ phức tạp tính toán vì với mỗi điểm ảnh input, cần một bước sắp xếp để tìm giá trị trung vị cho các lân cận của nó. Bên cạnh đó các cấu trúc chiếm ít hơn một nửa kernel filter sẽ có xu hướng bị loại bỏ.

2.2.2. Alpha-trimmed mean filter (ATM filter)

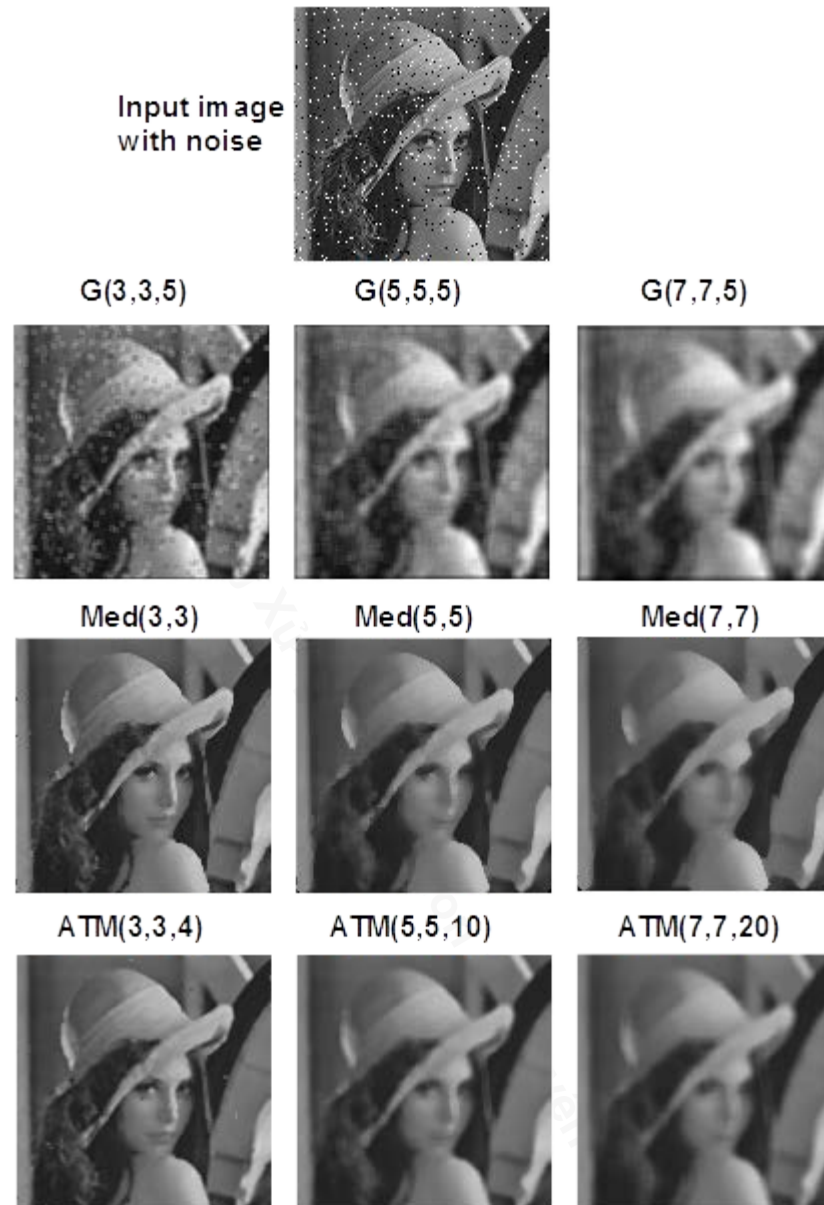
Bộ lọc Alpha-trimmed mean là bộ lọc kết hợp các đặc điểm của cả hai bộ lọc trung bình và lọc trung vị. Xét các điểm trong lân cận S_{xy} của một điểm ảnh $f(x,y)$, giá trị output sẽ được tính bằng trung bình cộng của $m \times n - \alpha$ giá trị thuộc lân cận đang xét sau khi bỏ đi $\alpha/2$ giá trị lớn nhất và $\alpha/2$ giá trị nhỏ nhất của lân cận. Công thức cụ thể như sau:

$$g(x, y) = \frac{1}{mn - \alpha} \sum_{(s,t) \in S_{x,y}} f(s, t)$$

Giá trị của α nằm trong khoảng từ 0 tới $mn-1$ và phải chẵn. Với $\alpha=0$, bộ lọc này chính là bộ lọc trung bình, khi $\alpha=mn-1$, bộ lọc trở thành bộ lọc trung vị.



Hình 3.10: Bộ lọc Alpha-Trimmed Mean



Hình 3.11: Lọc Gaussian, lọc trung vị và lọc ATM

2.2.3. Bộ lọc Bilateral

Bộ lọc Bilateral là bộ lọc dựa trên ý tưởng kết hợp một bộ lọc tuyến tính với các trọng số dựa trên quan hệ của mỗi điểm ảnh với các điểm lân cận của nó và loại bỏ các điểm mà giá trị của nó (màu sắc) quá xa so với điểm đang xét. Xét theo quan điểm loại bỏ điểm thì bộ lọc này giống với các bộ lọc Median. Cụ thể, mỗi điểm ảnh output sẽ được tính dựa vào các điểm lân cận của nó (thường là 3x3) bằng công thức:

$$g(i, j) = \frac{\sum_{k,l} f(k, l) w(i, j, k, l)}{\sum_{k,l} w(i, j, k, l)}$$

Với $w(i, j, k, l)$ là các trọng số được tính dựa trên hai nhân tố, nhân tố thứ nhất nói lên mối tương quan về không gian (vị trí) (gọi là spatial kernel):

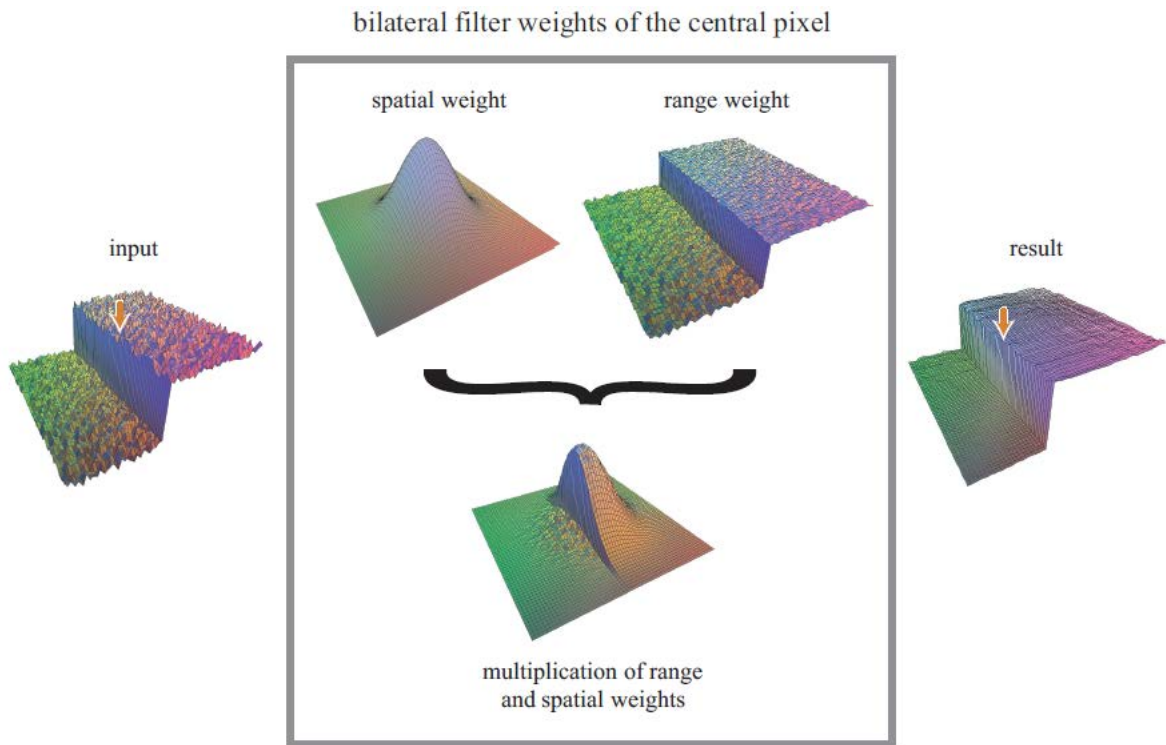
$$s(i, j, k, l) = \exp\left(-\frac{(i - k)^2 + (j - l)^2}{2 \sigma_s^2}\right)$$

Có thể xem đây là một mở rộng của Gaussian kernel.

Nhân tố thứ hai là tương quan về giá trị của mỗi điểm ảnh (màu sắc) (gọi là range kernel):

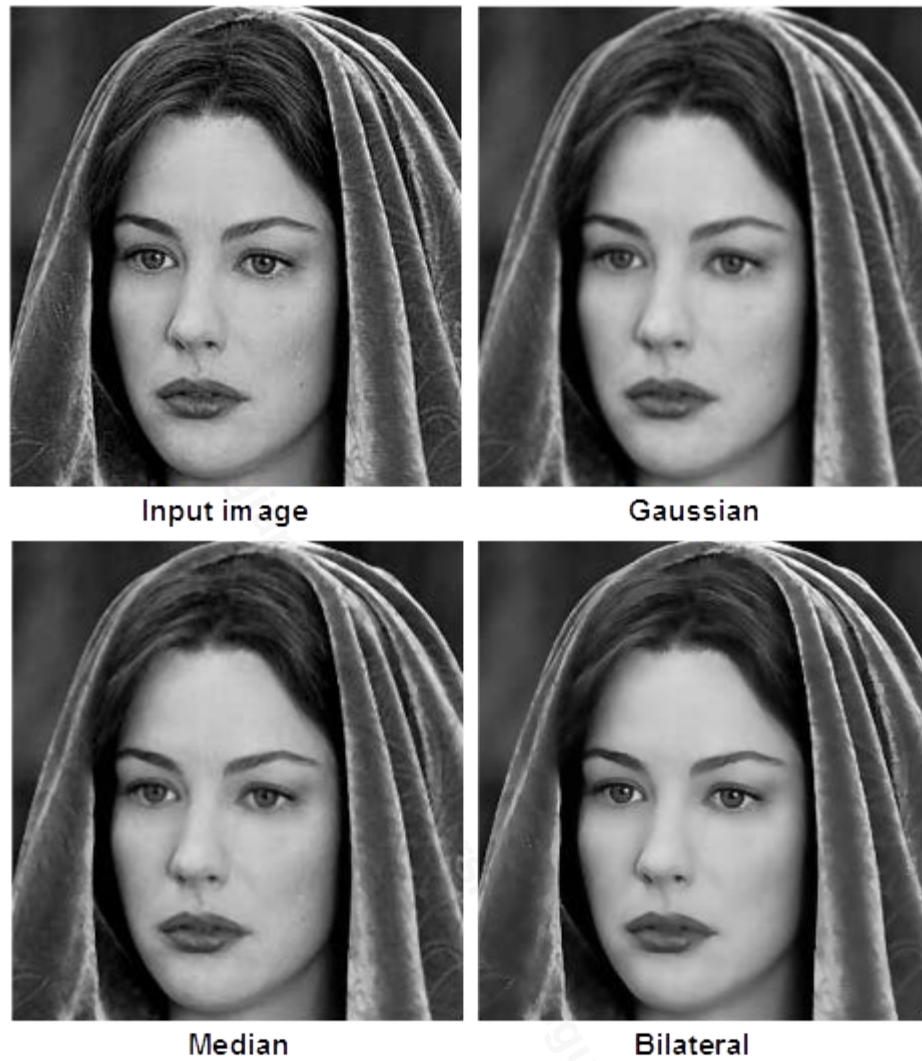
$$r(i, j, k, l) = \exp\left(-\frac{\|f(i, j) - f(k, l)\|^2}{2\sigma_r^2}\right)$$

$$w(i, j, k, l) = s(i, j, k, l) * r(i, j, k, l) = \exp\left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_s^2} - \frac{\|f(i, j) - f(k, l)\|^2}{2\sigma_r^2}\right)$$



Hình 3.12: Sự kết hợp của bộ lọc spatial kernel và range kernel

Có thể thấy tác dụng của spatial kernel $s(i, j, k, l)$ là lọc nhiễu còn tác dụng của range kernel $r(i, j, k, l)$ là giữ lại các đặc điểm sắc nét của ảnh (xem hình minh họa 3.12 và 3.13). Nhược điểm của bộ lọc Bilateral là tốc độ chậm vì cần phải tính toán nhiều.



Hình 3.13: Lọc Bilateral so với lọc Gaussian và Median

2.2.4. Các phép lọc phi tuyến khác



Hình 3.14: Các bộ lọc phi tuyến Min, Max và Mid-point.

Khi giá trị của điểm ảnh output bằng với min, max hoặc min+max của các điểm lân cận ta sẽ có các bộ lọc phi tuyến min, max và mid-point tương ứng (xem hình minh họa 3.11). Nhưng các bộ lọc này không hiệu quả bằng bộ lọc trung vị, ATM và Bilateral.

3. Các phép lọc làm sắc nét ảnh

Các bộ lọc làm sắc nét ảnh (còn gọi là sharpening filters hoặc bộ lọc thông cao trong miền không gian, spatial highpass filters) nhằm mục đích nhấn mạnh vào các chuyển đổi về cường độ sáng của các điểm ảnh chẳng hạn như các biên (edges). Output của một phép lọc làm sắc nét ảnh thường là số âm và do đó cần phải thực hiện một ánh xạ vào khoảng [0, 255]. Tổng các phần tử trong bộ lọc làm sắc nét ảnh bằng 0.

3.1. Bộ lọc Linear sharpening

Các bộ lọc làm mờ ảnh được trình bày ở phần trước có thể được dùng để làm sắc nét ảnh với một kỹ thuật gọi là unsharp masking hay linear sharpening. Nguyên tắc của kỹ thuật này là ảnh bị làm mịn sẽ đồng thời làm mất đi các thành phần tần số cao của ảnh gốc nên một bản sắc nét của ảnh có thể nhận được bằng cách cộng ảnh gốc với hiệu giữa nó và ảnh bị mờ theo công thức:

$$I_{sharp} = I + \gamma(I - I_{smooth}) = (1 + \gamma)I - \gamma I_{smooth}$$

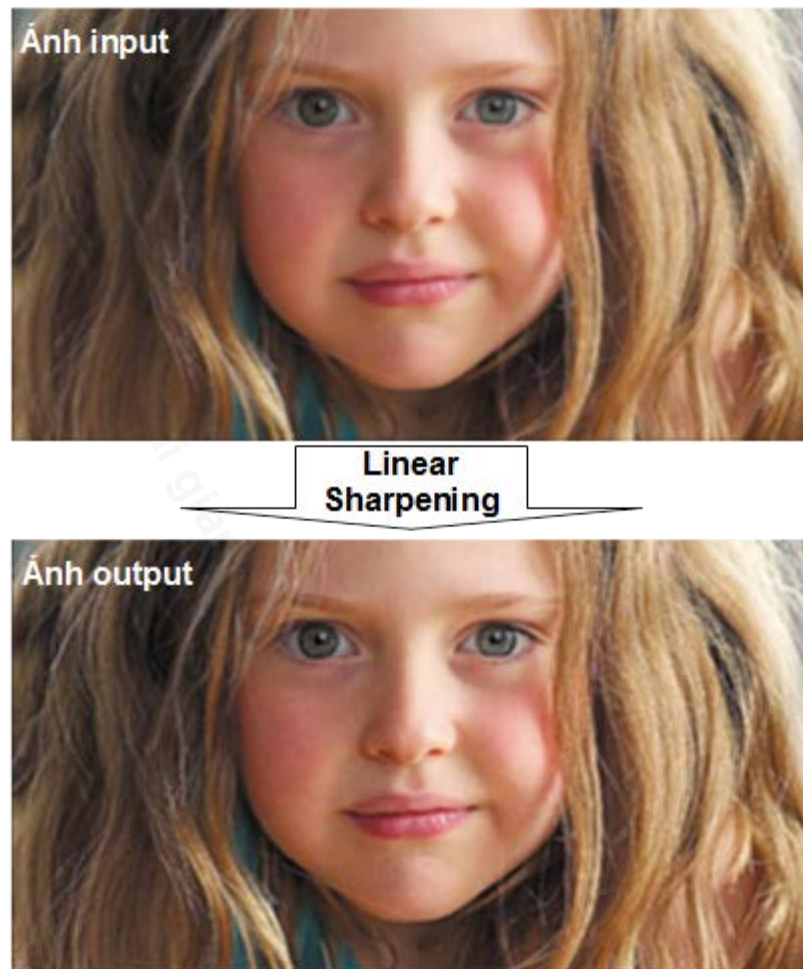
Trên thực tế bộ lọc này thường được sử dụng bởi các thiết bị chụp ảnh với giá trị của γ là 0.5. Khảo sát về mặt lý thuyết cho thấy bộ lọc này còn có tác dụng loại bỏ các quầng sáng (halos) của ảnh chụp. Hình 3.15 minh họa kết quả lọc linear sharpening với bộ lọc Gaussian được sử dụng để làm mờ ảnh.

3.2. Bộ lọc High boost

Việc làm cho ảnh sắc nét có thể làm mất các chi tiết nên cần phải tăng cường vai trò của ảnh gốc trong ảnh kết quả theo công thức sau:

$$I_{HP} = AI - I_{smooth} = (A - 1)I + I_{sharp}$$

Khi $A=2$, bộ lọc High boost chính là một bộ lọc làm sắc nét unsharp masking, còn với $A>1$, các phần của ảnh gốc sẽ thêm các chi tiết màu đen vào trong bộ lọc làm sắc nét tương ứng.



Hình 3.15: Kỹ thuật linear sharpening

3.3. Đạo hàm của ảnh (Image derivative)

Lấy đạo hàm của ảnh là một phương pháp phổ biến để làm sắc nét ảnh.

3.3.1. Khái niệm

Một bức ảnh có thể được xem như một mặt với độ cao tương ứng với các mức xám của ảnh. Biểu diễn của ảnh có thể xem là một hàm hai chiều với giá trị tại mỗi vị trí điểm ảnh (x, y) là độ sáng của ảnh. Trong phần lọc ảnh (lọc trung bình, lọc Gaussian) chúng ta đã thấy giá trị trung bình của mỗi lân cận luôn làm cho ảnh mờ đi nên sự khác biệt giữa các mức xám giữa các điểm lân cận có thể mang lại hiệu ứng ngược và làm cho ảnh thêm sắc nét.

Các biên là các điểm ảnh mà tại đó độ sáng của ảnh thay đổi đột ngột. Các thay đổi này có thể được biểu diễn bằng khái niệm đạo hàm của ảnh. Với một ảnh $f(x, y)$, đạo hàm bậc một của ảnh theo các hướng x và y được tính theo công thức sau:

$$g_x = \frac{\partial f}{\partial x} = f(x, y) - f(x + 1, y + 1), g_y = \frac{\partial f}{\partial y} = f(x + 1, y) - f(x, y + 1)$$

Hai công thức trên tương ứng với việc nhân chập ảnh $f(x, y)$ với hai bộ lọc Roberts (bảng 3.1) theo các hướng x và y tương ứng. Như vậy có thể biểu diễn gradient của ảnh như một cặp của phép nhân chập :

$$g_x = h_x \otimes f(x, y), g_y = h_y \otimes f(x, y)$$

Kết quả của hai phép nhân chập này sẽ là ứng cử viên cho các biên của ảnh theo các hướng x và y.

Ngoài toán tử Roberts, các toán tử Prewitt và Sobel cũng có thể dùng cho việc làm sắc nét ảnh hoặc tính gradient của ảnh.

X-direction		Y-direction	
1	0	0	1
0	-1	-1	0
Roberts operator			
-1	0	1	-1
-1	0	1	0
-1	0	1	1
Prewitt operator			
-1	0	1	-1
-2	0	2	0
-1	0	1	1
Sobel operator			

Bảng 3.1: Các bộ lọc dùng để tính đạo hàm

Tiếp tục lấy các đạo hàm bậc nhất của đạo hàm bậc nhất của ảnh, ta sẽ có đạo hàm bậc hai của ảnh.

3.3.2. Derivative of Gaussian

Dựa vào khái niệm đạo hàm của ảnh (derivative) rất nhiều các khái niệm khác nảy sinh. Đầu tiên là khái niệm đạo hàm của một Gaussian kernel. Việc áp dụng một bộ lọc Gaussian lên ảnh sẽ làm mờ ảnh đồng thời giảm nhiễu nên có thể sẽ tốt hơn nếu ta áp dụng một bộ lọc Gaussian trước sau đó mới lấy đạo hàm:

$$\frac{\partial f}{\partial x} = \frac{\partial (G_{\sigma} * f)}{\partial x}$$

Theo tính chất của phép nhân chập công thức trên được viết lại là:

$$\frac{\partial f}{\partial x} = \frac{\partial G_{\sigma}}{\partial x} * f$$

Theo công thức này, đạo hàm của ảnh có thể được tính bằng phép nhân chập ảnh với một đạo hàm của bộ lọc Gaussian $\frac{\partial G_{\sigma}}{\partial x}$. Một cách đầy đủ (theo cả hai trục x và y) đạo hàm của một bộ lọc Gaussian là:

$$\nabla G_{\sigma} = \left[\frac{\partial G_{\sigma}}{\partial x} \frac{\partial G_{\sigma}}{\partial y} \right]^T$$

3.3.3. Difference of Gaussian (DoG)

Một phương pháp khác để có thể làm ảnh trở nên sắc nét hơn nhằm phục vụ cho việc phát hiện biên là sử dụng hiệu của hai bộ lọc Gaussian, gọi là Difference of Gaussian (DoG) theo công thức sau:

$$\nabla^2 G_\sigma = \text{DoG}_{\sigma_1, \sigma_2}(x, y) = G_{\sigma_1}(x, y) - G_{\sigma_2}(x, y) = \frac{1}{2\pi} \left[\frac{e^{-\frac{(x^2+y^2)}{2\sigma_1^2}}}{\sigma_1^2} - \frac{e^{-\frac{(x^2+y^2)}{2\sigma_2^2}}}{\sigma_2^2} \right]$$

Với $\sigma_1 > \sigma_2$, tỉ lệ thường dùng giữa hai giá trị này (đề nghị bởi Marr và Hildreth) là 1.6:1 trong khi đối với hệ thống thị giác của người tỉ lệ này là 1.75:1.

DoG thường được dùng trong phát hiện biên (edge detection) mà không cần phải tính đạo hàm của ảnh.

3.3.4. Toán tử Laplacian

Toán tử Laplacian (đề xuất bởi Rosenfeld và Kak năm 1982) dựa trên đạo hàm bậc 2 của ảnh theo công thức sau:

$$\nabla^2 f = \nabla f \nabla f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Vì:

$$\frac{\partial f}{\partial x} = f(x, y) - f(x+1, y+1), \frac{\partial f}{\partial y} = f(x+1, y) - f(x, y+1)$$

Nên ta có:

$$\frac{\partial^2 f}{\partial x^2} = f(x, y+1) - 2f(x, y) + f(x, y-1), \frac{\partial^2 f}{\partial y^2} = f(x+1, y) - 2f(x, y) + f(x-1, y)$$

Do đó:

$$\nabla^2 f = -4f(x, y) + f(x, y+1) + f(x, y-1) + f(x+1, y) + f(x-1, y)$$

Như vậy có thể biểu diễn kernel để tính Laplacian bằng phép nhân chập của một ảnh như sau:

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Bảng 3.2: Toán tử Laplacian

Dựa vào toán tử Laplacian, ảnh sắc nét sẽ thu được bằng cách lấy ảnh gốc trừ đi Laplacian của nó:

$$I_{\text{sharp}} = I - \nabla^2 I = 5f(x, y) - f(x, y+1) - f(x, y-1) - f(x+1, y) - f(x-1, y)$$

Bộ lọc tương ứng với công thức trên là:

0	-1	0
-1	5	-1
0	-1	0

Bảng 3.3: Bộ lọc làm sắc nét ảnh bằng toán tử Laplacian

So với đạo hàm bậc 1, đạo hàm bậc 2 của ảnh cho ta các chi tiết biên mạnh hơn, phản ánh tốt các chi tiết của ảnh, nhưng lại yếu đối với các thay đổi mức xám cục bộ. Do đó đạo hàm bậc 2 thích hợp hơn trong việc làm sắc nét ảnh.

Trên thực tế ta có thể sử dụng các kernel khác cho việc tính Laplacian của ảnh, khi đó nếu điểm trung tâm của kernel dương thì ta sẽ phải cộng ảnh gốc với Laplacian của nó chứ không phải phép trừ như thông thường.



Ảnh Input

Ảnh output

Hình 3.16: Kết quả lọc ảnh bằng toán tử Laplacian

Khi đạo hàm của ảnh dùng để tính Laplacian là Gaussian ta sẽ có toán tử Laplacian of Gaussian (LoG). Cụ thể ta có công thức để tính một kernel LoG là:

$$LoG_{\sigma} = -\frac{1}{\pi\sigma^4} \left[1 - \frac{x^2 + y^2}{2\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

Để giữ sự tương thích giữa DoG và LoG, giá trị σ của LoG thường được chọn là:

$$\sigma = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 - \sigma_2^2} \ln \frac{\sigma_1^2}{\sigma_2^2}$$

3.4. Giới thiệu một số bộ lọc phổ biến

Phần này giới thiệu một số các bộ lọc tuyến tính được thực hiện qua phép nhân chập được sử dụng rộng rãi cho các mục đích khác nhau như làm mờ ảnh, làm sắc nét, tạo các hiệu ứng như motion blur, emboss và các toán tử dò biên phổ biến. Kết quả thực hiện thuật toán có thể xem trong hình minh họa 3.13.

3.4.1. Làm mờ ảnh

Diagram illustrating the construction of a 1/5 SRS from a 1/13 SRS. The 1/5 SRS is represented by a 3x3 grid of 0s and 1s. The 1/13 SRS is represented by a 5x5 grid of 0s and 1s. The 1/9 SRS is represented by a 5x5 grid of 0s and 1s.

0	1	0
1	1	1
0	1	0

1/5

0	0	1	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	1	0	0

1/13

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

1/9

Bảng 3.4: Bộ lọc làm mờ ảnh

Các bộ lọc làm mờ ảnh có tổng các phần tử bằng 1.

3.4.2. Hiệu ứng motion blur

1	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	0
0	0	0	0	0	0	0	0	1

Bảng 3.5: Bộ lọc tạo hiệu ứng blur motion

Bộ lọc tạo hiệu ứng blur motion bằng cách chỉ làm mờ ảnh theo một hướng. Điều này giống như khi camera di chuyển từ góc trên bên trái xuống góc dưới bên phải tạo thành một chuyển động nhưng bị mờ.

3.4.3. Làm sắc nét ảnh

Diagram illustrating the transformation of a 3x3 matrix A into a 5x5 matrix B through a series of row operations. The transformation is indicated by a large arrow labeled $1/8$.

Matrix A (3x3):

-1	-1	-1
-1	9	-1
-1	-1	-1

Matrix B (5x5):

0	0	0	0	0
0	0	-1	0	0
1	-1	5	-1	0
0	0	-1	0	0
0	0	0	0	0

Bảng 3.6: Bộ lọc làm sắc nét ảnh

Các bộ lọc làm sắc nét ảnh có tổng cách phần tử bằng 1.

3.4.4. Hiệu ứng emboss (chạm nổi)

-1	-1	0	-2	-1	0
-1	0	1	-1	1	1
0	1	1	0	1	2

-1	-1	-1	-1	0
-1	-1	-1	0	1
-1	-1	0	1	1
-1	0	1	0	1
0	1	1	1	1

Bảng 3.7: Bộ lọc tạo hiệu ứng Emboss

Input image



Làm mờ ảnh



Motion blur



Làm sắc nét



Hiệu ứng Emboss

Hình 3.17: Một số hiệu ứng với các bộ lọc phổ biến

4. Phát hiện biên (edge detection)

4.1. Giới thiệu

Các biên của ảnh (edges) thường nằm ở các vùng ảnh mà độ tương phản về cường độ sáng rất cao. Thường thì đây là những vị trí biểu diễn đường biên giới ngăn

cách giữa các đối tượng khác nhau trong ảnh nên các thuật toán phát hiện biên được sử dụng rất nhiều trong phân đoạn ảnh (image segmentation) khi chúng ta cần chia ảnh thành các vùng con tương ứng với các đối tượng khác nhau. Biểu diễn ảnh qua các biên cũng có ưu điểm là lượng dữ liệu cần để biểu diễn ảnh nhỏ trong khi vẫn giữ được hầu hết các thông tin cần thiết từ ảnh.

Vì các biên được tạo thành chủ yếu từ các thông tin có tần số cao, nên theo lý thuyết hoàn toàn có thể dùng các bộ lọc thông cao (highpass filter) trong miền tần số hoặc dùng phép nhân chập với một bộ lọc thích hợp để thực hiện phát hiện biên của ảnh. Tuy nhiên trên thực tế việc phát hiện biên thường dùng đều sử dụng các kỹ thuật xử lý ảnh trong miền không gian vì kết quả thu được tốt hơn và chi phí tính toán thấp hơn.

Về cơ bản có 3 bước để thực hiện phát hiện biên của một ảnh:

Bước 1: Lọc nhiễu (noise reduction). Ở bước này các nhiễu sẽ bị loại bỏ càng nhiều càng tốt nhưng không được ảnh hưởng quá nhiều tới các biên (do bị làm mờ).

Bước 2: Tăng cường chất lượng các biên. Ở bước này một bộ lọc mạnh đối với các biên sẽ được sử dụng để xác định các biên, là các cực trị cục bộ của kết quả phép lọc. Thường một bộ lọc làm sắc nét sẽ được sử dụng.

Bước 3: Xác định vị trí các biên. Ở bước này, các điểm cực trị cục bộ sẽ được xác định là các biên hay chỉ là kết quả của nhiễu tác động.

4.2. Phát hiện biên dựa vào gradient của ảnh.

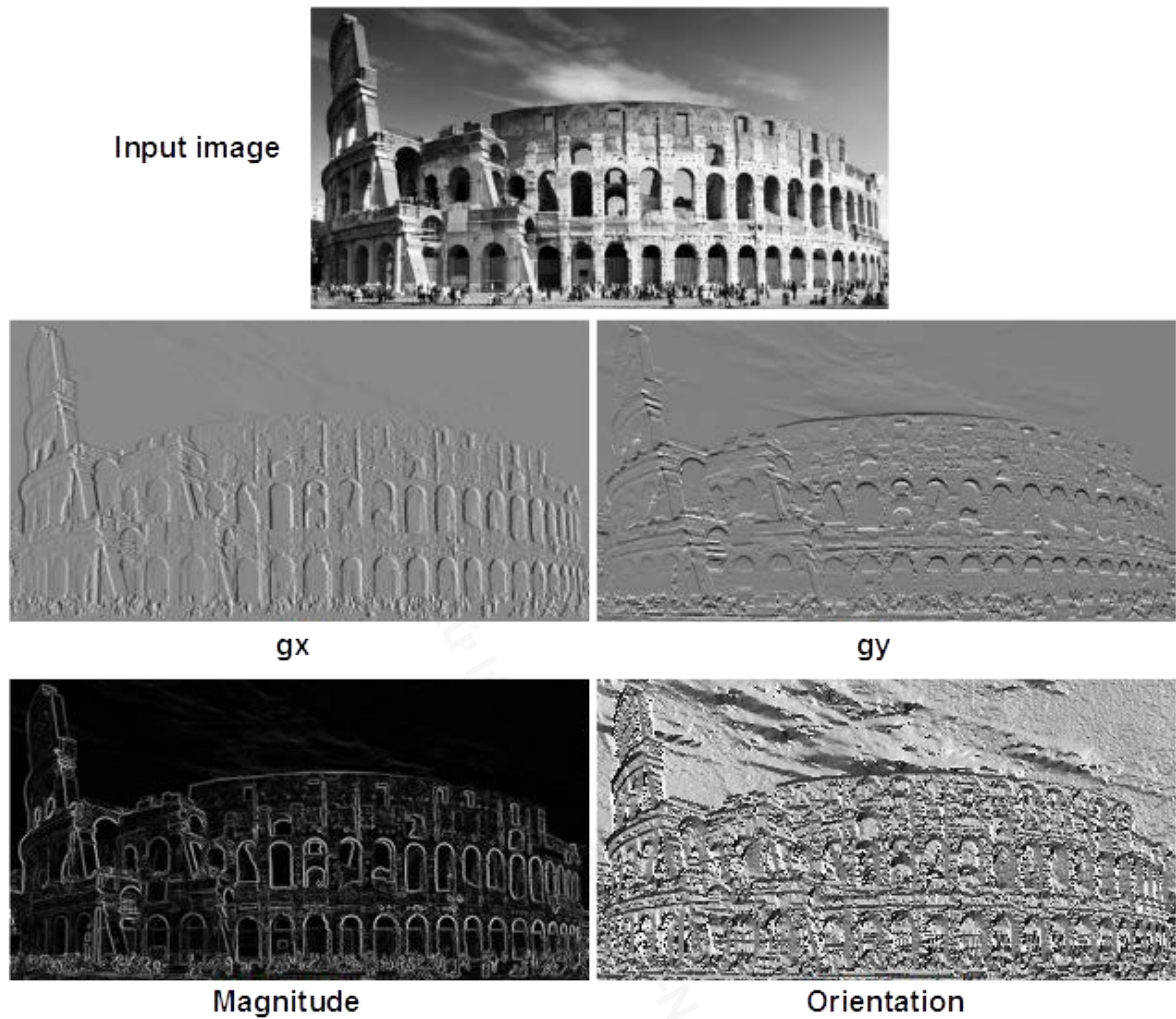
Dựa vào gradient của ảnh là phương pháp khá phổ biến để phát hiện biên của ảnh vì nó khá dễ hiểu cũng như dễ cài đặt. Ảnh input sẽ được nhân chập với hai kernel filter theo các hướng x và y để sinh ra đạo hàm bậc nhất theo các hướng này, khi đó gradient của ảnh sẽ được xem như một vector hai thành phần:

$$\Delta f(x, y) = g = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} h_x \otimes f(x, y) \\ h_y \otimes f(x, y) \end{bmatrix}$$

Hai thành phần quan trọng để biểu diễn gradient của ảnh là magnitude và orientation (phase, direction) được tính như sau:

$$|\Delta f(x, y)| = |g| = \sqrt{g_x^2 + g_y^2}, \theta = \text{atan}\left(\frac{g_y}{g_x}\right)$$

Trong khi magnitude thể hiện độ lớn của các biên tại mỗi điểm thì orientation thể hiện hướng của gradient mà theo hướng đó tốc độ thay đổi hàm ảnh $f(x, y)$ là lớn nhất (xem hình minh họa bên dưới).

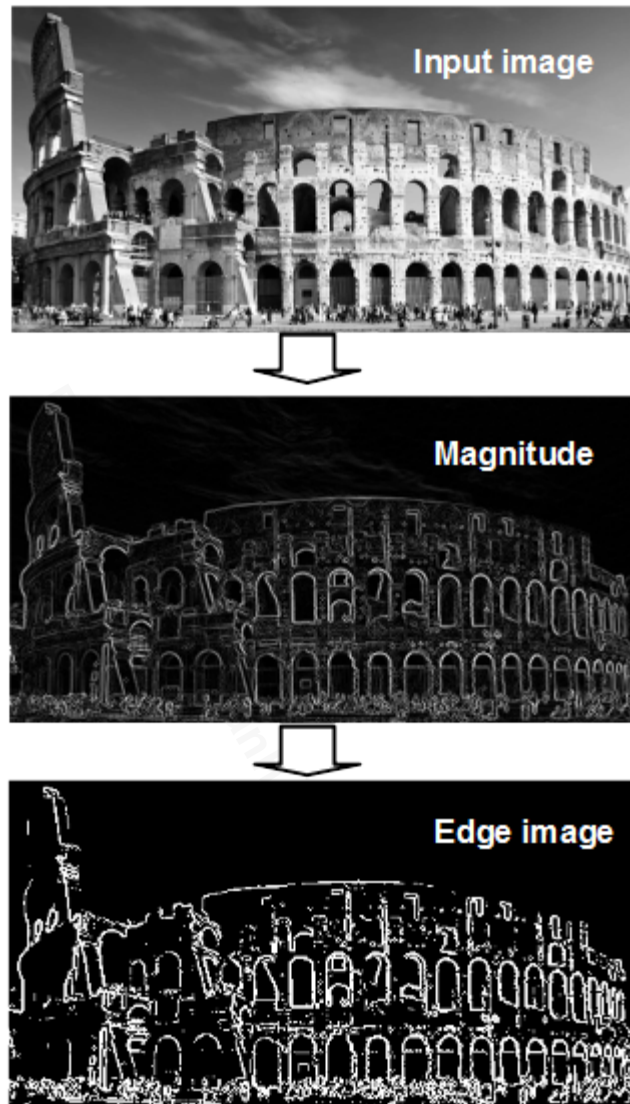


Hình 3.18: Ảnh và các thành phần Gradient

Ở bước tiếp theo để xác định các biên của ảnh ta dùng thuật toán lọc theo ngưỡng, một điểm của ảnh magnitude sẽ được xét là nằm trên một biên nếu giá trị magnitude của nó lớn hơn một giá trị ngưỡng, ngược lại điểm đó không nằm trên biên của ảnh và sẽ bị loại bỏ.

$$g(x, y)_{edge\ point} = \begin{cases} 1 & \text{nếu } |g(x, y)| > T \\ 0 & \text{nếu } |g(x, y)| < T \end{cases}$$

Có thể xem kết quả qua hình minh họa bên dưới:



Hình 3.19: Phát hiện biên dựa trên gradient của ảnh

Phương pháp phát hiện biên dựa vào gradient có một số nhược điểm sau:

- + Biên của các phần tử trong ảnh không phải lúc nào cũng đạt được độ sắc nét cần thiết.
- + Giá trị magnitude của nhiễu đôi khi có thể cao hơn so với magnitude của điểm nằm trên biên.
- + Việc thực hiện một phép lọc ngưỡng đơn giản ở bước cuối cùng cho kết quả là các biên có độ dày lớn.

Một phương pháp tốt hơn để phát hiện biên là phương pháp Canny có thể khắc phục các nhược điểm này, tuy nhiên phương pháp này vượt qua ra ngoài phạm vi của tài liệu này.

Bài tập

Bài tập 1: Viết chương trình thực hiện cài đặt phép nhân chập 1 ảnh với 1 kernel và minh họa với các bộ lọc được giới thiệu ở phần 3.3.

Bài tập 2: Viết chương trình mở một file ảnh và thực hiện lọc ảnh theo các phương pháp

- + Lọc trung bình
- + Lọc Gaussian
- + Lọc trung vị
- + Lọc ATM
- + Lọc Bilateral

Cho phép ghi kết quả vào file ảnh có tên tương ứng với thao tác thực hiện.

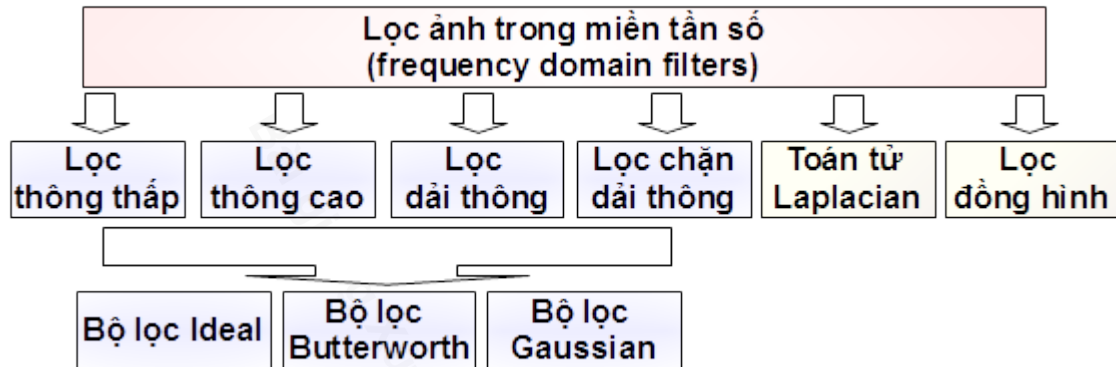
Bài tập 3: Viết chương trình mở một file ảnh và thực hiện:

- + Lọc ảnh theo bộ lọc High boost
- + Tính đạo hàm theo các bộ lọc Roberts, Sobel, Prewitt
- + Tính magnitude và phase của đạo hàm tương ứng
- + DoG, Laplacian, LoG của ảnh

Cho phép ghi kết quả vào file có tên tương ứng với thao tác thực hiện.

CHƯƠNG IV: XỬ LÝ ẢNH TRONG MIỀN TẦN SỐ

Chương này trình bày các phương pháp về xử lý ảnh trong miền tần số (*frequency domain*) với kỹ thuật cơ bản là biến đổi Fourier (rời rạc) hai chiều (2D DFT). Cùng với biến đổi Fourier hai chiều, các thuật toán lọc ảnh (lọc thông thấp, thông cao, lọc dải thông, chặn dải thông, lọc bằng toán tử Laplacian, lọc đồng hình) cũng sẽ được trình bày.



Hình 4.1: Các bộ lọc ảnh trong miền tần số

1. Miền tần số và biến đổi Fourier hai chiều

1.1. Các khái niệm cơ sở

Số phức (complex number): Một số phức x có dạng $x = a + jb$, trong đó $j^2 = -1$, $a, b \in \mathbb{R}$, a được gọi là phần thực (real part) còn b được gọi là phần ảo (imaginary part).

Phép cộng và nhân hai số phức được thực hiện như sau:

$$(a + jb) + (c + jd) = (a + c) + j(b + d)$$

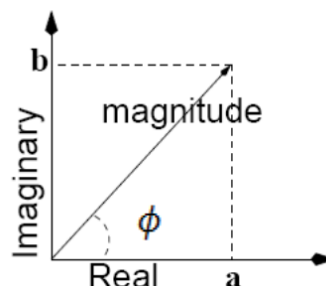
$$(a + jb) * (c + jd) = (ac - bd) + j(ad + bc)$$

Biểu diễn số phức được dưới dạng vector qua magnitude và phase được tính như sau:

$$\text{Magnitude: } |x| = \sqrt{a^2 + b^2}$$

$$\text{Phase: } \phi(x) = \tan^{-1}\left(\frac{b}{a}\right)$$

$$x = |x|e^{j\phi(x)}$$



Hình 4.2: Biểu diễn số phức

Phép nhân số phức qua biểu diễn magnitude-phase:

$$xy = |x|e^{j\phi(x)}|y|e^{j\phi(y)} = |x||y|e^{j(\phi(x)+\phi(y))}$$

Số phức liên hợp và tính chất: $x^* = a - jb$, $|x| = |x^*|$, $\phi(x) = -\phi(x^*)$, $xx^* = |x|^2$.

Công thức Euler và các tính chất:

$$e^{\pm j\theta} = \cos(\theta) \pm j\sin(\theta)$$

$$|e^{\pm j\theta}| = \sqrt{\cos^2(\theta) + \sin^2(\theta)} = 1$$

$$\phi(e^{\pm j\theta}) = \tan^{-1}\left(\pm \frac{\sin(\theta)}{\cos(\theta)}\right) = \tan^{-1}(\tan(\theta)) = \pm \theta$$

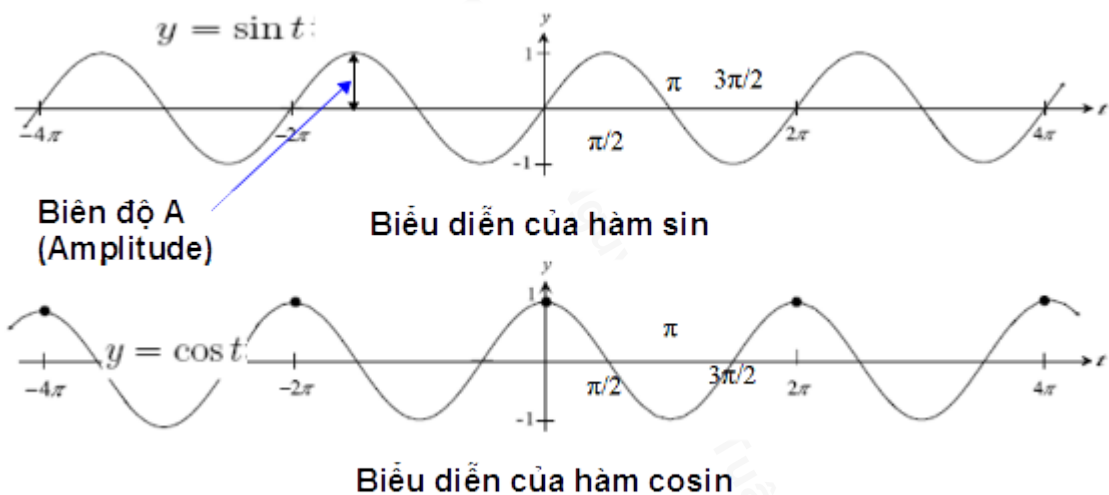
$$\sin(\theta) = \frac{1}{2}(e^{j\theta} - e^{-j\theta})$$

$$\cos(\theta) = \frac{1}{2}(e^{j\theta} + e^{-j\theta})$$

Các hàm sin và cosin: biểu diễn dạng hàm tổng quan của các hàm sin và cosin được cho bởi các công thức:

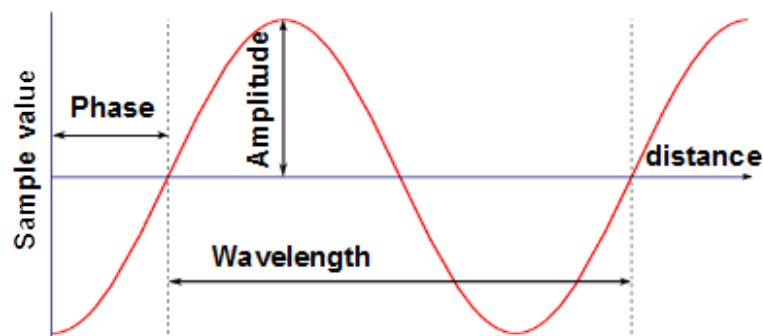
$$y(t) = A\sin[a(t + b)], y(t) = A\cos[a(t + b)]$$

Trong đó $|A|$ là biên độ của sóng, $2\pi/|a|$ là chu kỳ và b là pha ban đầu (phase shift). Trường hợp đặc biệt khi $A=1$, $b=0$ và $a=1$, các sóng hình sin và cosin được biểu diễn như hình sau:



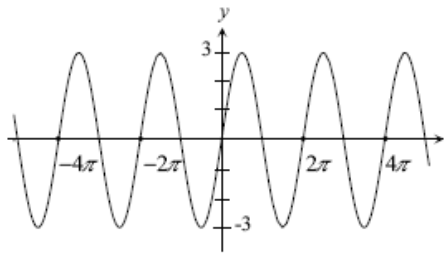
Hình 4.3: Biểu diễn của hàm sin và cosin

Có thể thấy biểu diễn của hàm cosin thực chất là biểu diễn của hàm sin khi dịch chuyển pha bằng hằng số $b = \frac{\pi}{2}$, $\cos(t) = \sin(t + \frac{\pi}{2})$, vì vậy các biểu diễn của hàm sin và cosin đều được gọi là sinusoid và được biểu diễn như sau:



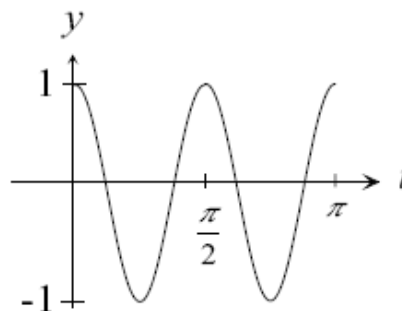
Hình 4.4: Sinusoid

Khi thay đổi biên độ A, biểu diễn của hàm sin sẽ thay đổi như sau:



Hình 4.5: Biểu diễn của hàm $y=3\sin(t)$

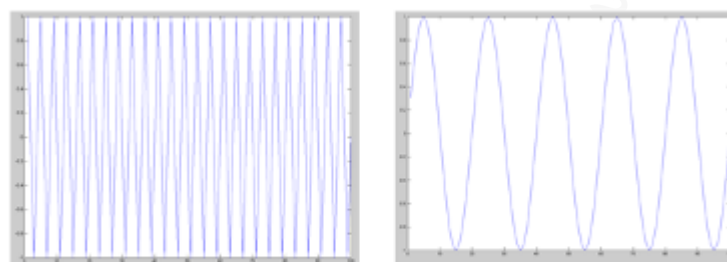
Khi thay đổi chu kỳ $T = 2\pi/|\alpha|$ với $A=1$, $b=0$, $\alpha=4$, hàm $y = \cos(\alpha t)$ được biểu diễn như sau:



Hình 4.6: Biểu diễn của hàm $y=\cos(4t)$

Hàm $y=\cos(4t)$ có chu kỳ bằng $T = \frac{2\pi}{|\alpha|} = \pi/2$. Hàm có chu kỳ nhỏ sẽ tương ứng với tần số cao (dao động nhanh hơn, hàm $\sin(nx)$, $\cos(nx)$ với n lớn) và ngược lại chu kỳ lớn sẽ tương ứng với tần số thấp (hàm $\sin(nx)$, $\cos(nx)$ với n nhỏ). Tần số của hàm được định nghĩa là $f = 1/T$.

Tần số (Frequency): là số dao động trong một giây, tần số càng cao (high) thì chu kỳ càng ngắn (xem hình 4.7). Đối với ảnh, tần số biểu diễn cho tốc độ (nhanh hay chậm) thay đổi của cường độ của mức xám. Các tần số thấp tương ứng với các thay đổi chậm như các vùng liên tục của ảnh còn các tần số cao tương ứng với các thay đổi nhanh, đột ngột của ảnh như các cạnh (edges).



Tín hiệu có tần số cao Tín hiệu có tần số thấp

Hình 4.7: Biểu diễn tần số của tín hiệu 1-D

1.2. Lý thuyết Fourier

Biến đổi Fourier, được đặt theo tên của nhà bác học người Pháp Fourier (tên đầy đủ là Jean Baptiste Joseph Fourier, sinh tại Auxerre năm 1768), là một trong những lý thuyết toán học quan trọng nhất và có ảnh hưởng sâu sắc tới các lĩnh vực phân tích toán,

vật lý và trong kỹ thuật hiện đại (thiết kế ăngten, phân tích giọng nói, xử lý ảnh vệ tinh ...). Lý thuyết Fourier (đưa ra năm 1807) dựa trên ý tưởng là **bất cứ một hàm tuần hoàn** (periodic function) nào cũng có thể biểu diễn dưới dạng một **tổng có trọng số của các hàm sin/cosin** có các tần số (và biên độ) khác nhau (xem hình 4.8).

Các trọng số trong biểu diễn Fourier của một hàm được gọi là dãy Fourier.

$$f(x) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} a_n \cos(nx) + \sum_{n=1}^{\infty} b_n \sin(nx),$$

$$a_0 = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) dx$$

$$a_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(nx) dx$$

$$b_n = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(nx) dx$$

Dạng tổng quát của chuỗi Fourier phức chính là biến đổi Fourier 1 chiều (Forward Fourier Transform-Forward FT) liên tục (hàm biểu diễn f là hàm liên tục):

$$F(f(x)) = F(u) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi ux} dx, \text{ với } e^{-j2\pi ux} = \cos(2\pi ux) - j\sin(2\pi ux)$$

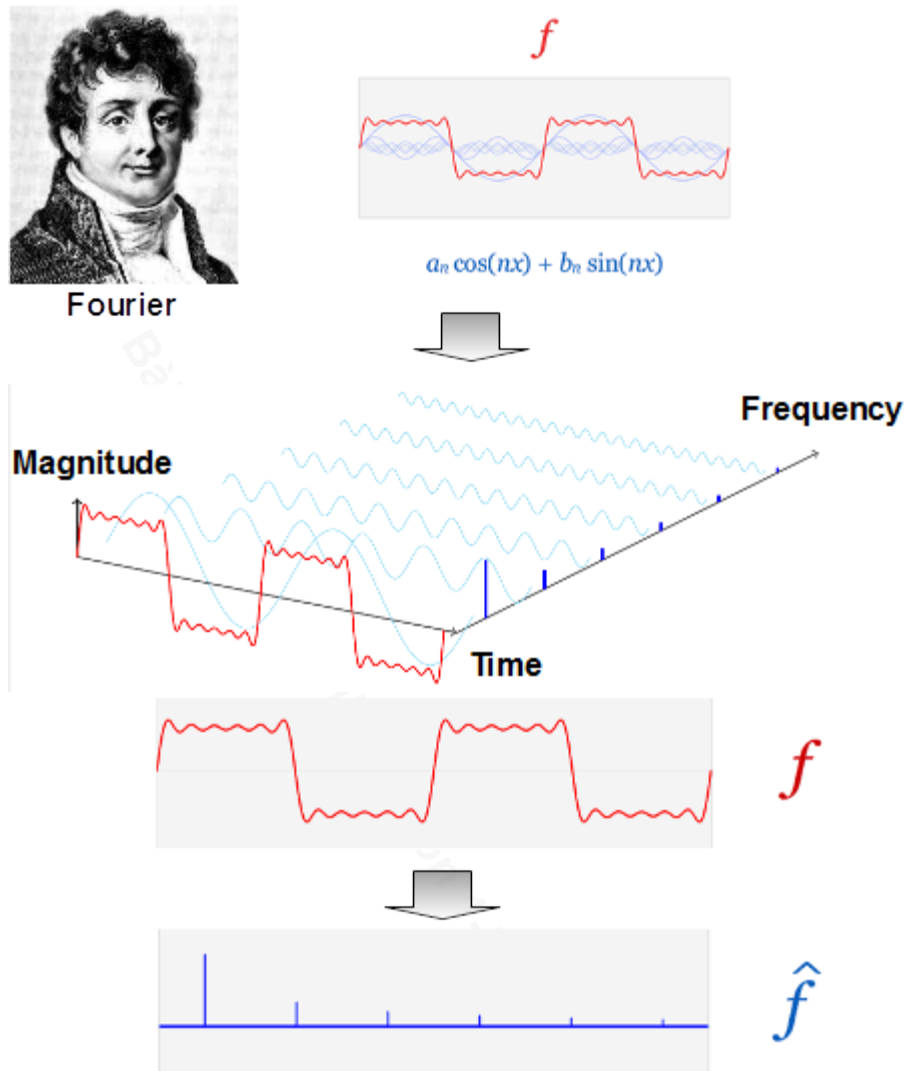
Biến đổi Fourier ngược (Inverse Fourier Transform-Inverse FT):

$$F^{-1}(F(u)) = f(x) = \int_{-\infty}^{\infty} F(u) e^{j2\pi ux} dx$$

Biến đổi Fourier liên tục được mở rộng cho các tín hiệu 2 chiều như sau:

$$\text{Biến đổi thuận: } F(f(x, y)) = F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-j2\pi(ux+vy)} dx dy$$

$$\text{Biến đổi ngược: } F^{-1}(F(u, v)) = f(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v) e^{j2\pi(ux+vy)} du dv$$



Hình 4.8: Lý thuyết Fourier

Tác dụng của biến đổi Fourier là biến các tín hiệu ở miền không gian/thời gian (time/spatial domain) thành các tín hiệu của miền tần số (bằng biến đổi thuận-Forward FT) và ngược lại (bằng biến đổi ngược-Inverse FT).

1.3. Biến đổi Fourier rời rạc (Discrete Fourier Transform-DFT)

1.3.1. Biến đổi Fourier rời rạc 1 chiều (1-D DFT)

Đối với các tín hiệu rời rạc được biểu diễn bằng một dãy N giá trị $f(x) = f(x_0 + x\Delta x)$, $x = 0, 1, \dots, (N-1)$ biến đổi Fourier được thực hiện như sau:

$$\text{Biến đổi thuận: } F(u) = \sum_{x=0}^{N-1} f(x) e^{-\frac{j2\pi ux}{N}}, u = 0, 1, \dots, (N-1)$$

$$\text{Biến đổi ngược: } F^{-1}(F(u)) = f(x) = \frac{1}{N} \sum_{u=0}^{N-1} F(u) e^{\frac{j2\pi ux}{N}}, x = 0, 1, \dots, (N-1)$$

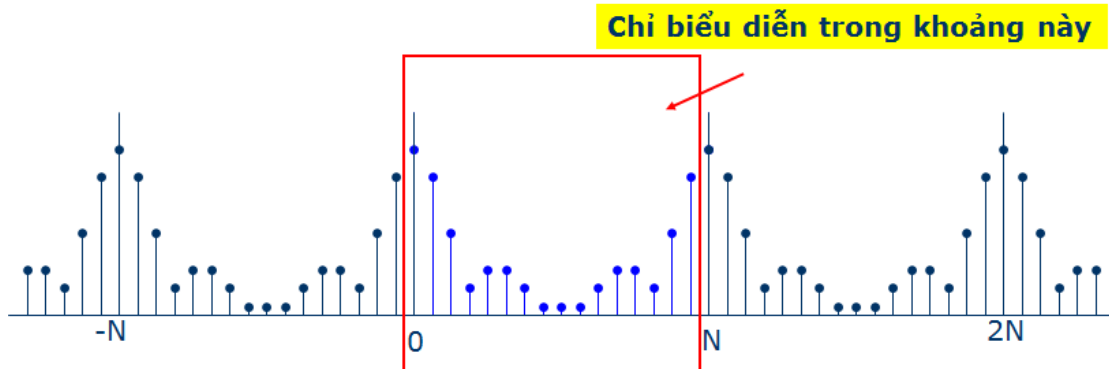
Dãy $F(u)$ cũng là một tín hiệu rời rạc trong miền tần số: $F(u) = F(u\Delta u)$, $u = 0, 1, \dots, (N-1)$, $\Delta u = 1/N\Delta x$ và có thể biểu diễn dưới dạng số phức như sau:

$$F(u) = R(u) + jI(u) = |F(u)|e^{-j\phi(u)}$$

$$\text{Trong đó: } |F(u)| = \sqrt{R(u)^2 + I(u)^2}, \phi(u) = \tan^{-1}(I(u)/R(u))$$

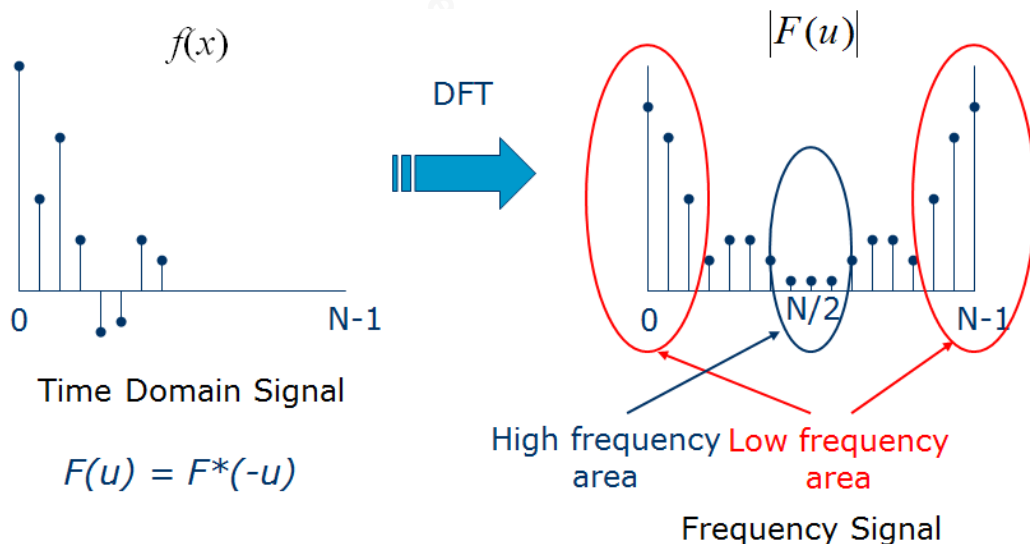
Tính tuần hoàn của biến đổi DFT:

Từ công thức $F(u) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-j2\pi ux/N} \Rightarrow F(u) = F(u + kN)$, điều này có nghĩa DFT là một biến đổi tuần hoàn và sẽ lặp lại sau mỗi N điểm (N chính là chu kỳ của DFT) vì vậy khi biểu diễn một biến đổi DFT ta chỉ cần biểu diễn hàm F trong một khoảng N điểm như hình vẽ sau:



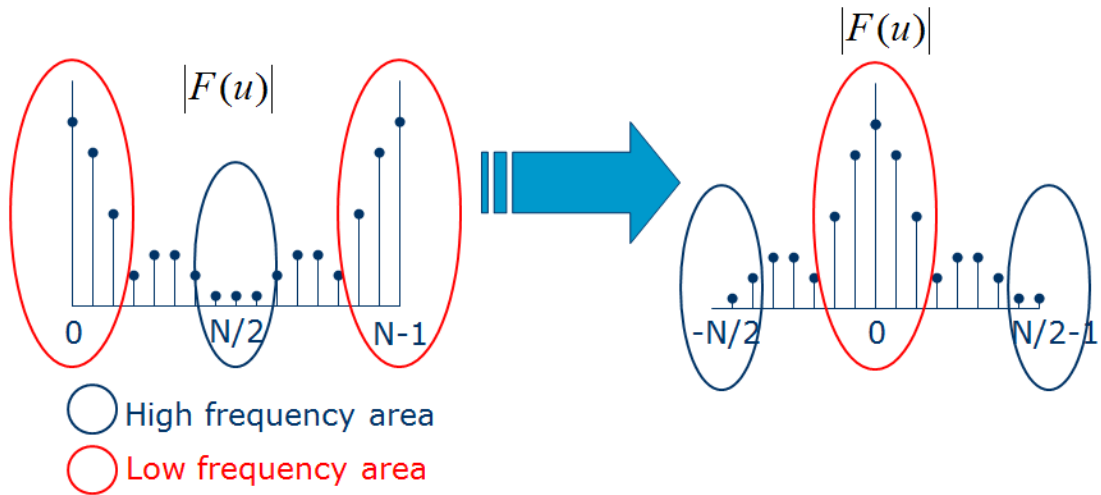
Hình 4.9: Tính tuần hoàn của DFT

Trong một chu kỳ, DFT được biểu diễn như sau:



Hình 4.10: Biểu diễn DFT trong một chu kỳ

Tuy nhiên rõ ràng là cách biểu diễn này không thực sự dễ hiểu nên một cách biểu diễn bằng cách dịch (shift) các vùng có tần số cao của biểu diễn $F(u)$ vào trung tâm (0) để nhận được biểu diễn dễ hiểu hơn đã được đề xuất như sau:



Hình 4.11: Biểu diễn của DFT sau phép dịch

1.3.2. Biến đổi Fourier rời rạc 2 chiều (2-D DFT)

Trong biến đổi 2-D liên tục, có thể xem tín hiệu một ảnh liên tục tồn tại trong một vùng hữu hạn của một mặt phẳng vô hạn còn khi chuyển sang biến đổi rời rạc, một ảnh được xem là nằm trên mặt phẳng đóng. Đối với một tín hiệu rời rạc 2 chiều, cụ thể là một ảnh $f(x,y)$ kích thước $M \times N$, biến đổi Fourier rời rạc được thực hiện như sau:

Biến đổi thuận (2D DFT): $F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$ với $u = 0,1..(M-1)$ là tần số theo hướng x và $v = 0,1..(N-1)$ là tần số theo hướng y.

Biến đổi ngược (2D IDFT): $F^{-1}(F(u,v)) = f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$ với $x = 0,1..(M-1)$ và $y = 0,1..(N-1)$.

Tương tự như đối với biến đổi cho tín hiệu 1 chiều, biến đổi DFT 2 chiều cũng có thể biểu diễn dưới dạng số phức:

$$F(u,v) = R(u,v) + jI(u,v) = |F(u,v)|e^{-j\phi(u,v)}$$

Trong đó: $|F(u,v)| = \sqrt{R(u,v)^2 + I(u,v)^2}$, $\phi(u,v) = \tan^{-1}(I(u,v)/R(u,v))$

Các tính chất biến đổi DFT 2 chiều:

+ Biến đổi DFT và IDFT 2-D là các biến đổi tuần hoàn vô hạn theo các hướng u và v:

$$F(u,v) = F(u + k_1M, v) = F(u, v + k_2N) = F(u + k_1M, v + k_2N)$$

+ Biến đổi DFT 2-D có tính chất tách biệt:

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} = \sum_{x=0}^{M-1} \left(\sum_{y=0}^{N-1} f(x,y) e^{-j2\pi(\frac{vy}{N})} \right) e^{-j2\pi(\frac{ux}{M})}$$

nghĩa là một biến đổi DFT 2 chiều thực chất là hai biến đổi DFT 1 chiều liên tiếp theo các chiều y, x hoặc x, y.

+ Biến đổi DFT có tính chất đối xứng liên hợp đối với $f(x,y)$ thực:

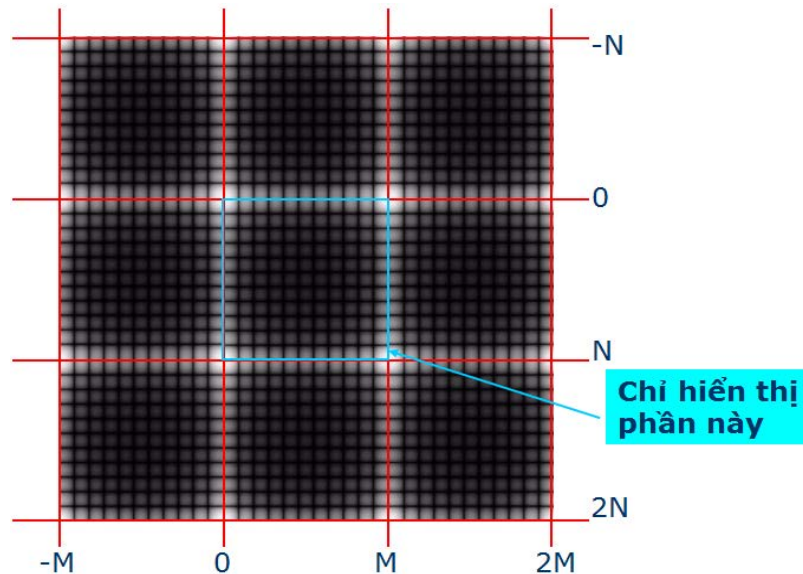
$$F(u,v) = F^*(-u,-v) \Rightarrow |F(u,v)| = |F(-u,-v)|$$

+ Tính dịch (translation):

$$f(x, y)e^{j2\pi(\frac{u_0x}{M} + \frac{v_0y}{N})} \Leftrightarrow F(u - u_0, v - v_0); f(x, y)(-1)^{x+y} \Leftrightarrow F(u - \frac{M}{2}, v - \frac{N}{2})$$

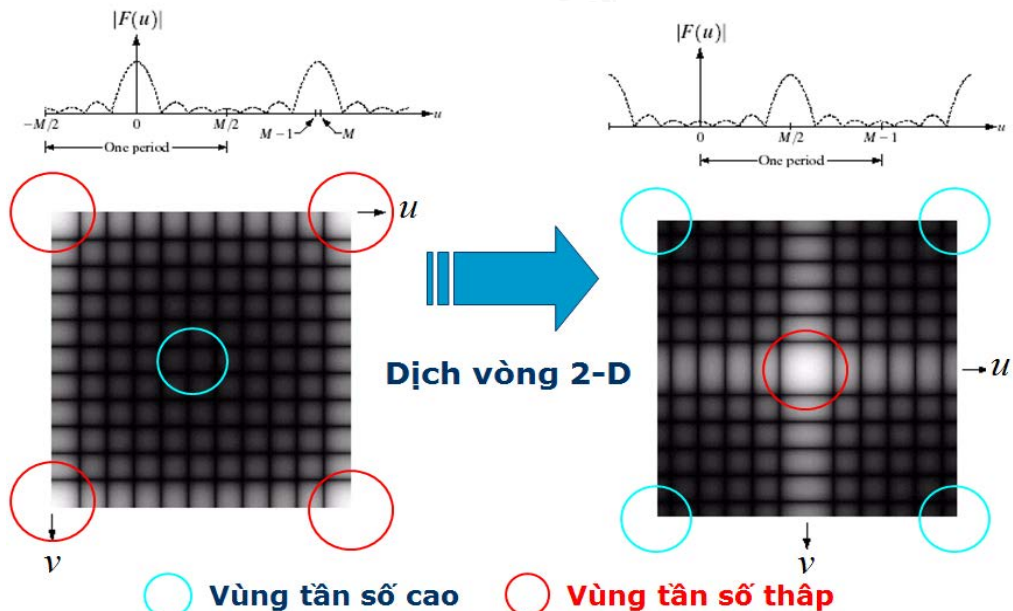
Biểu diễn biến đổi DFT:

Do tính tuần hoàn, một biến đổi DFT trên của một ảnh kích thước MxN sẽ lặp lại sau M điểm theo hướng x và N điểm theo hướng y:



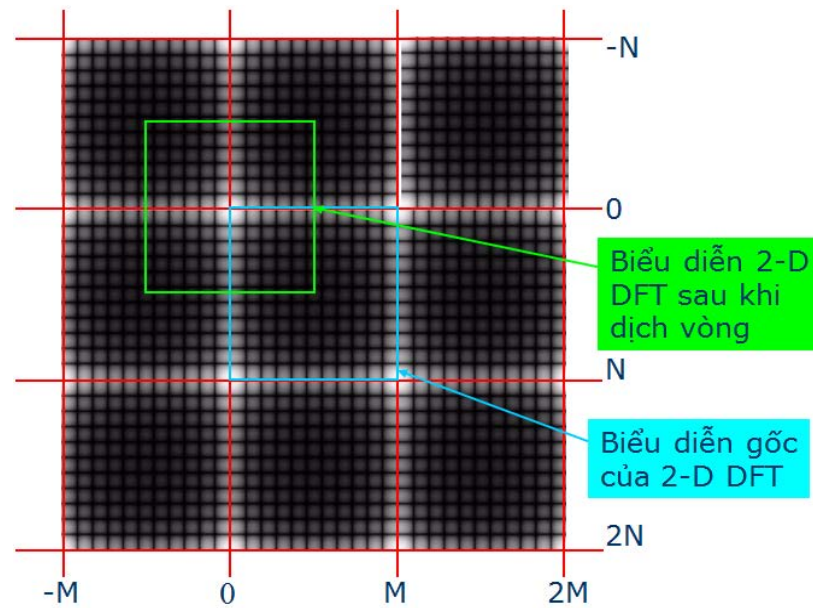
Hình 4.12: Tính tuần hoàn của biến đổi DFT 2-D

Tương tự như đối với biến đổi của tín hiệu 1 chiều, để biểu diễn biến đổi DFT với tín hiệu 2 chiều, ta cần một phép dịch vòng như hình sau:



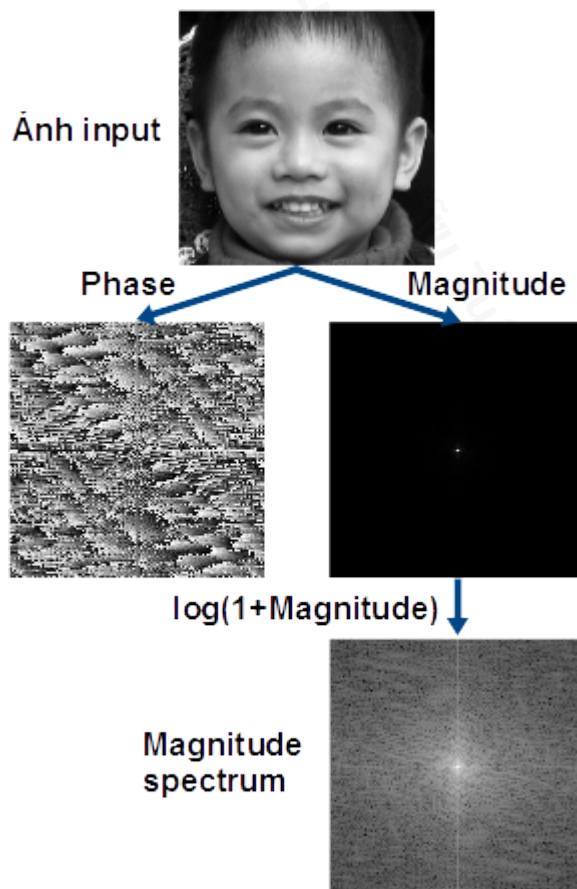
Hình 4.13: Biểu diễn 2-D DFT dùng dịch vòng

Phép dịch vòng được thực hiện như hình minh họa sau:



Hình 4.14: Phép dịch vòng trong biểu diễn 2-D DFT

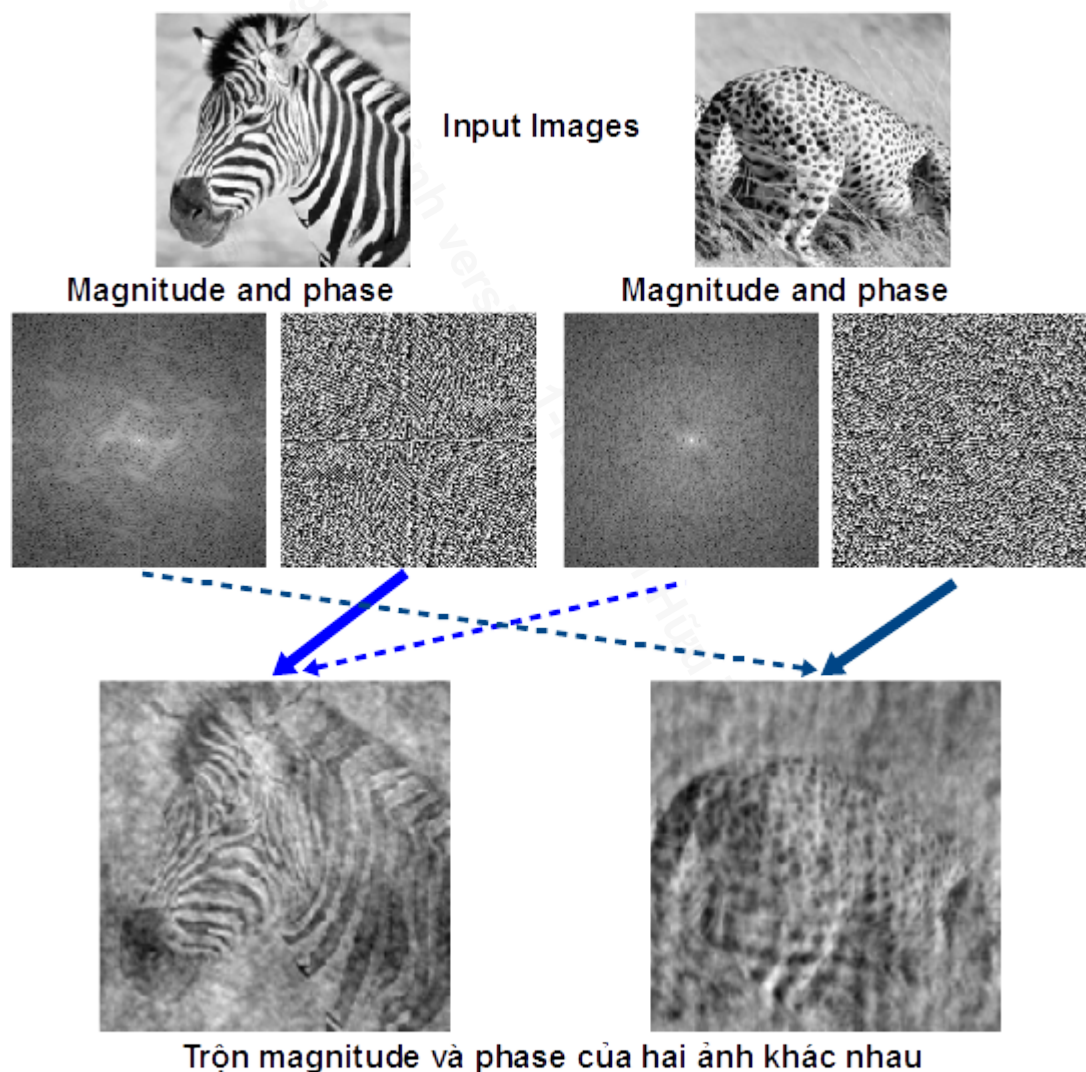
Các thành phần của biến đổi 2-D DFT: biến đổi DFT có hai thành phần quan trọng hay được sử dụng là magnitude $|F(u, v)|$ và phase $\phi(u, v)$. Biểu diễn của chúng được gọi là phổ (spectrum) của biến đổi Fourier. Giá trị phổ của các ảnh trong miền tần số có thể rất lớn (đối với thành phần magnitude) nên để biểu diễn trực quan người ta hay sử dụng phép biến đổi dựa trên hàm logarithm (xem thêm ở chương 2) để giảm các giá trị rồi thực hiện phép dịch vòng trước khi hiển thị (xem hình minh họa 4.15 bên dưới).



Hình 4.15: Phase và Magnitude của ảnh trong biến đổi 2-D DFT

Một bức ảnh luôn có các thành phần tương ứng với các tần số khác nhau nhưng magnitude thu được của các tần số cao hơn sẽ nhỏ hơn. Điều này dẫn đến một thực tế là các tần số thấp sẽ chứa nhiều thông tin về ảnh hơn so với các tần số cao. Hai chiều chính trong một ảnh magnitude là x và y . Giá trị của mỗi điểm ảnh xác định phase của tần số tương ứng tại điểm đó. Thông tin về phase là tối quan trọng để khôi phục lại ảnh gốc trong miền không gian. Có thể thấy trong hình minh họa 4.16 khi ta sử dụng thông tin Phase gốc kết hợp với thông tin Magnitude của ảnh khác thì hình ảnh thu được sẽ giống với hình ảnh cung cấp thông tin Phase.

Có thể thấy bằng mắt thường (qua các hình minh họa 4.15 và 4.16) rằng các ảnh đều có biểu diễn magnitude giống nhau. Nhưng thường thì thành phần magnitude sẽ được biểu diễn cho mục đích quan sát và dựa vào quan sát phổ magnitude chúng ta có thể phát hiện được nhiều trong ảnh (xem chương 5).



Hình 4.16: Vai trò của Magnitude và Phase trong khôi phục ảnh

1.3.3. Biến đổi Fourier nhanh (Fast Fourier Transform-FFT)

Biến đổi Fourier nhanh FFT là một thuật toán hiệu quả để cài đặt thuật toán DFT (cả xuôi và ngược). Có nhiều thuật toán FFT khác nhau nhưng thuật toán của Cooley-Turkey (dựa trên nguyên lý chia để trị và kỹ thuật đệ quy) là nổi tiếng nhất. Các thuật toán này đều có độ phức tạp là $O(N^2 \log N)$ với $N \times N$ là kích thước của ảnh. FFT làm việc tốt và

nhanh nhất đối với các trường hợp mà $N=2^K$. Chặn dưới của biến đổi DFT đã được chứng minh là $O(N)$ (tác giả Winograd, năm 1978). Trong phạm vi của tài liệu này chúng ta không bàn tới chi tiết của các thuật toán FFT nhưng cần biết rằng thực tế cài đặt của các hàm DFT là FFT.

Các thuật toán FFT được cài đặt sẵn trong các công cụ và môi trường, thư viện hỗ trợ xử lý ảnh như Matlab (dùng thư viện FFTW, các hàm `fft2` và `ifft2`), OpenCV (hàm `dft` và `idft`), thư viện MKL của hãng Intel hoặc CUDA của Nvidia.

Biến đổi Fourier không phải là phương pháp xử lý ảnh duy nhất trong miền tần số, ngoài biến đổi Fourier còn có một số kỹ thuật khác như biến đổi Cosin rời rạc (Discrete Cosine Transform – DCT), các kỹ thuật dựa trên biến đổi của các sóng nhỏ (wavelets based transforms) v.v. Biến đổi Fourier cũng có một số nhược điểm như dựa trên các sóng sin/cosin vô hạn, ngoài ra nó cũng không hỗ trợ các biểu diễn multiscale/multiresolution, các yếu điểm này có thể khắc phục với các biến đổi wavelets.

Các ứng dụng quan trọng của biến đổi Fourier gồm có: là biểu diễn của ảnh trong miền tần số để làm cơ sở cho các biến đổi phức tạp hơn (như biến đổi wavelets, monogenic), cho phép thực hiện phép nhân chập với độ phức tạp thấp hơn trong miền tần số (đặc biệt là khi các bộ lọc trong miền không gian có kích thước lớn) và các phép lọc tăng cường chất lượng ảnh. Trong phạm vi của tài liệu này, chúng ta sẽ xem xét chủ yếu các bộ lọc trong miền tần số dựa vào biến đổi Fourier rời rạc hai chiều.

2. Các phép lọc ảnh trong miền tần số

2.1. Lọc ảnh trong miền tần số

Trong rất nhiều trường hợp các bài toán xử lý ảnh sẽ tìm được lời giải tốt nhất trong miền tần số thay vì miền không gian vì các lý do sau:

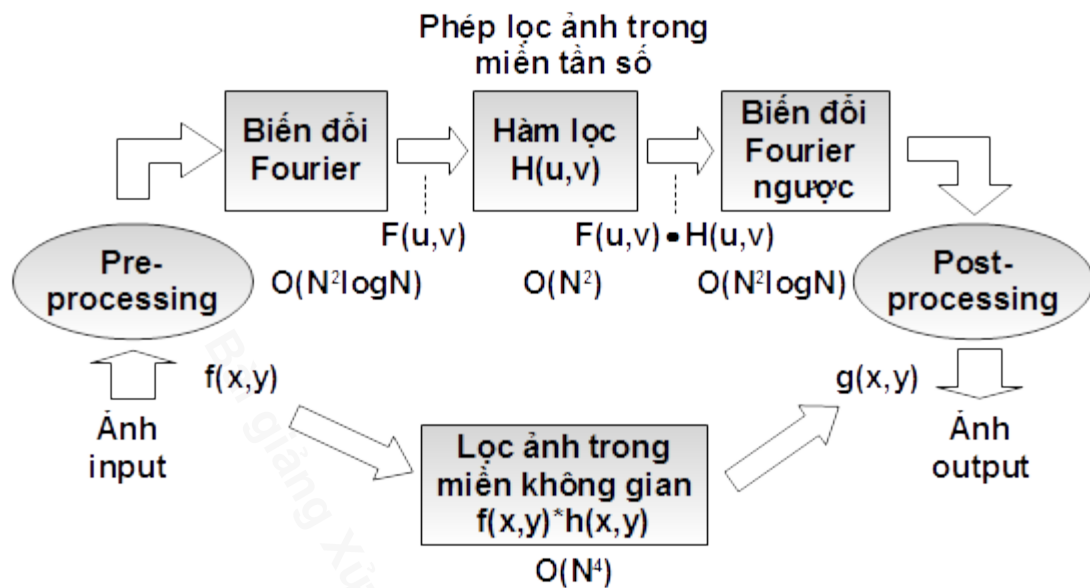
- + Miền tần số biểu diễn tốc độ thay đổi của tín hiệu, các tần số này có thể gắn với các mẫu về thay đổi cường độ trong ảnh để có thể trích xuất và xử lý các thông tin khác nhau của ảnh.

- + Trong miền tần số, việc loại bỏ các tần số không mong muốn (nhiều) là dễ dàng hơn.

- + Một số thao tác với ảnh thực hiện trong miền tần số sẽ nhanh hơn trong miền không gian. Ví dụ như thao tác áp dụng bộ lọc tương đương trong miền không gian sẽ liên quan tới phép nhân chập (convolution), một thao tác đòi hỏi số lượng tính toán lớn (xem hình 4.17) có thể được thực hiện trong miền tần số với số thao tác ít hơn nhiều.

- + Một số thao tác sẽ là rất khó và hầu như không thể thực hiện được một cách trực tiếp trong miền không gian ví dụ như làm sắc nét một ảnh mờ gây ra bởi chuyển động.

Phép lọc trong miền không/thời gian với bộ lọc $h(x,y)$ qua phép nhân chập giữa $f(x,y)$ với $h(x,y)$ tương đương với phép lọc trong miền tần số giữa $F(u,v)$ với bộ lọc $H(u,v)$ qua phép nhân từng phần tử: $g(x,y) = f(x,y) * h(x,y) \Leftrightarrow F(u,v) \bullet H(u,v) = G(u,v)$ trong đó F và H là DFT của hàm f và h tương ứng. Dựa vào tính chất này, mô hình của các phép lọc ảnh dựa vào biến đổi DFT trong miền tần số sẽ gồm các bước như sau:

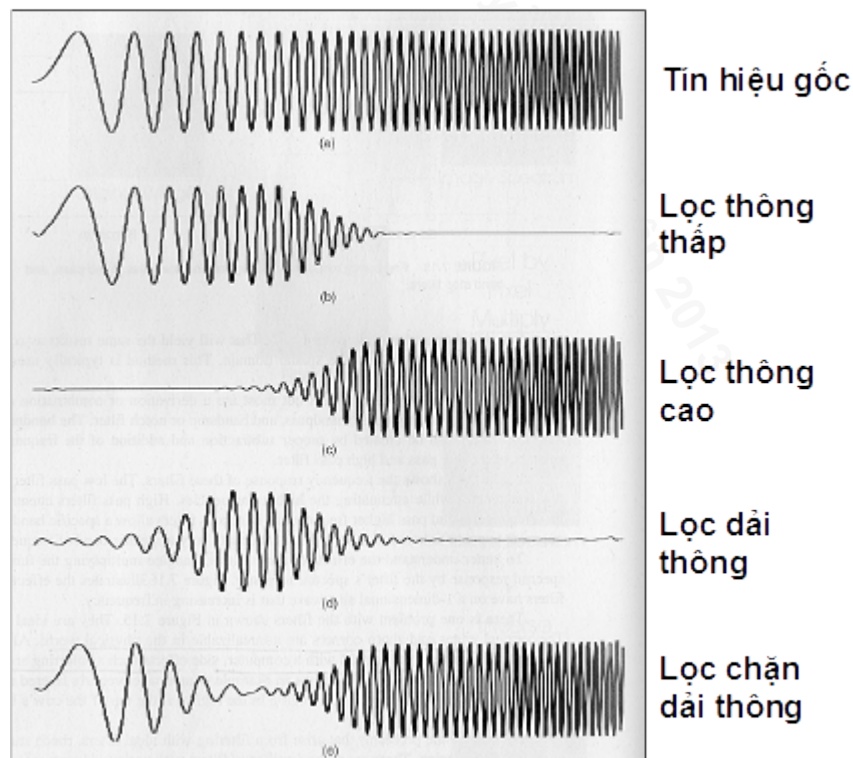


Hình 4.17: Các bước của phép lọc ảnh trong miền tần số

Mô hình trên có 3 bước chính là:

- + Tính $F(u,v)$ là biến đổi Fourier của ảnh input $f(x,y)$
- + Nhân $F(u,v)$ với bộ lọc $H(u,v)$
- + Thực hiện phép biến đổi Fourier ngược để nhận được ảnh kết quả $g(x,y)$.

Trong phần tiếp theo chúng ta sẽ xem xét 4 kỹ thuật lọc trong miền tần số là lọc thông thấp, lọc thông cao, lọc dải thông và lọc chặn dải thông (xem sự khác nhau giữa các phép lọc trong hình minh họa 4.18 bên dưới).



Hình 4.18: Các phép lọc trong miền tần số

Có hai trường hợp cần xét khi thực hiện một phép lọc trong miền tần số theo mô hình 4.17: $h(x,y)$ được cho trong miền không gian và $H(u,v)$ được cho trong miền tần số. Ta sẽ xem xét các bước đối với trường hợp $H(u,v)$ được cho trong miền tần số trước. Thuật toán gồm các bước sau:

Input: ảnh $f(x, y)$ có kích thước $M \times N$ và $H(u, v)$ là bộ lọc trong miền tần số

Output: ảnh $g(x, y)$ với chất lượng ảnh đã được tăng cường

Bước 1: Chọn $P = 2M, Q = 2N$

Bước 2: Sinh một ảnh $f_p(x,y)$ (gọi là padded image) có kích thước $P \times Q$ bằng cách thêm các số 0 vào ảnh gốc $f(x,y)$: $f_p(x, y) \leftarrow f(x, y) \text{ padded } 0s$

Bước 3: Nhân $f_p(x,y)$ với $(-1)^{x+y}$ để đảm bảo tính đối xứng trong DFT ở bước sau

Bước 4: Tính $F(u,v)$ là kết quả của biến đổi DFT với ảnh kết quả ở bước 3:

$$F(u,v) = R(u,v) + jI(u,v)$$

Bước 5: Sinh một bộ lọc đối xứng $H(u,v)$ trong miền tần số có kích thước $P \times Q$, đối xứng tại điểm trung tâm $(P/2, Q/2)$. Tính tích $G(u,v)$:

$$G(u,v) = H(u,v)F(u,v) = H(u,v)R(u,v) + jH(u,v)I(u,v)$$

Bước 6: Lấy kết quả từ thành phần thực (real part) của $G(u,v)$ theo công thức:

$$g_p(x,y) = \{real[IDFT[G(u,v)]]\}(-1)^{x+y}$$

Bước 7: Lấy kết quả cuối cùng là ảnh $g(x,y)$ bằng cách tách phần phía trên, bên trái có kích thước $M \times N$ của ảnh $g_p(x,y)$.

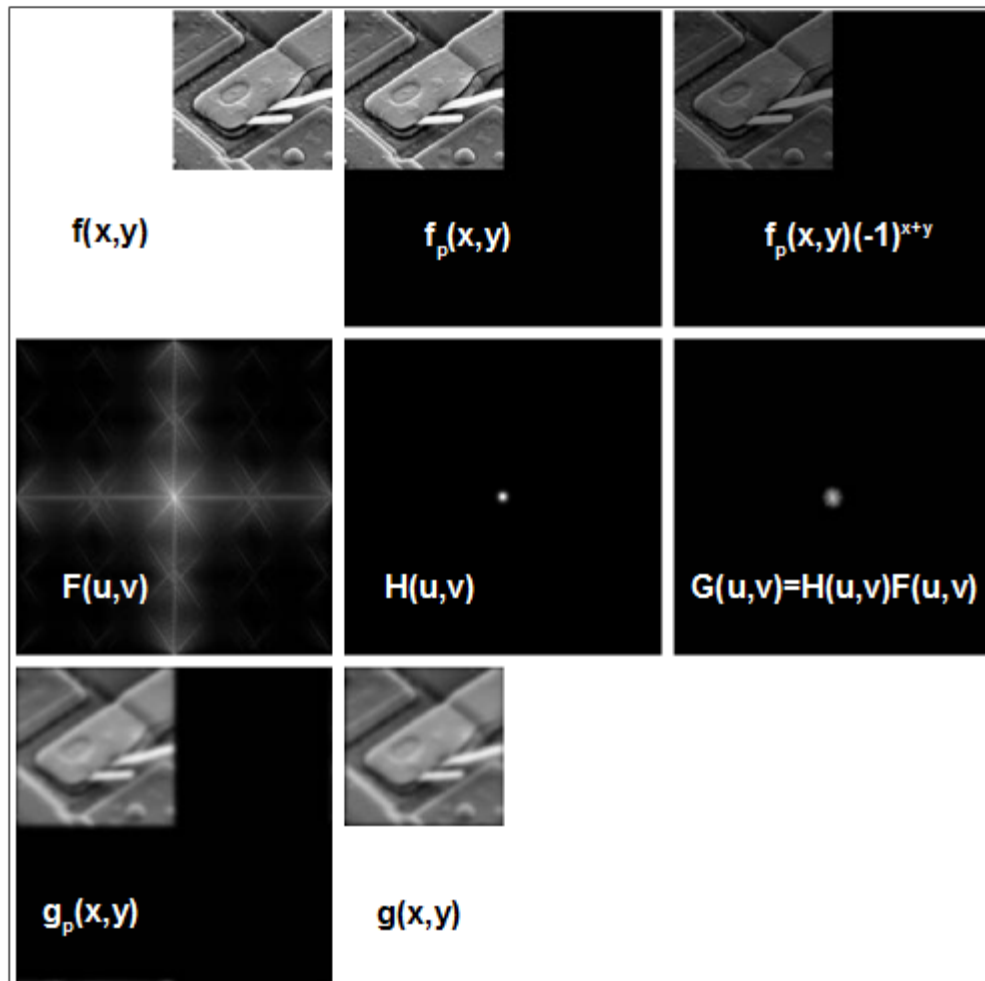
Có thể xem kết hoạt động của các bước trên qua hình minh họa 4.19 bên dưới.

Khi bộ lọc $h(x, y)$ được cho trong miền không gian, một thuật toán sẽ được dùng để sinh $H(u, v)$ từ $h(x,y)$ như sau:

Bước 1: Sinh $h_p(x, y)$ từ $h(x, y)$ bằng cách thêm các số 0 (padding step).

Bước 2: Nhân $h_p(x, y)$ với $(-1)^{(x+y)}$.

Bước 3: Tính DFT của kết quả thu được từ bước 2.



Hình 4.19: Các bước của phép lọc ảnh trong miền tần số

2.2. Làm mượt ảnh bằng phép lọc thông thấp (Lowpass filter)

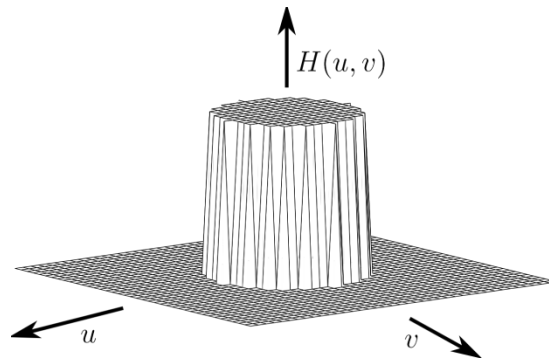
Kỹ thuật lọc thông thấp sẽ giới hạn dữ liệu ảnh chỉ còn các thành phần có tần số thấp bằng cách làm suy yếu các thành phần có tần số cao và giữ nguyên đối với các thành phần ở tần số thấp (cho qua). Lọc thông thấp là phép lọc quan trọng và phổ biến nhất trong các kỹ thuật lọc ảnh dựa vào DFT. Kết quả trong miền không gian của phép lọc thông thấp tương đương với việc làm mượt ảnh (smoothing filter) và loại bỏ các tần số cao tương ứng với các thay đổi lớn về cường độ, hoặc các nhiễu. Vì vậy bộ lọc thông thấp hay được sử dụng để làm mượt (smooth) hoặc làm mờ (blur) ảnh.

2.2.1. Bộ lọc thông thấp lý tưởng (Ideal Lowpass Filter – ILPF)

Trong bộ lọc ILPF, hàm lọc $H(u,v)$ được tính như sau:

$$H_{LP}(u,v) = \begin{cases} 1 & \text{nếu } D(u,v) \leq D_0 \\ 0 & \text{nếu } D(u,v) > D_0 \end{cases} \text{ trong đó } D(u,v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$$

D_0 được gọi là giá trị cắt tần số (cut-off frequency) hoặc bán kính cắt tần số (cut-off radius). Đối với các bộ lọc thông thấp, giá trị này càng giảm thì các tần số cao sẽ bị cắt bỏ càng nhiều.

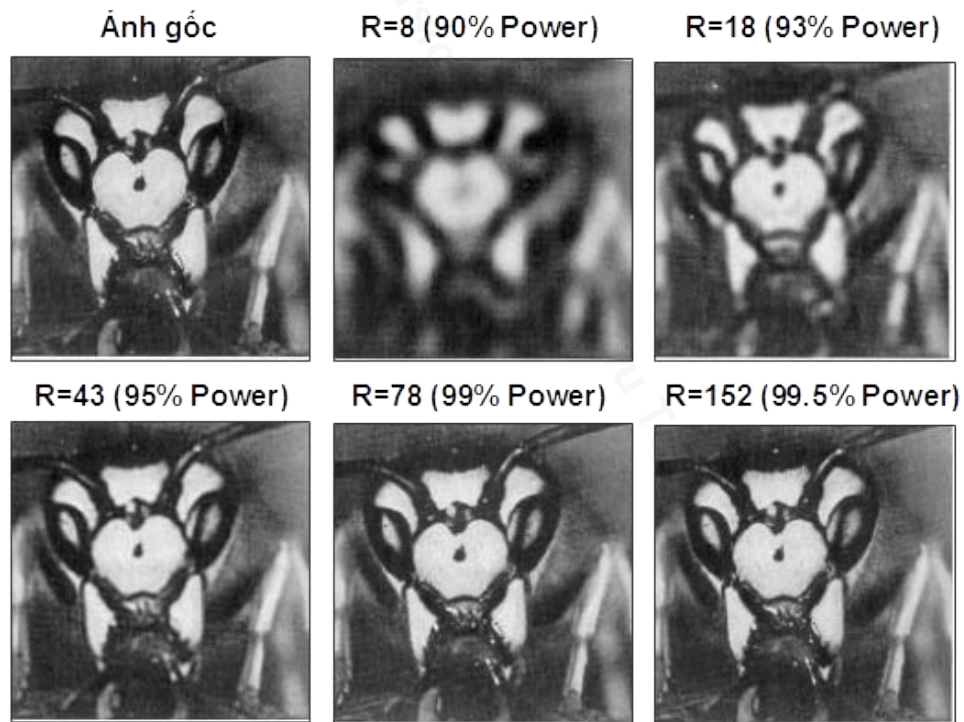


Hình 4.20: Tần số bị loại bỏ trong lọc thông thấp ILPF

Do các tần số bị loại bỏ nằm trên một hình tròn (hình 4.20) nên mỗi bộ lọc thông thấp sẽ tương ứng với một giá trị là bán kính của đường tròn có tâm tại điểm $(M/2, N/2)$ trong miền tần số. Giá trị bán kính này được chọn bằng cách lấy tương ứng với phần trăm của tổng năng lượng (Power/Total Power) bao gói trong toàn bộ hình tròn tính bởi các công thức:

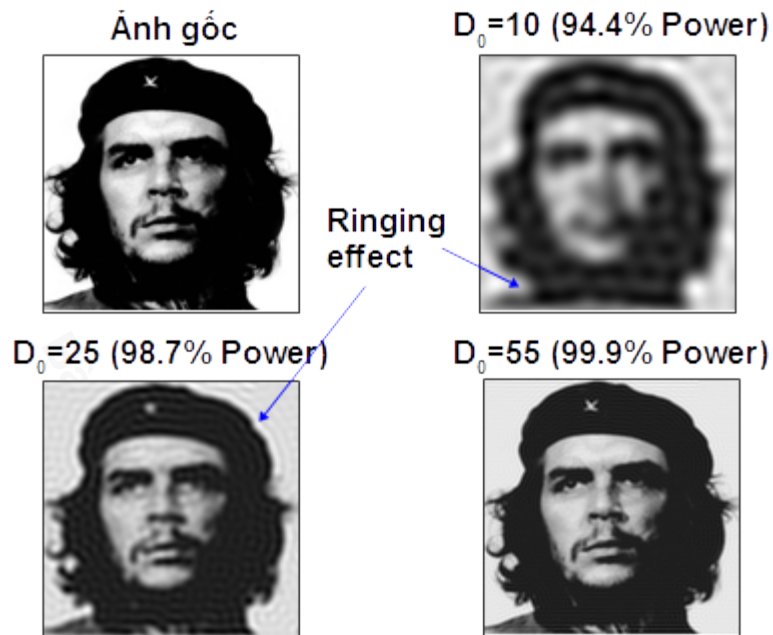
$$\text{Power: } P(u, v) = |F(u, v)|^2 = R^2(u, v) + I^2(u, v)$$

$$\text{Total Power: } P_T(u, v) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} P(u, v)$$

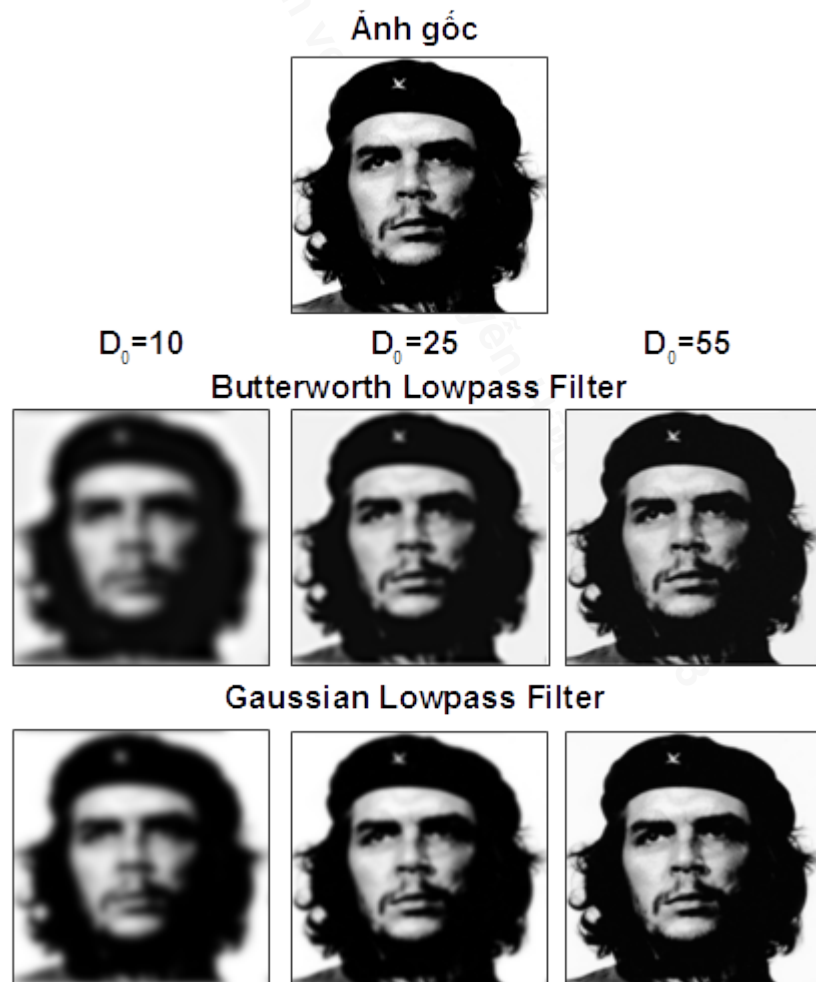


Hình 4.21: Bán kính năng lượng trong bộ lọc thông thấp ILPF

Có thể thấy (trên hình 4.21) rằng hầu hết năng lượng tập trung xung quanh tâm của phổ. Đồng thời ta cũng thấy rõ tác dụng của hằng số D_0 trong hai ảnh có $R=8$ và $R=78$: D_0 kiểm soát mức độ mờ của ảnh trong quá trình lọc.



Hình 4.22: Hiện tượng Ringing effect



Hình 4.23: Lọc thông thấp Butterworth và Gaussian

Một nhược điểm của phép lọc ILPF là khi thực hiện có thể dẫn tới tình trạng sinh ra ảnh kết quả với các đặc điểm mà tần số của chúng gần với các tần số bị loại bỏ, hiện

tượng này gọi là **ringing effect** (xem minh họa trong hình 4.22). Nguyên nhân của hiện tượng này là do sự không liên tục của bộ lọc Ideal (xem hình 4.20)

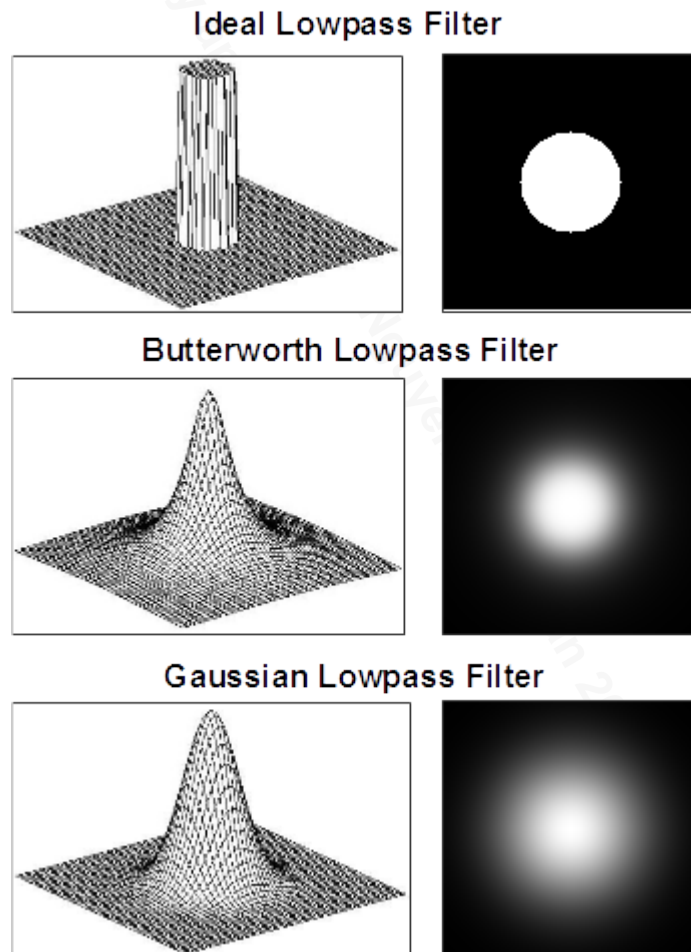
Để làm giảm hiện tượng này trên thực tế người ta sử dụng các bộ lọc làm suy yếu các tần số cao một cách trơn tru hơn. Chúng ta sẽ xem xét 2 loại bộ lọc thông thấp có thể khắc phục hiện tượng này trong các phần tiếp theo.

2.2.2. Bộ lọc thông thấp Butterworth (Butterworth Lowpass Filter – BLPF)

Trong bộ lọc ILPF, hàm lọc $H(u,v)$ được tính như sau:

$$H_{LP}(u,v) = \frac{1}{1 + \left[\frac{D(u,v)}{D_0} \right]^{2n}} = \frac{1}{1 + \left[\frac{(u - M/2)^2 + (v - N/2)^2}{D_0^2} \right]^n}$$

Trong đó n được gọi là bậc (order) của phép lọc, giá trị thường dùng của n là 2. Hàm lọc này khắc phục hiện tượng **ringing effect** bằng việc làm trơn quá trình chuyển đổi giữa các tần số cao và thấp. Có thể thấy rõ điều này khi so sánh kết quả thực hiện các phép lọc thông thấp Ideal với Butterworth, Gaussian trong hai hình 4.22 và 4.23.



Hình 4.24: Biểu diễn trực quan 2D và 3D của các bộ lọc thông thấp

2.2.3. Bộ lọc thông thấp Gaussian (Gaussian Lowpass Filter – GLPF)

Với bộ lọc Gaussian, hàm lọc $H(u,v)$ được cho bởi công thức:

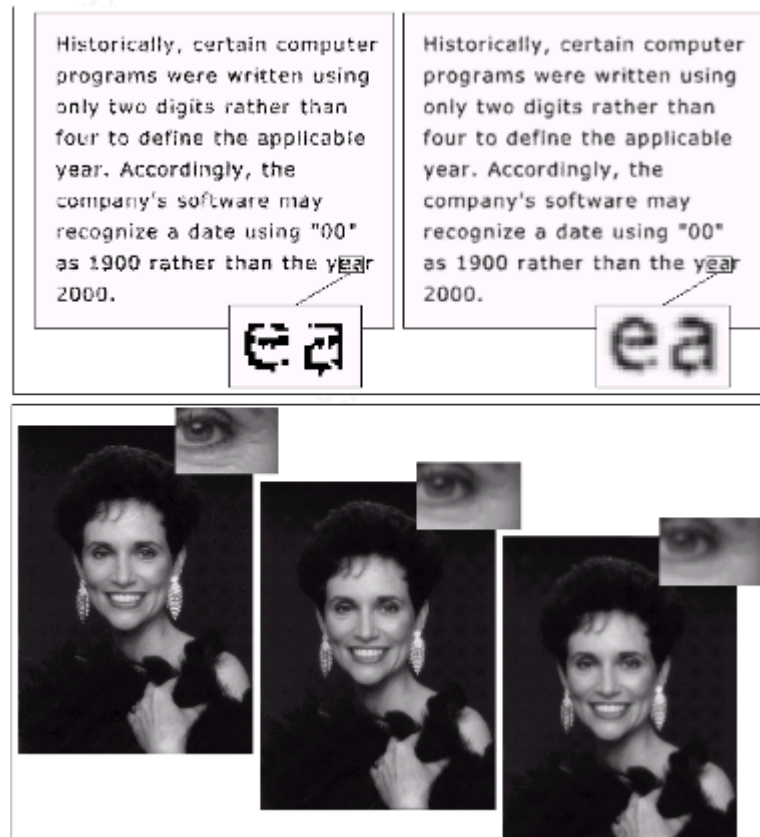
$$H_{LP}(u,v) = e^{-\frac{D(u,v)^2}{2\sigma^2}}, \text{ với } \sigma = D_0 \text{ ta có } H_{LP}(u,v) = e^{-\frac{D(u,v)^2}{2D_0^2}}$$

$$\text{trong đó } D^2(u, v) = (u - \frac{M}{2})^2 + (v - \frac{N}{2})^2$$

Với các giá trị bậc nhỏ, BLPF gần với GLPF, còn với giá trị bậc lớn, nó lại gần với ILPF hơn. So với BLPF, GLPF loại bỏ hiện tượng ringing effect triệt để hơn. Có thể xem sự khác biệt của ba bộ lọc thông thấp trong hình 4.24.

2.2.4. Ứng dụng của lọc thông thấp

Với tác dụng làm mượt ảnh, các bộ lọc thông thấp (chủ yếu là GLPF) có thể được dùng để khôi phục các nét đứt trong ảnh hoặc loại bỏ các chi tiết trong chỉnh sửa ảnh mặt người (xem hình minh họa bên dưới). Ngoài ra chúng còn có thể dùng để lọc nhiễu.



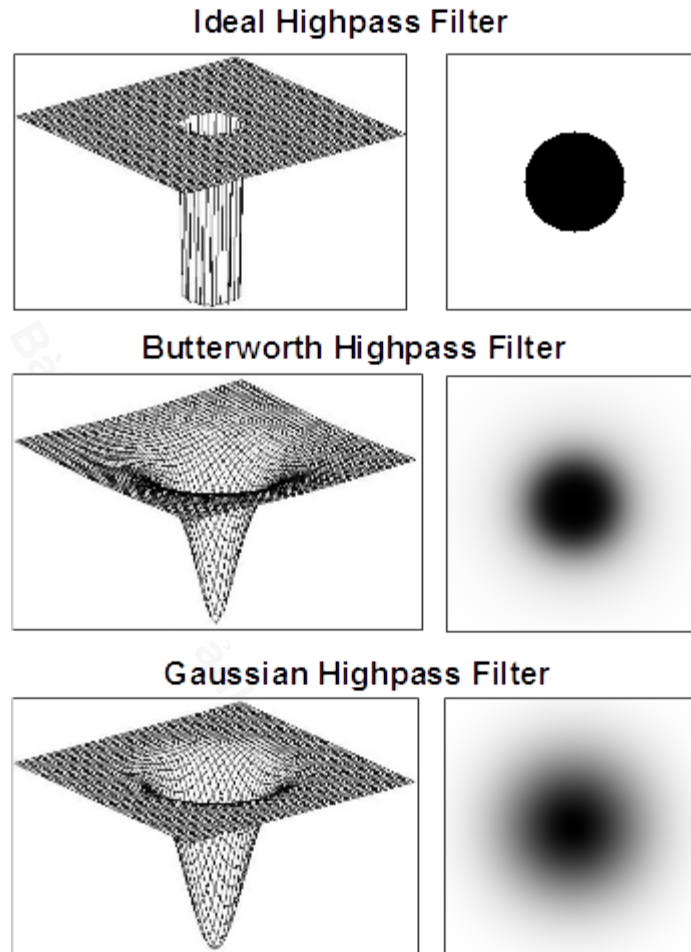
Hình 4.25: Ứng dụng thực tế của lọc thông thấp

2.3. Làm sắc nét ảnh bằng phép lọc thông cao (Highpass filter)

Ngược lại với lọc thông thấp, các bộ lọc thông cao làm suy yếu các tín hiệu tần số thấp và giữ nguyên các tín hiệu tần số cao. Các tần số cao tương ứng với các đường (edges) và các chi tiết (fine detail) của ảnh, vì vậy các bộ lọc thông cao thường được sử dụng để làm sắc nét các chi tiết ảnh, hoặc làm giảm độ mờ của ảnh. Một bộ lọc thông cao (Ideal, Butterworth, Gaussian) có thể tính được từ bộ lọc thông thấp tương ứng theo công thức:

$$H_{HP}(u, v) = 1 - H_{LP}(u, v)$$

Có thể thấy minh họa trực quan của công thức này khi so sánh các hình minh họa 4.24 và 4.26. Tương tự như lọc thông thấp, trong lọc thông cao cũng có hiện tượng ringing effect và phương pháp khắc phục cũng là dùng các bộ lọc Butterworth hoặc Gaussian (trong hình 4.27).



Hình 4.26: Biểu diễn trong miền không gian của các bộ lọc thông cao

2.3.1. Bộ lọc thông cao lý tưởng (Ideal Highpass Filter – IHPF)

$$H_{HP}(u, v) = \begin{cases} 0 & \text{nếu } D(u, v) \leq D_0 \\ 1 & \text{nếu } D(u, v) > D_0 \end{cases} \text{ trong đó } D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$$

2.3.2. Bộ lọc thông cao Butterworth (Butterworth Highpass Filter – BHPF)

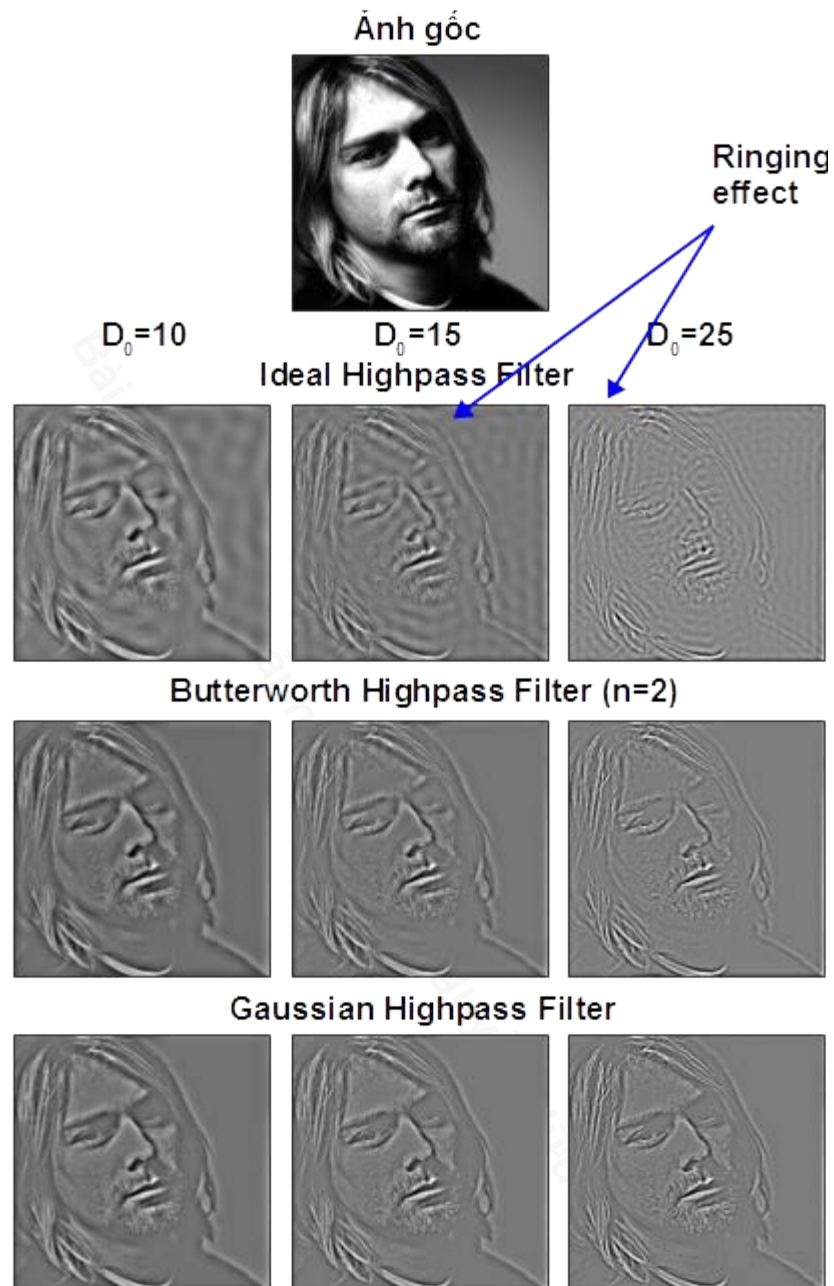
$$H_{HP}(u, v) = \frac{1}{1 + \left[\frac{D_0}{D(u, v)} \right]^{2n}} = \frac{1}{1 + \left[\frac{D_0^2}{(u - M/2)^2 + (v - N/2)^2} \right]^n}$$

2.3.3. Bộ lọc thông cao Gaussian (Gaussian Highpass Filter – GHPF)

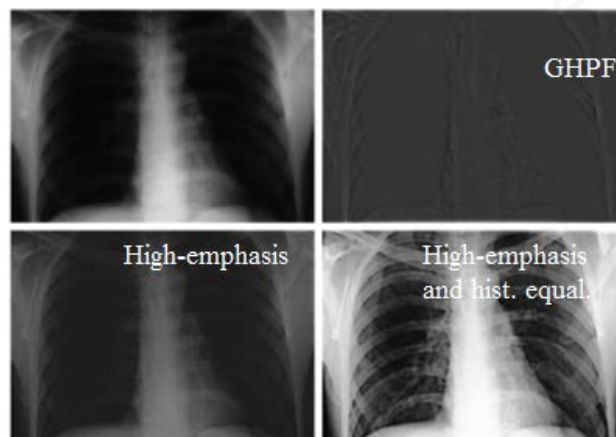
$$H_{HP}(u, v) = 1 - e^{-\frac{D(u, v)^2}{2D_0^2}}$$

Ngược lại với lọc thông thấp, trong lọc thông cao, giá trị D_0 càng giảm thì các tần số càng được giữ lại càng nhiều và ảnh càng gần với ảnh gốc trong miền không gian (xem hình minh họa 4.27).

Với chức năng làm sắc nét, lọc thông cao hay được sử dụng để làm nổi bật các chi tiết của ảnh, ví dụ như trong các ảnh chụp y học (chụp X-quang, hình 4.28).



Hình 4.27: Kết quả thực hiện các phép lọc thông cao

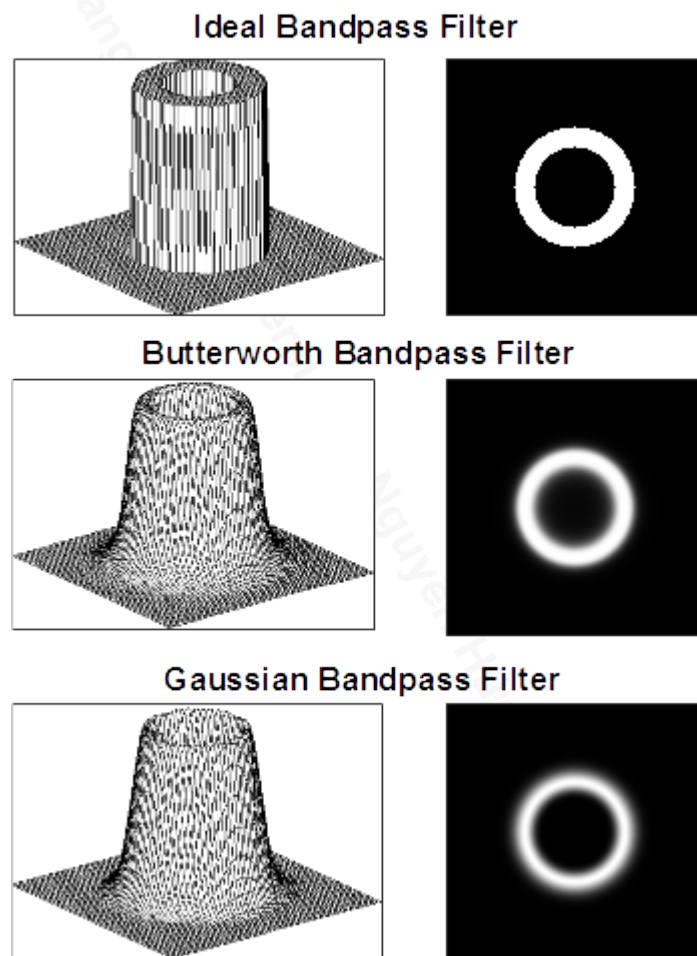


Hình 4.28: Một ứng dụng của lọc thông cao

2.4. Lọc dải thông (Bandpass filter)

Lọc thông thấp và lọc thông cao làm việc với toàn bộ dải tần số của mỗi ảnh. Tuy nhiên có những ứng dụng đòi hỏi việc chỉ xử lý một dải tần số nhất định. Để thực hiện điều này chúng ta có thể dùng bộ lọc dải thông (bandpass) hoặc bộ lọc chặn dải thông (bandstop hay bandreject filter).

Một bộ lọc dải thông sẽ chỉ giữ lại các tần số nằm trong khoảng cho phép, các tần số nằm ngoài dải này sẽ bị loại bỏ. Với nguyên tắc này, bộ lọc chặn dải thông cân bằng giữa hai yếu tố độ mờ (blur, gây ra do lọc thông thấp) và nhiễu trong ảnh (gây ra do lọc thông cao). Một bộ lọc dải thông có thể được tạo thành bằng cách kết hợp (hợp nhất) một bộ lọc thấp và một bộ lọc cao với các tham số phù hợp.



Hình 4.29: Biểu diễn các bộ lọc dải thông

2.4.1. Bộ lọc dải thông lý tưởng (Ideal Bandpass Filter – IBPF)

$$H_{BP}(u, v) = \begin{cases} 1 & \text{nếu } D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 0 & \text{nếu ngược lại} \end{cases}$$

Trong đó D_0 là cut-off frequency và W là độ rộng của dải thông.

2.4.2. Bộ lọc dải thông Butterworth (Butterworth Bandpass Filter – BBPF)

$$H_{BP}(u, v) = \frac{1}{1 + \left[\frac{D^2 - D_0^2}{DW} \right]^{2n}}$$

2.4.3. Bộ lọc dải thông Gaussian (Gaussian Bandpass Filter – GBPF)

$$H_{BP}(u, v) = e^{-[\frac{D^2 - D_0^2}{DW}]^2}$$

Ảnh gốc



$D_0=20, W=20$

$D_0=30, W=20$

$D_0=50, W=30$

Ideal Bandpass Filter



Butterworth Bandpass Filter (n=2)



Gaussian Bandpass Filter



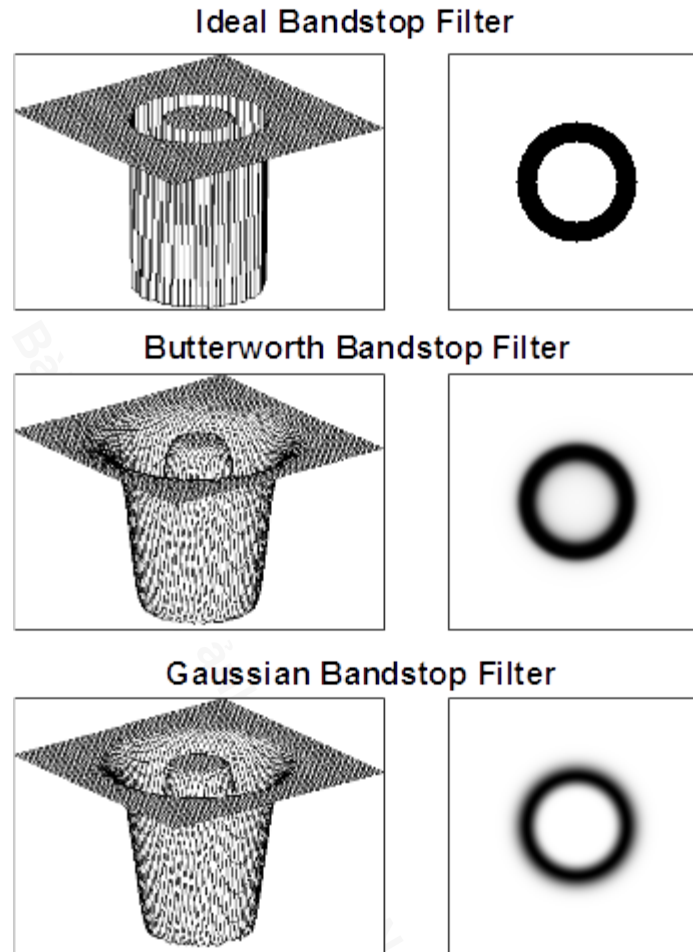
Hình 4.30: Kết quả thực hiện phép lọc dải thông

2.5. Lọc chặn dải thông (Band stop filter)

Ngược lại với bộ lọc dải thông, bộ lọc chặn dải thông sẽ không cho qua các tần số nằm trong một dải nhất định, do đó với cùng các tham số, bộ lọc chặn dải thông sẽ cho bởi công thức:

$$H_{BS}(u, v) = 1 - H_{BP}(u, v)$$

Có thể so sánh hai hình 4.29 và 4.31 để thấy sự tương phản của hai loại bộ lọc làm việc theo dải thông.



Hình 4.31: Biểu diễn các bộ lọc chặn dải thông

2.5.1. Bộ lọc chặn dải thông lý tưởng (Ideal Band Stop Filter – IBSF)

$$H_{BS}(u, v) = \begin{cases} 0 & \text{nếu } D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 1 & \text{nếu ngược lại} \end{cases}$$

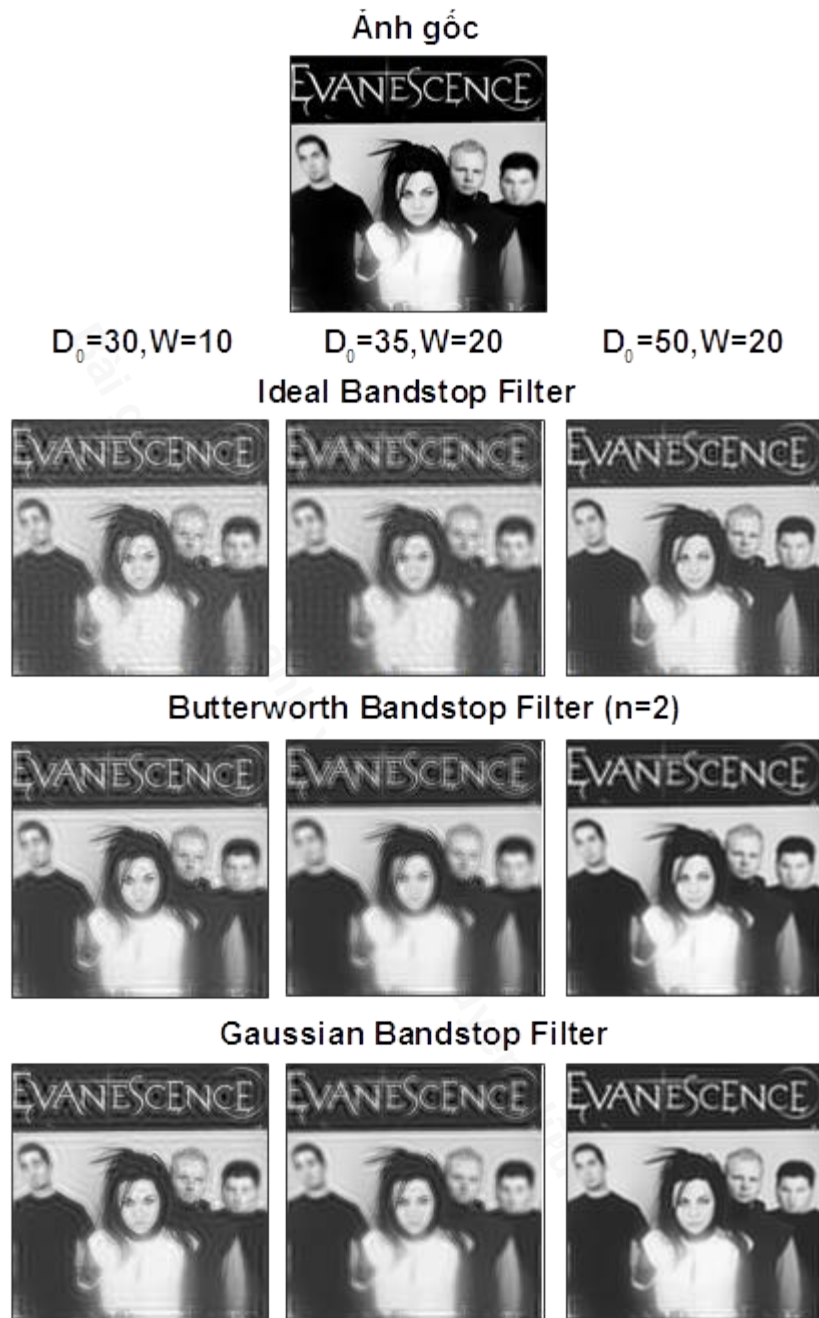
2.5.2. Bộ lọc chặn dải thông Butterworth (Butterworth Band Stop Filter – BBSF)

$$H_{BS}(u, v) = \frac{1}{1 + \left[\frac{DW}{D^2 - D_0^2} \right]^{2n}}$$

2.5.3. Bộ lọc chặn dải thông Gaussian (Gaussian Band Stop Filter – GBSF)

$$H_{BS}(u, v) = 1 - e^{-\left[\frac{D^2 - D_0^2}{DW} \right]^2}$$

Sau đây là hình minh họa kết quả thực hiện lọc chặn dải thông:



Hình 4.32: Phép lọc chặn dải thông

2.6. Toán tử Laplacian trong miền tần số

Giả sử $f(x)$ là một hàm liên tục một chiều và f có đạo hàm tới bậc n , ta có công thức biểu diễn biến đổi Fourier của đạo hàm bậc n của f như sau:

$$DFT\left(\frac{\partial^n f}{\partial x^n}\right) = (2\pi ju)^n F(u)$$

Chứng minh: theo biến đổi Fourier ngược ta có:

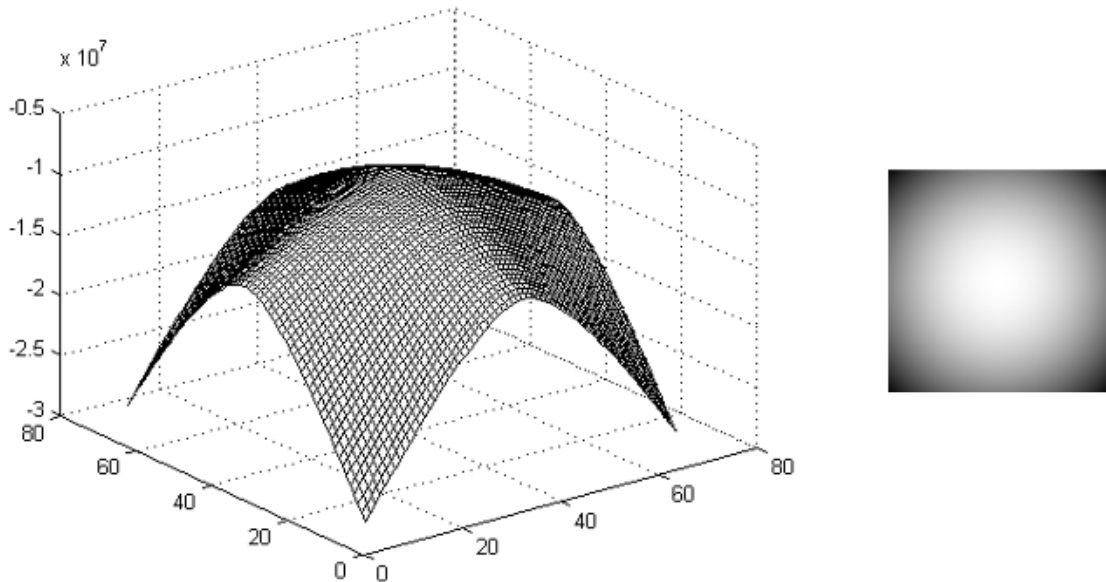
$$f(x) = \int_{-\infty}^{\infty} F(u) e^{2\pi jux} du$$

Lấy đạo hàm bậc nhất hai vế của đẳng thức trên ta được:

$$\frac{\partial f}{\partial x} = \int_{-\infty}^{\infty} [F(u)(2\pi ju)] e^{2\pi jux} du$$

Vì thế theo tính chất duy nhất của biến đổi Fourier ngược ta sẽ có:

$$DFT\left(\frac{\partial f}{\partial x}\right) = (2\pi ju)F(u)du$$



Biểu diễn trong miền không gian của toán tử Laplacian



Ảnh Input

Ảnh output theo biến đổi Laplacian

Hình 4.33: Toán tử Laplacian trong miền tần số

Lặp lại việc lấy đạo hàm hai vế của biểu thức trên (n-1) lần nữa ta sẽ có điều phải chứng minh.

Khi đó theo công thức tính toán tử Laplacian ở chương 3 ta có:

$$DFT(\nabla^2 f) = DFT\left(\frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}\right) = DFT\left(\frac{\partial^2 f}{\partial x^2}\right) + DFT\left(\frac{\partial^2 f}{\partial y^2}\right) = -4\pi^2(u^2 + v^2)F(u, v)$$

Do đó công thức bộ lọc để tính toán tử Laplacian trong miền tần số sẽ là:

$$H(u, v) = -4\pi^2(u^2 + v^2)$$

Xét với tâm của miền tần số (M/2, N/2) ta sẽ có công thức:

$$H(u, v) = -4\pi^2 \left[\left(u - \frac{M}{2}\right)^2 + \left(v - \frac{N}{2}\right)^2 \right] = -4\pi^2 D^2(u, v)$$

Ảnh output thực hiện lọc bằng toán tử Laplacian thu được sẽ là:

$$g(x, y) = f(x, y) - IDFT[F(u, v) \bullet H(u, v)] = IDFT[F(u, v) \bullet (1 - 4\pi^2 D^2(u, v))]$$

Như vậy ta có thể thay việc lấy ảnh input $f(x, y)$ trừ đi kết quả biến đổi ngược IDFT của phép nhân từng phần tử giữa $F(u, v)$ với $H(u, v)$ bằng bộ lọc $(1 - H(u, v))$. Tuy nhiên trên thực tế do kết quả của $IDFT[F(u, v) \bullet H(u, v)]$ có thể âm trong khi $f(x, y)$ luôn không âm nên người ta sẽ thực hiện chuẩn hóa các giá trị của ảnh $f(x, y)$ về khoảng $[0, 1]$ trước khi lọc và kết quả của $IDFT[F(u, v) \bullet H(u, v)]$ sẽ được chuẩn hóa về khoảng $[-1, 1]$ trước khi bị trừ bởi $f(x, y)$ (đã chuẩn hóa) để có kết quả cuối cùng.

Biểu diễn của toán tử Laplacian và kết quả lọc ảnh được minh họa trong hình 4.33. Tương tự như trong miền tần số, việc áp dụng lọc với toán tử Laplacian cho ta một phiên bản sắc nét của ảnh với các chi tiết được làm rõ hơn.

2.7. Lọc đồng hình (Homomorphic filter)

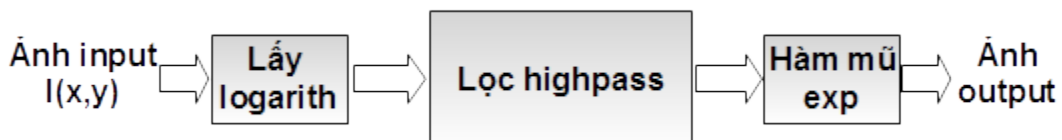
Lọc đồng hình là phép lọc trong miền tần số cho phép loại bỏ các nhiễu nhân tính theo mô hình thu nhận ảnh: $I(x, y) = L(x, y)\eta(x, y)$ trong đó $I(x, y)$ là ảnh thu nhận được, $L(x, y)$ là ảnh gốc không nhiễu và $\eta(x, y)$ là nhiễu. Cụ thể hơn lọc đồng hình thường được dùng để xử lý ánh sáng không nhất quán tác động lên ảnh theo mô hình phản chiếu ánh sáng:

$$I(x, y) = L(x, y)R(x, y)$$

Trong đó $L(x, y)$ là ánh sáng, còn $R(x, y)$ là phản chiếu của các đối tượng trong cảnh chụp lên ảnh. Thông thường thì thành phần ánh sáng $L(x, y)$ thay đổi rất chậm và chỉ ảnh hưởng tới các chi tiết có tần số thấp của ảnh. Ngược lại thành phần $R(x, y)$ thay đổi nhanh và ảnh hưởng hầu hết tới các thành phần có tần số cao. Ý tưởng của phép lọc đồng hình là tách các thành phần tần số thấp khỏi các thành phần có tần số cao sau đó áp dụng bộ lọc tần số cao để loại bỏ các thành phần tần số thấp trong ảnh, chỉ giữ lại các thành phần có tần số cao. Tuy nhiên việc xử lý riêng rẽ hai thành phần này theo mô hình nhân tính một cách đồng thời là khó, vì thế trước hết cần tách chúng bằng một phép lấy logarithm:

$$\ln[I(x, y)] = \ln[L(x, y)R(x, y)] = \ln[L(x, y)] + \ln[R(x, y)]$$

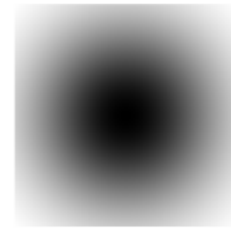
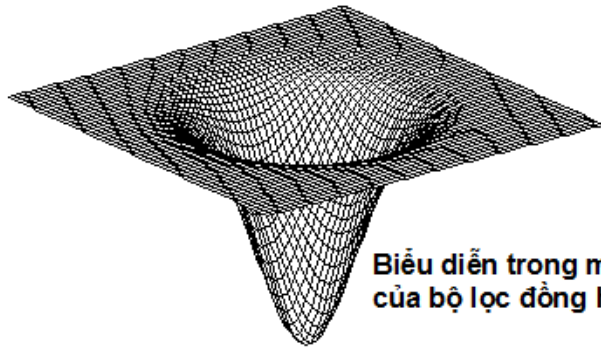
Tiếp đến bộ lọc thông cao sẽ được áp dụng, kết quả cuối cùng sẽ nhận được sau phép lấy hàm mũ của trên kết quả phép lọc thông cao (vì trước đó ta đã thực hiện lấy logarithm). Mô hình có phép lọc đồng hình được minh họa qua hình sau:



Hình 4.34: Các bước của phép lọc đồng hình

Bộ lọc đồng hình Gaussian cho bởi công thức:

$$H(u, v) = (\gamma_H - \gamma_L)H_{HP, Gaussian} + \gamma_L = (\gamma_H - \gamma_L) \left[1 - e^{-c \left[\frac{D^2(u, v)}{D_0^2} \right]} \right] + \gamma_L$$



Biểu diễn trong miền không gian của bộ lọc đồng hình Gaussian

Ảnh Input



$$\gamma_L = 0.9$$

$$\gamma_H = 1.5$$

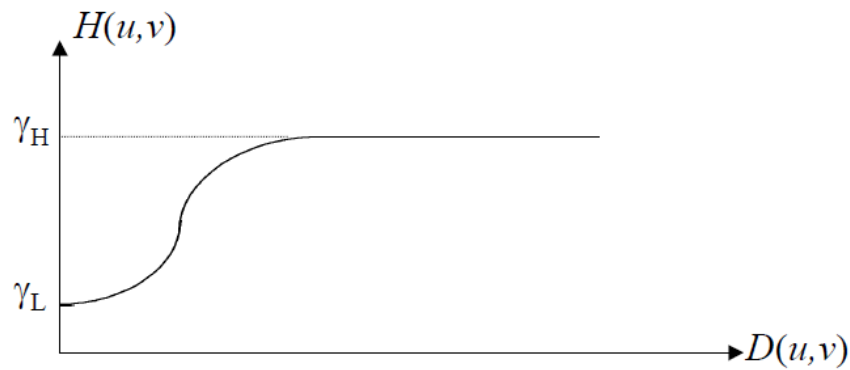
$$D_0 = 55$$

Ảnh Output



Hình 4.35: Kết quả phép lọc đồng hình Gaussian

Tham số $\gamma_H (>1)$ là chặn trên loại bỏ các thành phần tần số thấp trong khi $\gamma_L (<1)$ cho phép giữ lại các thành phần có tần số cao, có thể hình dung như sau:



Hình 4.36: Loại bỏ tần số trong lọc đồng hình

Hình sau 4.35 cho chúng ta hình dung về kết quả của phép lọc đồng hình Gaussian với $c=1$. Hoàn toàn tương tự ta có thể có các phép lọc đồng hình Ideal, Butterworth. Trên thực tế phép lọc đồng hình được dùng như một kỹ thuật tăng độ tương phản đồng thời chuẩn hóa độ sáng (*toàn cục*) của ảnh.

Lưu ý về cài đặt phép lọc đồng hình và các phép lọc ảnh trong miền tần số:

Hàm thực hiện IDFT OpenCV đều làm việc với các giá trị tham số mặc định với các giá trị output sẽ không được chuẩn hóa (chia cho kích thước của ảnh – MN) và do đó có thể sinh ra các giá trị rất lớn, cần tham khảo chi tiết với các version cụ thể để thực hiện điều này. Đối với phép lọc đồng hình khi thực hiện lấy logarithm cơ số tự nhiên của ảnh ta thực hiện với giá trị $1+I(x,y)$ vì có thể có điểm ảnh với mức xám bằng 0 trong ảnh làm cho kết quả của việc lấy logarithm bị sai.

Bài tập

Bài tập 1: Viết chương trình mở một file ảnh, tính và hiển thị các thành phần magnitude và phase trong biểu diễn DFT của ảnh.

Bài tập 2: Viết chương trình mở 2 ảnh, sử dụng thành phần phase của ảnh này với thành phần magnitude của ảnh kia để khôi phục lại ảnh gốc.

Bài tập 3: Viết chương trình mở một file ảnh và thực hiện các phép lọc:

- + Lọc thông thấp
- + Lọc thông cao
- + Lọc dải thông
- + Lọc chặn dải thông
- + Lọc bằng toán tử Laplacian
- + Lọc đồng hình

CHƯƠNG V: KHÔI PHỤC ẢNH

1. Khái niệm cơ bản

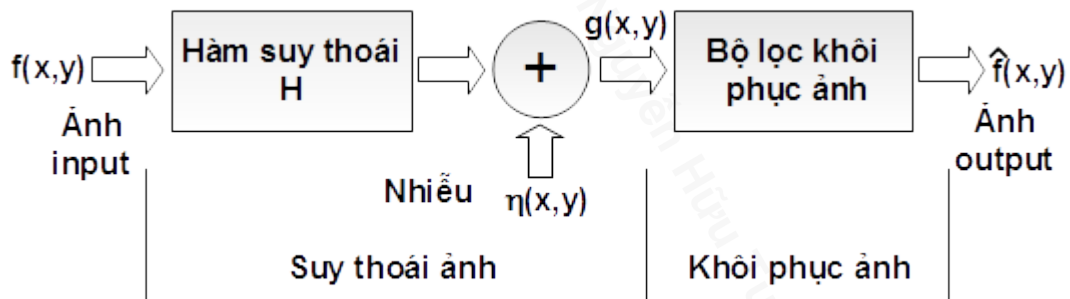
1.1. Phân biệt khôi phục ảnh và tăng cường chất lượng ảnh

Khôi phục ảnh (*Image restoration*) bao gồm các kỹ thuật nhằm mục đích khôi phục một ảnh bị suy thoái (degraded) hoặc méo mó (distorted) về nội dung và chất lượng ban đầu của nó (là tốt). Điều này nghe có vẻ giống với tăng cường chất lượng ảnh (mà ta đã xét trong 3 chương trước của tài liệu này) nhưng trên thực tế vẫn có những khác biệt cơ bản sau:

- + Khôi phục ảnh dựa trên giả thuyết là một mô hình suy thoái (degradation model) là biết rõ hoặc có thể ước lượng được (can be estimated).
- + Nội dung và chất lượng ban đầu không đồng nghĩa với việc có một hình ảnh đẹp.
- + Tăng cường chất lượng ảnh thường chủ yếu dựa vào kinh nghiệm trong khi khôi phục ảnh đòi hỏi một tri thức biết trước về quá trình suy thoái.
- + Tăng cường ảnh nhằm mục đích tạo ra một bức ảnh kết quả phù hợp hơn cho một tác vụ cụ thể nào đó trong khi khôi phục ảnh nhằm đưa nó về trạng thái nguyên bản ban đầu. Như vậy so với tăng cường chất lượng ảnh thì khôi phục ảnh nhằm nhiều mục đích hơn.

1.2. Mô hình suy thoái chất lượng ảnh (degradation model)

Quá trình suy thoái chất lượng ảnh có thể được mô hình hóa qua một hàm suy thoái H và nhiễu cộng tính $\eta(x,y)$ như sau:



Hình 5.1: Mô hình suy thoái ảnh

Với mô hình này, biểu diễn hàm tương ứng trong miền không gian và miền tần số sẽ là:

$$g(x,y) = h[f(x,y)] + \eta(x,y) = h(x,y) * f(x,y) + \eta(x,y)$$

$$G(x,y) = H[F(x,y)] + N(x,y) = H(x,y)F(x,y) + N(x,y)$$

Có hai cách tiếp cận để thực hiện khôi phục ảnh là cách tiếp cận dựa trên các mô hình nhiễu và cách tiếp cận dựa trên độ mờ của ảnh (blur). Đối với cách tiếp cận thứ nhất, các bộ lọc ảnh sẽ được sử dụng cho các loại mô hình nhiễu tương ứng. Trong cách tiếp cận thứ hai, một ước lượng về độ suy thoái sẽ được thực hiện trước khi áp dụng một bộ lọc ngược để khôi phục ảnh. Trong phần tiếp theo chúng ta sẽ xem xét các thuật toán đánh giá hiệu năng của việc khôi phục ảnh và các phương pháp sử dụng phép lọc nhiễu để khôi phục ảnh theo cách tiếp cận thứ nhất. Các thuật toán khôi phục dựa trên các mô

hình về độ mờ và một số kỹ thuật tiên tiến khác (super resolution và sparse representation) cũng được giới thiệu.

1.3. Đánh giá hiệu năng

Để đánh giá hiệu năng của các thuật toán khôi phục ảnh khác nhau cần có một công cụ để đo đếm ảnh thu được của các thuật toán với ảnh input $f(x,y)$. Hai hàm thường được sử dụng cho mục đích này là hàm Mean Square Error(MSE) và Signal to Noise Ratio (SNR) cho bởi các công thức sau:

$$MSE = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x,y) - \hat{f}(x,y)]^2$$
$$SNR = \frac{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \hat{f}(x,y)^2}{\sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x,y) - \hat{f}(x,y)]^2}$$

Giá trị MSE cho biết lỗi bình phương trung bình nhận được khi thực hiện khi khôi phục ảnh, nên thuật toán có giá trị MSE nhỏ hơn sẽ tốt hơn. Giá trị SNR cho biết tỉ lệ tín hiệu trên nhiễu thu được trong ảnh kết quả $g(x,y)$ so với nhiễu, tỉ lệ này lớn hơn tương ứng với thuật toán hiệu quả hơn.

2. Khôi phục ảnh bằng loại bỏ nhiễu

2.1. Mô hình nhiễu (Noise model)

Nhiễu (noise) có trong ảnh phát sinh trong quá trình thu nhận, số hóa ảnh và truyền ảnh có thể gây ra bởi các yếu tố như sự tác động của môi trường xung quanh tới các bộ cảm biến thu nhận ảnh hoặc các nhân tố can thiệp vào nội dung ảnh trong quá trình truyền. Theo mô hình suy thoái nhiễu tác động tới chất lượng ảnh được biểu diễn bằng công thức:

$$g(x,y) = f(x,y) + \eta(x,y)$$

Trong đó $f(x,y)$ là ảnh gốc, $\eta(x,y)$ là nhiễu còn $g(x,y)$ là ảnh bị nhiễu thu được cuối cùng. Nếu chúng ta có thể ước lượng được mô hình nhiễu trong ảnh thì việc khử nhiễu để thu được ảnh gần với ảnh gốc $f(x,y)$ sẽ cho một kết quả tốt hơn. Để đơn giản hóa quá trình lọc nhiễu, chúng ta giả sử:

- + Nhiễu là độc lập với các tọa độ không gian
- + Không có sự tương quan nào giữa các giá trị điểm ảnh với nhiễu.

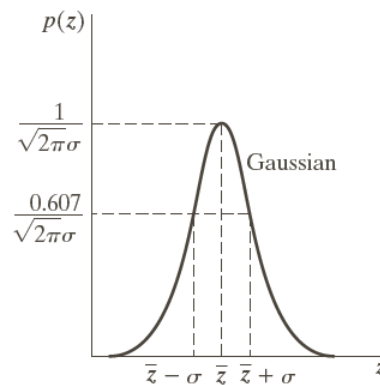
Các giả thiết này không đúng với các nhiễu định kỳ (periodic noise).

2.1.1. Các mô hình nhiễu

Hầu hết các mô hình nhiễu được mô hình hóa dựa trên một hàm mật độ xác suất (probability density function – PDF) và đòi hỏi có thông tin trước về nguồn gốc nhiễu. Có rất nhiều mô hình nhiễu nhưng mô hình Gaussian và mô hình xung (impulse) là phổ biến nhất.

Mô hình nhiễu Gaussian: Nhiễu Gaussian phát sinh do các mạch điện tử hoặc do nhiễu của bộ cảm biến tác động bởi nguồn sáng yếu hoặc nhiệt độ cao. Hàm mật độ xác suất của mô hình nhiễu Gaussian là:

$$p(z) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(z-\bar{z})^2/2\sigma^2}$$



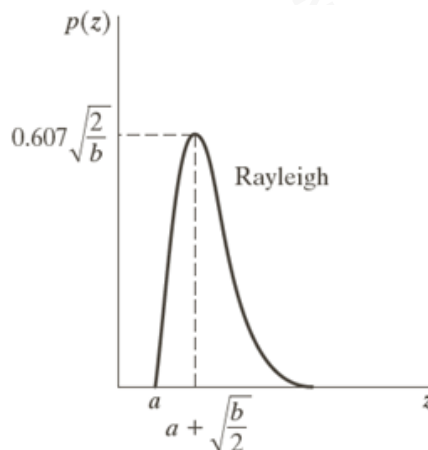
Hình 5.2: Biểu diễn của hàm mật độ xác suất nhiễu Gaussian

Mô hình nhiễu Rayleigh: là mô hình thường được dùng để biểu diễn các nhiễu trong lĩnh vực *range imaging* (các phương pháp dùng để tạo ra một hình ảnh 2D biểu diễn khoảng cách tới các điểm trong một khung cảnh từ một điểm xác định trước, thường liên quan tới thiết bị cảm ứng). Hàm mật độ xác suất cho bởi mô hình này là:

$$p(z) = \begin{cases} \frac{2}{b}(z-a)e^{-\frac{(z-a)^2}{b}} & \text{với } z \geq a \\ 0 & \text{với } z < a \end{cases}$$

Trong đó: $\bar{z} = a + \sqrt{\pi b/4}$, $\sigma^2 = b(4 - \pi)/4$

là các giá trị trung bình và phương sai của hàm $p(z)$ được biểu diễn như sau:

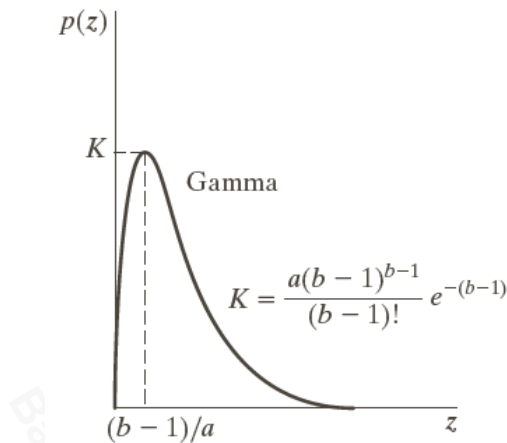


Hình 5.3: Biểu diễn hàm mật độ của mô hình nhiễu Rayleigh

Mô hình nhiễu Gamma (Erlang): Mô hình này thường dùng trong các ảnh laser với hàm mật độ xác suất cho bởi công thức:

$$p(z) = \begin{cases} \frac{a^b z^{b-1}}{(b-1)!} e^{-az} & \text{với } z \geq 0 \\ 0 & \text{với } z < 0 \end{cases}$$

Trong đó: $\bar{z} = \frac{b}{a}$, $\sigma^2 = \frac{b}{a^2}$

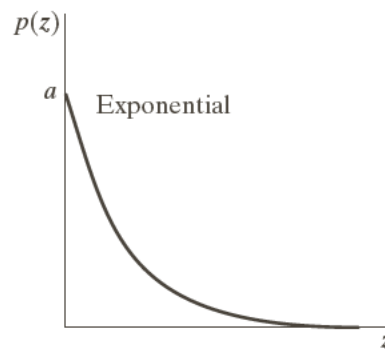


Hình 5.4: Biểu diễn hàm mật độ của mô hình nhiễu Gamma

Mô hình nhiễu hàm mũ (exponential): Mô hình này cũng thường được dùng với các ảnh laser với hàm mật độ cho bởi công thức:

$$p(z) = \begin{cases} ae^{-az} & \text{với } z \geq 0 \\ 0 & \text{với } z < 0 \end{cases}$$

Trong đó: $\bar{z} = \frac{1}{a}$, $\sigma^2 = \frac{1}{a^2}$

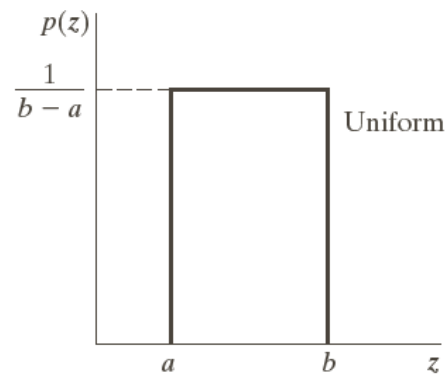


Hình 5.5: Mô hình nhiễu hàm mũ

Mô hình nhiễu Uniform: Mô hình này ít dùng trong thực tế nhưng là một cơ sở để so sánh với mô hình Gaussian và làm công cụ để sinh các số ngẫu nhiên. Hàm mật độ xác suất của mô hình Uniform là:

$$p(z) = \begin{cases} \frac{1}{b-a} & \text{với } a \leq z \leq b \\ 0 & \text{nếu ngược lại} \end{cases}$$

Trong đó: $\bar{z} = \frac{a+b}{2}$, $\sigma^2 = \frac{(b-a)^2}{12}$

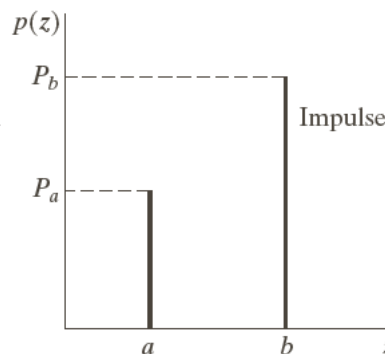


Hình 5.6: Biểu diễn của hàm mật độ mô hình nhiễu Uniform

Mô hình nhiễu xung (Impulse noise model): Các nhiễu gây ra bởi các chuyển đổi nhanh, đột ngột diễn ra trong quá trình thu nhận ảnh. Hàm mật độ xác suất của nhiễu xung là:

$$p(z) = \begin{cases} P_a & \text{nếu } z = a \\ P_b & \text{nếu } z = b \\ 0 & \text{trong các trường hợp khác} \end{cases}$$

Do các nhiễu xung thường lớn hơn nhiễu so với cường độ sáng của điểm ảnh (tín hiệu) nên luôn có thể giả thiết rằng a và b được số hóa bằng với các giá trị bão hòa là màu trắng (salt-255) và màu đen (pepper-0). Có thể thấy là với $P_a=P_b$, $a=0$, $b=255$ thì nhiễu này biểu hiện trên ảnh dưới dạng các điểm ảnh trắng và đen nên gọi là nhiễu “salt and pepper”, về mặt hình ảnh giống như màu tóc *muối tiêu* (salt and pepper), có thể xem trong hình 5.4 để thấy rõ hơn.

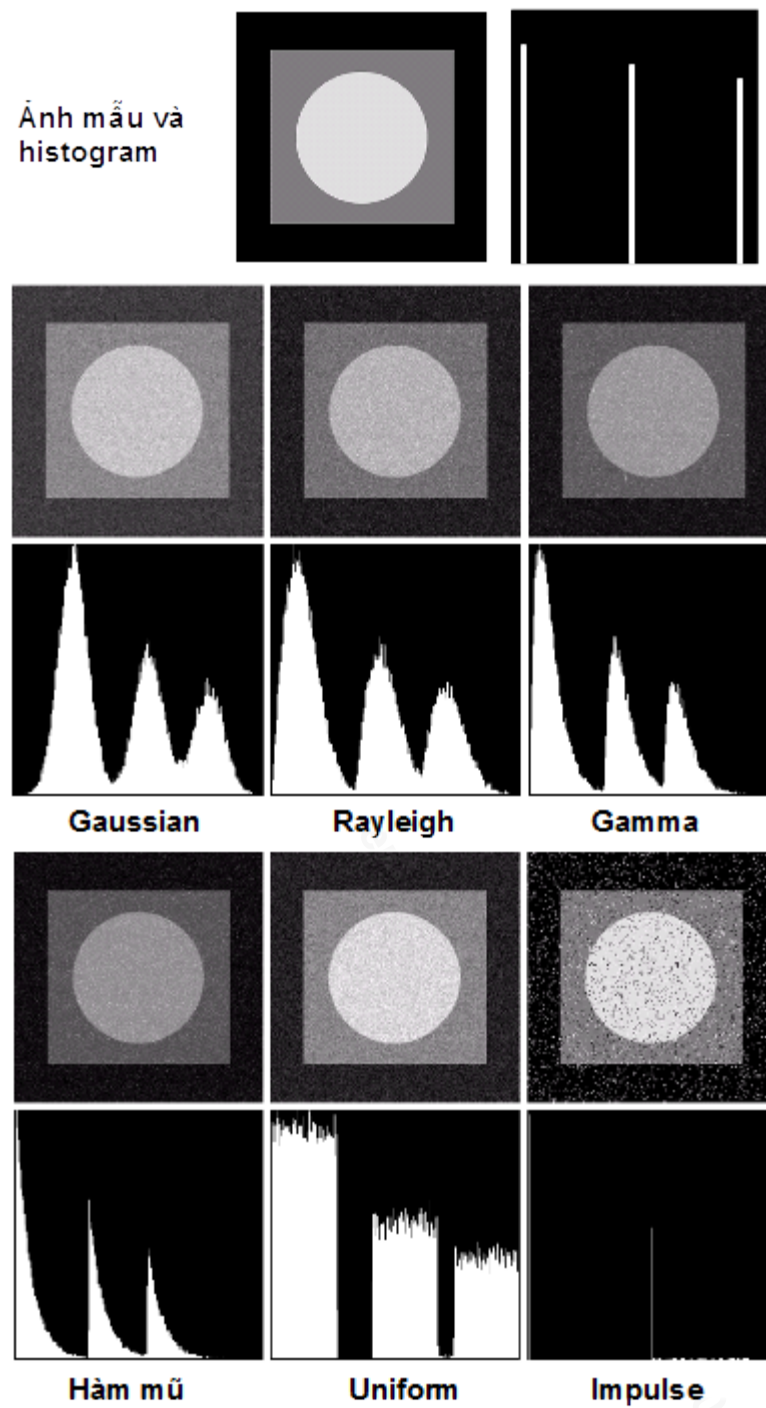


Hình 5.7: Biểu diễn hàm mật độ xác suất của nhiễu xung

2.1.2. Tác động của nhiễu và dự đoán nhiễu

Để đánh giá tác động của nhiễu, một ảnh mẫu được sử dụng và sau đó các ảnh bị nhiễu sẽ thu được từ việc áp dụng các hàm PDF lên ảnh mẫu (xem trong hình 5.8).

Để xác định nhiễu người ta thực hiện ước lượng các tham số của hàm PDF từ các vùng ảnh nhỏ (patch) được lựa chọn với tiêu chí có cường độ sáng của nền là hằng số và từ đó tính các giá trị a , b . Có thể xem ví dụ trong hình 5.9.



Hình 5.8: Ảnh hưởng của các mô hình nhiễu lên ảnh



FIGURE 5.6 Histograms computed using small strips (shown as inserts) from (a) the Gaussian, (b) the Rayleigh, and (c) the uniform noisy images in Fig. 5.4.

Hình 5.9: Ước lượng tham số nhiễu.

2.2. Bộ lọc trung bình cho lọc nhiễu

Các bộ lọc trong miền không gian có thể sử dụng để loại bỏ các loại nhiễu khác nhau. Các bộ lọc phi tuyến như median, ATM, min, max, mid-point có thể dùng hiệu quả để loại bỏ các nhiễu theo mô hình xung, gọi là các nhiễu salt and pepper. Các bộ lọc dựa vào giá trị trung bình có thể dùng để loại bỏ các nhiễu theo mô hình Gaussian. Trong phần tiếp theo chúng ta sẽ xem xét một số bộ lọc trung bình cho phép lọc nhiễu hiệu quả hơn so với bộ lọc trung bình đơn giản đã xét ở phần đầu chương 3.

2.2.1. Bộ lọc trung bình hình học (Geometric mean)

$$\hat{f} = \left[\prod_{(s,t) \in S_{xy}} g(s,t) \right]^{\frac{1}{mn}}$$

Bộ lọc này cho kết quả khá giống với bộ lọc trung bình nhưng giữ lại được nhiều chi tiết ảnh hơn.

2.2.2. Bộ lọc trung bình điều hòa (Harmonic mean)

$$\hat{f} = \frac{mn}{\sum_{(s,t) \in S_{xy}} \frac{1}{g(s,t)}}$$

Làm việc tốt với các nhiễu salt và Gaussian, nhưng không tốt với các nhiễu pepper.

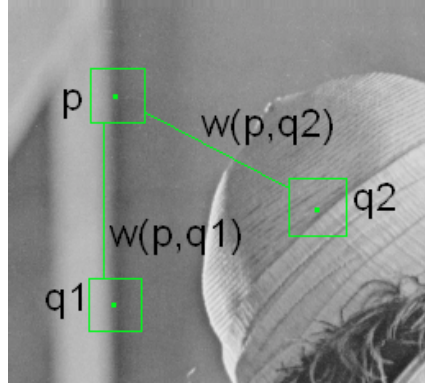
2.2.3. Bộ lọc trung bình Contraharmonic

$$\hat{f} = \frac{\sum_{(s,t) \in S_{xy}} g(s,t)^{Q+1}}{\sum_{(s,t) \in S_{xy}} g(s,t)^Q}$$

Q là bậc của bộ lọc và sẽ được gán giá trị dương để lọc nhiễu pepper, nhiễu salt tương ứng với các giá trị âm.

2.3. Loại bỏ nhiễu bằng bộ lọc Non-Local Means

Trong phần này chúng ta sẽ xem xét một bộ lọc khá hiệu quả cho việc lọc nhiễu nói chung có tên là Non-Local Means (NLM) được đề xuất năm 2005 bởi các tác giả Antoni Buades. Bộ lọc này dựa trên giả thiết là bất cứ một bức ảnh nào cũng chứa một lượng lớn các đặc điểm gọi là self-similarity, một khái niệm xuất phát từ việc tổng hợp các vân ảnh (texture). Các đặc điểm này là các vùng con (patch, sub-region) của ảnh giống nhau, ví dụ như hình sau:



Hình 5.10: Tính chất self-similarity của ảnh

Phương pháp NLM tính giá trị của mỗi điểm ảnh khôi phục (output) p là một hàm của tất cả các điểm ảnh có trong ảnh theo công thức:

$$NLM(f)(p) = \sum_{q \in f} w(p, q) f(q)$$

Trong đó f là ảnh bị nhiễu còn $w(p, q)$ là các trọng số thỏa mãn điều kiện:

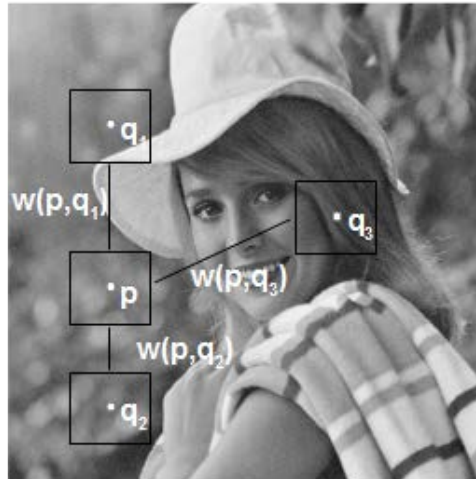
$$0 \leq w(p, q) \leq 1 \text{ và } \sum_{q \in f} w(p, q) = 1$$

Các giá trị trọng số này dựa trên độ tương đồng giữa các lân cận của hai điểm p và q với mỗi lân cận N_i là một sub-region của ảnh có trung tâm là điểm i và được xác định bởi một bán kính R_{sim} . Độ tương đồng giữa hai lân cận của điểm p và q được tính bằng tổng bình phương khoảng cách giữa các điểm thuộc hai lân cận theo công thức:

$$ssd(i, j) = \|f(N_i) - f(N_j)\|_{\sigma}^2 = \|(f(N_i) - f(N_j)) * G_{\sigma}\|^2$$

Trong đó G_{σ} là bộ lọc Gaussian có kích thước kernel filter bằng N_i (có thể thay bằng một bộ lọc dựa trên lân cận khác như bộ lọc trung bình). Khi đó các trọng số $w(i, j)$ sẽ được tính theo công thức:

$$w(i, j) = \frac{1}{Z(i)} e^{-\frac{ssd(i, j)}{h^2}}$$



Hình 5.11: Minh họa cách làm việc của thuật toán NLM

Với $Z(i)$ là hằng số chuẩn hóa và $Z(i) = \sum_j e^{-\frac{ssd(i,j)}{h^2}}$, h là bậc của bộ lọc. Để tránh lặp lại trường hợp $p=q$, các trọng số $w(x,y)$ được gán $w(x,y) = \max\{w(p,q) | p \neq q\}$.

Có thể quan sát cách thức làm việc của bộ lọc NLM qua hình 5.11.

Trên lý thuyết mỗi giá trị trọng số $w(p,q)$ có thể được tính dựa trên tất cả các điểm ảnh nhưng trên vì các lý do thực tế và chi phí tính toán, số lượng điểm ảnh được dùng để tính mỗi trọng số được giới hạn nằm trong một cửa sổ có kích thước 21×21 . Bên cạnh đó bán kính R_{sim} cũng được cố định và chỉ xét các điểm nằm trong lân cận kích thước 7×7 của mỗi điểm ảnh. Giá trị bậc h thường được lấy bằng 10σ .

Hình 5.11 cho ta một so sánh khi lọc nhiễu ảnh bằng các phương pháp lọc Gaussian, lọc Median và NLM. Có thể nhận thấy bằng mắt thường rằng bộ lọc NLM cho kết quả tốt nhất với hình ảnh mượt nhưng vẫn giữ được các chi tiết ảnh trong khi các nhiễu đã bị loại bỏ. Bộ lọc Median cho kết quả tốt hơn Gaussian. Về số liệu thực tế NLM cũng đạt kết quả tốt nhất vì có MSE nhỏ nhất.



Hình 5.12: So sánh bộ lọc NLM với bộ lọc Gaussian và Median

Một nhược điểm của NML là chi phí tính toán lớn vì ta phải tính các trọng số $w(p,q)$ rất nhiều lần và cho từng điểm ảnh. Ngoài ra NML không làm cho ảnh sắc nét hơn mà còn làm cho một số biên của ảnh bị mờ đi.

2.4. Bộ lọc trung vị linh hoạt (Adaptive median filter) cho lọc nhiễu xung

Bộ lọc trung vị linh hoạt dựa trên các đặc điểm thống kê của các điểm ảnh trong lân cận của mỗi điểm ảnh thuộc ảnh input để đưa ra kết quả. Bộ lọc này là một biến thể tốt hơn của bộ lọc trung vị được trình bày ở chương 2 khi làm việc với các nhiễu xung. Đối với các nhiễu không phải là nhiễu xung, bộ lọc trung vị linh hoạt có tác dụng là mượt ảnh. Ngoài ra so với các bộ lọc khác (trung bình, Gaussian, trung vị, ATM) nó còn làm giảm sự biến dạng (distortion) của ảnh sau khi lọc. Ý tưởng cơ bản của bộ lọc trung vị linh hoạt là kích thước của filter kernel sẽ thay đổi dựa trên đặc điểm của ảnh.

Với các ký hiệu sau:

- $Z_{min}, Z_{max}, Z_{med}$ là mức xám nhỏ nhất, lớn nhất, trung vị trong lân cận S_{xy}
- z_{xy} là mức xám tại vị trí (x, y)
- S_{max} là kích thước lớn nhất của S_{xy}

Giá trị output của bộ lọc tại điểm (x, y) sẽ được tính như sau:

Mức A:

+ Tính $A1=Z_{med} - Z_{min}, A2=Z_{med} - Z_{max}$

+ Nếu $A1>0$ và $A2<0$ thì chuyển sang mức B

Ngược lại tăng kích thước của kernel filter lên 2 đơn vị

+ Nếu kích thước của kernel filter $\leq S_{max}$ thì lặp lại mức A, ngược lại trả về Z_{med}

Mức B:

+ Tính $B1=z_{xy} - Z_{min}, B2=z_{xy} - Z_{max}$

+ Nếu $B1>0$ và $B2<0$ thì trả về z_{xy} ngược lại trả về Z_{med} .

Input image
with salt and
pepper noise

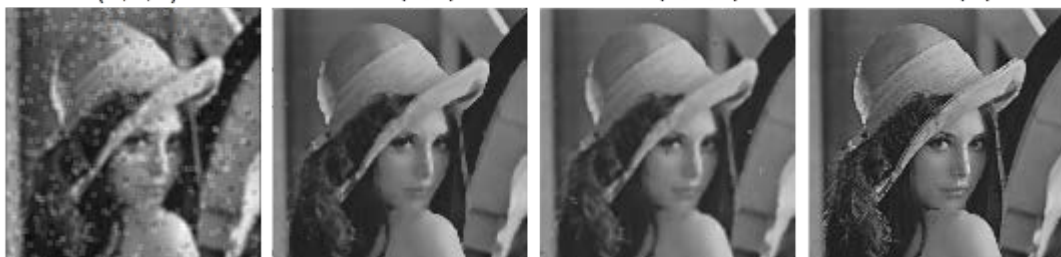


G(3,3,5)

Med(3,3)

ATM(3,3,4)

Amed(5)



Hình 5.13: Bộ lọc trung vị linh hoạt (Amed) vs. Gaussian (G), Median(M) và ATM

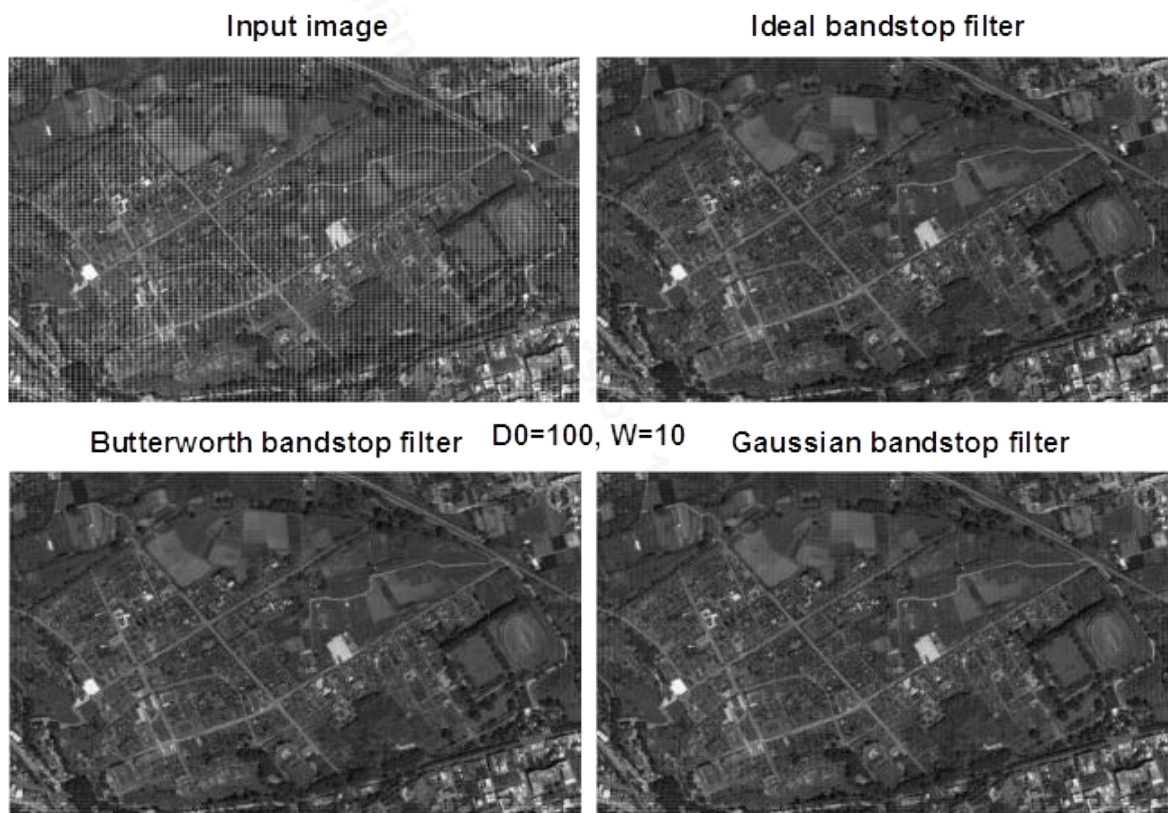
Có thể thấy rõ trong hình 5.13 là so với các bộ lọc khác, bộ lọc trung vị linh hoạt cho kết quả tốt nhất, vừa lọc được nhiễu đồng thời không làm biến dạng ảnh mà vẫn giữ được các chi tiết sắc nét của ảnh.

2.5. Lọc ảnh trong miền tần số để khử nhiễu định kỳ

Các nhiễu định kỳ (periodic noise) gây ra bởi các tác nhân điện hoặc điện từ (electromagnetic) tạo mẫu nhiễu có tính chất lặp lại trong ảnh có thể được lọc rất hiệu quả bằng cách sử dụng các phép lọc trong miền tần số.

2.5.1. Bộ lọc chặn dải thông (Bandpass stop filter)

Bộ lọc này đã được trình bày trong chương 4. Ở đây chỉ nêu lại một ví dụ minh họa trong hình bên dưới. Có thể thấy rõ sự hiệu quả của bộ lọc chặn dải thông cho các ảnh chụp vệ tinh.



Hình 5.14: Bộ lọc Bandstop cho nhiễu định kỳ

2.5.2. Bộ lọc Notch reject (stop) filter

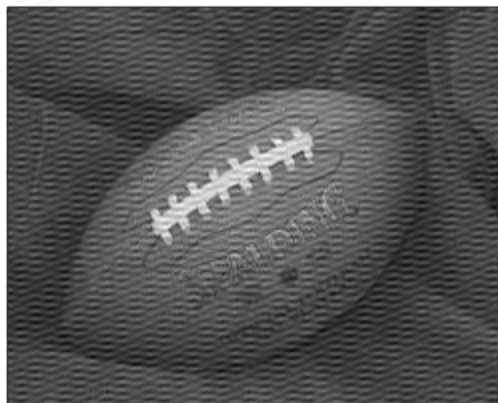
Bộ lọc Notch trong miền tần số là một bộ lọc hiệu quả khác cho các nhiễu định kỳ. Bộ lọc này còn được gọi là bộ lọc khử các đỉnh trong biểu diễn phổ của ảnh. Trong số các bộ lọc có tính chất lựa chọn (selective) (như dải thông-bandpass, bandstop hay notch reject, notch pass) thì bộ lọc notch reject hiệu quả hơn cả. Một bộ lọc Notch sẽ cho qua (pass) hoặc loại bỏ (reject, stop) các tần số nằm trong một lân cận được xác định trước dựa trên một tần số trung tâm. Các nhiễu định kỳ có tính chất lặp lại có thể được nhận thấy dưới dạng các đỉnh màu trắng trong biểu diễn phổ magnitude (trong biểu diễn DFT) của ảnh (xem hình minh họa bên dưới, các đỉnh trắng được đánh dấu bằng các hình tròn màu đỏ là nhiễu). Các nhiễu trong ảnh gốc có thể được loại bỏ một cách hiệu quả bằng cách xóa bỏ các đỉnh này qua việc sử dụng các bộ lọc notch. Kỹ thuật này rất phổ biến

trong việc lọc các âm thanh có các nhiễu cơ khí hoặc điện tử. Tuy nhiên việc sử dụng bộ lọc Notch là một quá trình phi thể thức và đòi hỏi cần có kinh nghiệm để xác định tần số cần loại bỏ.

Do tính chất đối xứng của biến đổi Fourier các bộ lọc Notch phải đi theo cặp đối xứng qua gốc tọa độ (xem hình minh họa bên dưới). Các hàm của các bộ lọc Ideal, Butterworth, Gaussian Notch Reject Filters với các bán kính $D0_k$ ($k=1..Q$) và Q điểm trung tâm (u_k, v_k) và $(-u_k, -v_k)$ cho bởi các công thức sau:

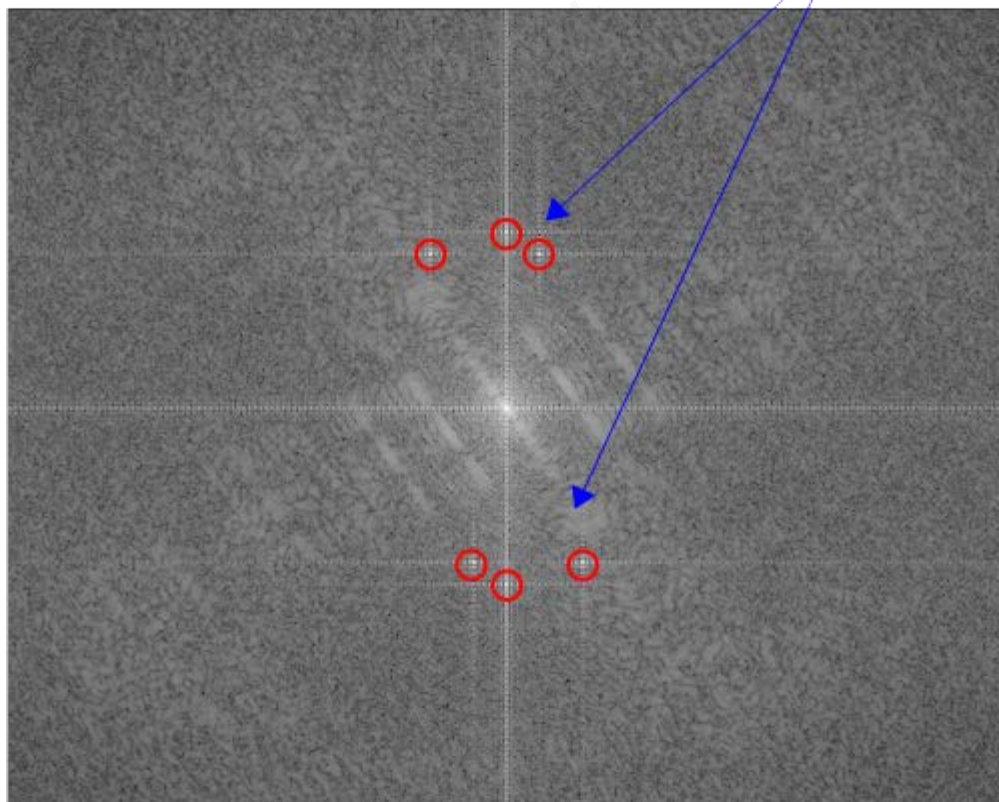
$$H_{NR}(u, v) = \prod_{k=1}^Q H_{HP,k}(u, v) H_{HP,-k}(u, v)$$

Ảnh Input bị nhiễu



Nhiễu cần loại bỏ

Magnitude spectrum của biến đổi DFT

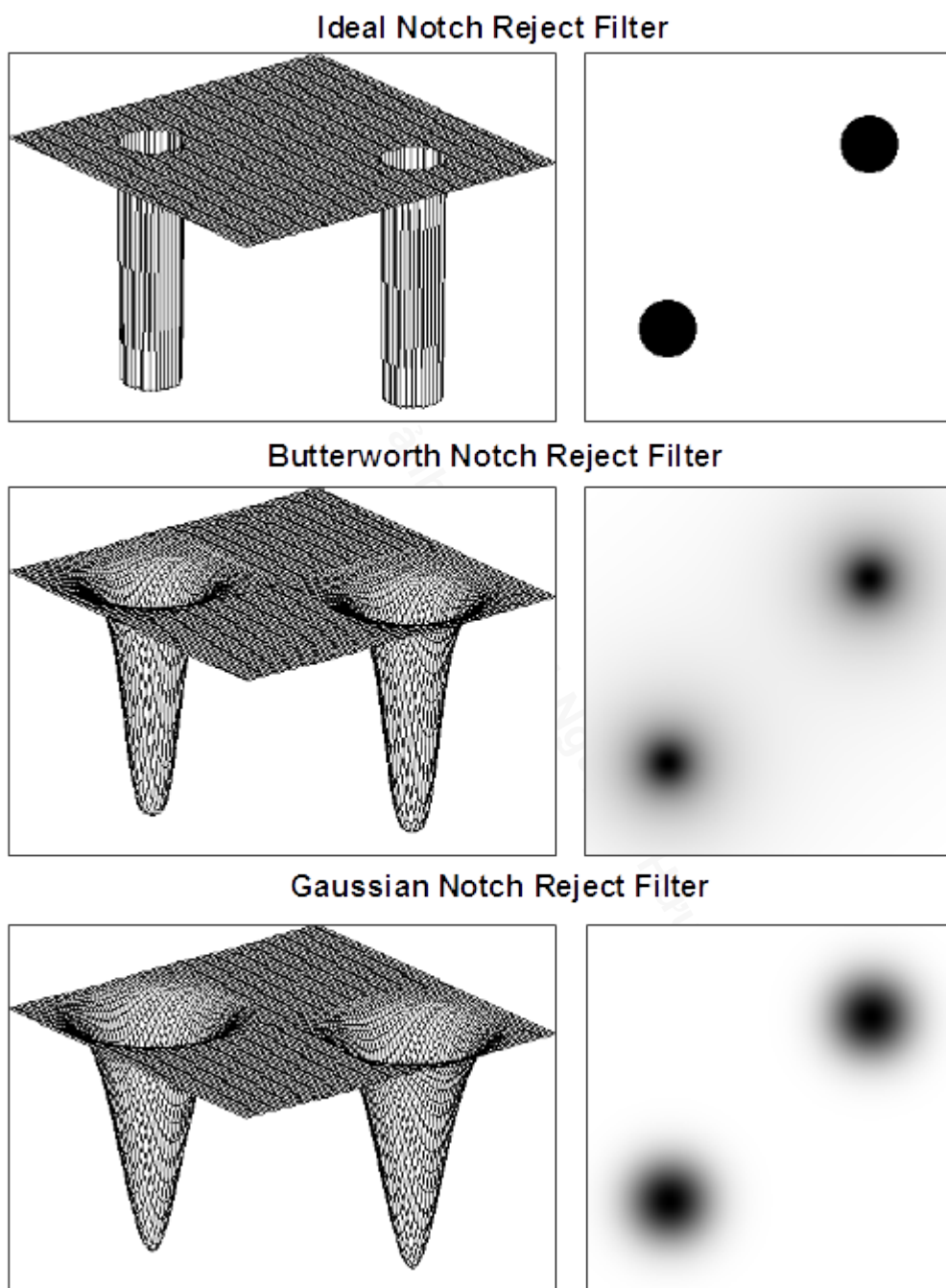


Hình 5.15: Nhiễu định kỳ của ảnh trên biểu diễn phổ DFT

Trong đó $H_{HP,k}(u,v)$ và $H_{HP,-k}(u,v)$ lần lượt là hai bộ lọc thông cao (highpass filter) tương ứng (Ideal, Butterworth, Gaussian) của điểm trung tâm thứ (u_k, v_k) . Hàm khoảng cách D tương ứng với mỗi điểm trung tâm (u_k, v_k) được tính như sau:

$$D_k(u, v) = \sqrt{(u - M/2 - u_k)^2 + (v - N/2 - v_k)^2},$$

$$D_{-k}(u, v) = \sqrt{(u - M/2 + u_k)^2 + (v - N/2 + v_k)^2}$$



Hình 5.16: Các bộ lọc Notch Reject

Cụ thể đối với bộ lọc Butterworth Notch Reject Filter (BNRF), ta có:

$$H_{NR}(u, v) = \prod_{k=1}^Q \left[\frac{1}{1 + [D0_k/D_k(u, v)]^{2n}} \right] \left[\frac{1}{1 + [D0_k/D_{-k}(u, v)]^{2n}} \right]$$

Các bán kính $D0_k$ có thể khác nhau cho mỗi điểm trung tâm (u_k, v_k) nhưng là giống nhau cho các cặp (u_k, v_k) và $(-u_k, -v_k)$. Các công thức cho bộ lọc Ideal và Gaussian hoàn toàn có thể suy ra từ công thức của bộ lọc thông cao một cách tương tự.

Một bộ lọc Notch Pass $H_{NP}(u,v)$ có thể nhận được từ bộ lọc Notch Reject $H_{NR}(u,v)$ tương ứng theo công thức: $H_{NP}(u,v) = 1 - H_{NR}(u,v)$.

Do tính chất đối xứng trong biểu diễn phổ DFT của ảnh (xem hình 5.15) ta chỉ cần chọn một nửa các điểm trung tâm nằm phía trên trục hoành để thực hiện lọc nhiễu (với ảnh minh họa tương ứng là 3).

Có thể thấy rõ hiệu quả tuyệt vời của các bộ lọc khi so sánh kết quả trong hình 5.17 với ảnh gốc trong hình 5.15, đồng thời các đỉnh trắng trong DFT spectrum của ảnh cũng bị loại bỏ. Cũng cần lưu ý là sau bước lọc bằng Notch filters, độ tương phản của ảnh thường bị ảnh hưởng nên cần có thêm một thao tác xử lý để tăng cường độ tương phản (chẳng hạn như bằng cân bằng histogram).

2.5.3. Bộ lọc Notch tối ưu (Optimum Notch Filter)

Khi có nhiều yếu tố tác động lên ảnh, các bộ lọc đã xem xét ở các phần trước thường cho kết quả tồi vì chúng có thể làm mất rất nhiều chi tiết của ảnh trong quá trình xử lý. Trong phần này chúng ta sẽ xem xét một loại bộ lọc tối ưu theo nghĩa làm giảm tối thiểu các phương sai cục bộ của ảnh khôi phục $\hat{f}(x,y)$. Quá trình lọc bằng bộ lọc tối ưu gồm hai bước: bước thứ nhất là xác định mẫu nhiễu dựa trên việc phát hiện các thành phần tần số chính gây ra nhiễu (giống như đối với bộ lọc Notch thông thường), bước thứ hai sẽ trừ ảnh bị nhiễu $g(x,y)$ cho một phần có trọng số của nhiễu để nhận được ảnh gốc.

Bước thứ nhất được thực hiện bằng cách đặt một bộ lọc H_{NP} vào các đỉnh nhiễu trong phổ của ảnh. Khi đó nhiễu nhận được trong miền tần số là:

$$N(u,v) = H_{NP}(u,v)G(u,v)$$

Nhiễu trong miền không gian tương ứng sẽ là:

$$\eta(x,y) = DFT^{-1}\{H_{NP}(u,v)G(u,v)\}$$

Với nhiễu $\eta(x,y)$ đã biết việc tìm $\hat{f}(x,y)$ rất đơn giản:

$$\hat{f}(x,y) = g(x,y) - \eta(x,y)$$

Tuy nhiên $\eta(x,y)$ trên thực tế chỉ là một xấp xỉ của nhiễu và không thể hiện hết các nhân tố gây ra nhiễu thực sự. Để giảm thiểu sự ảnh hưởng của các nhân tố không được thể hiện trong nhiễu $\eta(x,y)$, ta trừ $g(x,y)$ cho một phần có trọng số của $\eta(x,y)$:

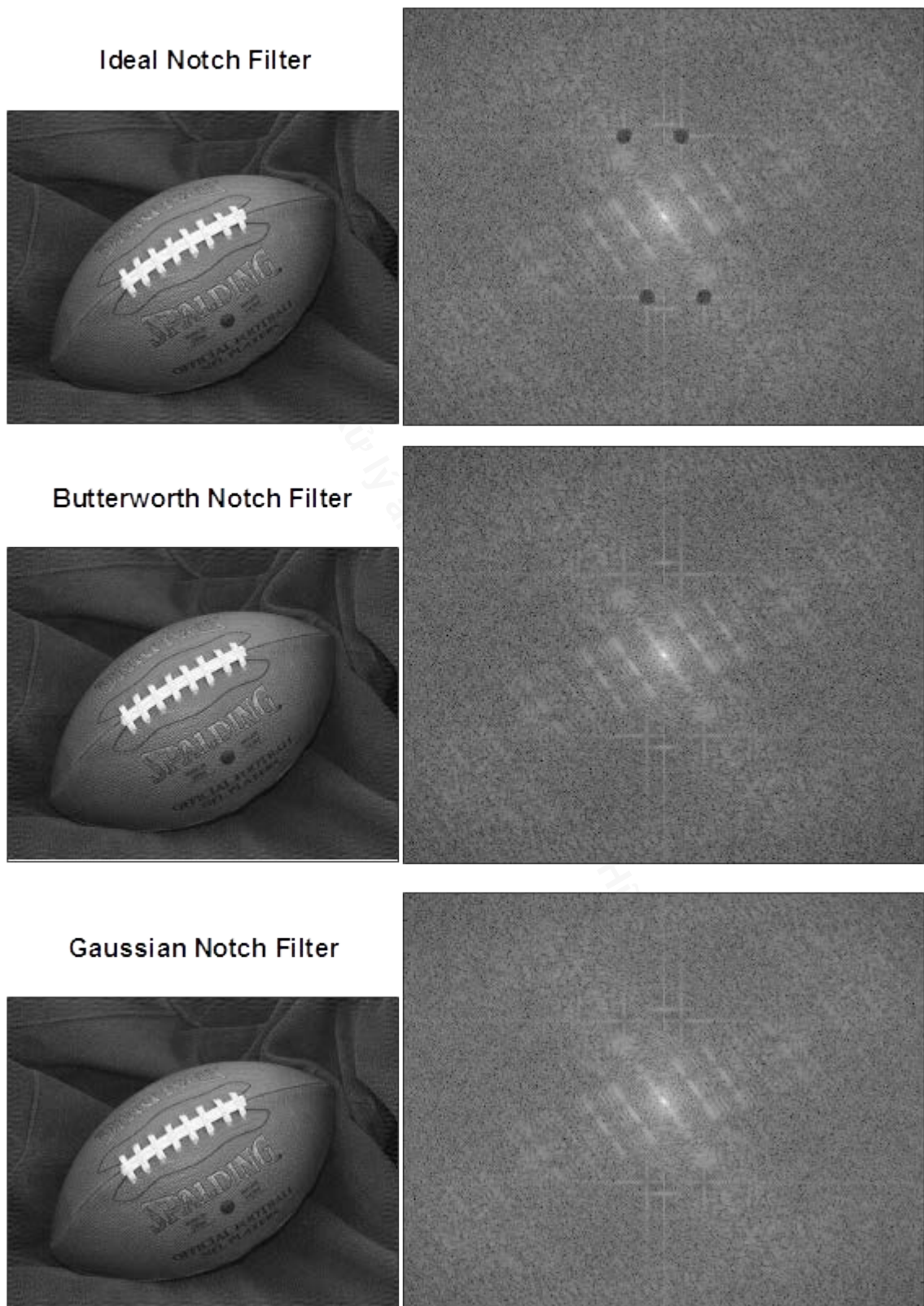
$$\hat{f}(x,y) = g(x,y) - w(x,y)\eta(x,y)$$

Với:

$$w(x,y) = \frac{\overline{g(x,y)\eta(x,y)} - \bar{g}(x,y)\bar{\eta}(x,y)}{\bar{\eta}^2(x,y) - \bar{\eta}^2(x,y)}$$

là các trọng số có các giá trị $w(x,y)$ được tính trong một lân cận kích thước axb của ảnh. Với mỗi lân cận $w(x,y)$ được xem là hằng số nên chỉ cần tính cho một điểm (thường là điểm trung tâm của lân cận) và sau đó sử dụng để tính cho tất cả các điểm khác trong lân cận.

cận đó (tính $\hat{f}(x, y)$). Xem minh họa các bước thực hiện bộ lọc Notch tối ưu ở hình 5.8 bên dưới (kích thước cửa sổ sử dụng là 20x25).

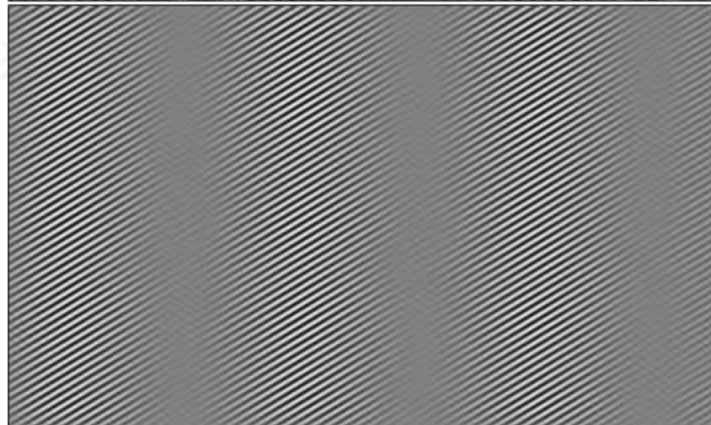


Hình 5.17: Lọc nhiễu bằng bộ lọc Notch Reject

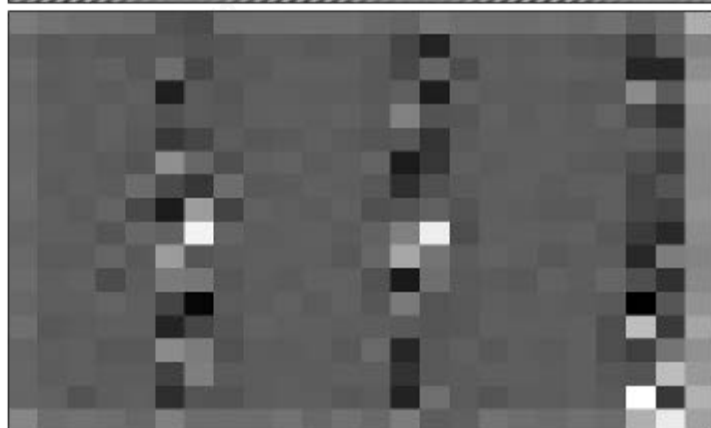
Input Image
 $g(x,y)$



Noise Image
 $\eta(x,y)$



Weight $w(x,y)$



Ảnh khôi phục



Hình 5.18: Khôi phục ảnh bằng bộ lọc tối ưu Gaussian Notch

3. Khôi phục ảnh dựa vào mô hình độ mờ ảnh

3.1. Mô hình tuyến tính (Linear Blur Model)

Cách tiếp cận thứ hai để khôi phục ảnh là dựa vào mô hình độ mờ của ảnh. Khi đó ảnh $g(x,y)$ thu được giả sử bị mờ đi dưới tác động của một yếu tố môi trường nào đó. Thông thường ảnh bị mờ do chuyển động (của đối tượng chụp so với vị trí tương đối của thiết bị chụp) hoặc do hiện tượng out-of-focus của ống kính của thiết bị chụp.

Có hai cách tiếp cận để thực hiện khôi phục ảnh trong trường hợp này là cách tiếp cận non-blind (có thông tin) và blind (không có thông tin). Với các tiếp cận có thông tin, ta có ảnh $g(x,y)$ và biết cơ chế làm ảnh bị mờ qua hàm $h(x,y)$, mục đích là tìm một bộ lọc khôi phục ảnh sao cho độ biến dạng giữa ảnh thu được $\hat{f}(x,y)$ và ảnh $f(x,y)$ là nhỏ nhất. Với cách tiếp cận không có thông tin, ta chỉ có ảnh mẫu thu được $g(x,y)$ bị mờ, mục đích là tìm hàm khôi phục sao cho độ biến dạng giữa ảnh thu được và mẫu ban đầu là nhỏ nhất. Ta sẽ xem xét hai kỹ thuật đơn giản nhất để thực hiện theo các cách tiếp cận này là bộ lọc ngược và bộ lọc Wiener.

3.2. Bộ lọc ngược (Inverse filter)

Cách đơn giản nhất để tiến hành khôi phục ảnh khi ta biết hàm suy thoái (làm cho ảnh mờ) $H(u,v)$ là sử dụng bộ lọc ngược một cách trực tiếp theo công thức:

$$\hat{F}(u,v) = \frac{G(u,v)}{H(u,v)}$$

Theo mô hình suy thoái ảnh (hình 5.1) thì ta sẽ có:

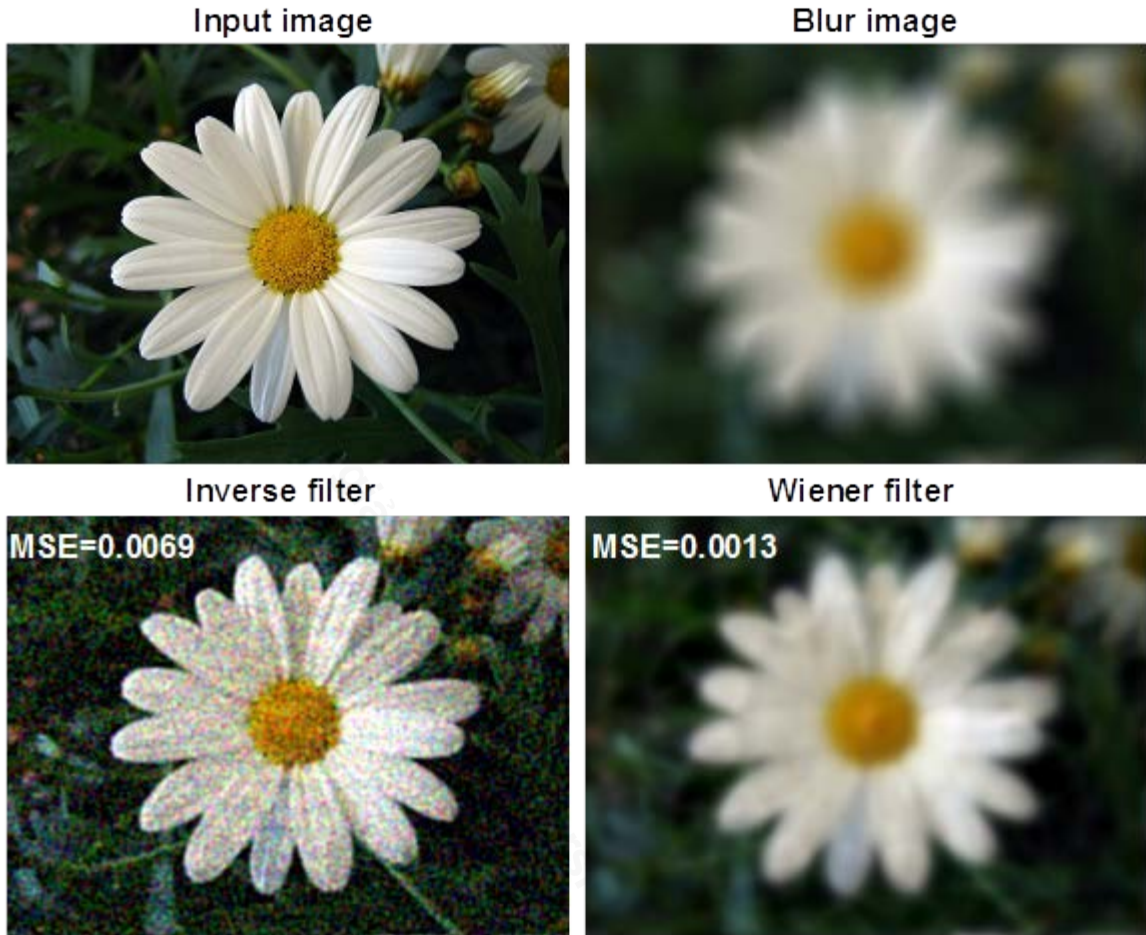
$$\hat{F}(u,v) = \frac{H(u,v)F(u,v) + N(u,v)}{H(u,v)} = F(u,v) + \frac{N(u,v)}{H(u,v)}$$

Công thức này có nghĩa là ta không thể khôi phục được ảnh gốc thực sự $F(u,v)$ vì nhiễu $N(u,v)$ là không biết rõ. Thêm nữa, với các giá trị $H(u,v)$ nhỏ, nhiễu sẽ càng rõ trong ảnh thu được. Để tránh điều này, ta có thể áp dụng cách lọc ngược này với các tần số nằm trong một bán kính D_0 tính từ trung tâm của $H(u,v)$ vì vùng trung tâm sẽ ứng với các giá trị $H(u,v)$ lớn. Trên thực tế bộ lọc ngược cũng ít được áp dụng vì kết quả không đáp ứng được yêu cầu (xem minh họa ở hình 5.19).

3.3. Bộ lọc Wiener

Cách tiếp cận sử dụng cả hàm suy thoái $H(u,v)$ và các thuộc tính thống kê của nhiễu được Norbert Wiener (một nhà toán học tài năng người Mỹ, đã lấy bằng PhD tại Harvard ở tuổi 18) đề xuất nhằm mục đích tìm ảnh $\hat{f}(x,y)$ từ ảnh gốc $f(x,y)$ sao cho giá trị MSE (Mean Square Error) giữa chúng là nhỏ nhất có thể. Bằng cách giả thiết rằng:

- + Nhiễu và ảnh là không có tương quan
- + Một trong hai đối tượng (ảnh hoặc nhiễu) có trung bình bằng 0.
- + Các giá trị cường độ trong việc phục hồi ảnh là một hàm tuyến tính của các mức xám trong ảnh bị suy thoái.



Hình 5.19: Kết quả thực hiện bộ lọc ngược và bộ lọc Wiener

Khi đó giá trị MSE của ảnh khôi phục và ảnh gốc sẽ đạt được giá trị nhỏ nhất với công thức trong miền tần số sau:

$$\begin{aligned}\hat{F}(u, v) &= \left[\frac{H^*(u, v)S_f(u, v)}{S_f(u, v)|H(u, v)|^2 + S_\eta(u, v)} \right] G(u, v) = \left[\frac{H^*(u, v)}{|H(u, v)|^2 + \frac{S_\eta(u, v)}{S_f(u, v)}} \right] G(u, v) \\ &= \left[\frac{1}{H(u, v)} \frac{|H(u, v)|^2}{|H(u, v)|^2 + \frac{S_\eta(u, v)}{S_f(u, v)}} \right] G(u, v) = G_W G(u, v)\end{aligned}$$

Trong đó $\hat{F}(u, v)$ là ảnh khôi phục trong miền tần số, $G(u, v)$ là biến đổi DFT của ảnh bị suy thoái $g(x, y)$, $H(u, v)$ là biểu diễn trong miền tần số của hàm suy thoái H , $H^*(u, v)$ là liên hợp (conjugate) của $H(u, v)$, $|H(u, v)|^2 = H^*(u, v)H(u, v)$, $S_\eta(u, v) = N^2(u, v)$ là giá trị phổ của nhiễu, $S_f(u, v) = |F(u, v)|^2$ là giá trị phổ của ảnh không bị suy thoái $F(u, v)$.

Kết quả của công thức trên là một bộ lọc G_W gọi là bộ lọc Wiener, còn có tên khác là bộ lọc MSE cực tiểu, bộ lọc này không có vấn đề về chia 0 như bộ lọc ngược. Nếu như nhiễu bằng 0, bộ lọc Wiener chính là bộ lọc ngược (xem minh họa ở hình 5.19). Khi ảnh không bị các yếu tố gây mờ tác động ($H=1$), bộ lọc Wiener chính là bộ lọc làm mờ ảnh. Khi có cả nhiễu và ảnh bị mờ, bộ lọc Wiener sẽ là một sự cân bằng giữa lọc thông thấp

để loại bỏ nhiễu và lọc thông cao để làm ảnh sắc nét hơn, kết quả là một bộ lọc dải thông. Có thể thấy trên hình 5.19 là bộ lọc Wiener cho kết quả tốt hơn bộ lọc Inverse vì giá trị MSE nhỏ hơn. Trên thực tế bộ lọc Wiener còn là một giải pháp tốt đối với các ảnh bị mờ gây ra do chuyển động (motion blur).

Tuy nhiên một vấn đề với bộ lọc Wiener là giá trị của phân số $\frac{S_{\eta}(u,v)}{S_f(u,v)}$ không dễ xác định, ngay cả khi $S_{\eta}(u,v)$ là hằng số. Vì thế một cách phổ biến cho các trường hợp này là sử dụng một hằng số thay thế cho giá trị này như sau:

$$\hat{F}(u,v) = \left[\frac{1}{H(u,v)} \frac{|H(u,v)|^2}{|H(u,v)|^2 + K} \right] G(u,v)$$

4. Một số kỹ thuật khôi phục ảnh khác

4.1. Kỹ thuật Super resolution

Kỹ thuật super-resolution gần đây được quan tâm nhiều trong các lĩnh vực có liên quan tới xử lý ảnh và thị giác máy tính. Về cơ bản đây là một kỹ thuật cho phép tạo ra các ảnh có độ phân giải cao (High Resolution) từ nhiều ảnh có độ phân giải thấp dựa vào một quá trình học (learning) trước đó. Ban đầu từ dữ liệu là các ảnh có độ phân giải cao và các ảnh có độ phân giải thấp có liên quan, một thuật toán học sẽ được sử dụng để tìm ra mối liên quan (biểu diễn tuyệt tính) giữa một chi tiết ảnh (điểm ảnh) trong ảnh có độ phân giải thấp với các chi tiết trong ảnh có độ phân giải cao. Sau đó các chi tiết ảnh có độ phân giải cao bị ẩn sẽ được khôi phục.

4.2. Khôi phục ảnh bằng biểu diễn thưa (Sparse representation)

Biểu diễn thưa là một kỹ thuật khác cũng được phát triển gần đây và đạt được nhiều thành công đáng kể khi áp dụng cho khôi phục ảnh. Biểu diễn thưa dựa trên việc mỗi phần tử thông tin được biểu diễn dưới dạng tổ hợp tuyến tính thưa (đa số các hệ số bằng 0) của các phần tử thuộc về một ma trận các mẫu (patterns) nhỏ có kích thước lớn.

Bài tập

Bài tập 1: Viết chương trình thực hiện mở file ảnh và tiến hành khôi phục ảnh theo các phương pháp đã trình bày từ phần 1 tới phần 3.

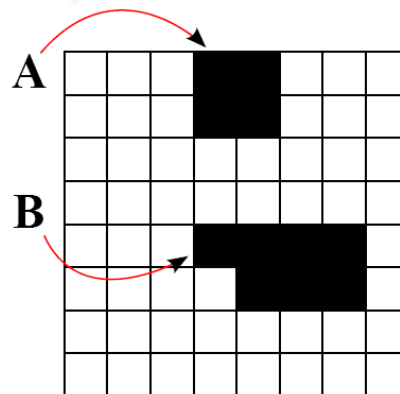
CHƯƠNG VI: CÁC PHÉP XỬ LÝ HÌNH THÁI

1. Giới thiệu

Thuật ngữ hình thái (*morphological*) xuất phát từ một ngành khoa học nghiên cứu về các dạng (form) và cấu trúc (structure) của các loài động vật, cây cối.

Trong xử lý ảnh, các phép xử lý hình thái (*morphological operations*) nhằm mục đích trích xuất các thành phần của ảnh được sử dụng cho việc biểu diễn (representation) và mô tả (description) các vùng cục bộ của ảnh, chẳng hạn như các đường bao (boundary) hoặc các xương (skeleton), hay bao lồi (convex hull). Như vậy mục tiêu chính của các phép xử lý hình thái là xác định, biểu diễn và xử lý hình dạng của các đối tượng trong ảnh. Các kết quả của thao tác xử lý hình thái có thể có chứa các thông tin hình học và hữu ích cho các bước xử lý tiếp theo trong lĩnh vực thị giác máy tính. Ngoài ra xử lý hình thái còn được sử dụng để lọc ảnh, làm mảnh (thinning) hoặc cắt tỉa (pruning).

Một thành phần của ảnh là một đối tượng (hoặc hình dáng) của đối tượng nằm trong ảnh. Ví dụ ảnh sau có hai thành phần:



Hình 6.1: Ví dụ về các thành phần của ảnh

Một trong những thao tác thường thấy là làm thế nào để xác định và gán nhãn cho tất cả các thành phần của ảnh.

Vậy thế nào là phép biến đổi hình thái? Đó chính là biến đổi logic dựa trên điểm ảnh để xử lý các đối tượng cục bộ trong ảnh. *Thường các thao tác xử lý hình thái đều thực hiện trên các ảnh nhị phân và các biến đổi logic dựa trên việc so sánh các điểm trong cùng một lân cận với một mẫu.*

1.1. Lý thuyết tập hợp

Kỹ thuật xử lý hình thái xuất phát từ lý thuyết về tập hợp nên chúng ta sẽ xem xét các khái niệm cơ bản về tập hợp. Một ảnh có thể được biểu diễn dưới dạng các tập trong không gian Z^2 .

Với A là một tập hợp trong không gian Z^2 và một phần tử $a=(a_1, a_2)$ ta có các khái niệm sau đây:

- + a là một phần tử của tập A : $a \in A$.
- + a không phải là một phần tử thuộc tập A : $a \notin A$.

+ Với hai tập A, B , A được gọi là tập con của B nếu mọi phần tử của tập A đều nằm trong B và ký hiệu là: $A \subseteq B$.

Chương VI: Các phép xử lý hình thái

+ Hợp nhất của A và B là tập C gồm tất cả các phần tử hoặc thuộc về A hoặc thuộc về B: $C = A \cup B$

+ Giao của A và B là tập C gồm các phần tử vừa thuộc về A, vừa thuộc về B: $C = A \cap B$.

+ Hai tập A và B gọi là không giao nhau nếu chúng không có phần tử nào chung: $A \cap B = \emptyset$.

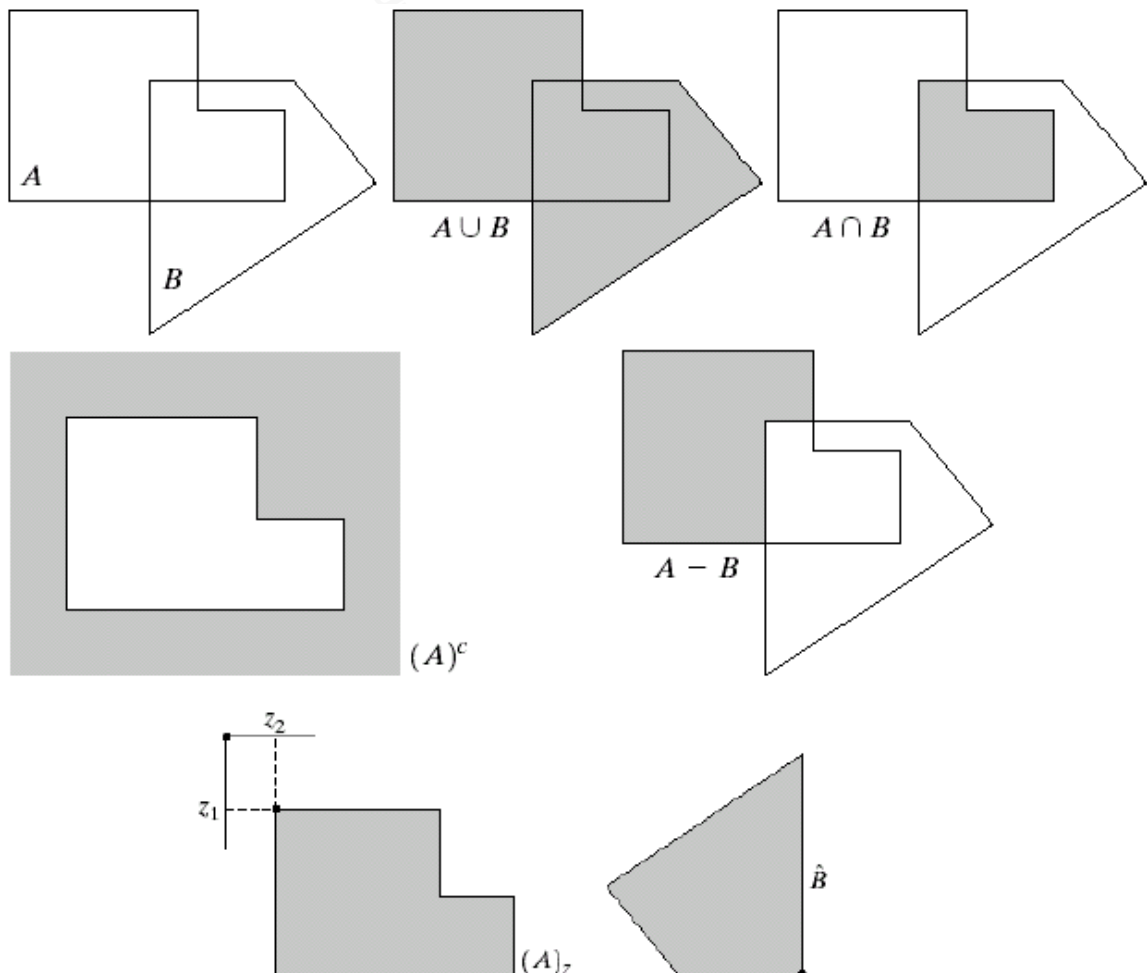
+ Phần bù của tập A là tập gồm tất cả các phần tử không nằm trong A: $A^c = \{\omega | \omega \notin A\}$.

+ Hiệu của hai tập hợp A và B là tập gồm các phần tử thuộc A nhưng không thuộc B: $A - B = \{\omega | \omega \in A, \omega \notin B\}$.

+ Phản chiếu của một tập B là tập $\hat{B}$ được định nghĩa: $\hat{B} = \{\omega | \omega = -b \ \forall b \in B\}$.

+ Phép dịch một tập A bởi một điểm $z = (z_1, z_2)$: $(A)_z = \{\omega | \omega = a + z \ \forall a \in A\}$.

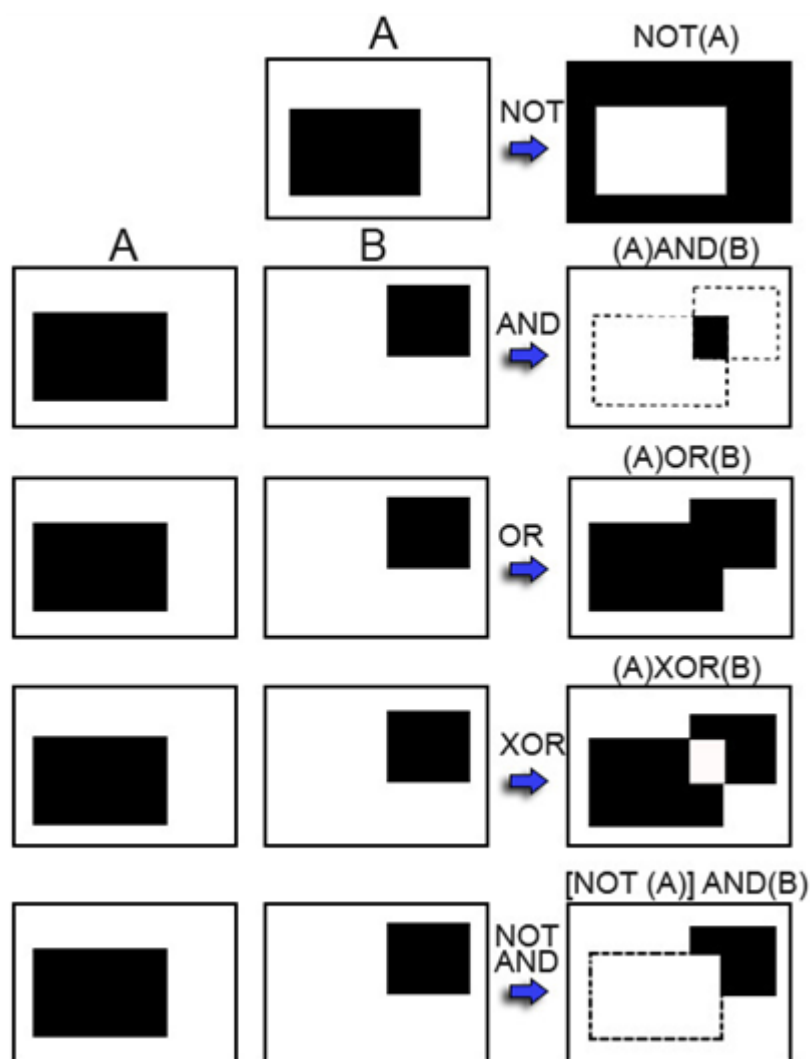
Có thể hình dung về mặt hình ảnh như sau:



Hình 6.2: Các phép toán trên tập hợp

1.2. Phép toán logic trên ảnh nhị phân

Các thao tác xử lý hình thái còn liên quan tới các thao tác logic trên ảnh nhị phân nên phần này cũng nhắc lại một số phép toán logic NOT, AND, OR, XOR, NOT AND qua minh họa bằng hình ảnh như sau:



Hình 6.3: Phép toán logic trên ảnh nhị phân

Lưu ý là trong hình vẽ trên, phần màu đen biểu thị cho giá trị 1, phần màu trắng biểu thị cho giá trị 0.

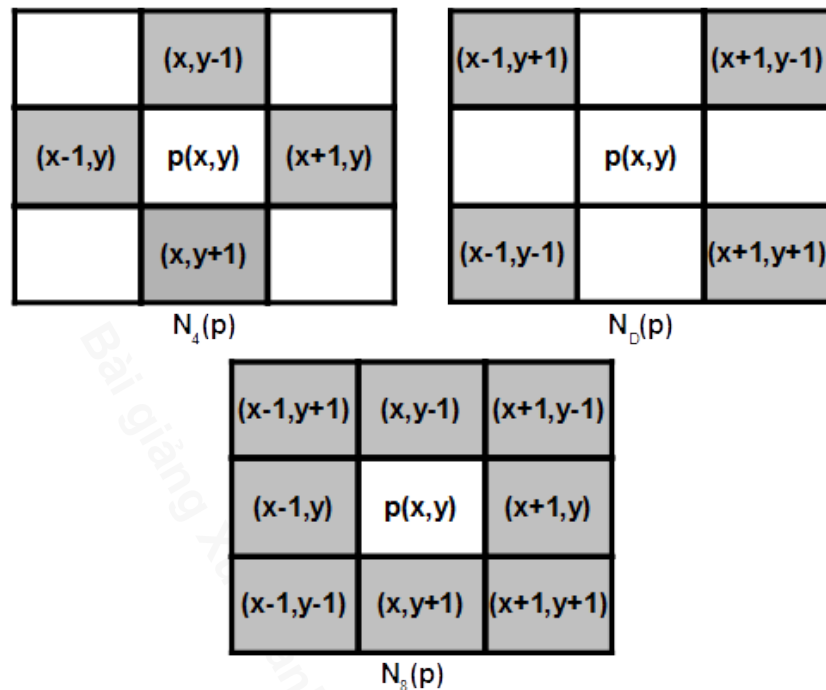
1.3. Tính liên thông

Các điểm ảnh tạo thành một thành phần cục bộ của ảnh được kết nối với nhau dựa trên khoảng cách gần nhau và sự tương đương về màu sắc. Trong phần này chúng ta sẽ xem xét các khái niệm về tính liên thông giữa các điểm ảnh, khái niệm liên kề, khái niệm vùng và đường bao.

Một điểm ảnh p tại vị trí (x,y) có bốn điểm lân cận theo các chiều ngang và dọc được cho bởi các tọa độ: $(x+1,y)$, $(x-1,y)$, $(x,y+1)$, $(x,y-1)$. Tập các điểm này được gọi là 4-neighbors hoặc ký hiệu là $N_4(p)$.

Bốn điểm lân cận theo đường chéo của p có tọa độ là: $(x+1,y+1)$, $(x+1,y-1)$, $(x-1,y+1)$, $(x-1,y-1)$. Tập các điểm này ký hiệu là $N_D(p)$.

Tập tám điểm tạo thành từ $N_4(p)$ và $N_D(p)$ tạo thành một lân cận 8 điểm của p ký hiệu là $N_8(p)$. Các lân cận này được minh họa trong hình sau:



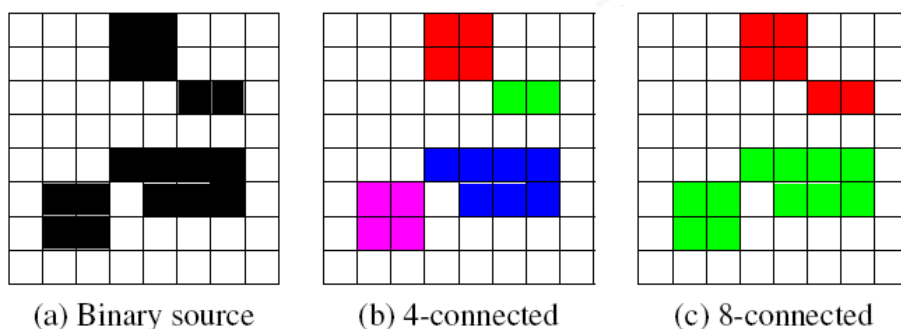
Hình 6.4: Các mẫu lân cận của một điểm ảnh

Khái niệm liên kề (adjacency): hai điểm ảnh p và q được gọi là 4-adjacent nếu q nằm trong tập $N_4(p)$. p và q được gọi là 8-adjacent nếu q nằm trong tập $N_8(p)$.

Khái niệm liên thông (connectivity): Một tập các điểm màu đen P là một thành phần liên thông 4-connected nếu với mỗi cặp p_i, p_j thuộc P đều tồn tại một dãy các điểm $p_1, \dots, p_j$ sao cho:

- + tất cả các điểm đều là màu đen
- + tất cả các cặp điểm liên kề nhau trong dãy đều là 4-neighbors.

Tương tự ta có định nghĩa về thành phần liên thông 8-connected.



Hình 6.5: Ví dụ về các thành phần liên thông

Khái niệm vùng (region): Cho R là một tập các điểm trong một ảnh, R được gọi là một vùng của ảnh nếu nó là tập điểm liên thông với nhau.

Khái niệm đường bao (Boundary): Đường bao (còn gọi là contour hay border) của một vùng R là một tập các điểm trong vùng (R) có ít nhất một điểm lân cận (neighbor) không nằm trong R .

2. Các phép toán hình thái trên ảnh nhị phân

Hai thao tác biến đổi hình thái cơ bản là phép co ảnh (erosion) và phép giãn ảnh (dilation), các thao tác phức tạp hơn đều được thực hiện bằng sự kết hợp giữa hai thao tác cơ bản này.

2.1. Phép giãn ảnh (dilation)

Phép giãn ảnh được sử dụng để mở rộng một phần tử A bằng cách sử dụng một phần tử cấu trúc B theo công thức:

$$A \oplus B = \{z | (\hat{B})_z \cap A \neq \emptyset\}$$

Để thực hiện công thức này trước hết cần thực hiện tính tập phản chiếu của B dựa vào điểm gốc tọa độ của nó và dịch tập phản chiếu này dựa trên z. Có thể thấy rằng phép giãn ảnh của A và B là một tập các chuyển vị z sao cho $\hat{B}$ và A giao nhau (overlap) ít nhất một phần tử, do đó công thức trên có thể viết lại như sau:

$$A \oplus B = \{z | [(\hat{B})_z \cap A] \subseteq A\}$$

Giá trị output sẽ là đen nếu như ít nhất một điểm đen của phần tử cấu trúc B được đặt trên một điểm ảnh màu đen. Khi phần tử cấu trúc làm việc trên điểm nằm trên cạnh (edge) của một thành phần của ảnh, nó sẽ được giãn ra.

2.2. Phép co ảnh (erosion)

Ngược lại với phép giãn ảnh, thao tác co ảnh sẽ thu hẹp một phần tử A bằng cách sử dụng một phần tử cấu trúc B theo công thức sau:

$$A \ominus B = \{z | [(B)_z \cap A] \subseteq A\}$$

Theo công thức này, kết quả của phép co ảnh của A bởi B là tập các điểm z mà khi dịch B theo z thì ta được một điểm thuộc A.

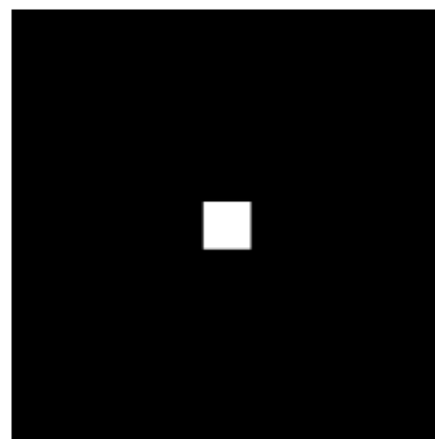
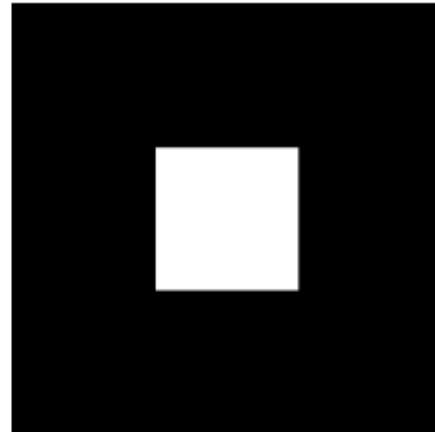
Giá trị output sẽ là đen nếu như tất cả các điểm đen của phần tử cấu trúc đều đặt được lên trên một điểm đen. Khi phần tử cấu trúc làm việc trên các điểm nằm trên cạnh của một thành phần của ảnh sẽ làm cho nó bị co lại.

Một ứng dụng đơn giản nhất của phép co ảnh là loại bỏ các chi tiết không phù hợp (về mặt kích thước) trong ảnh. Trong ví dụ minh họa ở hình 6.7, trước hết ảnh được áp dụng phép co để loại bỏ các phần tử có kích thước nhỏ, sau đó phép giãn được thực hiện để khôi phục các đặc điểm cần giữ lại.

$$\begin{matrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{matrix}$$
 Phần tử cấu trúc Ảnh Input



Phép giãn ảnh



Phép co ảnh

Hình 6.6: Phép giãn ảnh và co ảnh



Hình 6.7: Ứng dụng phép co ảnh để loại bỏ các phần tử không phù hợp

2.3. Phép đóng ảnh (closing) và phép mở ảnh (opening)

Phép đóng và phép mở ảnh là hai thao tác dựa trên phép co và giãn ảnh. Phép mở ảnh sẽ chia các thành phần có các kết nối hẹp trong khi phép đóng ảnh sẽ hợp hai thành phần gần nhau lại thành một.

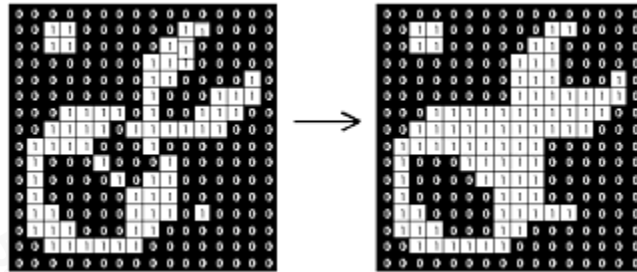
Phép mở một ảnh A bởi một phần tử cấu trúc B được thực hiện bằng phép co ảnh giữa A và B , sau đó là phép giãn ảnh giữa A và B theo công thức sau:

$$A \circ B = (A \ominus B) \oplus B$$

Hình 6.7 chính là minh họa cho một phép mở ảnh.

Phép đóng một ảnh A bởi một phần tử cấu trúc B được thực hiện bằng phép giãn ảnh A bởi B, sau đó là phép co ảnh giữa với A với B theo công thức sau:

$$A \cdot B = (A \oplus B) \ominus B$$



Hình 6.8: Phép đóng ảnh

Phép mở ảnh thường được dùng để làm mịn các đường bao (contour), hay loại bỏ các chi tiết lỗi của một đối tượng trong ảnh. Trong khi đó phép đóng ảnh thường dùng để làm mịn các giao của các đường bao, loại bỏ các lỗ nhỏ và lấp đầy các khoảng trống của các đường bao. Các thao tác này có thể áp dụng nhiều lần nhưng chỉ có tác dụng một lần.

2.4. Biến đổi hit-and-miss

Đây là phép biến đổi cơ bản dựa trên phép toán hình thái để thực hiện bài toán shape detection. Giả sử chúng ta cần tìm vị trí của một đối tượng X trong một ảnh lớn hơn A:

- + Giả sử X được bao bởi một hình chữ nhật W
- + Nền cục bộ của X được xác định dựa trên W là tập hiệu $W-X$.
- + Áp dụng phép co ảnh A bởi X sẽ cho chúng ta một tập các vị trí gốc của X. Tập các điểm này có thể xem (về mặt hình học) là các điểm gốc của X mà tại đó có sự khớp nối (hit) giữa X với A.
- + Áp dụng phép co ảnh trên tập bù của A với nền cục bộ ($W-X$)
- + Tập các điểm mà tại đó X khớp với vị trí thực sự của nó trong ảnh A chính là giao của hai tập kết quả của hai bước cuối cùng ở trên.

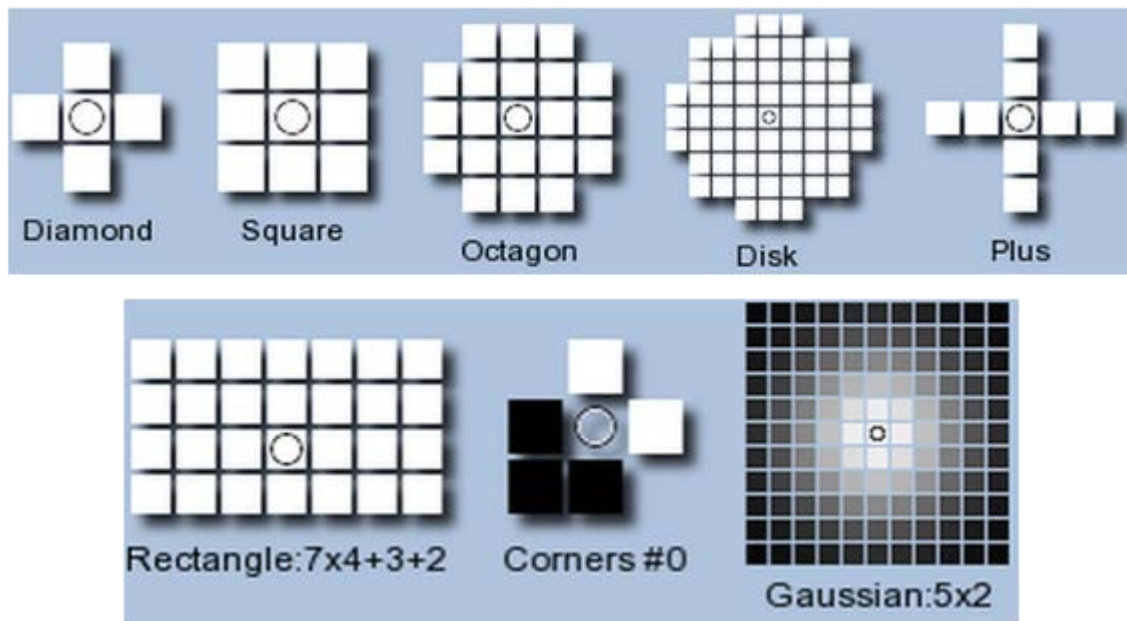
Giả sử X được tạo bởi hai tập là chính nó và nền B_1, B_2 ($B_1=X, B_2=W-X$) ta sẽ có các công thức sau:

$$A * B = (A \ominus B_1) \cap (A^c \ominus B_2)$$

Tập kết quả sẽ chứa tất cả các điểm gốc mà tại đó B_1 khớp (hit) với vị trí của nó trong A còn B_2 khớp với các vị trí của nó trong A^c .

Trên thực tế chúng ta đôi khi phải tìm các mẫu cụ thể gồm các phần tử 0 và 1 chứ không tìm các đối tượng một cách riêng rẽ, và trong các trường hợp đó, nền là không cần thiết, khi đó phép biến đổi hit-and-miss sẽ chỉ còn là phép toán co ảnh đơn giản. Mô hình phát hiện mẫu đơn giản này được dùng trong các thuật toán để phát hiện các ký tự trong văn bản.

2.5. Một số kiểu phần tử cấu trúc thông dụng



Hình 6.9: Các kiểu phần tử cấu trúc

Như đã trình bày ở phần đầu chương, các phép xử lý hình thái đều thực hiện với các ảnh nhị phân và làm việc dựa trên một phần tử cấu trúc theo nguyên lý so sánh phần tử cấu trúc với lân cận của mỗi điểm ảnh để đưa ra kết quả. Mỗi phần tử cấu trúc về mặt hình ảnh có thể xem là một ảnh (trong hình 6.9, các ô trắng được biểu diễn tương ứng với giá trị phần tử dương, các ô không được biểu diễn tương ứng với phần tử có giá trị bằng 0) nhưng trên thực tế là một mảng các số thực với một phần tử của mảng được gán là gốc (hay trung tâm, được biểu diễn qua hình tròn nhỏ trong hình 6.9). Các kiểu phần tử cấu trúc được dùng trong xử lý hình thái được chia theo hình dáng và được dùng vào các mục đích khác nhau.

Diamond: kiểu phần tử cấu trúc có dạng hình kim cương (mặc dù trong hình 6.6 chưa thể hiện rõ điều này vì kích thước nhỏ). Kiểu phần tử này hay được dùng với thao tác giãn ảnh.

Square: phần tử cấu trúc hình chữ nhật, thường được dùng để lấy trung bình cộng của một lân cận, hoặc mở rộng (giãn) hình dáng các đối tượng trong ảnh lên 1 giá trị điểm ảnh (với kích thước 3x3).

Octagon: kiểu phần tử cấu trúc có hình 8 cạnh.

Disk: kiểu phần tử cấu trúc có hình tròn, thường được dùng trong các trường hợp cần sử dụng phần tử cấu trúc có kích thước lớn.

Plus: kiểu phần tử cấu trúc hình dấu cộng (+), thường ít được sử dụng nhưng lại rất hữu ích khi cần phải nhấn mạnh các điểm đơn trong một ảnh.

Cross: kiểu phần tử cấu trúc hình gạch chéo như hai đường chéo chính của một ma trận, là một bản xoay 45° của kiểu Plus. Kiểu này cũng giống như kiểu Plus, thích hợp để nhấn mạnh các điểm ảnh nhằm đánh dấu vị trí của các điểm đơn trong ảnh.

Ring: kiểu cấu trúc hình dạng chiếc nhẫn, được dùng để xác định vị trí của các vùng ảnh quan tâm (region of interest – ROI) trong ảnh.

Rectangle: khá giống với kiểu cấu trúc Square, thường được dùng để xác định độ dài của các đường thẳng ngang và dọc, cho phép tìm kiếm các đối tượng trong ảnh.

Corner: kiểu phân tử được dùng để xác định (tìm) một mẫu (pattern) đặc biệt trong ảnh, đó là các góc.

Gaussian: kiểu phân tử giống với một Gaussian kernel, được dùng để làm mờ ảnh.

3. Các phép toán hình thái trên ảnh xám

Hoàn toàn tương tự như đối với ảnh nhị phân, các thao tác xử lý hình thái trên ảnh xám cũng dựa vào hai thao tác chính là giãn ảnh và co ảnh, nhưng qui tắc tính giá trị mức xám output cho mỗi điểm ảnh khi áp dụng phân tử cấu trúc thay đổi như sau:

+ Đối với phép giãn ảnh: giá trị output sẽ là *giá trị lớn nhất* trong các điểm trong lân cận của điểm đang xét. Đối với ảnh nhị phân, nếu tồn tại một điểm lân cận nào đó bằng 1 thì giá trị này là 1. Cụ thể công thức thực hiện giãn ảnh f với phân tử cấu trúc b là:

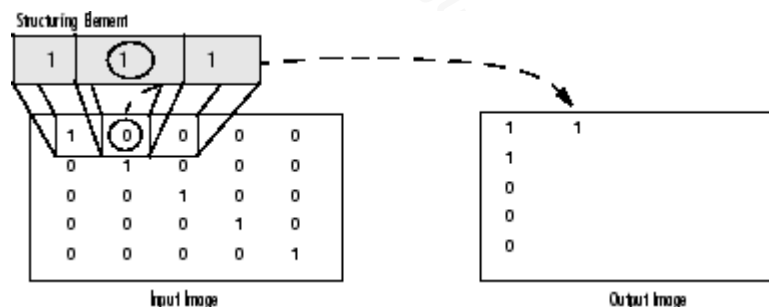
$$(f \oplus b)(s, t) = \max \{f(s - x, t - y) + b(x, y) | (s - x), (t - y) \in D_f, (x, y) \in D_b\}$$

Trong đó D_f và D_b tương ứng là miền giá trị của f và b .

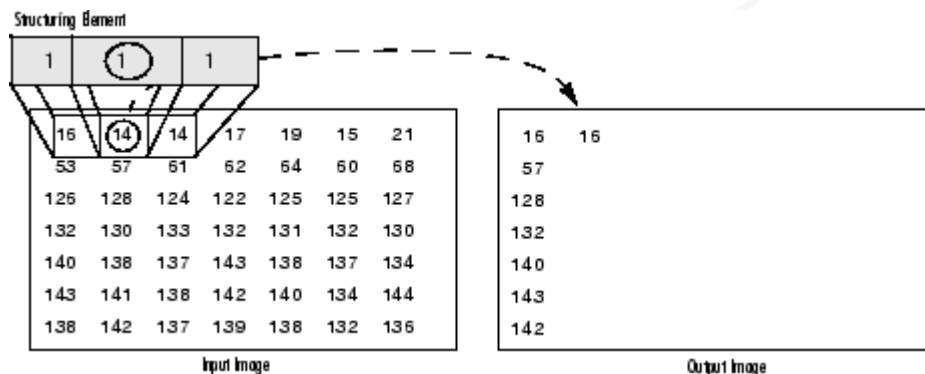
+ Đối với phép co ảnh: giá trị output là *giá trị nhỏ nhất* của tất cả các điểm trong lân cận của điểm đang xét. Đối với ảnh nhị phân, nếu tồn tại một điểm lân cận nào đó bằng 0 thì giá trị này là 0. Phép co ảnh giữa ảnh f và phân tử cấu trúc b được biểu diễn dưới dạng công thức:

$$(f \ominus b)(s, t) = \min \{f(s + x, t + y) - b(x, y) | (s + x), (t + y) \in D_f, (x, y) \in D_b\}$$

Có thể xem các hình minh họa sau để thấy rõ sự khác nhau khi áp dụng các thao tác xử lý hình thái đối với ảnh nhị phân và ảnh xám.



Hình 6.10: Qui tắc áp dụng phép toán hình thái với ảnh nhị phân



Hình 6.11: Qui tắc áp dụng phép toán hình thái với ảnh xám

Đối với thao tác giãn ảnh, nếu các giá trị của phần tử cấu trúc đều dương thì ảnh output sẽ sáng hơn so với ảnh input. Các chi tiết tối sẽ bị giảm hoặc loại bỏ tùy theo giá trị của nó và mối tương quan đối với phần tử cấu trúc sử dụng.

Ngược lại đối với thao tác co ảnh nếu tất cả các giá trị của phần tử cấu trúc là dương thì ảnh output sẽ tối hơn so với ảnh input. Các chi tiết sáng trong ảnh input có kích thước nhỏ hơn phần tử cấu trúc sẽ bị giảm với mức độ giảm phụ thuộc vào các giá trị lân cận của chi tiết và hình dáng, giá trị cụ thể của phần tử cấu trúc.

Ảnh Input



Dilate



Erode



Opening



Closing

Hình 6.12: Các phép xử lý hình thái với ảnh xám

Các thao tác mở ảnh và đóng ảnh đối với ảnh xám cũng được định nghĩa tương tự như đối với ảnh nhị phân dựa trên hai thao tác cơ sở là giãn ảnh và co ảnh. Thao tác mở ảnh sẽ loại bỏ các chi tiết nhỏ sáng, trong khi các chi tiết sáng lớn cũng như mức xám tổng thể của ảnh bị ảnh hưởng rất ít. Ngược lại thao tác đóng ảnh sẽ loại bỏ các chi tiết tối nhỏ và không ảnh hưởng nhiều tới các chi tiết tối lớn và mức xám tổng thể của ảnh.

Các thao tác giãn, co, đóng và mở ảnh xám được minh họa qua hình 6.12.

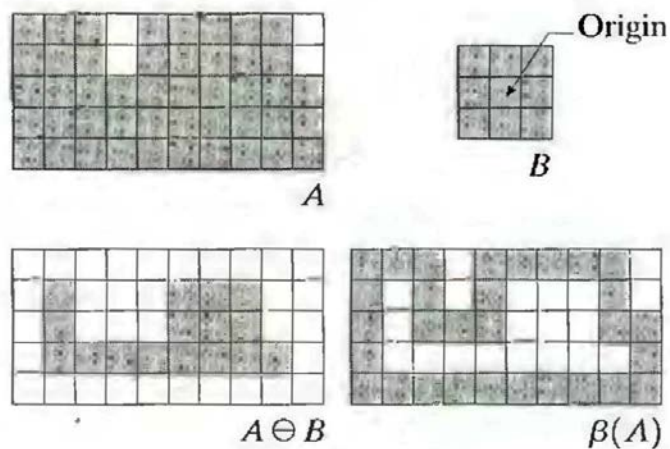
4. Ứng dụng các phép toán hình thái trong xử lý ảnh

4.1. Trích chọn đường bao (contour) sử dụng phép toán hình thái

Để trích chọn đường bao của một đối tượng A sử dụng phần tử cấu trúc B, trước hết ta dùng phép co giữa A và B, sau đó lấy hiệu của A với kết quả của phép co. Độ dày của biên sẽ phụ thuộc vào kích thước của phần tử cấu trúc B. Công thức của phép trích chọn đường bao theo cách này là:

$$\beta(A) = A - (A \ominus B)$$

Ví dụ minh họa:



Hình 6.13: Trích chọn đường bao sử dụng phép toán hình thái

Một ví dụ thực tế hơn:



Hình 6.14: Ví dụ trích chọn đường bao thực tế

4.2. Làm mảnh (thinning) sử dụng phép toán hình thái

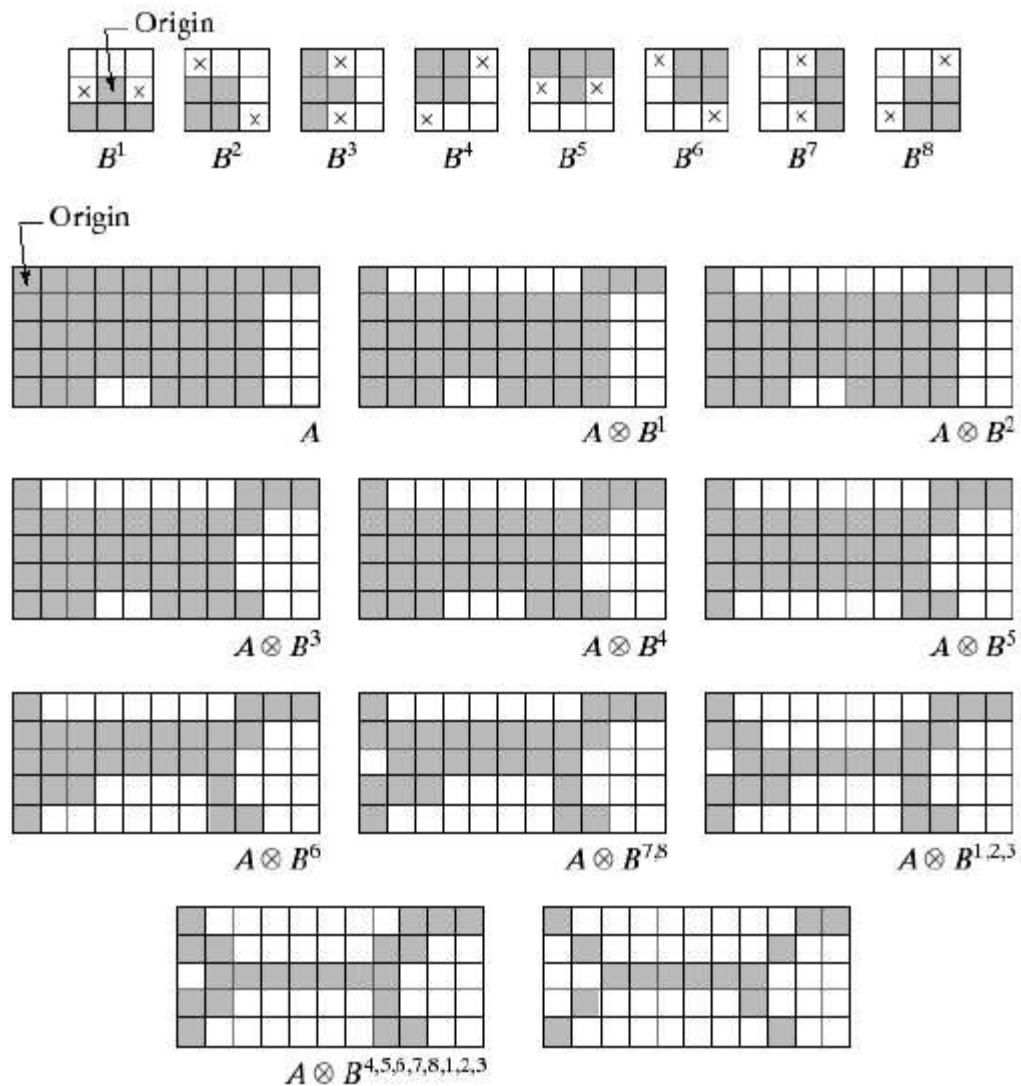
Phép toán làm mảnh (thinning) của một đối tượng A sử dụng phần tử cấu trúc B có thể được định nghĩa dưới dạng một biến đổi hit-and-miss như sau:

$$A \otimes B = A - (A \odot B) = A \cap (A \odot B)^c$$

Một cách khác hiệu quả hơn là biểu diễn B dưới dạng một tập các phần tử cấu trúc: $\{B\} = \{B^1, B^2, \dots, B^n\}$. Trong đó B^i là bản xoay của B^{i-1} . Khi đó phép làm mảnh được định nghĩa lại như sau:

$$A \otimes \{B\} = (((A \otimes B^1) \otimes B^2) \dots) \otimes B^n$$

Quá trình này làm mảnh với B^1 , sau đó tiếp tục với B^2 , và cứ thế cho tới B^n . Cả quá trình sẽ dừng khi A không có thay đổi gì nữa.



Hình 6.15: Các bước của phép làm mảnh

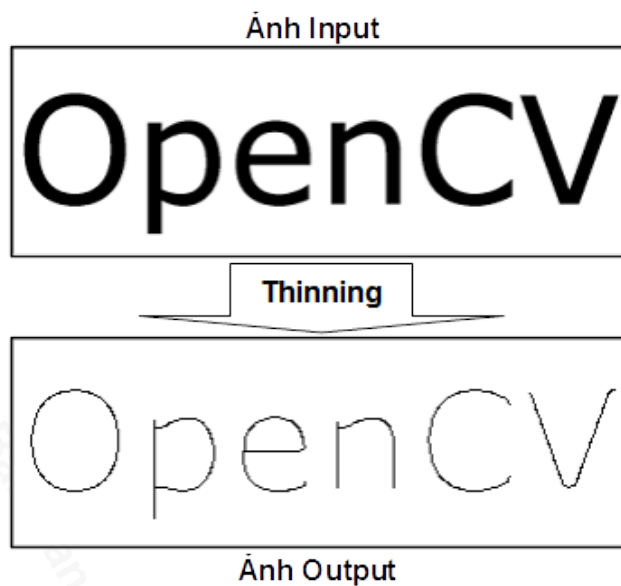
Minh họa của thuật toán được thể hiện ở hình 6.16.

4.3. Trích chọn thành phần liên thông

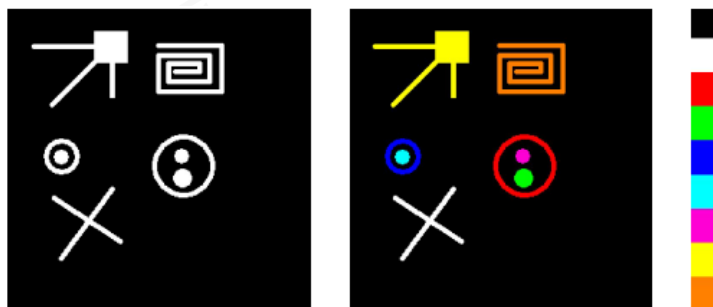
Thuật toán gán nhãn được sử dụng để tìm các thành phần liên thông trong ảnh (các thành phần liên thông thường thuộc về các đối tượng khác nhau). Giả sử Y là một thành phần liên thông của ảnh A, một điểm $p \in Y$ đã biết. Khi đó để gán nhãn cho các thành phần liên thông của A ta làm như sau:

- + Sử dụng phần tử cấu trúc B là một ma trận 3x3 gồm các phần tử đơn vị
- + Đặt $X_0 = p$ và thực hiện lặp: $X_k = (X_{k-1} \oplus B) \cap A$ cho tới khi $X_k = X_{k-1}$.

Minh họa của thuật toán được thể hiện ở hình 6.17.



Hình 6.16: Làm mảnh ảnh bằng phép xử lý hình thái



Hình 6.17: Trích chọn thành phần liên thông

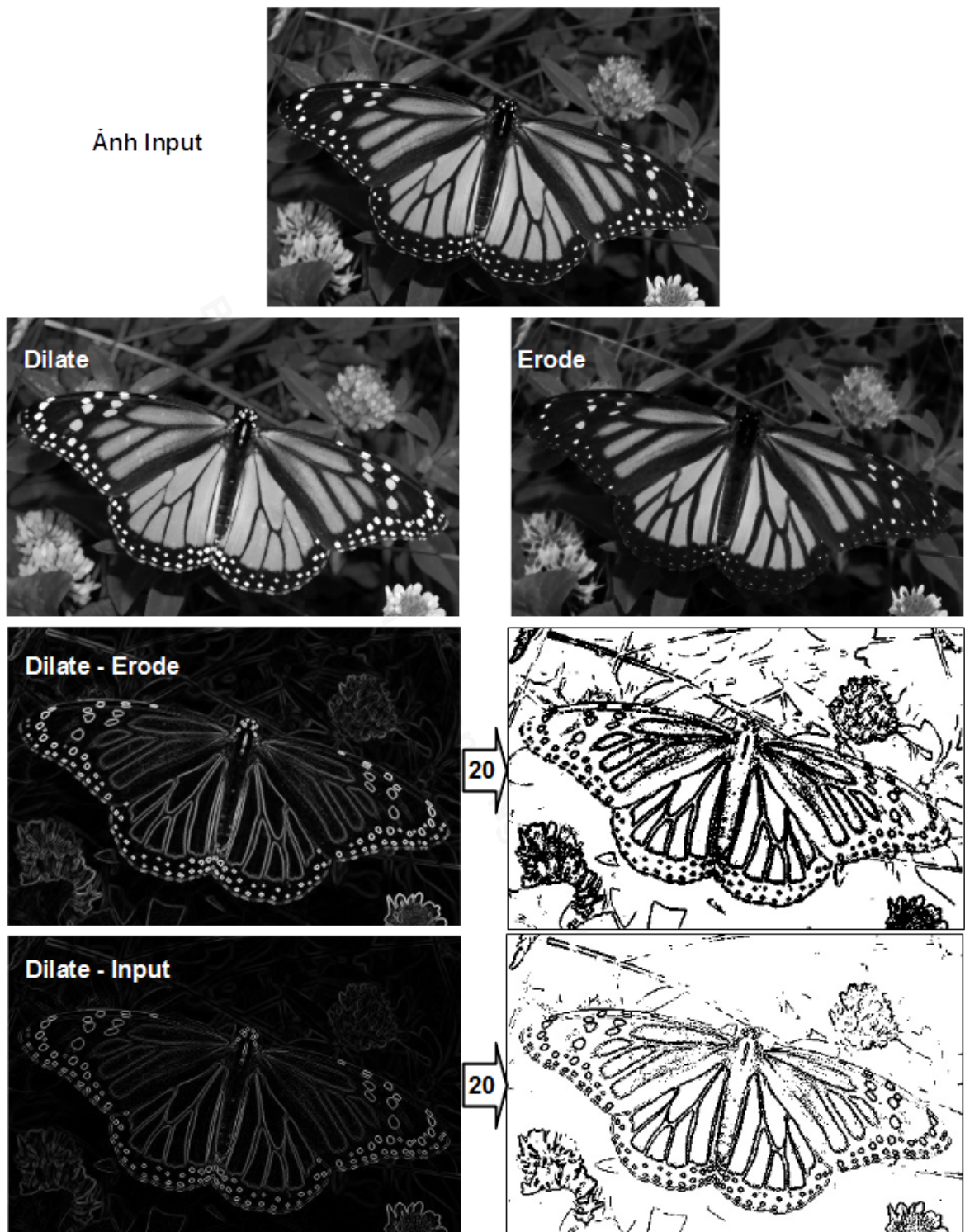
4.5. Làm mờ ảnh dựa vào phép toán hình thái

Ảnh xám có thể được làm mờ nhờ phép xử lý hình thái bằng cách thực hiện một phép mở ảnh, sau đó đóng ảnh. Kết quả của việc kết hợp này loại bỏ hoặc làm suy yếu nhiều hoặc các chi tiết lạ sáng hoặc tối. Xem minh họa ở hình 6.18.



Hình 6.18: Làm mờ ảnh bằng phép xử lý hình thái

4.4. Phát hiện biên (edge detection) dựa vào phép toán hình thái



Hình 6.19: Phát hiện biên bằng phép xử lý hình thái (ngưỡng=20)

Phép toán hình thái có thể được dùng để trích chọn biên của ảnh xám bằng cách lấy hiệu của phép giãn ảnh với phép co ảnh, kết quả ta được ảnh output chứa các biên của ảnh ban đầu. Thao tác này dựa trên thực tế là kết quả của hai phép giãn ảnh và co ảnh chỉ khác nhau ở các điểm thuộc về các biên (edges) của ảnh và còn được gọi bằng thuật ngữ morphological gradient. Một cách hình thức, biên của ảnh xám f được phát hiện dựa vào phần tử cấu trúc b sẽ được tính như sau:

$$g = (f \oplus b) - (f \ominus b)$$

Sau đó ảnh sẽ được tách ngưỡng (chuyển về ảnh nhị phân) để làm nổi bật các biên. Một cách tương tự là lấy kết quả của phép giãn ảnh và trừ đi ảnh gốc, sau đó tách ngưỡng. Có thể xem cụ thể các bước của việc phát hiện biên được minh họa qua hình 6.19.

4.5. Tìm xương (skeleton) của ảnh

Tìm xương của ảnh là giảm kích thước của tất cả các đối tượng trong ảnh thành các đường mảnh (line) mà không làm thay đổi cấu trúc nội tại cơ bản của ảnh, kết quả của cả quá trình này gọi là xương của ảnh. Khái niệm xương (skeleton) $S(A)$ của một đối tượng ảnh A được định nghĩa như sau:

+ Nếu z là một điểm thuộc $S(A)$ và đĩa $(D)z$ là (hình) đĩa lớn nhất có tâm là z và chứa được trong A (với nghĩa là không thể tìm được đĩa có kích thước lớn hơn) thì đĩa $(D)z$ gọi là đĩa lớn nhất.

+ Đĩa $(D)z$ chạm đường bao của A tại hai vị trí hoặc nhiều hơn.

Xương của ảnh A được định nghĩa dưới dạng phép co ảnh và mở ảnh như sau:

$$S(A) = \bigcup_{k=0}^K S_k(A)$$

Với:

$$S_k(A) = (A \ominus kB) - (A \ominus kB) \circ B$$

Trong đó B là phần tử cấu trúc và $A \ominus kB$ là k lần thực hiện phép co ảnh A liên tiếp với phần tử cấu trúc B :

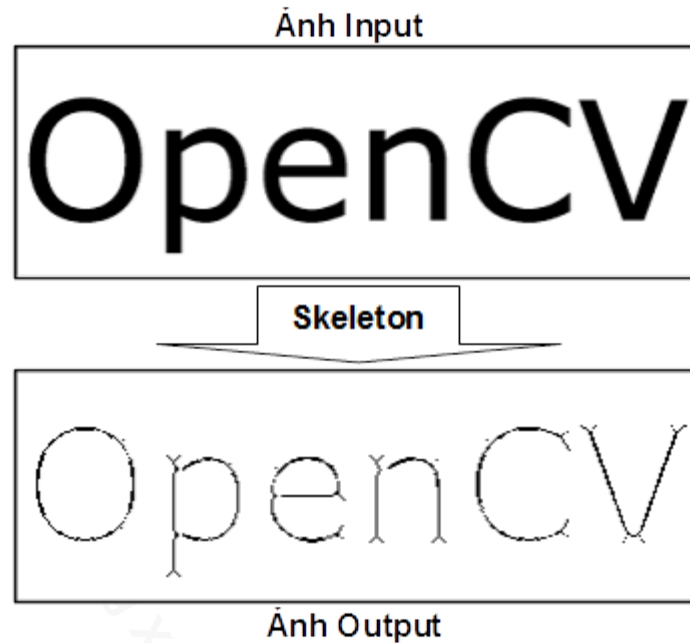
$$(A \ominus kB) = (\dots ((A \ominus B) \ominus B) \ominus \dots) \ominus B$$

Còn K là lần lặp cuối cùng trước khi A bị co thành một tập rỗng:

$$K = \max \{k | (A \ominus kB) \neq \emptyset\}$$

Có thể nói xương $S(A)$ của ảnh A là kết quả của phép hợp các tập xương con $S_k(A)$.

Ví dụ thực hiện tìm xương của ảnh xám:



Hình 6.20: Tìm xương của ảnh bằng phép xử lý hình thái

Bài tập

Bài tập 1: Viết chương trình mở một file ảnh và thực hiện các phép biến đổi hình thái cơ bản với ảnh xám của ảnh đã mở:

- + Giãn ảnh
- + Co ảnh
- + Đóng ảnh
- + Mở ảnh

Bài tập 2: Viết chương trình mở một file ảnh và thực hiện:

- + Làm mảnh
- + Trích chọn biên
- + Tìm xương
- + Làm mượt ảnh

CHƯƠNG VII: PHÂN ĐOẠN ẢNH

1. Khái niệm

Phân đoạn ảnh (Image segmentation) là phép xử lý chia một ảnh thành các phần khác nhau gồm tập hợp các điểm ảnh có quan hệ chặt chẽ với các đối tượng trong ảnh. Đây thường là một bước quan trọng trong phân tích ảnh, biểu diễn các đối tượng ảnh cũng như nhiều thao tác xử lý ảnh và thị giác máy tính khác. Thường phân đoạn ảnh được thực hiện trên ảnh màu thay vì ảnh đen trắng. Lĩnh vực phân đoạn ảnh vẫn là một ngành nghiên cứu khó và đang phát triển. Mặc dù hiện nay chưa có một lý thuyết thực sự cho phân đoạn ảnh song đã có rất nhiều các phương pháp phân đoạn ảnh được đề xuất và trong tài liệu này chúng ta sẽ tập trung vào các cách tiếp cận sau:

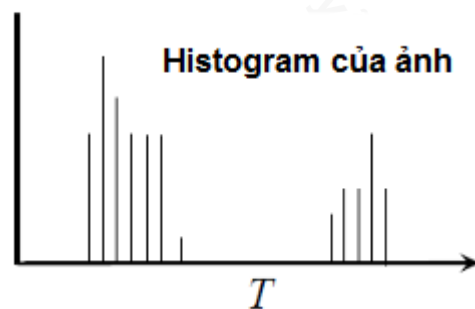
- + Phân đoạn ảnh dựa vào tách ngưỡng (thresholding based): Kỹ thuật cắt và phân ngưỡng histogram được dùng để phân đoạn ảnh.

- + Phân đoạn dựa trên các vùng con của ảnh (region based): Kỹ thuật bắt đầu bằng một điểm nằm giữa đối tượng và mở rộng dần cho tới khi chạm tới đường bao của đối tượng.

- + Kỹ thuật dựa vào thuật toán phân cụm dữ liệu (clustering based): các kỹ thuật phân cụm dữ liệu (data clustering) sẽ được sử dụng để phân ảnh thành các đối tượng.

2. Phân đoạn dựa trên tách ngưỡng

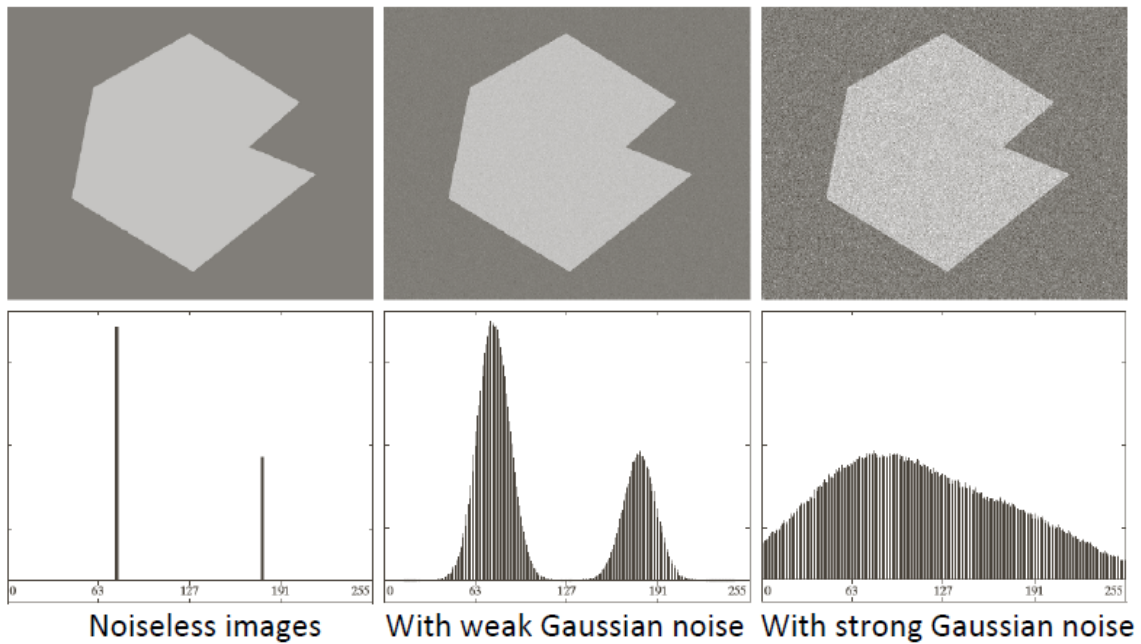
Do tính chất trực quan, đơn giản trong cài đặt, tốc độ, các kỹ thuật phân đoạn ảnh dựa trên tách ngưỡng được sử dụng khá phổ biến. Về nguyên tắc cơ bản giống như kỹ thuật tách ngưỡng nhị phân mà ta đã xem xét ở chương 2. Ảnh sẽ được tách thành hai phần dựa trên một giá trị ngưỡng T được chọn, một phần màu đen (giá trị điểm ảnh bằng 1) tương ứng với nền là các điểm màu tối và một phần màu trắng (0) tương ứng với đối tượng có màu sáng. Có thể quan sát cách mà ngưỡng T được chọn dựa vào phân bố histogram của ảnh gốc như sau:



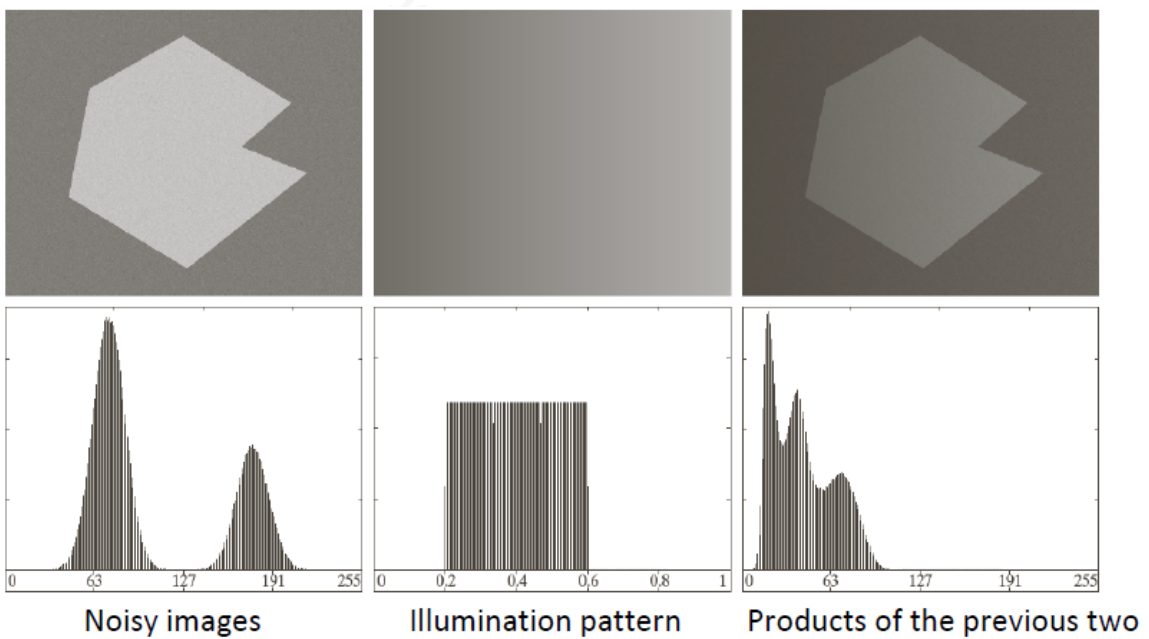
Hình 7.1: Chọn ngưỡng trong phân đoạn ảnh dựa trên kỹ thuật tách ngưỡng

Tuy nhiên có thể thấy rằng kết quả nhận được sẽ phụ thuộc nhiều vào giá trị ngưỡng được chọn như thế nào, đôi khi kết quả sẽ không như chúng ta mong muốn. và kết quả sẽ còn tồi tệ hơn nữa trong các trường hợp ảnh có nhiễu (xem trong hình 7.2 và 7.3). Khi đó nếu không có các bước tiền xử lý để lọc nhiễu thì việc tìm một ngưỡng thích hợp là bất khả thi.

Sự hiệu quả của việc lọc ngưỡng dựa trên cường độ sáng của ảnh liên quan tới hình dáng (chiều cao và độ rộng) của các hình thung lũng trong biểu diễn histogram của ảnh.



Hình 7.2: Ảnh hưởng của nhiễu đối với việc chọn ngưỡng



Hình 7.3: Ảnh hưởng của nhiễu và ánh sáng đối với việc chọn ngưỡng

Các yếu tố chính cần xem xét khi chọn ngưỡng là:

- + Sự tách biệt giữa các đỉnh trong biểu diễn histogram của ảnh, các đỉnh càng xa nhau thì khả năng chọn được ngưỡng tốt càng lớn.
- + Nhiễu trong ảnh.
- + Tính nhất quán của ánh sáng và các đặc điểm phản xạ lại ánh sáng trong ảnh.

2.1. Kỹ thuật chọn ngưỡng toàn cục cơ bản

Kỹ thuật chọn ngưỡng toàn cục cơ bản phân hoạch histogram của ảnh bằng cách sử dụng một ngưỡng toàn cục. Chất lượng của phân đoạn ảnh sẽ phụ thuộc chủ yếu vào việc chọn ngưỡng có tốt hay không. Ngưỡng T được chọn sao cho chênh lệch giữa hai

giá trị trung bình của hai tập điểm sau tách ngưỡng là nhỏ nhất. Theo kỹ thuật này, việc T sẽ được thực hiện theo các bước sau:

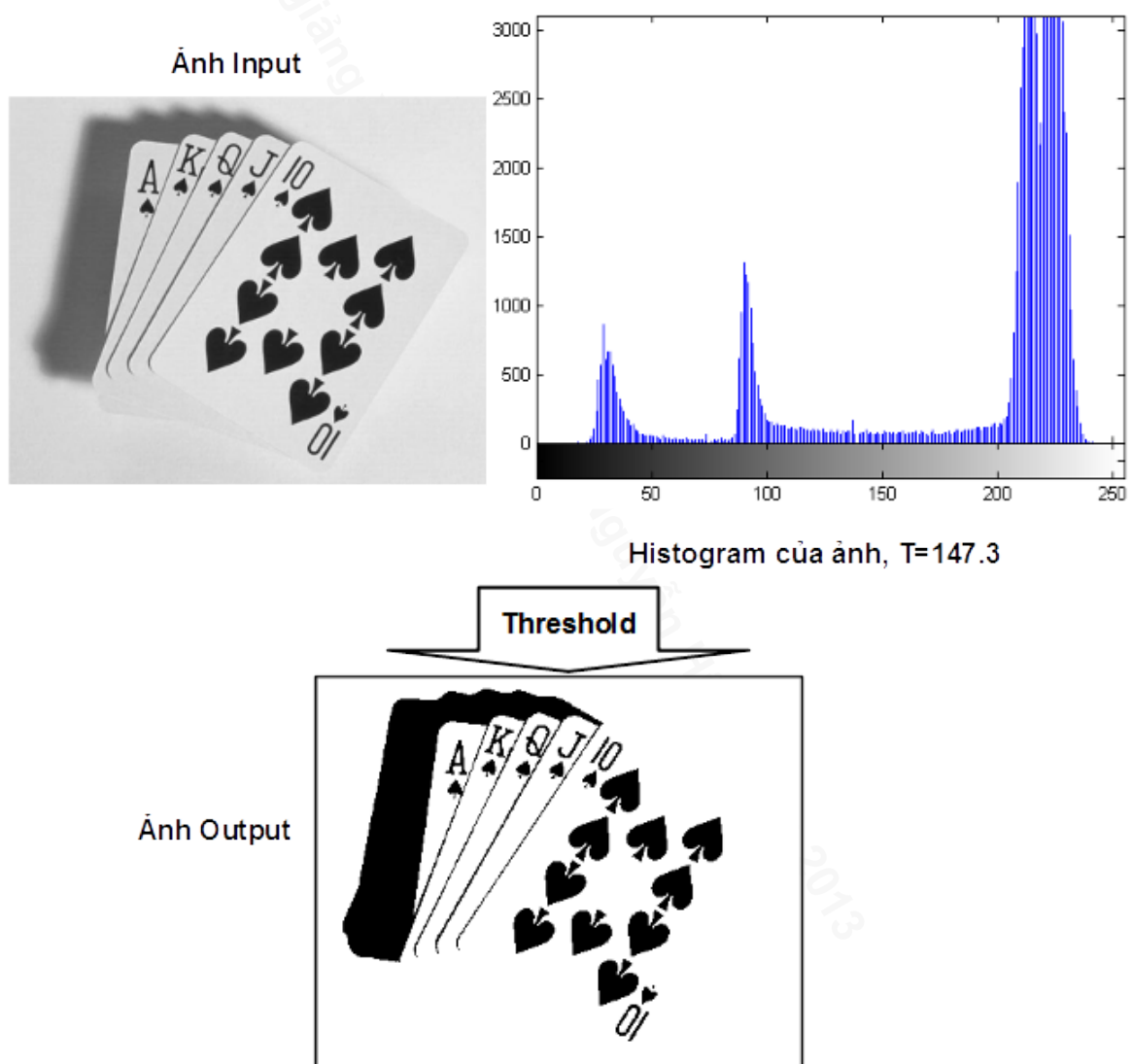
Bước 1: Chọn một giá trị khởi tạo mang tính chất ước lượng cho T, thường là giá trị trung bình của tất cả các mức xám của ảnh.

Bước 2: Phân hoạch ảnh sử dụng T để sinh hai nhóm điểm ảnh: nhóm G_1 gồm các điểm có mức xám $> T$, nhóm G_2 là các điểm còn lại, có mức xám $\leq T$.

Bước 3: Tính giá trị trung bình m_1 và m_2 tương ứng của hai tập G_1, G_2 .

Bước 4: Tính giá trị mới của ngưỡng $T = (m_1 + m_2) / 2$

Bước 5: Lặp lại các bước 2-4 cho tới khi trị hiệu tuyệt đối của hai giá trị T liên tiếp nhỏ hơn một hằng số ∇T chọn trước.



Hình 7.4: Phân đoạn sử dụng chọn ngưỡng toàn cục

Minh họa kết quả của thuật toán chọn ngưỡng toàn cục được thể hiện trong hình 7.4.

2.2. Chọn ngưỡng tối ưu theo phương pháp của Otsu

Thuật toán Otsu (đặt theo tên của Nobuyuki Otsu) cho phép chọn ngưỡng tối ưu hơn phương pháp chọn ngưỡng cơ bản với tiêu chí làm cho giá trị phương sai giữa các lớp (between-class variance) của ảnh đạt cực đại.

Gọi L là số giá trị cường độ sáng của một ảnh có kích thước $M \times N$ và n_i là số điểm ảnh có cường độ sáng i . Khi đó histogram chuẩn hóa sẽ có các thành phần:

$$p_i = \frac{n_i}{MN}, \sum_{i=0}^{L-1} p_i = 1, p_i \geq 0$$

Giả sử một ngưỡng k được chọn sao cho C_1 là tập các điểm có các mức cường độ sáng $[0, 1, \dots, k]$ và C_2 là tập các điểm với các mức cường độ sáng $[(k+1), \dots, (L-1)]$. Giá trị phương sai giữa lớp của hai tập là:

$$\sigma_B^2 = P_1[m_1(k) - m_G]^2 + P_2[m_2(k) - m_G]^2$$

Trong đó $P_1(k)$ là xác suất của C_1 và $P_1(k) = \sum_{i=0}^k p_i$, $P_2(k)$ là xác suất của C_2 và $P_2(k) = 1 - P_1(k)$, $m_1(k)$ và $m_2(k)$ là trung bình cường độ sáng của hai tập C_1 , C_2 tương ứng còn m_G là giá trị trung bình toàn cục $m_G = \sum_{i=0}^{L-1} ip_i$.

Khi đó giá trị ngưỡng tối ưu sẽ là $k^* = \underset{0 \leq k \leq L-1}{\operatorname{argmax}} \sigma_B^2(k)$.

Cụ thể các bước của thuật toán sẽ như sau:

Bước 1: Tính giá trị chuẩn hóa histogram của ảnh với các thành phần p_i , $i=0, 1, \dots, L-1$.

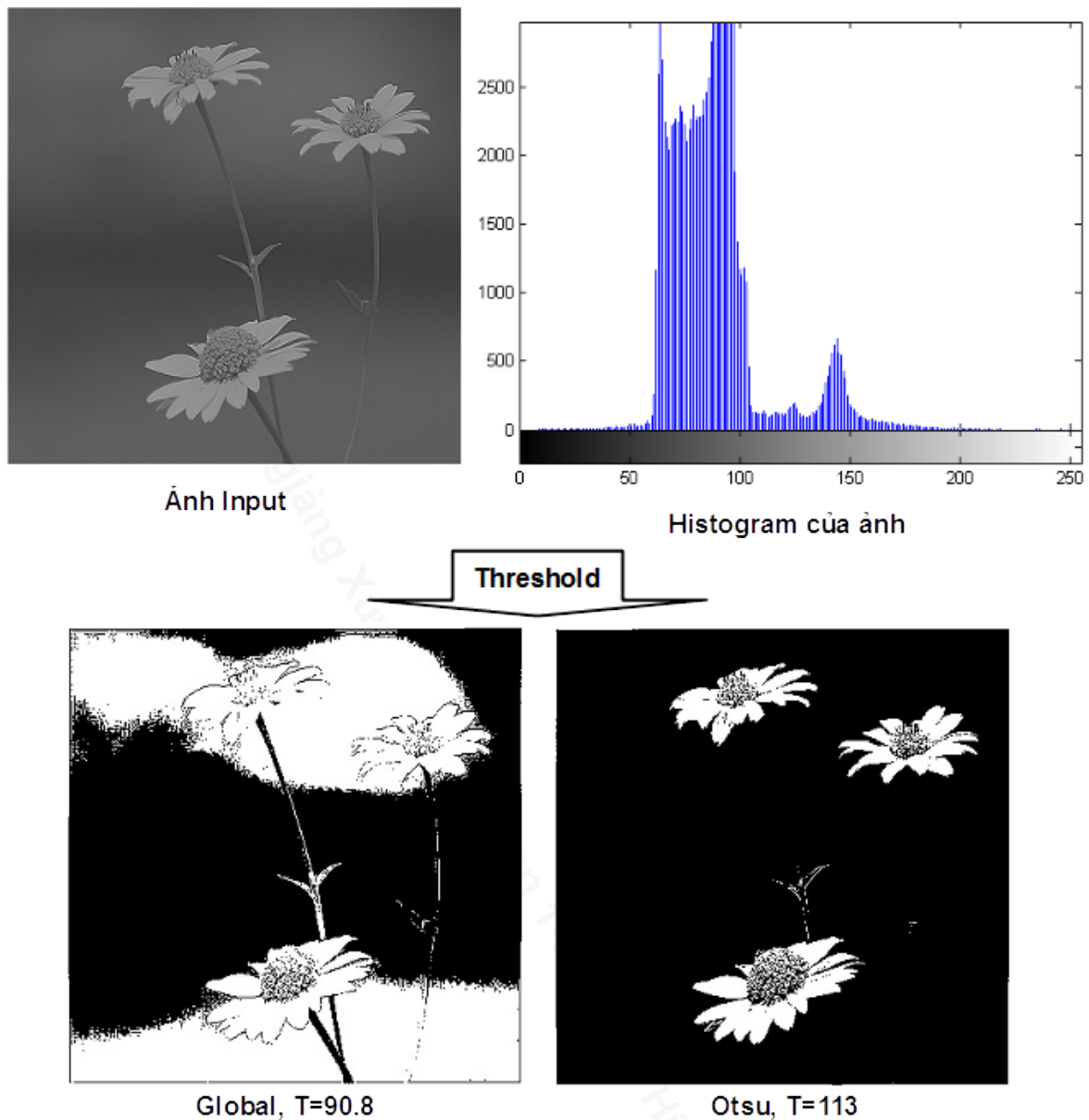
Bước 2: Tính giá trị trung bình toàn cục m_G .

Bước 3: Với mỗi giá trị $0 \leq k \leq L-1$, tính σ_B^2 .

Bước 4: Tính giá trị k^* .

Về cơ bản cài đặt thuật toán Otsu chỉ mất thời gian tính các p_i , tức là mất một vòng lặp đơn chạy MN lần qua các điểm ảnh, các tính toán còn lại là khá đơn giản. Cũng cần lưu ý là việc giá trị between class variance đạt cực đại cũng tương đương với trị within class variance đạt cực tiểu.

Có thể thấy trên hình minh họa 7.5 kết quả của phương pháp Otsu tốt hơn so với phương pháp Global Threshold.



Hình 7.5: Phân ngưỡng bằng phương pháp Otsu

2.3. Phân ngưỡng biến thiên (Variable thresholding)

Các yếu tố nhiễu và ánh sáng ảnh hưởng rất lớn tới hiệu năng của một thuật toán phân ngưỡng. Trong một số trường hợp, mặc dù rất đơn giản, nhưng việc sử dụng một ngưỡng toàn cục không mang lại kết quả khả quan, khi đó chúng ta cần áp dụng một kỹ thuật có tính chất mềm dẻo hơn gọi là kỹ thuật phân ngưỡng biến thiên.

Kỹ thuật phân ngưỡng biến thiên sử dụng giá trị độ lệch chuẩn (standard deviation) và giá trị trung bình của các điểm ảnh trong một lân cận tại tất cả các vị trí của ảnh. Với mỗi điểm ảnh, một giá trị ngưỡng riêng sẽ được dùng để phân ngưỡng.

Gọi σ_{xy} và m_{xy} lần lượt là độ lệch chuẩn và giá trị trung bình của các điểm lân cận có trung tâm là điểm (x, y) của ảnh, giá trị ngưỡng sẽ được tính theo công thức:

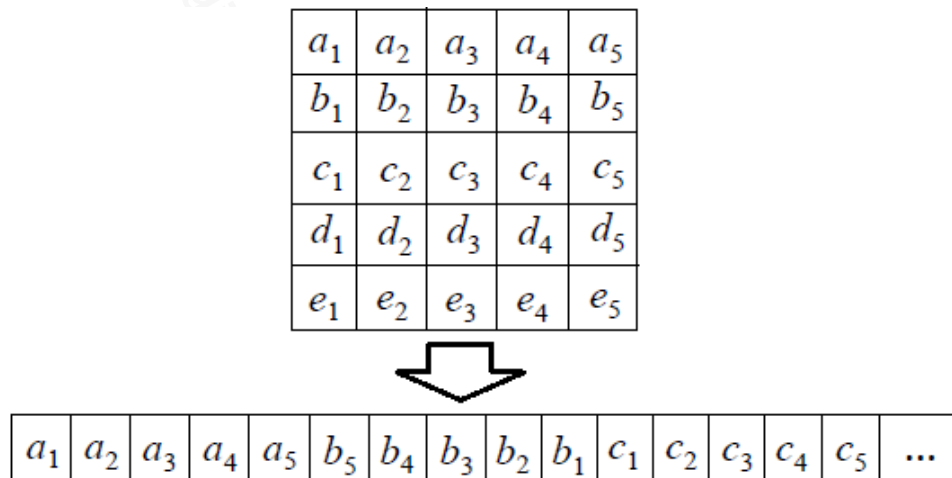
$$T_{xy} = a\sigma_{xy} + bm_{xy}$$

Trong đó a và b là các hằng số không âm. Việc phân ngưỡng được tiến hành cho điểm (x, y) như sau:

$$g(x, y) = \begin{cases} 1 & \text{nếu } f(x, y) > T_{xy} \\ 0 & \text{nếu } f(x, y) \leq T_{xy} \end{cases}$$

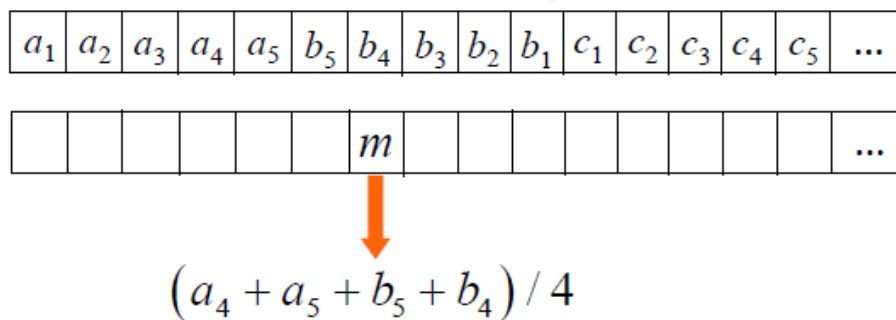
Kỹ thuật này còn gọi là phân ngưỡng cục bộ, khi $a=0$, kỹ thuật này được gọi là moving average, với việc di chuyển giá trị trung bình của một cửa sổ (có chiều cao bằng 1 hàng) dọc theo các đường quét của một ảnh. Moving average đặc biệt hiệu quả khi xử lý các ảnh văn bản khi thực hiện xử lý theo từng dòng với một mẫu hình zigzag để giảm ảnh hưởng của ánh sáng. Có thể hình dung quá trình thực hiện phương pháp moving average như sau:

+ chuyển các điểm ảnh thành một dãy zigzag như hình sau:



Hình 7.6: Biến dữ liệu ảnh thành dãy zigzag

+ tính giá trị ngưỡng tại các điểm của dãy theo qui tắc: ngưỡng của một điểm bằng trung bình cộng của điểm đó với $n-1$ điểm đứng trước nó trong dãy.



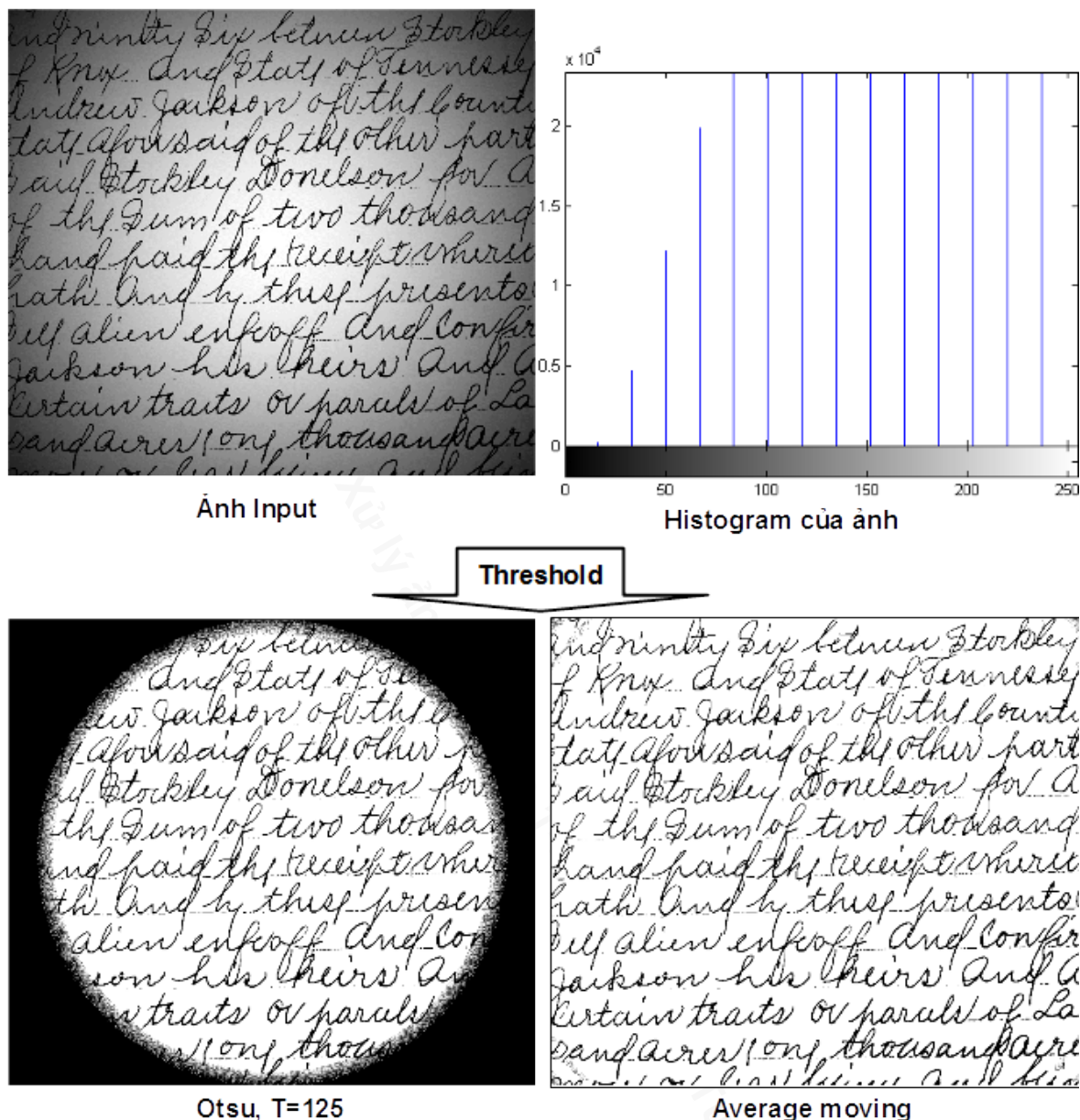
Hình 7.7: Tính ngưỡng cho dãy zigzag

+ chuyển các giá trị ngưỡng ngược về dạng ma trận 2 chiều với các phần tử $m(x, y)$

+ thực hiện phân ngưỡng ảnh ban đầu bằng các ngưỡng đã tính được theo công thức:

$$g(x, y) = \begin{cases} 1 & \text{nếu } f(x, y) > Km(x, y) \\ 0 & \text{nếu } f(x, y) \leq Km(x, y) \end{cases}$$

Có thể thấy rõ hiệu quả của kỹ thuật phân ngưỡng cục bộ qua so sánh kết quả thực hiện với phương pháp Otsu qua hình sau:



Hình 7.8: So sánh phân ngưỡng cục bộ và Otsu

3. Phân đoạn dựa trên vùng con của ảnh

Trong đa số các trường hợp phân ngưỡng ảnh trên thực tế, histogram của ảnh không phân bố một cách đơn giản để có thể chọn một ngưỡng tối ưu. Tuy vậy các điểm thuộc về cùng một đối tượng được quan tâm lại có sự đồng nhất ở một số khía cạnh khác như màu sắc, cường độ sáng, vân ảnh ... Kỹ thuật dựa vào vùng ảnh (region growing) khai thác các yếu tố đồng nhất trong ảnh để phân đoạn ảnh. Kỹ thuật này chỉ phân đoạn được một đối tượng ảnh với mỗi lần thực hiện nên nếu có k đối tượng, ta cần phải áp dụng k lần.

Gọi $P(x,y)$ là một điểm bắt đầu nằm trong một đối tượng trong ảnh. Mục đích của phân đoạn là tìm ra vị trí của tất cả các điểm thuộc về đối tượng. Gọi S là tập vị trí của các điểm. Khi khởi tạo, S chỉ gồm 1 điểm có vị trí (x, y) . Thuật toán gồm các bước sau:

Bước 1: Xét các điểm lân cận (xN, yN) của (x, y) .

Bước 2: Đánh giá giá trị điểm ảnh $P(xN, yN)$ bằng cách so sánh nó với $P(x, y)$.

+ nếu có sự tương đồng thì đưa vị trí (xN, yN) vào tập S

Bước 3: Lặp lại bước 1 với các điểm mới được thêm vào tập S.

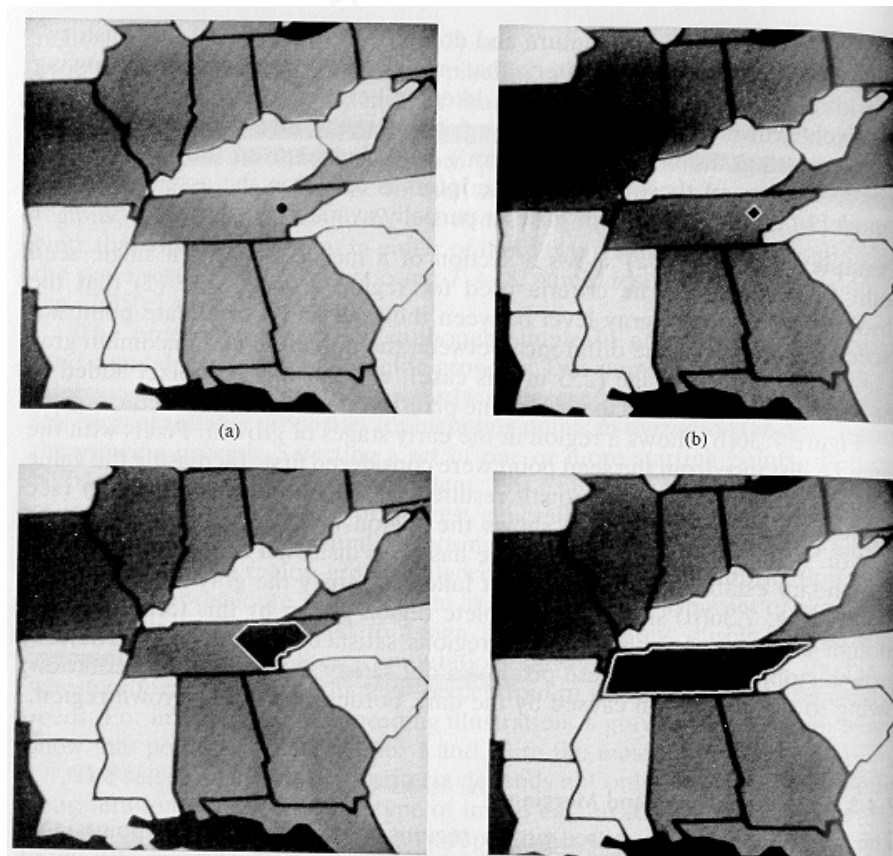
Bước 4: Nếu không có điểm mới nào được thêm vào thì kết thúc thuật toán.

Tập S sẽ chứa vị trí của các điểm thuộc về vùng hoặc đối tượng cần quan tâm. Các vị trí mới được thêm vào tập S sẽ dựa trên tiêu chí về độ tương đồng dùng để đánh giá hai điểm ảnh. Có hai tiêu chí thường được sử dụng:

+ dựa trên giá trị trung bình: nếu giá trị cường độ sáng của điểm đang xét nằm trong một khoảng được xác định trước của giá trị trung bình của tập S thì nó được xem là tương đồng.

+ dựa vào phương sai: nếu việc thêm một điểm vào tập S không làm thay đổi phương sai của nó trong một khoảng xác định trước thì được xem là tương đồng.

Việc xác định các khoảng cho hai tiêu chí trên thường là không tự động và dựa trên ảnh và đối tượng cần quan tâm. Có thể xem cách thức hoạt động của phân ngưỡng dựa trên vùng trong hình minh họa sau:



Hình 7.9: Phân đoạn ảnh dựa trên vùng Region Growing

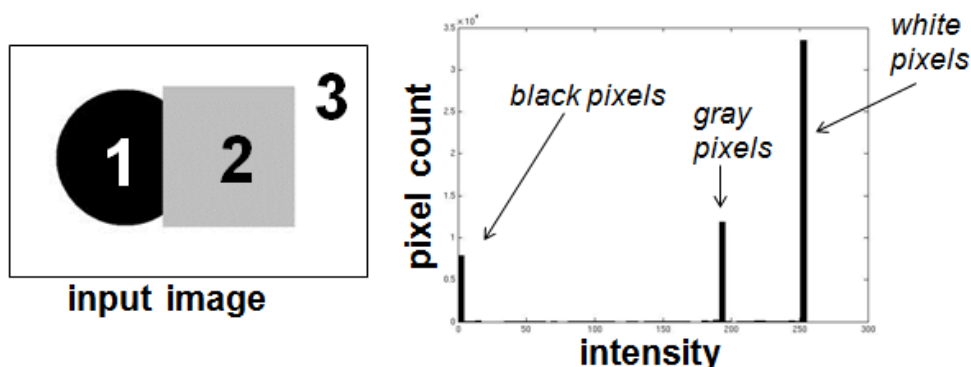
Một số nhược điểm của phương pháp phân đoạn ảnh dựa trên vùng:

+ giả thiết chính cho cách tiếp cận này là các vùng gần như có cường độ sáng là hằng số.

+ phương pháp này không thể áp dụng cho các ảnh có các vùng mà sự thay đổi cường độ không diễn ra một cách trơn tru (smoothly) như các vùng vân.

4. Phân đoạn ảnh dựa vào kỹ thuật phân cụm

Chúng ta hãy xem ví dụ về phân đoạn ảnh trong hình minh họa sau:



Hình 7.10: Cơ sở của phân đoạn ảnh dựa vào phân cụm

Có thể thấy rõ ràng rằng các cường độ sáng của ảnh cần phân đoạn gồm 3 nhóm. Chúng ta có thể gán nhãn (phân đoạn) cho tất cả các điểm của ảnh bằng cách dựa vào các giá trị cường độ sáng chủ yếu của nó. Vậy làm thế nào chúng ta có thể gán các điểm ảnh của ảnh input trong hình 7.8 cho các thành phần của ảnh, cụ thể ở đây là 3 thành phần, câu trả lời là dùng một thuật toán phân cụm dữ liệu trên giá trị cường độ sáng của các điểm ảnh. Đây chính là nguyên lý cơ bản của việc phân đoạn ảnh dựa vào phân cụm dữ liệu.

Mục tiêu của việc phân cụm giờ đây là làm thế nào để chọn ra các giá trị trung tâm (centroid) làm đại diện cho các cường độ sáng của mỗi thành phần, khi đó các điểm thuộc về cùng một thành phần sẽ có khoảng cách gần với trung tâm của thành phần đó nhất.

Một phân cụm tốt sẽ giảm tối thiểu giá trị Sum of Square Error (SSE), là đại lượng đo độ tán xạ (scatter) của dữ liệu-tính bằng tổng bình phương các khoảng cách Euclidean từ các điểm thuộc cùng một cụm (cluster) tới trung tâm của nó.

$$J_{SSE} = \sum_{i=1}^k \sum_{x \in D_i} \|x - \mu_i\|^2$$

Với mục tiêu này chúng ta sẽ có một bài toán non-parametric (tức là loại bài toán chỉ biết các ràng buộc về nghiệm chứ không có mô tả cụ thể cho nghiệm của bài toán-loại parametric) thuộc loại “con gà và quả trứng”:

+ Nếu biết các điểm trung tâm, chúng ta có thể xác định được các điểm thuộc về mỗi cụm bằng cách gán các điểm gần nhất cho nó.

+ Ngược lại nếu biết các điểm thuộc về một cụm, chúng ta có thể dễ dàng tính ra điểm trung tâm bằng cách lấy trung bình cộng của chúng.

Trong phần tiếp theo chúng ta sẽ xem xét hai kỹ thuật phân cụm nổi tiếng nhất áp dụng cho bài toán phân đoạn ảnh là kỹ thuật K-mean và kỹ thuật Mean-shift.

4.1. Thuật toán K-mean

Về mặt ý tưởng, thuật toán K-mean tương đối đơn giản, nhưng đây lại là một trong 10 thuật toán data mining hiệu quả nhất. Ban đầu chúng ta chọn ngẫu nhiên k điểm trung tâm của mỗi cụm và lặp lại hai thao tác tính toán, cụ thể như sau:

Bước 1: Khởi tạo ngẫu nhiên các giá trị trung tâm của k cụm: $c_1, c_2, \dots, c_k$.

Bước 2: Với mỗi điểm trung tâm, xác định các điểm thuộc về cụm mà nó là trung tâm bằng cách:

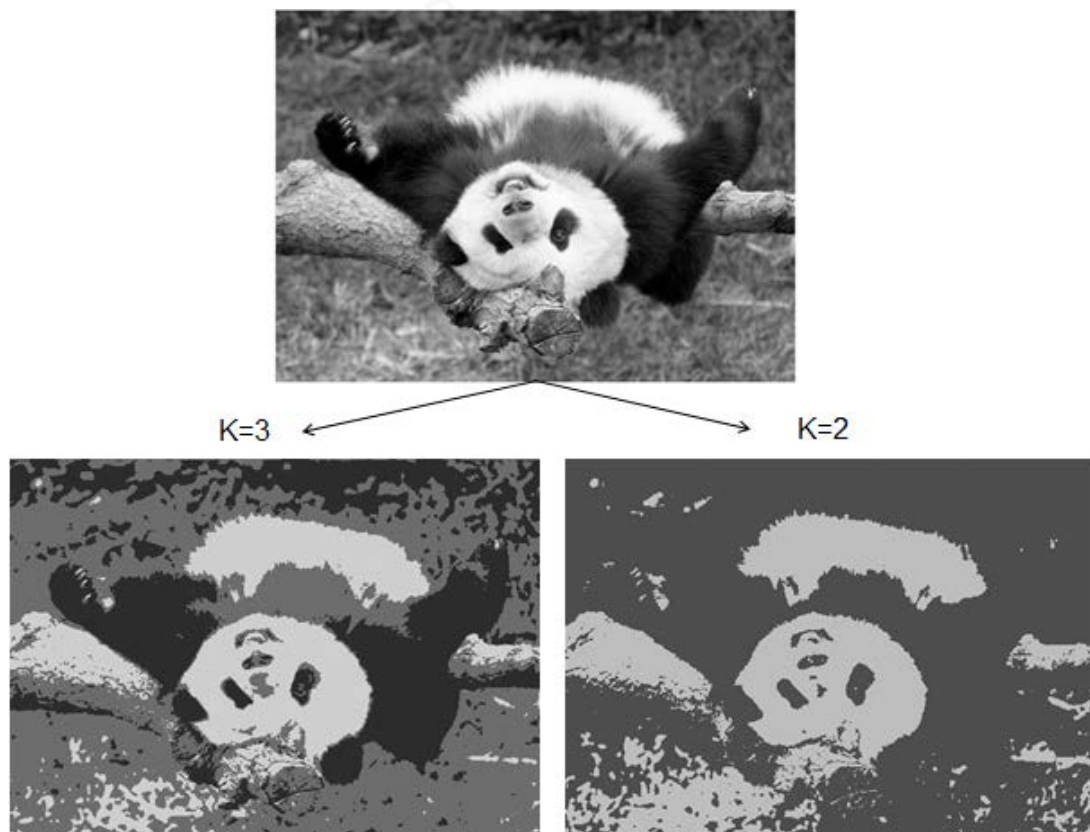
+ với mỗi điểm p , tìm c_i gần với nó nhất, gán p vào cụm thứ i

Bước 3: Với các điểm đã được phân cho các cụm, tính lại các giá trị cho các điểm trung tâm $c_1, c_2, \dots, c_k$ bằng cách:

+ gán c_i bằng trung bình cộng của các điểm thuộc về cụm i .

Bước 4: Nếu có giá trị c_i nào thay đổi, lặp lại bước 2.

Thuật toán K-mean nhanh, luôn luôn hội tụ và cho một nghiệm của bài toán phân cụm, tuy nhiên các nghiệm này có thể không phải là tối ưu toàn cục mà là tối ưu cục bộ. Thêm nữa, nghiệm của bài toán là không duy nhất, khi thay đổi các giá trị khởi tạo, ta sẽ có các nghiệm khác nhau, và làm thế nào có thể chọn k cũng là một vấn đề.



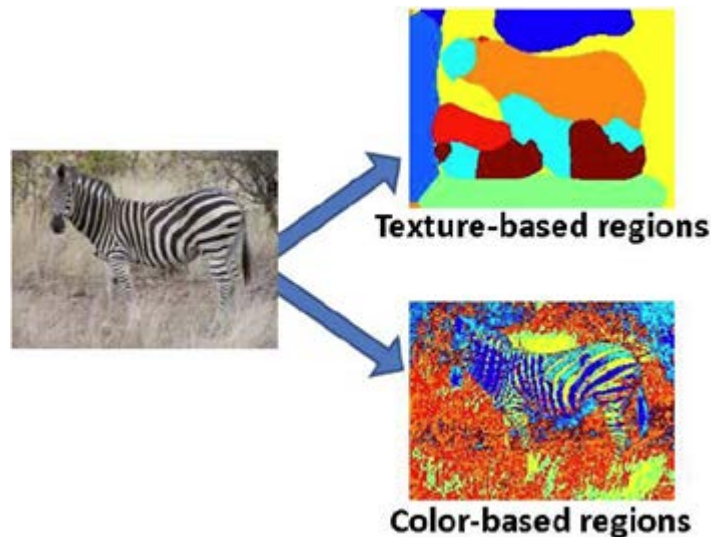
Hình 7.11: Phân đoạn ảnh bằng thuật toán K-mean

Dựa vào các tiêu chí khác nhau, các điểm ảnh có thể được phân cụm theo các cách khác nhau:

+ Dựa vào màu sắc, khi đó không gian phân cụm sẽ là 3-D cho các tổ hợp màu RGB.

- + Dựa vào độ tương đồng của cường độ sáng.
- + Dựa vào độ tương đồng giữa cường độ sáng và vị trí.
- + Dựa vào độ tương đồng của vân ảnh (texture).

Hình sau cho ta một minh họa về các kết quả phân đoạn khác nhau với các tiêu chí chọn độ tương đồng khác nhau:



Hình 7.12: Phân đoạn ảnh K-mean theo các tiêu chí độ tương đồng khác nhau

4.2. Thuật toán Mean-shift

Thuật toán Mean-shift có thể coi là thuật toán hiệu quả nhất hiện nay cho bài toán phân đoạn ảnh. Các bước của thuật toán để tìm vị trí hội tụ cho mỗi cửa sổ mean-shift như sau:

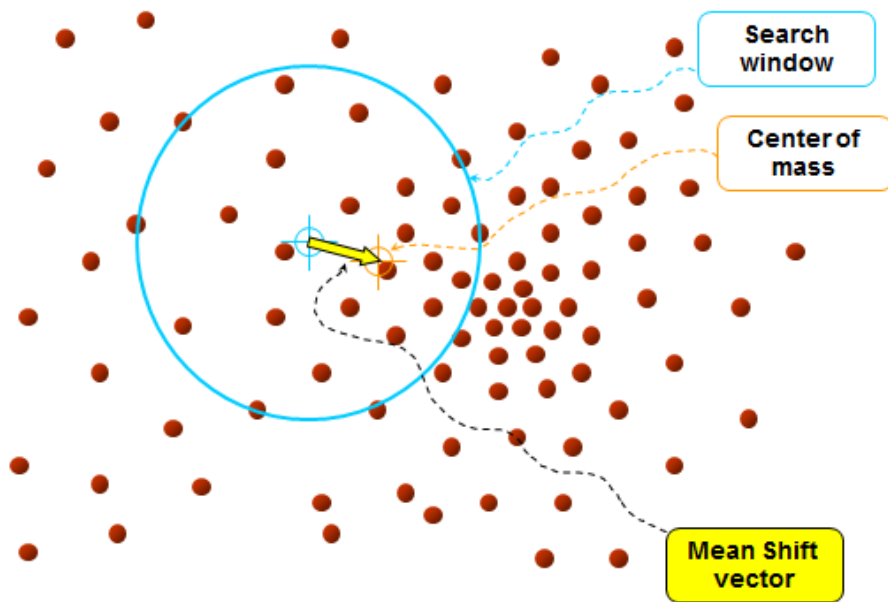
Bước 1: Chọn một kích thước của sổ tìm kiếm

Bước 2: Chọn một vị trí khởi tạo trong cửa sổ tìm kiếm

Bước 3: Tính vị trí của giá trị trung bình trong cửa sổ tìm kiếm

Bước 4: Đặt lại trung tâm của cửa sổ tìm kiếm vào giá trị trung bình tính được ở bước 3.

Bước 5: Lặp lại bước 3 và 4 cho tới khi hội tụ.



Hình 7.13: Thuật toán phân cụm Mean-shift

Về cơ bản, kỹ thuật mean-shift tìm kiếm các điểm có mật độ dữ liệu phân bố cao nhất. Sau đây là thuật toán mean-shift cho phân đoạn ảnh:

Bước 1: Chuyển ảnh thành không gian đặc điểm (với mỗi điểm ảnh được biểu diễn bằng một phần tử dữ liệu qua màu sắc, cường độ sáng, texture, gradient ...).

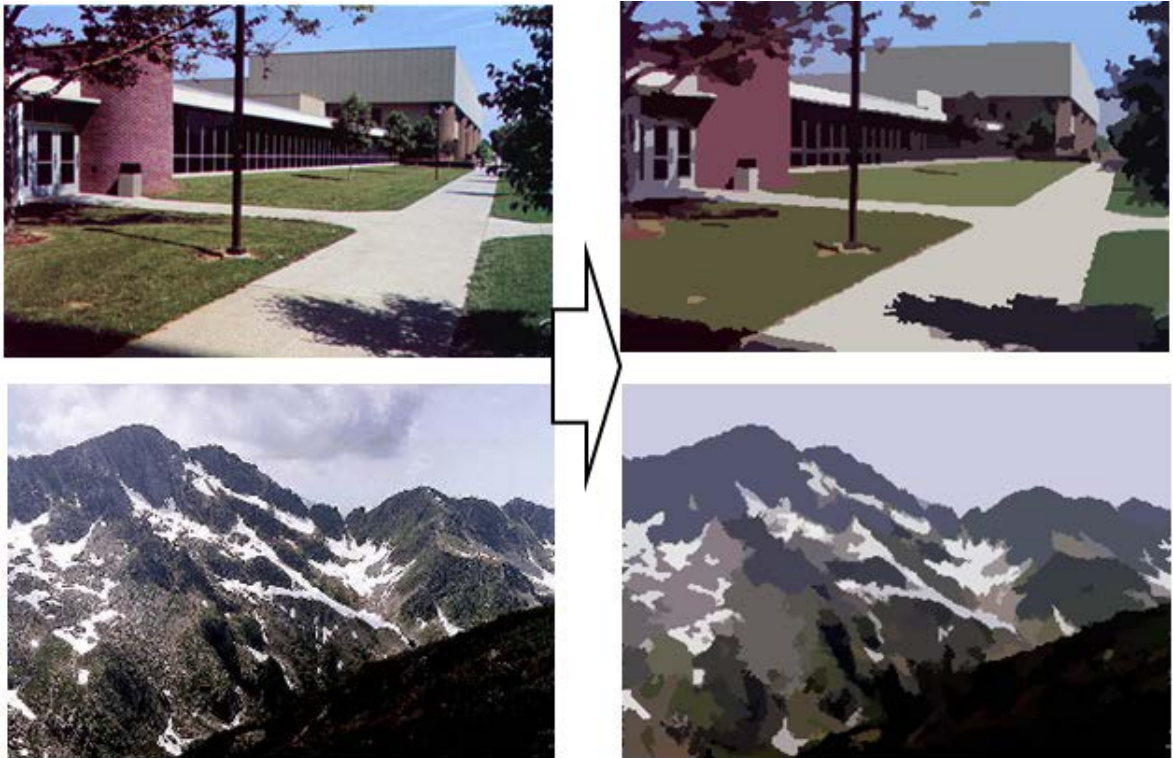
Bước 2: Chọn các vị trí khởi tạo của các cửa sổ tìm kiếm trong không gian đặc điểm.

Bước 3: Tìm vị trí của cửa sổ mean-shift cho mỗi vị trí khởi tạo.

Bước 4: Trộn các cửa sổ có cùng đỉnh (vị trí của cửa sổ mean-shift).

Bước 5: Dữ liệu nhận được sau khi trộn các cửa sổ mean-shift chính là kết quả phân đoạn ảnh.

Có thể xem kết quả thực hiện phân đoạn bằng thuật toán Mean-shift trong hình sau:



Hình 7.14: Kết quả phân đoạn ảnh bằng thuật toán Mean-shift

Tuy kết quả là khá tốt song thuật toán Mean-shift vẫn có những vấn đề sau:

- + làm thế nào để chọn được kích thước cửa sổ phù hợp?
- + việc chọn kích thước cửa sổ và vị trí khởi tạo khác nhau sẽ cho các kết quả khác nhau.

Bài tập

Bài tập 1: Viết chương trình mở một file ảnh và thực hiện phân đoạn theo các phương pháp đã học:

- + Dựa vào tách ngưỡng: tách ngưỡng toàn cục, Otsu, average moving
- + Region growing
- + K-mean
- + Mean-shift

TÀI LIỆU THAM KHẢO

- [1] Rafael C. Gonzalez, Richard E. Woods, “*Digital Image Processing*”, 3rd Edition, Prentice Hall, 2007
- [2] R.C. Gonzalez, R.E. Woods, and S. Eddins, “*Digital Image Processing Using MATLAB*”, 2nd edition, Pearson Prentice Hall, 2008
- [3] C. Solomon and T. Breckon, “*Fundamentals of Digital Image processing: A Practical Approach with Examples in MATLAB*”, John Wiley & Sons, 2011.
- [4] Oge Marques, “*Practical Image and Video Processing Using MATLAB*”, Wiley, 2011.
- [5] Shervin Emami, Khvedchenia Levgen, Naureen Mahmood, Jason Saragih, Roy Shilkrot, David Millan Escriva, and Daniel Lelis Baggio, “*Mastering OpenCV with Practical Computer Vision Projects*”, Packt Publishing, 2012.
- [6] Tomáš Svoboda , Jan Kybic , and Václav Hlaváč, “*Image Processing, Analysis, and Machine Vision: A MATLAB Companion*”, Thomson Learning, 2007
- [7] “OpenCV”. <http://www.opencv.org>
- [8] Itseez, “*The OpenCV Tutorial 2.4.3*”, http://docs.opencv.org/opencv_tutorials.pdf
- [9] Peter Kovesi, “*MATLAB and Octave Functions for Computer Vision and Image Processing*”, <http://www.csse.uwa.edu.au/~pk/research/matlabfns/>.
- [10] “Wikipedia”, <http://en.wikipedia.org/wiki/>